



**This electronic thesis or dissertation has been
downloaded from the University of Bristol Research
Portal, <http://research-information.bristol.ac.uk>**

Author:

Shen, Zichao

Title:

**Advanced Human Activity Recognition and Identification Using Edge Processors and
Millimeter-Wave Radar Technology**

General rights

Access to the thesis is subject to the Creative Commons Attribution - NonCommercial-No Derivatives 4.0 International Public License. A copy of this may be found at <https://creativecommons.org/licenses/by-nc-nd/4.0/legalcode>. This license sets out your rights and the restrictions that apply to your access to the thesis so it is important you read this before proceeding.

Take down policy

Some pages of this thesis may have been removed for copyright restrictions prior to having it been deposited on the University of Bristol Research Portal. However, if you have discovered material within the thesis that you consider to be unlawful e.g. breaches of copyright (either yours or that of a third party) or any other law, including but not limited to those relating to patent, trademark, confidentiality, data protection, obscenity, defamation, libel, then please contact collections-metadata@bristol.ac.uk and include the following information in your message:

- Your contact details
- Bibliographic details for the item, including a URL
- An outline nature of the complaint

Your claim will be investigated and, where appropriate, the item in question will be removed from public view as soon as possible.

Advanced Human Activity Recognition and Identification Using Edge Processors and Millimeter-Wave Radar Technology

By

ZICHAO SHEN



Department of Electrical and Electronic Engineering
UNIVERSITY OF BRISTOL

A dissertation submitted to the University of Bristol in accordance with the requirements of the degree of DOCTOR OF PHILOSOPHY in the Faculty of Engineering.

SEPTEMBER 2024

Word count: 44321

ABSTRACT

Human Activity Recognition (HAR) has progressed significantly with various sensors and edge processors. Deploying HAR on edge devices like wearables requires fast processing and low energy use, as well as enhancing privacy by keeping data local. Meanwhile, researchers are shifting from vision-based methods to radar-based approaches for non-intrusive HAR, with Millimeter-Wave (mmWave) radars becoming popular for their high-resolution spatial data. This thesis provides a comprehensive review of HAR using different sensors and edge processors, focusing on deep learning and mmWave techniques, and covers the fundamentals of these methods that support HAR applications.

This thesis initially demonstrates the feasibility of deploying neural networks on various edge processors for HAR tasks, accompanied by a performance comparison. An adaptive neural network solution is proposed to accelerate the HAR inference process by a factor of 5, reduce energy consumption by approximately 80%, and maintain a comparable level of HAR accuracy (above 90%) compared to the baseline method on the UCI-HAR dataset.

Subsequently, this thesis investigates the application of mmWave technology for human tracking and fall detection. A framework that collaborates three mmWave radars in real time is proposed to track multiple individuals. It achieves an F1 score of 98.4% (a harmonic mean of precision and sensitivity) for tracking a single target, and 97.3% and 95.9% for tracking two and three targets, respectively. Furthermore, for human fall detection, the overall accuracy reaches 96.3%, with a sensitivity of 99.0% for detecting the fall category.

Building upon this framework, this thesis introduces the MmWave-based Human Identification Network (MMIDNet) to investigate indoor identification further. MMIDNet, a neural network architecture specifically designed for mmWave radar point cloud data, is able to classify the identities of the 12 individuals with an overall accuracy of 92.4%. With an extended observation duration from 3 seconds to 7 seconds, the accuracy surpasses 98%.

DEDICATION AND ACKNOWLEDGEMENTS

I would like to begin by expressing my deepest gratitude to my supervisors, without whom I could not have completed my PhD. To Professor Naim Dahnoun, your expert guidance and insightful ideas have been instrumental in ensuring the smooth progress of my project. Your support with equipment and teaching me how to conduct research have been crucial to my success. To Professor Jose Nunez-Yanez, thank you for serving as my supervisor during the first two years, which were the most challenging period of my PhD journey. Your valuable suggestions and technical support acted as a guiding light, helping me navigate through those struggles and setting a strong foundation for my PhD. To my annual progress assessor, Professor Martin Cryan, your insightful feedback has been invaluable throughout my research. Thank you for encouraging me to reach my full potential and for believing in my work. I also extend my heartfelt thanks to the paper reviewers and editors for their constructive feedback, which greatly helped in refining my work. Your contributions have been essential to the completion of this thesis.

This thesis is dedicated to my family, whose unwavering belief in me has been a constant source of strength and motivation. Your love, encouragement, and support have been the cornerstone of my academic journey. To my parents, Chengwu and Hongjie, thank you for your unwavering faith in my potential and for providing me with the funding supports to succeed. I am deeply grateful for your unwavering belief in my potential and your steadfast support throughout my journey. Thank you for providing me with the financial resources and opportunities necessary to pursue my aspirations, as well as for instilling in me the values of perseverance and determination. To my cousin, Qichen, and his parents, Qiao and Hong, thank you for always standing by my side and providing me with valuable feedback. To my other family members, Yuhang, Jian, and Tao, your belief in me has helped me overcome challenges, and your sacrifices have motivated me to keep striving. You have been my rock, my inspiration, and my greatest advocates.

I would like to express my deepest gratitude to the academics, colleagues, and friends at the University of Bristol who have played an integral role in my academic journey. Special thanks to Guangwei, Zhihua, Lichang, Shujun, Qijia, Lianghao, Qi, Minghui, Elliott, Kaan, Victor, Kris, Matt, Charles, Xingtang, Han, Jiacheng, Binyu, Juefei, Yichang, Yiming, Yue, and many others for their invaluable guidance, camaraderie, and support. Your valuable feedback, ideas, and collaboration have been instrumental in my progress. Together, we accomplished numerous projects, contributed to teaching units as teaching assistants, participated in academic meetings and conferences, and celebrated countless milestones. From pizza parties to memorable dinners at homes and bars, we shared meaningful moments that fostered both connection and friendship. These past four years have been an incredible and truly unforgettable experience.

I am also deeply grateful to my dear friends who have supported me beyond the academic sphere. To Minxuan, Rentian, Renle, Mingke, Maggie, Minxi, Di, Mengying, Xiaohan, Xiaoxuan, Zhi, Juncheng, Hongjia, Zhonghui, Krystal, Lesley, Dominic, Timo, CarefulLee, Ariel, Chengen, Ai, Xiaohan, Chenghao, Chenxiao, Shuye, Jingrong, Xinyi, Fan, Yutong, Yumu, Jinzhe, Shuran,

Qingyu, Kaixuan, and many others—you have been an incredible part of my life. We shared interesting stories through online chats and spent meaningful time together in person over meals, travels, and countless adventures. From picnics and barbecues to seaside trips, skiing, kart racing, and more, these moments brought so much happiness into my life, like a refreshing break during PhD heavy workloads. You have supported me both physically and emotionally, comforting me during times of sadness and stress. Your presence and encouragement have helped me overcome challenges with less pain and more strength. Your friendship has been a profound source of comfort, joy, and inspiration. Thank you all for being such an integral part of my life and journey. To my ex-girlfriend, Mingxin, whose love and support brought joy and strength during the past three years. I'm sorry to have lost you—this is dedicated to the youth we once shared.

Finally, I am grateful for the enriching experiences and opportunities provided by life at the University of Bristol over the past eight years. The vibrant community and dynamic academic atmosphere have played a significant role in my personal and professional growth. This thesis is a testament to the collective support and encouragement I have received from all those mentioned, and I am profoundly grateful to each one of you. Our stories will never end.

AUTHOR'S DECLARATION

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is my own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

SIGNED: DATE:

TABLE OF CONTENTS

	Page
List of Tables	xiii
List of Figures	xv
List of abbreviations	xxi
 1 Introduction	 1
1.1 Contribution	3
1.2 Publications	4
1.3 Thesis Outline	4
 2 Background and related work	 7
2.1 HAR with Low-power Edge Devices	8
2.1.1 Hardware for Edge Applications	9
2.1.2 Algorithmic Techniques for Edge AI	10
2.1.3 Frameworks for Edge AI Deployment	11
2.2 Sensors for HAR	12
2.2.1 Wearable Sensors	12
2.2.2 Non-wearable Sensors	13
2.2.2.1 Camera	13
2.2.2.2 WiFi	14
2.2.2.3 MmWave Radar	15
2.2.2.4 LiDAR	16
2.2.2.5 Environment Sensors	16
2.2.3 Multi-modality Fusion Methods	17
2.3 HAR Applications	17
2.3.1 Human Detection and Position Tracking Approaches	17
2.3.2 Human Activities and Postures Recognition Approaches	18
2.3.3 Human Fall Detection Approaches	19
2.3.4 Human Identification Approaches	19

3	Fundamentals	23
3.1	MmWave Radar Fundamentals	23
3.1.1	Processing Chain for FMCW Radar	24
3.1.2	Range Measurement	25
3.1.2.1	Range Resolution	26
3.1.2.2	Range Maximum	27
3.1.3	Velocity Measurement	27
3.1.3.1	Velocity Resolution	28
3.1.3.2	Velocity Maximum	29
3.1.4	Angle Measurement	29
3.1.4.1	Angle Resolution	30
3.1.4.2	Angle Maximum	31
3.1.4.3	Virtual Antennas	31
3.1.5	TI Radar Comparison	32
3.1.6	Radar Configuration	33
3.1.7	Point Cloud Packet Parse	35
3.2	Deep Learning Fundamentals	36
3.2.1	Multilayer Perceptron (MLP)	36
3.2.1.1	Activation Functions	37
3.2.1.2	Softmax	38
3.2.2	Convolutional Neural Network (CNN)	38
3.2.2.1	Convolutional Kernel and Operation	40
3.2.2.2	Pooling Layer	41
3.2.3	Long Short-Term Memory Network (LSTM)	41
3.2.4	PointNet	42
3.2.4.1	Invariance under Rigid Transformations	43
3.2.4.2	Permutation Invariance	43
3.2.5	Network Training and Hyperparameters	44
3.2.5.1	Normalization	44
3.2.5.2	Overfitting	45
3.2.6	Deep Learning Framework	46
3.2.7	DBSCAN Clustering	47
4	Human activity recognition with low power processors	51
4.1	Overview	52
4.2	Low-Power Microcontroller Evaluation	53
4.3	Adaptive Neural Network Methodology	57
4.3.1	Dataset	57
4.3.2	"Big" + Six "Little" Configuration	58

4.3.3	"Big" + "Dual" Configuration	62
4.3.4	"Big" + Distance Configuration	64
4.4	Neural Network Microcontroller Deployment	66
4.4.1	STM32L4R5ZI	66
4.4.2	Apollo2 Blue	68
4.4.3	SparkFun Edge (Apollo3 Blue)	68
4.4.4	ECM3532	69
4.5	Results and Discussion	70
4.5.1	"Big"-Only	71
4.5.2	"Big" + Six "Little"	71
4.5.3	"Big" + "Dual"	73
4.5.4	"Big" + Distance	74
4.6	Conclusions	76
5	Human tracking and fall detection using mmWave radar	79
5.1	Overview	80
5.2	Radar System Evaluation	81
5.2.1	Radar Resolution	81
5.2.2	Multiple Radar Arrangement	83
5.2.2.1	Radar Interference	83
5.2.3	Radar Placement and Coverage Evaluation	84
5.3	Experimental Setup	87
5.3.1	Radar Characterisation	87
5.3.2	Data Collection	88
5.4	System Design	89
5.4.1	Radar Raw Data Preprocessing	91
5.4.1.1	Rotation Compensation	91
5.4.1.2	Position Compensation	92
5.4.2	Multiple Radar Data Lines Synchronization	92
5.4.2.1	Synchronization via Accumulation Removal	94
5.4.2.2	Synchronization via Timestamp	95
5.4.2.3	First-In-First-Out Strategy	96
5.4.3	Background Noise Reduction	96
5.4.4	Human Target Detection	97
5.4.5	Human Target Tracking	100
5.4.6	Fall Detection	102
5.4.7	Adaptive Computing	103
5.5	System Evaluation	104
5.5.1	Multiple Human Tracking Evaluation	104

TABLE OF CONTENTS

5.5.2	Human Fall Detection Evaluation	107
5.5.3	Human Fall Posture Estimation	109
5.5.4	System Comparison	111
5.6	Conclusion	111
6	Human identification using mmWave radar	113
6.1	Overview	114
6.2	FAUST Evaluation	114
6.2.1	Point Cloud Generation	115
6.2.1.1	Data Augmentation	115
6.2.2	Network Design	116
6.2.2.1	MLP	116
6.2.2.2	CNN + MLP	117
6.2.3	Results and Comparison	118
6.3	Data Preparation	119
6.3.1	Data Collection	119
6.3.2	Data Preprocessing	119
6.4	System Design	121
6.4.1	Network Structure	121
6.4.1.1	Transformation Block	121
6.4.1.2	Residual CNN Block	123
6.4.1.3	Global Max Pooling	123
6.4.1.4	Bi-LSTM Block	124
6.4.1.5	Dense Block	124
6.4.2	Network Training	125
6.4.3	Prediction with Window Sliding	125
6.5	System Evaluation	126
6.5.1	Identification Performance	126
6.5.2	Evaluation of Network Structure	127
6.5.3	Evaluation of Observation Duration	128
6.5.4	Evaluation of Number of People	129
6.5.5	System Comparison	130
6.6	Conclusion	131
7	Conclusions and future work	133
7.1	Conclusions	133
7.2	Future Work	136
A	Implementation Code for MMIDNet	139

B Configuration Code for MmWave Radar	147
Bibliography	151

LIST OF TABLES

TABLE	Page
2.1 Edge devices comparison.	9
3.1 TI mmWave radar hardware comparison.	33
3.2 Radar configuration and parameters.	34
4.1 Low-power MCU comparison.	53
4.2 "Big" (left) and "little" (right) model parameter details. The pooling layers are hidden.	62
4.3 "Dual" model parameter details. The pooling layers are hidden.	64
5.1 Data collection composition in minutes.	89
5.2 Human tracking performance of the real-time system.	105
5.3 Human tracking & fall detection system comparison.	111
6.1 Different network structure accuracy comparison on the test set.	128
6.2 Human identification system comparison.	130

LIST OF FIGURES

FIGURE	Page
3.1 A single chirp sent from the radar in both amplitude and frequency versus time. . . .	24
3.2 Data processing chain of the TI mmWave radar.	25
3.3 Chirps of TX (purple) and RX (green) modules in FMCW. Two reflected chirps are collected by the RX module, resulting in two corresponding IF signals.	26
3.4 A frame of chirps in FMCW.	28
3.5 AoA estimation through the utilization of multiple antennas.	29
3.6 The antenna layout of TI IWR1843 mmWave radar.	31
3.7 The antenna distribution including TXs, RXs and Virtual Antennas (VAs) for the IWR1843 mmWave radar.	32
3.8 The structure of chirps within a frame (a) and the structure of an individual chirp (b).	35
3.9 An example of the packet structure.	36
3.10 An example of an MLP with an input layer, two hidden layers, and an output layer. A zoomed-in view of a neuron is shown on the right.	37
3.11 Comparison between Sigmoid, Tanh, and ReLU activation functions.	37
3.12 The calculation flow diagram for Softmax activation function.	38
3.13 The CNN structure with convolutional layers, pooling layers and dense layers.	39
3.14 The residual block structure.	39
3.15 The overview of the convolutional layer propagation process with dimensions labelled.	40
3.16 Comparison for max pooling and average pooling approaches.	41
3.17 The cell view of the LSTM structure.	42
3.18 The classification part of PointNet architecture.	43
3.19 Comparison between Batch Normalization (BN) and Layer Normalization (LN) in 2D and 3D.	45
3.20 An example for the overfitting issue. The green line represents an overfitted model, while the black line represents a regularized model.	46
3.21 An example for DBSCAN algorithm, where C is Core points (red), E is Edge (non-Core) points (yellow) and O is Outliers (blue)	48
4.1 CNN structure for the initial performance and energy evaluation of the MCU.	54

LIST OF FIGURES

4.2	MCU initial evaluation in terms of power consumption.	54
4.3	MCU initial evaluation in terms of time cost.	55
4.4	MCU initial evaluation in terms of energy consumption.	56
4.5	The test accuracy of three sensors on the secondary network. The total accelerometer sensor which achieves the highest overall accuracy is chosen as input for the secondary network.	58
4.6	The flowchart for the "big" + six "little" configuration of the adaptive neural network system. The dotted line means only one "little" network model of six is invoked at each loop.	59
4.7	The processing steps for the "big" + six "little" configuration of the adaptive neural network system. Dark blue and brown represent two "little" network models corresponding to the input activities.	59
4.8	"Big" network model structure.	61
4.9	"Little" network model structure.	61
4.10	The flow chart for the "big" + "dual" configuration of the adaptive neural network system. Registers store the previous data and label, ready to be used in the processing within the "dual" network.	63
4.11	The processing steps for the "big" + "dual" configuration of the adaptive neural network system. The two input data blocks represent a pair of adjacent data samples required by the "dual" network.	63
4.12	"Dual" model structure.	64
4.13	The flow chart for the "big" + distance configuration of the adaptive neural network system. Registers store the previous data and label, ready to be used in the processing within the distance calculator.	65
4.14	The processing steps for the "big" + distance configuration of the adaptive neural network system. The two input data blocks represent a pair of adjacent data samples required by the distance calculator.	65
4.15	Comparison of the software tools used in each deployment phase for different MCUs. The colours indicate different sources.	67
4.16	The steps for deploying neural network models on target MCU boards. Black boxes denote trained models, brown boxes denote the library source codes used, and red boxes denote the MCU boards.	67
4.17	The accuracy and the "big" inference counts for four configurations (quantized TFLite format) on the test data set (2947 samples) are evaluated on a PC.	70
4.18	The time evaluation of four adaptive configurations on MCU boards with the "big" inference counts. A total of 60 data samples extracted from the UCI-HAR test data set are tested to form the evaluation.	72

4.19	The power and energy evaluation of four adaptive configurations on MCU boards. A total of 60 data samples extracted from the UCI-HAR test data set are tested to form the evaluation.	73
4.20	The output comparison of two configurations when an error occurs at the moment of an "activity changes". Errors are marked in red.	74
4.21	Distance results for different pairs of adjacent data samples. Values in one distance measurement (one row) are comparable.	74
4.22	The overall accuracy and time cost (on PC) against seven levels of Manhattan distance, with the number of "Big" inferences labelled.	75
4.23	The overall accuracy and time cost (on PC) against seven levels of Euclidean distance, with the number of "Big" inferences labelled.	75
4.24	The overall accuracy and time cost (on PC) against seven levels of Mahalanobis distance, with the number of "Big" inferences labelled.	76
5.1	The system overview with modules (left), 2D/3D reconstruction (middle), and real images (right).	81
5.2	The image of the corner reflection experiment for radar range resolution.	82
5.3	The distance of points reflected by the corner reflector in y-axis versus steps. The corner reflector is moved 1 cm per step away from the radar.	83
5.4	The sending chirp and the reflection chirp from the target.	84
5.5	The comparison of the cover area between the radar installed at a height of 1 meter (green) and the radar positioned at 2.6 meters with a 55° angle to the wall (red). . . .	85
5.6	The radar placed on the ceiling at a 55° angle to the wall shown on the left side. Both the white radar case and the brown support are 3D printed, shown on the right side.	85
5.7	The point clouds acquired from the 1 m height radar (left), the 2.6 m height radar with a tilt of 55° (middle), and the 2.6 m height radar with a tilt of 50° (right), along with the corresponding histograms depicting the number of points recorded over a duration of 10 s when a target is positioned approximately 1 m in front of the radars.	86
5.8	The 2D comprehensive layout for the radar room.	87
5.9	The 3D view simulation with measurement details.	88
5.10	The working flowchart of the software system for real-time human tracking and fall detection. The blue boxes represent each module, while the arrows represent data transfer between modules. Each dotted box denotes a concurrent working process. . .	90
5.11	The demonstration of the synchronization challenge over time. Error packets highlighted in red are incomplete packets or corrupted packets which are discarded by <i>Data_Reader</i> . The system is expected to work at 20 FPS.	93
5.12	The workflow for resolving the synchronization issue through accumulation removal. The <i>Queue_Monitor</i> remove the accumulated packets to ensure the synchronization between these <i>Queues</i>	94

LIST OF FIGURES

5.13	The histogram for the number of dropped packets over 2 hours across 10 trials.	95
5.14	The workflow for resolving the synchronization issue through using timestamps. All packets are marked with timestamps by <i>Data_Reader</i> before sending to <i>Queues</i> . Error packets highlighted in red are discarded. <i>Queue_Monitor</i> merges all packets for each time interval.	96
5.15	The proposed dynamic DBSCAN strategy based on the signal SNR level (unit 0.1dB) performs multiple DBSCAN clustering for each data frame.	98
5.16	The demonstration of the dynamic DBSCAN method. On the left side, the DBSCAN algorithm is implemented using default parameters, whereas on the right, the dynamic DBSCAN strategy is applied, with dashed lines indicating the clusters.	99
5.17	The probability matrix for <i>TrackingSys</i> module. According to the global maximum value selected from the probability matrix, cluster 2 is updated to person 1. Then cluster 2 and its neighbors 1 and 3 are removed, while cluster 4 is allocated to person 2 as shown on the right.	101
5.18	The target state blurring process with a sliding window of a certain length (5 clusters in this figure).	103
5.19	The flowchart of adaptive computing.	104
5.20	Two scenarios in the field are presented, each accompanied by the top view of the point cloud (left) and the real image from the top camera (right).	105
5.21	Three examples involving multiple individuals in the field are presented, each accompanied by the top view of the point cloud (left), the target trajectory of the whole period (middle), and the real image from the camera (right).	106
5.22	Confusion matrix of three states: walking (or standing), sitting, and fall (or lying on the ground).	107
5.23	Three scenarios of human fall detection are presented, each accompanied by the 3D view of the point cloud (left), the target trajectory of the whole period (middle), and the real image from the camera (right).	108
5.24	Three fall posture scenarios are presented, each accompanied by point clouds captured from both the top view (left) and side view (middle) of the point cloud, along with corresponding ground truth obtained from the camera (right).	110
6.1	The example images for FAUST mesh models and point clouds. Red dots are points sampled on the mesh models.	115
6.2	The MLP network structure with N samples and K categories. The dotted boxes represent the layers removed for various parameters in evaluation.	116
6.3	The network structure for CNN and MLP with N samples and K categories. The dotted boxes represent the layers removed for various parameters in evaluation. . . .	117
6.4	The accuracy (a) and loss (b) for a single MLP network on the validation set over epochs.	118
6.5	The accuracy (a) and loss (b) for CNN + MLP network on the validation set over epochs.	119

6.6	The system flowchart encompasses the experimental setup, point cloud collection, data filtering & clustering, and dataset preparation for network training.	120
6.7	The proposed MMIDNet structure for human identify classification. K represents the number of identities ($K = 12$).	122
6.8	The same point distribution in the geometry in (a) can share multiple point cloud representations. Two representations are shown in (b) as an example.	124
6.9	The comparison is depicted between the network prediction and the result obtained after applying a window sliding operation, with a window length of 5 predictions as an example.	125
6.10	The confusion matrix for the classification prediction results of 12 people's identities.	126
6.11	The accuracy (a) and loss (b) for different network configurations on the validation set over epochs.	127
6.12	The system accuracy across various lengths of observation time, ranging from 3 to 7 seconds.	129
6.13	The system accuracy across various scenarios involving the classification of different numbers of people, ranging from 4 to 12.	130

LIST OF ABBREVIATIONS

AdaBoost	Adaptive Boosting
Adam	Adaptive Moment Estimation
ADC	Analog-to-Digital Converter
AHAR	Adaptive CNN for energy-efficient HAR
AI	Artificial Intelligence
AoA	Angle of Arrival
API	Application Programming Interface
BES Filter	Boundary-Energy-Speed Filter
BGN Filter	Background Noise Filter
Bi-LSTM	Bidirectional Long Short-Term Memory
BN	Batch Normalization
BNN	Binarized Neural Network
BPM	Binary Phase Modulation
CARM	Channel State Information (CSI)-based human Activity Recognition and Monitoring
CCTV	Closed-Circuit Television
CDC	Centers for Disease Control and Prevention
CFAR	Constant False Alarm Rate
CIFAR-10	Canadian Institute for Advanced Research 10-class dataset
CIFAR-100	Canadian Institute for Advanced Research 100-class dataset
CLGNet	Combined Local and Global information encoding Network

LIST OF ABBREVIATIONS

CMSIS Cortex Microcontroller Software Interface Standard framework

CNN Convolutional Neural Network

COM Communication

CPU Central Processing Unit

CSI Channel State Information

CUDA Compute Unified Device Architecture

CVFS Continuous Voltage and Frequency Scaling

DBSCAN Density-Based Spatial Clustering of Applications with Noise

DC Direct Current

DenseNet Dense Convolutional Network

DMM Depth Motion Maps

DSP Digital Signal Processor

FANN Fast Artificial Neural Network

FAUST Fine Alignment Using Scan Texture

FFT Fast Fourier Transform

FIFO First-In-First-Out

FMCW Frequency Modulated Continuous Wave

FP False Positives

FP32 Single-Precision Floating-Point 32-bit

FPS Frames Per Second

FPU Floating-Point Unit

GCC GNU Compiler Collection

GPS Global Positioning System

GPU Graphics Processing Unit

HAR Human Activity Recognition

HMM Hidden Markov Model

HOG Histograms of Oriented Gradient

HRNet High-Resolution Network

IDE Integrated Development Environment

IDF Identity F1 Score

IF Intermediate Frequency

ILSVRC ImageNet Large Scale Visual Recognition Challenge

IMU Inertial Measurement Unit

INT8 Integer 8-bit

IoT Internet of Things

IoU Intersection over Union

KNN K-Nearest Neighbors

LiDAR Light Detection And Ranging

LN Layer Normalization

LSTM Long Short-Term Memory

LVDS Low-Voltage Differential Signaling

MAC Multiply-Accumulate

mAP Mean Average Precision

MCU Microcontroller Unit

MDPI Multidisciplinary Digital Publishing Institute publisher

MECO Mediterranean Conference on Embedded Computing conference

MHAD Multimodal Human Action Dataset

MIMO Multiple-Input and Multiple-Output

MIT Massachusetts Institute of Technology

MLP Multi-Layer Perceptron

LIST OF ABBREVIATIONS

MMIDNet MmWave-based Human Identification Network

mmWave Millimeter-Wave

MNIST Modified National Institute of Standards and Technology database

MOTA Multiple Object Tracking Accuracy

MSPS Mega-Samples Per Second

MSRC Microsoft Research Cambridge-12 Kinect gesture data set

MUSIC Multiple Signal Classification

NCS2 Neural Compute Stick 2

NLP Natural Language Processing

NN Neural Network

NNoM Neural Network on Microcontroller framework

PC Personal Computer

PULP Parallel Ultra-Low Power

RAM Random Access Memory

RDM Range-Doppler Map

ReLU Rectified Linear Unit

ResNet Residual Neural Network

RF Radio Frequency

RFID Radio Frequency Identification

RGB-D Red, Green, Blue, and Depth

RNN Recurrent Neural Networks

RTC Real-Time Clock

RX Receiver

SDK Software Development Kit

SGDR Stochastic Gradient Descent with Restarts

SNR Signal Noise Ratio

SoC System on Chip

SPOT Subthreshold Power Optimized Technology

SRAM Static Random Access Memory

SVHN Street View House Numbers dataset

SVM Support Vector Machine

SYSU-3D Sun Yat-Sen University 3D dataset

T-Net Transformation Network

Tanh Hyperbolic Tangent

TCN Temporal Convolutional Network

TDM Time-Division Multiplexing

TF TensorFlow

TFLite TensorFlow Lite

TI Texas Instruments

TLV Type-Length-Value

TP True Positives

TPU Tensor Processing Unit

TUM-GAID Technical University of Munich, Gait from Audio, Image and Depth dataset

TX Transmitter

UCI-HAR University of California, Irvine Human Activity Recognition dataset

UINT8 Unsigned Integer 8-bit

USB Universal Serial Bus

UWB Ultra-Wideband

VA Virtual Antenna

VCO Voltage-Controlled Oscillator

WiSDAR WiFi-based Spatial Diversity-aware device-free Activity Recognition system

YOLO You Only Look Once

INTRODUCTION

Human Activity Recognition (HAR) is a widely studied research field that plays an important role in applications such as smart homes [1–4], industrial security [5, 6], and health monitoring [7–10]. By applying various sensors and edge processors, HAR applications have achieved significant advancements in human presence detection [1, 11–15], position tracking [16–19], activity classification [7, 20–22], fall detection [8, 9, 23, 24], identification [25–27], and more. Although the complexity and diversity of human activities in real-world scenarios present challenges for HAR tasks, with the development of machine learning techniques, HAR applications are boosted to a new level in terms of performance and efficiency [7, 28–31].

At the same time, with the rapid development of the Internet of Things (IoT) and deep learning, the demand for deploying deep neural networks on edge devices for HAR tasks such as activity classification is also increasing [17, 32]. HAR applications are usually deployed with IoT techniques to interact with humans and gather information about human targets. Edge devices are hardware systems located at the "edge" of a network, enabling local data processing without the need for cloud transmission, a concept known as edge processing. This capability reduces response latency and enhances user privacy, both of which are crucial for HAR applications. Therefore, edge devices frequently replace host PCs as processing units in mature HAR applications with properties of low cost and low energy consumption [32–34]. However, due to the limitations of storage and computing power, large neural networks are difficult to deploy directly on Microcontroller Units (MCUs) and other edge devices [35–37]. Therefore, how to optimize the deployment and inference efficiency of deep neural networks on edge devices while maintaining high accuracy for HAR has become an important research topic.

Human tracking is also an important HAR task, serving as a fundamental function because many HAR tasks depend on accurate position information. Camera-based methods are commonly used in this field to capture human target positions [38–40]. However, traditional vision-based

methods for HAR suffer from lighting conditions and privacy leakage issues. As a result, in recent years, researchers have begun to focus on non-invasive radar-based systems. Among them, the Millimeter-Wave (mmWave) radar has attracted much attention due to its ability to provide high-resolution spatial information [18, 19, 41]. Recently, Xiaomi, a Chinese electronics company, has launched a new product called People Presence Sensor for human position tracking in smart home applications. When linked with other smart devices such as smart lights and smart air conditioners, this sensor significantly enhances home automation based on captured human position and demonstrates potential in IoT applications [1, 11].

Human fall detection is a crucial application in healthcare, particularly for the elderly. According to a report from the US Centers for Disease Control and Prevention (CDC), over 14 million adults aged 65 and older, or one in four, report falling each year in the United States. About 37% of those falls result in injuries requiring medical treatment or restricting activity for at least one day [42]. Therefore, a reliable fall detection system is necessary for alerting caregivers or medical staff, and researchers are exploring mmWave technology as a less intrusive alternative to camera-based methods to achieve the goal [43–45].

Finally, home security and intruder detection are critical tasks in HAR applications, as burglaries are a persistent issue worldwide. For instance, there were 192,060 residential home break-ins in England and Wales, UK in 2021, as reported by the UK security company ADT [46]. Home security solutions often rely on camera-based methods. ADT recommends that citizens protect themselves from residential break-ins by installing a comprehensive home security system, which includes an alarm system and CCTV. However, in contrast, mmWave radars show great potential in terms of performing reliably under various lighting conditions, remaining unaffected by face camouflage, and preserving user privacy [27, 47–49]. This thesis explores how mmWave radar can be utilized for identifying individuals in indoor environments.

This thesis aims to explore the use of edge processors and mmWave radar technology to achieve efficient various HAR tasks including human tracking, activity classification, fall detection, and human identification. The feasibility of deploying neural networks for HAR tasks on different edge processors in home scenarios is first assessed, along with an adaptive neural network solution designed to improve inference speed and reduce power consumption. Subsequently, the application of mmWave radar technology for human tracking and fall detection is investigated, presenting a framework that integrates three mmWave radars in real time to track multiple individuals. Finally, a neural network structure called MmWave-based Human Identification Network (MMIDNet) is developed, utilizing mmWave radar data for human identity recognition. These contributions provide valuable technical support for future intelligent monitoring and HAR applications.

1.1 Contribution

The contribution of this thesis can be summarized as following points:

- An adaptive deep neural network system is presented on near-threshold processors for HAR applications. This system classifies six action categories in the UCI-HAR dataset with a "big-little" structure, achieving a maintained accuracy of 92.8%, while reducing inference time and energy consumption by approximately 80%. The work is open source at https://github.com/DarkSZChao/Big-Little_NN_Strategies (accessed on 9 May 2024), to further promote advancements in this field. (Chapter 4)
- State-of-the-art near-threshold processors with adaptive voltage scaling are evaluated and compared to a standard edge processor in terms of performance, speed, and energy consumption. Utilizing the proposed adaptive network with these processors, the system achieves an increase in battery life by up to 62 times on the UCI-HAR dataset compared to the base method using a standard MCU. (Chapter 4)
- A real-time multiple human tracking system using three mmWave radars is presented. This thesis introduces a dynamic Density-Based Spatial Clustering of Applications with Noise (DBSCAN) algorithm for enhanced target detection when the human target is static. The system achieves an F1 score of 98.4% for single-person scenarios, and 97.3% and 95.9% for two-person and three-person scenarios, respectively. For future work, the work is available online at https://github.com/DarkSZChao/MMWave_Radar_Human_Tracking_and_Fall_detection (accessed on 19 May 2024). (Chapter 5)
- By modifying the mmWave human tracking system, fall detection can be distinguished from standing and sitting activities with a sensitivity of 99.0% based on the analysis of the target's body position. To ensure real-time performance, the system is designed to avoid computationally intensive algorithms, such as deep neural networks. (Chapter 5)
- Using point clouds provided by mmWave radars, this thesis introduces a deep neural network structure called MMIDNet for human identification, addressing privacy intrusiveness and lighting condition issues suffered by camera-based methods. The system achieves an overall accuracy of 92.4% in classifying 12 individuals in an indoor environment. The code is accessible at https://github.com/DarkSZChao/MMWave_Radar_Human_ID_Classification (accessed on 2 June 2024). (Chapter 6)

1.2 Publications

Journal Papers

- **Z. Shen**, J. Nunez-Yanez, and N. Dahnoun, “Advanced millimeter-wave radar system for real-time multiple-human tracking and fall detection,” *Sensors*, vol. 24, no. 11, p. 3660, 2024. Contributions include: design, coding, implementation, experiment, writing.
- **Z. Shen**, N. Howard, and J. Nunez-Yanez, “Big–little adaptive neural networks on low-power near-subthreshold processors,” *Journal of Low Power Electronics and Applications*, vol. 12, no. 2, p. 28, 2022. Contributions include: design, coding, implementation, experiment, writing.

Conference Papers

- **Z. Shen**, J. Nunez-Yanez, and N. Dahnoun, “MMIDNet: Secure human identification using millimeter-wave radar and deep learning,” in 2024 13th Mediterranean Conference on Embedded Computing (MECO), pp. 1–7, IEEE, 2024. Contributions include: design, coding, implementation, experiment, writing.
- **Z. Shen**, J. Nunez-Yanez, and N. Dahnoun, “Multiple human tracking and fall detection real-time system using millimeter-wave radar and data fusion,” in 2023 12th Mediterranean Conference on Embedded Computing (MECO), pp. 1–6, IEEE, 2023. Contributions include: design, coding, implementation, experiment, writing.
- H. Cui, S. Zhong, J. Wu, **Z. Shen**, N. Dahnoun, and Y. Zhao, “MiliPoint: A point cloud dataset for mmwave radar,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 62713–62726, Curran Associates, Inc., 2023. Contributions include: experiment.
- O. Hanton, **Z. Shen**, M. Fraser, and A. Roudaut, “FabricatINK: Personal fabrication of bespoke displays using electronic ink from upcycled e readers,” in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pp. 1–15, 2022. Contributions include: implementation, experiment, drawing figures.

1.3 Thesis Outline

The chapters in this thesis can be summarized as follows:

Chapter 2: Background and Related Work

This chapter provides a review of the HAR background and explores related works using various sensors, including wearable devices, cameras, and radio frequency-based sensors. It also discusses

edge near-threshold processors and their use with machine learning methods for HAR tasks. Additionally, the chapter covers four primary HAR applications including human tracking, activity classification, fall detection, and identification and illustrates how the works in this thesis contribute to these areas.

Chapter 3: Fundamentals

This chapter introduces the fundamentals of deep learning, including Multi-Layer Perceptron (MLP), Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), and PointNet network structures. It illustrates how these methods are implemented for HAR tasks using the popular TensorFlow framework. Additionally, the chapter details the operating principles of mmWave radar using Frequency Modulated Continuous Wave (FMCW) technology in terms of distance, velocity, and angle estimations. Following this, the configuration of the radar and retrieval of the point cloud data it generates are demonstrated to support subsequent HAR applications.

Chapter 4: Human Activity Recognition with Low Power Processors

In this chapter, the performance and energy efficiency of state-of-the-art edge near-threshold processors are first evaluated by running a CNN network and comparing them to a standard edge processor. Additionally, this chapter introduces an adaptive system utilizing CNN with three different configurations for classifying six human activities. These approaches are implemented and deployed on four processors, three for near-threshold and one for standard, using various vendor toolchains and compilers. Then, the approaches are evaluated and compared with a baseline method on these processors. The proposed novel "big-little" configuration achieves approximately 80% reductions in inference time and energy consumption while maintaining HAR accuracy on the published UCI-HAR dataset. Furthermore, leveraging the energy efficiency of near-threshold processors, the system attains a battery life increase of up to 62 times. This work has contributed to a paper published in the Journal of Low Power Electronics and Applications, MDPI [50].

Chapter 5: Human Tracking and Fall Detection using MmWave Radar

Unlike using a published dataset in Chapter 4, this chapter introduces a real-time human tracking and fall detection system using three mmWave radars. It begins with basic evaluations of radar detection capabilities, including resolution, coverage, and potential interference between multiple radars. Next, a system that integrates all three radars is proposed to capture point cloud data at 20 frames per second (FPS) from the experimental environment. To enhance target detection and tracking, novel strategies such as dynamic DBSCAN for improved detection of static human targets and a probability matrix for tracking multiple targets are introduced.

Building on this, a fall detection subsystem that can send fall alerts by analyzing the position and height of the target body is developed. The system is evaluated through over 300 minutes of experimentation, encompassing single and multi-person scenarios with activities such as walking, sitting, and fall categories. The results demonstrate robust performance in both human target tracking and fall detection. This work has been featured in papers presented at the MECO conference, IEEE [51] and published in the Sensors journal, MDPI [52].

Chapter 6: Human Identification using MmWave Radar

In this chapter, an innovative mmWave radar approach for human identification using deep learning techniques is presented. The feasibility of using deep learning networks for human identification tasks is first evaluated by applying basic MLP and CNN networks on the FAUST 3D mesh dataset. Subsequently, point cloud data from 12 individuals is collected using the system developed in Chapter 5 to create a dataset for training the network. Inspired by PointNet, this chapter combines it with CNN and LSTM to propose a neural network structure called MMIDNet, specifically optimized for processing point cloud data. The comprehensive evaluations of the system's performance are conducted under various scenarios, considering factors such as network structure, observation duration, and the total number of individuals. The results demonstrate robust performance, with an overall accuracy of 92.4% in classifying 12 individuals. With an extended observation duration from 3 seconds to 7 seconds, accuracy exceeds 98%. This work has been published in a paper presented at the MECO conference, IEEE [53].

Chapter 7: Conclusions and Future Work

This chapter summarizes the contributions of this work to the field of HAR and outlines potential future directions. This research discusses the advantages and disadvantages of using edge devices and mmWave sensors to perform multiple HAR tasks. The contributions presented in this thesis verify the possibility and feasibility of deploying HAR systems on edge devices for IoT applications and emphasize the significant potential of applying mmWave technology to HAR tasks, as covered in Chapter 4 through Chapter 6. Furthermore, this chapter explores future work and discusses possible directions for each of these chapters separately.

BACKGROUND AND RELATED WORK

HAR is a field within the broader scope of Artificial Intelligence (AI) and signal processing. It involves the automatic identification and classification of human activities based on data collected from sensors. These sensors are often integrated into wearable devices such as smartphones and smartwatches [23, 54, 55] to capture various body metrics (e.g., acceleration, angle velocity and heart rate). In addition to wearable devices, vision-based approaches like cameras can provide clear footage of human targets, helping determine their status [7–9, 24, 56]. Radio-frequency-based methods, including WiFi [3, 21, 28] and mmWave techniques [57–60], have also been proposed to acquire data for recognizing human activities in recent years.

The HAR systems are usually deployed with IoT techniques for smart homes and other indoor environments. In order to balance the performance and energy consumption during usage in these edge environments, low-power embedded processors such as ECM3532 from Eta Compute [61] and Apollo3 Blue from Ambiq [62] are considered to recognize human activities by processing the sensor data. In the last decade, propelled by rapid advancements in microprocessor technology, numerous HAR systems were proposed and investigated in various domains including healthcare, sports, security, and human-computer interaction [63–66]. Such systems commonly employ deep learning methods to analyze sensor data and identify human activity patterns based on learned knowledge. To conclude, HAR systems can be roughly divided into three aspects:

- The sensor side for data collection.
- The algorithms for activity classification.
- The processor side for HAR tasks deployment.

Since it is frequently impractical to perform all tasks using a single system, a HAR system typically concentrates on one or two of the following issues (including, but not limited to):

- Human target(s) presence detection.
- Human target(s) position tracking.
- Human activities or postures classification.
- Human fall detection for healthcare.
- Human identification for security.

Detecting the presence of human subjects is a crucial preliminary step for a HAR system before achieving other functions [12–15]. Position tracking involves locating the targets within system coordinates and recording their individual trajectories [16–19]. Activity classification typically encompasses actions such as walking, standing, sitting, lying down, and others [7, 20–22]. Fall detection, a subset of activity classification, is particularly relevant for healthcare applications [8, 9, 23, 24]. Human identification is another important aspect; knowing the identities of the targets can be useful when recognizing human activities [25–27, 67, 68].

This chapter provides the literature reviews of the related work of this thesis. Section 2.1 discusses the potential of HAR applications using low-power edge devices and examines algorithmic techniques optimized for these devices. Section 2.2 demonstrates the common sensors used in HAR tasks, which can be broadly categorized into wearable and non-wearable sensors. Wearable sensors, placed directly on the human body, provide accurate data collection such as 3-axial linear acceleration and angular velocity. On the other hand, non-wearable sensors offer the advantage of convenience due to their contactless nature. Section 2.3 presents several examples of HAR applications, highlighting their contributions to humans in various aspects such as health care, human-computer interaction, room security, and more.

2.1 HAR with Low-power Edge Devices

Low-power edge devices are crucial in IoT applications. Computing at edge devices helps protect privacy, as data does not need to be sent to the cloud for processing. Moreover, many HAR applications are implemented in resource-constrained homes without a host PC or rely on battery power, increasing the demand for improved runtime speed and energy efficiency. For battery-powered applications with battery capacities around 1000 mAh, maintaining operation for at least several days requires a device working power profile limited to under 1 watt [69]. Therefore, energy-efficient edge devices are considered for processing HAR tasks in this work. In this section, an overview of current state-of-the-art hardware with power profiles in the order of 1 watt or even less (working mode) is presented for HAR applications using edge AI, followed by algorithmic techniques and frameworks optimized to target such hardware.

2.1.1 Hardware for Edge Applications

To reduce latency and enhance privacy protection, HAR application computation has gradually shifted from cloud platforms to edge devices in recent years [32, 33]. However, this transition often requires enhanced edge device specifications, including faster processing speeds and improved energy efficiency, to support complex algorithms like neural networks while extending battery life [70]. This rising demand for AI applications at the edge has led to the development of hardware optimized for low-power consumption. For instance, Google introduced a light version of the Tensor Processing Unit (TPU), known as the Edge TPU, which can deliver power-efficient inference at 2 trillion Multiply-Accumulate (MAC) operations per second per watt (2 TMAC/s/W) [71]. This advanced device can run mobile models, such as MobileNet V2 (able to compute at nearly 400 FPS), making it suitable for classifying categories in HAR tasks. While the Cloud TPU is designed for training complex models, the Edge TPU is intended for inference in low-power systems for home scenarios.

Battery-operated HAR applications often demand minimal energy consumption for sustained daily operation. Aiming for significantly lower power consumption than the Edge TPU, Ambiq released the Apollo family of near-threshold processors based on the 32-bit ARM Cortex-M4 processor. These devices achieve remarkably low energy usage, consuming only 6 $\mu\text{A}/\text{MHz}$ at 3.3 V in active mode and 1 $\mu\text{A}/\text{MHz}$ at 3.3 V in sleep mode (Table 2.1). The Apollo3 device, featured in the SparkFun board, includes 1 MB of flash memory and 384 KB of low-leakage Random Access Memory (RAM) [62] (more information can be found in Section 4.2). To deploy HAR applications on the SparkFun board, the network model must be compact enough to fit within the available storage space.

Table 2.1: Edge devices comparison.

Device	Manufacturer	Feature	Cost
Edge TPU	Google	Fast computation speed: 2 TMAC/s/W	High
SparkFun Edge (Apollo3 Blue)	Ambiq	Low working current: 6 $\mu\text{A}/\text{MHz}$	Low
ECM3532	Eta Compute	Low working current: 5 $\mu\text{A}/\text{MHz}$	Low

Similarly, Eta Compute has focused on energy-efficient endpoint AI solutions with its ECM3532 processor. This device features an ARM Cortex-M3 32-bit CPU and a separate CoolFlux Digital Signal Processor (DSP) to accelerate machine learning operations with high energy efficiency. The ECM3532, available in the AI vision board, consumes less than 5 $\mu\text{A}/\text{MHz}$ in working mode and 1 $\mu\text{A}/\text{MHz}$ in sleep mode. According to Eta Compute, its self-timed Continuous Voltage and Frequency Scaling (CVFS) technology achieves a power profile of just 1 mW [61, 72]. Notably,

in these near-threshold devices, voltage scaling is applied to the core but not to the Static Random Access Memory (SRAM) or flash.

The advent of these edge hardware MCU devices paves the way for edge applications, including HAR tasks, to be deployed within IoT systems at lower energy costs. To assess the performance of such devices, an evaluation using various MCUs on a HAR task is conducted in aspects of inference speed and energy consumption in Chapter 4.

2.1.2 Algorithmic Techniques for Edge AI

In addition to hardware improvements, algorithms can also be optimized to run HAR applications efficiently on edge devices. The common deep learning methods employing CNN or LSTM networks for HAR applications are typically too large to be implemented on edge devices, making them difficult to integrate with IoT systems in the home and office environments. Therefore, optimized network structures that are suitable for edge devices are essential for executing HAR applications at the edge. Over the years, various algorithmic approaches have emerged to improve inference speed on edge devices, such as quantization, pruning, heterogeneous models, and early termination.

The deep quantization of network weights and activations is a well-established method for optimizing network models for edge deployments [35, 73]. For instance, Hubara et al. [74] employed extremely low precision (e.g., 1-bit or 2-bits) for weights and activations, achieving 51% top-1 accuracy and 7 times speedup in AlexNet (a 12-layer CNN network designed for image classification). Similarly, the authors of Courbariaux et al. [36] introduced a Binarized Neural Network (BNN) that binarized both weights and activation operations to minimize memory access. This approach demonstrated a 32-fold compression ratio and a speed improvement of 7 times on MNIST, CIFAR-10, and SVHN datasets compared to a full precision deep neural network [36].

While these networks can ease the computational burden on edge devices by using integer numbers, the resultant loss in accuracy must be properly managed. To address this, Mocerino and Calimera [75] introduced CoopNet, which balanced computational efficiency and accuracy. CoopNet applied two convolutional models: a BNN for faster inference and an integer 8-bit (INT8) network for higher accuracy. This design targeted low-power Cortex-M microcontrollers with limited RAM (≤ 1 MB). They achieved approximately a compression ratio of 3 times and a speedup of around twice while maintaining accuracy on the CIFAR-10, GSC, and FER13 datasets. Unlike CoopNet, the approach in Chapter 4 uses a much simpler structure for secondary networks instead of lower precision, with each one trained to identify a single category in the HAR task. This optimization results in around 80% time and energy savings while maintaining similar accuracy across all evaluated MCU platforms.

The configuration of primary and secondary neural networks has been proposed to accelerate the inference process on edge devices in recent years. Similar to the approach in [75], Amiri et al.

[76] and Park et al. [77] developed "big" and "little" networks that shared the same input and output data structures. The "big" network is activated based on the score metric generated by the "little" network. A comparable configuration was proposed by Nunez-Yanez and Howard [78], employing a heterogeneous setup where the "big" network ran on state-of-the-art edge neural accelerators of NCS2 (developed for efficient edge AI by Intel), while the "little" network operated on near-threshold processors such as ECM3531 or Apollo3. This approach achieved 93% accuracy and very low energy consumption, around 4 J, on HAR tasks by dynamically switching between the "big" and "little" networks.

In contrast to the approach in [78], which deployed "big" and "little" models on NCS2 hardware accelerators and near-threshold processors separately, this thesis implements both neural network models on a near-threshold MCU for HAR across six categories in Chapter 4. A switching algorithm is applied to alternate between the "big" and "little" network models, achieving significantly lower energy consumption while maintaining comparable accuracy levels. A related study by Ordóñez and Roggen [63] demonstrated excellent accuracy and performance in HAR tasks using both CNN and LSTM layers. However, due to the limited flash memory size of the MCU used in Chapter 4, the LSTM layers which contain millions of parameters as noted in [63] are excluded. The proposed adaptive system in Chapter 4 is well-suited for real-world HAR tasks, where activities typically do not change frequently. According to [79], individuals usually perform the same action for a period of time, generally lasting tens of seconds. Therefore, it is unnecessary to maintain the system at full capacity by continuously using the primary "big" network for inference. Minimizing the invocation of the primary network reduces inference time and computational load, thereby enhancing the speed and lowering the energy requirements of the system [75–78].

2.1.3 Frameworks for Edge AI Deployment

In recent years, several frameworks have emerged to facilitate the deployment of neural network models on edge devices with limited resources. One such framework is the Cortex Microcontroller Software Interface Standard (CMSIS), developed by ARM for MCUs using the ARM Cortex-M family. This includes a neural network acceleration component called CMSIS-NN, which can achieve a 4.6 times speedup in runtime/throughput and a 4.9 times improvement in energy efficiency [80]. Building on top of this, FANN-on-MCU was designed for the rapid deployment of MLPs on low-power MCUs [81]. This framework supported not only the ARM Cortex-M series MCUs but also the RISC-V Parallel Ultra-Low Power (PULP) processors. According to the research in [81], its capabilities were demonstrated through three HAR tasks: hand gesture recognition, human fall detection, and activity classification. The results showed outstanding performance, with microsecond-level latency and milliwatt-level power consumption, achieving up to 22 times the inference speed on a PULP-based "Mr. Wolf" System on Chip (SoC) with multiple RI5CY cores compared to an ARM Cortex-M4 device.

Similar to FANN-on-MCU, Ma et al. [82] delivered a fast deployment framework for MCUs called the Neural Network on Microcontroller (NNoM) which supported more complex model topologies such as ResNet and DenseNet. This framework featured a user-friendly Application Programming Interface (API) and high-performance backend options, enabling embedded developers to deploy Keras models on low-power MCU devices. Additionally, several commercial companies also developed deployment frameworks targeting low-power edge devices. For instance, Google focused on low-power edge AI with its popular TensorFlow Lite framework [83]. Combined with the TensorFlow model training framework, Google offered a comprehensive solution from the step of neural network model training to deployment on edge devices. Another example is STM32Cube.AI from STMicroelectronics [84], an AI deployment framework specifically designed for STM family devices such as STM32 Nucleo-L4R5ZI and STM32 Nucleo-F411RE. Eta Compute created the TENSAT Flow deployment framework to optimize performance and efficiency for its MCU products, such as the ECM3531 and ECM3532 [85].

In the methodology described in Chapter 4, the limited support for specific devices across certain frameworks requires the integration of tools from various vendors. Frameworks of NNoM [82], TensorFlow Lite [83], and TENSAT Flow [85] are utilized for model deployment on MCUs like the Apollo3 and ECM3532. Based on these edge devices and frameworks, adaptive neural network structures for HAR are proposed and evaluated using the published UCI-HAR dataset in Chapter 4.

2.2 Sensors for HAR

Sensors are a critical component for acquiring data and performing HAR tasks. Many HAR applications integrate various types of sensors with machine learning techniques to achieve real-time human posture recognition. Common sensors include wearable accelerometers & gyroscopes, vision sensors (e.g., cameras), and radio-frequency-based sensors (e.g., WiFi and mmWave radar). This chapter presents a literature review of state-of-the-art approaches that utilize data collected from sensors to classify human activities and postures using machine learning methods.

2.2.1 Wearable Sensors

HAR tasks utilizing wearable sensors have been extensively studied over the years, as reviewed by Lara and Labrador [86], Ramanujam et al. [29], and Zhang et al. [30]. Wearable devices equipped with low-power sensors offer opportunities for monitoring human physiological signals, such as blood pressure, heart rate, and movement acceleration, improving natural and seamless interaction between humans and machines. The rapid proliferation of wearable technologies and advancements in sensing analytics have significantly driven the growth of HAR. There are numerous commercial smart wearable approaches proposed nowadays, such as smartwatches,

smart glasses [87], smart rings [88], smart gloves [89], smart necklaces [90], and electronic tattoos [91, 92].

Wearable sensors can be typically placed on common body areas such as the wrist, waist, head, and chest. For instance, Zhu et al. [31] placed an accelerometer on the waist of subjects and employed a CNN technique to estimate energy expenditure for fitness and lifestyle purposes. Similarly, Attal et al. [93] placed three Inertial Measurement Units (IMUs), which consist of accelerometers and gyroscopes, on the chest, right thigh, and left ankle to classify 12 activities. Healthcare and rehabilitation are other important areas of application. Gao et al. [10] proposed a method using wearable accelerometers on the ankle and wrist to monitor stroke in infants. Ghandeharioun et al. [94] focused on assessing depressive symptoms by using wrist-worn sensors to obtain biomedical signals such as electrodermal activity. Moreover, by enabling the fusion of sensor data from IMUs, human fall detection approaches were proposed for ensuring the safety of elderly individuals [23, 55, 95]. Human-computer interaction is also a significant subset, providing a flexible and convenient way to control and communicate with electronics and computers, thereby facilitating the development of smart homes and IoT. For example, wearable sensors with IMUs were deployed on the human wrist to detect hand gestures and movements such as spreading, V-shapes, and shaking [96, 97]. CNN and LSTM deep learning networks were used to classify these gestures and movements for controlling smart devices.

2.2.2 Non-wearable Sensors

Some wearable sensors can be intrusive or inconvenient for users. In contrast, non-wearable sensors are typically installed on walls or within the environment to operate from a distance, without physical contact with the human body. This approach addresses HAR problems from a third-person perspective. The following are some commonly used non-wearable sensors for HAR.

2.2.2.1 Camera

Vision-based methods are important alternatives for detecting human activities. As reviewed by Nguyen et al. [12], Kong and Fu [98], and Yadav et al. [99], these systems utilize images and videos captured by cameras to detect human presence [13–15], track human movement [16, 38, 39], recognize human activities [8, 20, 100, 101], postures [64, 65], and identify individuals [25, 67, 102].

Since the field of computer vision has been developed for decades, earlier methods utilized traditional algorithms such as Histograms of Oriented Gradient (HOG) and the Kalman filter to achieve their objectives. For example, using HOG descriptors, Dalal and Triggs [14] achieved excellent results for human detection on both the MIT pedestrian test set and the more challenging INRIA dataset. The Kalman filter, a classic recursive solution proposed by Kalman in 1960 [103], has also been widely used. Based on the Kalman filter, Choi et al. [20] proposed a new framework for collective activity classification, including queuing, waiting, talking, etc., by

utilizing spatio-temporal relationships among people. However, the classification accuracy of these methods rarely exceeded 80%.

With the rapid development of deep learning techniques, many camera-based approaches incorporating deep learning algorithms were proposed for HAR tasks. For instance, Cam et al. [101] developed and compared two fall detection algorithms using two pose estimation models, OpenPose and MoveNet, both of which employed deep neural networks such as CNN layers. Similarly, Sokolova and Konushin [102] adopted a neural network approach to explore human identification through gait analysis, providing a complementary method for scenarios where individuals were too far away for clear facial images.

In addition to using standard cameras, many researchers also focused on depth cameras [7, 56, 104], which provided depth information along with images. Depth information, which indicates the distance between objects and the camera, is highly valuable for HAR tasks. By using Kinect RGB-D cameras in conjunction with a robotic system, Koide et al. [104] contributed to human identification problems, thereby advancing IoT applications. Alternatively, Park et al. [7] focused on network structures and developed a HAR system using Recurrent Neural Networks (RNN). They achieved an outstanding performance in recognizing twelve human activities in the MSRC-12 dataset, which used depth cameras as input. Furthermore, Chen et al. [56] merged a depth camera with inertial sensors to improve HAR performance. The usage of a depth camera aided in generating Depth Motion Maps (DMM) and extracting motion features for classification decisions.

2.2.2.2 WiFi

Although camera-based solutions have achieved significant success in HAR, they face issues related to privacy intrusiveness and lighting conditions. Since images captured by cameras can clearly display human faces, it is not secure to deploy cameras in private spaces. Additionally, cameras perform poorly and struggle to recognize individuals in low-light conditions.

To address the issues above, many researchers have explored the use of radio-frequency-based methods such as WiFi [2, 3, 21, 28] as an alternative to cameras. Channel State Information (CSI) is crucial in WiFi approaches, as it captures detailed variations in wireless signals caused by human activities, enabling high-precision recognition. Based on CSI, two WiFi-based systems, CARM [21] and WiSDAR [28], were proposed to classify human activities. The key difference between them was that the former [21] utilized the traditional Hidden Markov Model (HMM) to describe the features of human activities, whereas the latter [28] combined CNN and LSTM deep learning methods to achieve the same goal. Both systems achieved over 90% accuracy on their respective datasets.

Besides the activity classification, the problem of human identification should also be investigated. In 2016, researchers proposed systems named WiWho [2] and WifiU [3] to capture human gait patterns by analyzing variations in CSI. Based on gait information, Zeng et al. [2] achieved

80% accuracy in classifying six individuals, while Wang et al. [3] attained a top-1 accuracy of 79% for identifying 50 human subjects.

However, these WiFi-based approaches require multiple separate transmitters and receivers, making deployment troublesome. Additionally, they require human subjects to move between the transmitter and receiver, which can be impractical. In real-life scenarios, users are often not positioned between the transmitter and receiver, which significantly diminishes identification performance.

2.2.2.3 MmWave Radar

MmWave radar is another type of radio-frequency-based method [105]. The high demand for automotive applications has driven significant use of mmWave radar with the FMCW technique. However, its applications extend beyond the automotive field, with many researchers exploring its potential in HAR tasks. Compared to traditional methods like cameras, mmWave radar sensors offer advantages in terms of high-resolution spatial ability, performance in low-light environments and user privacy, making them highly promising for IoT applications such as smart homes and smart medical care.

Regarding user privacy concerns, studies have generally agreed that facial recognition performance significantly degrades when the image resolution falls below 32×32 pixels [106, 107]. This implies that a resolution of at least 0.7 cm per pixel is required for effective facial recognition, given that the average human face is approximately 20 cm long [108]. In contrast, the mmWave radars used in this work provide a resolution of 4 cm (in Section 5.2.1), which is far too sparse and lacks colour information, thereby demonstrating their inherent privacy-preserving nature.

Recently, Cui and Dahnoun [19] established a real-time human detection and tracking system using Texas Instruments (TI) mmWave radars. They developed a software framework capable of communicating with mmWave radars, achieving over 90.4% sensitivity in human detection for indoor environments. Building on this system, Cui and Dahnoun [57, 109] combined mmWave radars with CNN techniques to estimate human postures accurately, achieving an average precision of 71.3%. Similarly, Li et al. [58] presented a human pose-capturing system using two 77 GHz mmWave radars, applying CNNs to radar heat maps to estimate human skeletons.

In the field of hospital clinical care, a patient activity classification system was provided based on the point clouds data from mmWave radar [66]. Six activities can be recognized with an accuracy level of around 80% on the dataset they collected. To further improve the accuracy of human posture estimation, Wang et al. [59] presented a novel CLGNet by introducing global spatial information encoding at the early stage in the neural network, while Sengupta et al. [60] focused on tracking human skeletons to classify activities.

Overall, unlike WiFi-based methods, mmWave radar systems integrate transmitters and receivers, simplifying deployment and eliminating the need for human targets to move between them. While wearable solutions like IMUs can provide accurate acceleration and velocity

information, they are inconvenient for deployment on human subjects, especially the elderly. Contactless solutions like cameras face challenges related to privacy intrusion and lighting conditions. MmWave radar has proven to be a viable alternative for HAR tasks, especially in scenarios with poor visibility conditions and where high-resolution spatial capability is required [19, 57, 58, 66, 109]. This radar system can generate point clouds with detailed information about the range, velocity, and angle of targets, making it ideal for HAR tasks in diverse environments [105]. As a result, mmWave radar is selected as the approach for performing HAR tasks in the experiments (Chapter 5 and Chapter 6).

2.2.2.4 LiDAR

Light Detection and Ranging (LiDAR), is a type of method using laser-based sensors to measure distances and generate capture dense point clouds with high precision. This provides a high-quality 3D map of the environment, offering an advantage over mmWave radar in terms of spatial resolution [110]. However, it generates a heavy data stream that demands more robust processing capabilities from devices [111] and consumes more than twice the power of mmWave radar [110, 112].

Moreover, although LiDAR systems produce high-resolution point clouds with detailed information under favorable visibility conditions, their performance degrades in adverse environments (e.g., smoke), where opaque particles scatter light and severely impair visibility [110, 113]. To address this limitation, Qian et al. [113] applied a complementary radar, which is less impacted by adverse visibility, and proposed a fusion system.

Luo et al. [114] proposed a 2D LiDAR sensing solution for human trajectory tracking and activity classification. Using the Kalman filter, they were able to track multiple human targets separately. By introducing LSTM and Temporal Convolutional Network (TCN) network structures, they achieved an accuracy of 99.49% in classifying 15 human activities. However, unfortunately, this kind of approach is not yet widely used due to the high cost of LiDAR technology. Compared to mmWave radar, the cost of LiDAR is more than 10 times higher [110]. Meanwhile, LiDAR can potentially cause permanent damage to the camera sensors of mobile phones by emitting powerful concentrated beams of infrared light [115, 116].

2.2.2.5 Environment Sensors

Environment sensors which include floor sensors, infrared sensors, thermal sensors, light sensors, pressure sensors, etc. also play a critical role in HAR tasks. They measure various information about the environment, which can help to recognize human activities if humans interact in the environment.

To eliminate privacy concerns, alternative methods such as floor sensors and infrared sensors were proposed by Middleton et al. [117] and Fang et al. [118], respectively. The former [117] employed a floor sensor matrix to record footprints, considered a less invasive feature, and

achieved 80% identification accuracy for 15 human subjects, whereas the latter [118] utilized a pyroelectric infrared detector array combined with HMM, reaching an accuracy of 78.5% in classifying 10 individuals.

To conclude, environmental sensors can capture data when humans interact with their surroundings. However, some of these sensors are complex and troublesome to deploy. Consequently, many researchers opt to merge these sensors as a complementary measure alongside wearable or vision-based methods.

2.2.3 Multi-modality Fusion Methods

Using multiple types of sensors can provide more comprehensive information to enhance the performance of HAR. Many researchers have attempted to combine data from various sensors to achieve better results in HAR tasks. For example, to overcome the limitations of cameras, such as lighting conditions and lack of distance measurement, Roche et al. [119] employed a LiDAR sensor to complement camera data with distance information. This approach achieved 95.4% accuracy on the SYSU-3D dataset, a more than 8.5% improvement over baseline methods. Chen et al. [56] proposed a method that integrated depth cameras and inertial sensors to improve the performance of human action recognition on the Berkeley MHAD dataset. By extracting features from depth images and acceleration signals, and using feature-level fusion and decision-level fusion methods, the results showed that the fusion method can improve the recognition rate by 2% to 23% compared to using each sensor alone.

While integrating data from multiple sensor types can enhance the accuracy of HAR systems, it also adds complexity to system development and deployment. In this experiment, mmWave radar is selected as the input sensor, as it offers comprehensive point cloud data along with target position and speed information. As a result, other sensor types are not integrated for HAR tasks in Chapter 5 and Chapter 6.

2.3 HAR Applications

This section demonstrates the work that has been accomplished in several mainstream applications within the HAR field. As previously mentioned, HAR encompasses various tasks, including but not limited to human tracking, activity classification, fall detection, and human identification. Accordingly, these applications are categorised into the following four aspects.

2.3.1 Human Detection and Position Tracking Approaches

Human detection and position tracking in indoor environments are critical and fundamental aspects of HAR tasks. Wearable devices such as IMUs [120], Ultra-Wideband (UWB) tags [121], and Radio Frequency Identification (RFID) tags [122, 123] are commonly used for this purpose.

While GPS is ideal for human tracking, it is unsuitable to locate candidates in indoor environments. Although wearable devices are popular for human tracking due to their functionality, they can be inconvenient for the elderly and individuals with limited mobility and poor memory.

Therefore, researchers have turned to using cameras to detect human presence [13–15] and track human movement [16, 38, 39]. By integrating the HOG algorithm [14, 39], AdaBoost algorithm [15], and Kalman filter [38], they achieved excellent results on published datasets [14–16, 38] or their own data collections [13, 39]. Some demonstrated strong real-time processing capabilities [15, 16, 39].

Radio-frequency-based methods such as WiFi [124, 125] have also shown great potential for indoor human tracking. By applying Doppler-MUSIC and Doppler-AoA on CSI, Li et al. [124] achieved a 0.35 m median error in human trajectory estimation, while Qian et al. [125] used a single WiFi link (a pair of transmitter and receiver) to achieve an average error of 0.75 m. Besides using WiFi, mmWave radar systems integrated with camera methods for object detection and tracking in outdoor environments were proposed in [17, 18]. In contrast, Cui and Dahnoun [19] focused on indoor environments and established a real-time human detection and tracking system using TI mmWave radars recently. They applied multiple radars and achieved over 90.4% sensitivity for single human detection. To avoid dead ends when multiple human targets are present in a room, Wu et al. [41] strategically placed one mmWave radar on the ceiling to monitor human targets from above. Therefore, drawing on the concepts of employing multiple mmWave radars and positioning one on the ceiling as discussed in [19, 41], a framework that collaborates three mmWave radars is developed to track multiple human targets for indoor environments in real time. The details are discussed in Chapter 5.

2.3.2 Human Activities and Postures Recognition Approaches

One of the primary focuses of this research is the estimation of human activity and posture. Many researchers have investigated this field from different entry points, including deployment on edge processors, data collection from camera and mmWave sensors and algorithm development. There are numerous well-established solutions for this topic. For instance, Rashid et al. [32] introduced a HAR system called AHAR, which utilized an adaptive CNN structure specifically designed for low-power edge devices. By applying an output block predictor to switch between two output branches to avoid unnecessary computations, this system saved 12.32% and 11.14% energy consumption on the Opportunity dataset and the w-HAR dataset, respectively, while maintaining accuracy comparable to the baseline method. Attal et al. [93] achieved over 90% accuracy in classifying 12 human activity categories using the K-Nearest Neighbors (KNN) algorithm, and compared it with other classifiers such as K-means and Support Vector Machine (SVM). Additionally, Toshev and Szegedy [64] introduced a deep neural network structure called DeepPose for estimating human skeletons from camera images, while Sun et al. [65] developed a high-resolution network named HRNet to estimate postures, achieving 74.9% Mean Average

Precision (mAP) on the PoseTrack2017 dataset.

The research in [109] provided an analysis of the capabilities of mmWave radar in human recognition. Building on this analysis, Cui and Dahnoun [57] combined mmWave radars with CNN to accurately estimate human postures, achieving an average precision of 71.3%. Moreover, Sengupta et al. [60] and Jin et al. [66] employed CNN for point cloud analysis to estimate human skeletons and the postures of patients, helping the assessment of their current status.

An adaptive neural network is developed in this work to classify six activity categories from the UCI-HAR dataset, specifically for deployment on edge devices. A comparison is also presented between this approach and the baseline method, focusing on performance and energy consumption (Chapter 4).

2.3.3 Human Fall Detection Approaches

Human falls are an important category in HAR due to their considerable health implications, particularly for elderly individuals. Falls pose a severe danger to this population, making real-time detection and timely medical assistance essential. Currently, mature fall detection methods are broadly classified into two categories: wearable and non-wearable solutions. Wearable devices, which incorporate sensors such as IMUs placed on the human body, as proposed in [23, 55, 95], enable the fall detection and the safeguard of elderly individuals. In daily life, commercial products such as the Apple Watch Series 4 and later models can monitor wrist trajectory and measure impact acceleration to detect falls effectively [126].

To address the inconvenience of wearable solutions, camera-based fall detection systems were proposed in [8, 40, 101], integrating deep learning and background subtraction techniques for indoor environments. However, due to privacy concerns associated with cameras, many researchers have turned to using mmWave radars. Sun et al. [45] proposed a privacy-preserving fall detection method using mmWave radar, leveraging an RNN with LSTM units to identify fall events within seven activity categories. Yao et al. [43] presented a robust fall detection system using mmWave radar and deep learning, achieving a 97.9% F1-score on a dataset comprising 52 non-fall actions and 12 fall actions, though it does not operate in real time. In this work, to propose a real-time mmWave radar system for human fall detection, computational algorithms such as deep learning techniques are avoided (Chapter 5).

2.3.4 Human Identification Approaches

In the realm of smart space and indoor security, precise human identification techniques are crucial. Incorporating human identification into smart space systems enables these systems to not only track human activities but also recognize individual identities. For instance, this capability is beneficial in scenarios where an elderly person falls, enabling the system to identify the individual before assistance arrives and allowing helpers to make decisions based on the provided information.

Vision-based methods, such as cameras, have been widely employed and performing well with clear, frontal views of the face [4, 25, 67, 102, 104]. For instance, Surie et al. [4] achieved 91.75% precision and 66% recall for identifying 4 human subjects by combining Fisher face recognition with skeletal tracking using a Kinect RGB-D camera. They also considered multi-occupant scenarios, but their performance in such settings was limited. Hong et al. [25] not only focused on human identification but also on position tracking by placing a dome camera centrally on the ceiling. Similarly, Ye et al. [67] utilized multiple cameras to achieve a 92.1% Identity F1 Score (IDF) and 91.6% Multiple Object Tracking Accuracy (MOTA), though this was limited to a small number of people. In addition to face recognition, Sokolova and Konushin [102] explored gait analysis as a complementary method for scenarios where individuals were too distant for clear face images, achieving over 98% accuracy on the TUM-GAID dataset using neural networks.

In contrast, radio-frequency-based methods are less intrusive. For example, variations in ambient WiFi signals have been leveraged to identify individuals by recognizing their gait during walking [2, 3, 26]. Xin et al. [26] demonstrated 88.9% identification accuracy for 6 people in a smart home environment, rising to 94.5% when only 2 individuals were required to be distinguished. Zeng et al. [2] reached 80% accuracy and 92% accuracy for 6 subjects and 2 subjects respectively, by analyzing WiFi CSI. Wang et al. [3] performed further research by extending the number of people to 50, and achieved top-1, top-2, and top-3 recognition accuracy of 79.28%, 89.52%, and 93.05% respectively. However, these methods require human subjects to move between the transmitter and receiver, which is not a typical scenario in real life.

In comparison to traditional approaches, mmWave radar sensors provide advantages in terms of high-resolution spatial ability and performance in low-light environments, making them suitable for human identification. Zhou et al. [127] evaluated typical classic deep CNN models, such as VGGNet and GoogLeNet, for the task of human identification using mmWave radar. The study demonstrated the feasibility of employing deep CNN models for identity recognition, achieving a test accuracy exceeding 96% for 5 individuals. Yao et al. [27] introduced a multi-modal learning framework that combined point clouds and range-velocity maps to identify human identity, achieving an accuracy of 93.7%. By using a semi-supervised training method, the system proposed was able to automatically collect data without manual labelling. However, its performance may decrease in scenarios involving multiple human targets due to insufficient point cloud. Zhao et al. [48, 49] successfully classified the identities of 12 individuals with 89% accuracy by voxelizing point clouds to generate occupancy grids as input for their Bidirectional-LSTM (Bi-LSTM) based deep neural network. Their system also presented the capability to identify intruders for room security. However, the voxelization they used increased the total size of the dataset due to unnecessarily voluminous sparse data, leading to an obstacle when performing the network training. To avoid mapping the point cloud to a 3D voxel representation, Qi et al. [128] proposed PointNet, an MLP-based network, which included a joint alignment network using Transformation Network (T-Net) for invariance under rigid transformations and a symmetry

function approach (Global Max Pooling) to address the permutation invariance of input points. They achieved a strong performance of 89.2% accuracy for object classification on ModelNet40 and 83.7% for object segmentation on ShapeNet by directly consuming the point cloud.

Unlike the method proposed in [48, 49], this thesis bypasses the need for point cloud voxelization. Drawing inspiration from PointNet [128], a network structure called MMIDNet is introduced, which can be directly applied to point clouds generated by mmWave radars for human identification of 12 individuals in an indoor environment (Chapter 6).

FUNDAMENTALS

This chapter offers an introductory overview of the basic concepts of mmWave radar and deep learning. Section 3.1 covers the fundamental concepts of mmWave radar with the FMCW technique. It thoroughly discusses how mmWave radar estimates object targets in terms of range, velocity, and angle, and demonstrates its capabilities regarding resolution and maximum range. Then, different mmWave radar models from TI are compared and the IWR1843 is selected as the input sensor for HAR tasks. Finally, the process for configuring the IWR1843 and parsing the point cloud data transmitted by the radars is explained. Section 3.2 delves into the fundamental principles of deep learning, including operational concepts and advancements across various basic architectures such as MLP, CNN, and LSTM. This section also introduces PointNet, a network specifically designed for point cloud data, which served as an inspiration and is instrumental in developing the mmWave radar system in this thesis. To implement these networks at the code level, a detailed subsection on using the Keras framework is provided to build and train these networks.

3.1 MmWave Radar Fundamentals

MmWave radar technology has gained significant traction in various applications including automotive systems, industrial sensing, and security surveillance due to its ability to provide high-resolution sensing capabilities. Typically, mmWave radar is deployed in the fields with the requirements of higher accuracy level due to the usage of short wavelength electromagnetic waves. This section covers the fundamentals of state-of-the-art mmWave radars utilizing the FMCW technique and explores their application for HAR tasks in the subsequent chapters. For more comprehensive information, please refer to [105].

MmWave radar refers to a type of radar system that operates within the frequency band of millimeter wavelength, typically between 30 GHz and 300 GHz. These radar systems utilize electromagnetic waves with wavelengths ranging from 1 mm to 10 mm. Due to the better support and ease of deployment offered by TI's mmWave radar in industrial applications compared to other manufacturers, the IWR1843 FMCW mmWave radars developed by TI are chosen in the experiments. IWR1843 operates at a frequency range of 76-81 GHz with a maximum available bandwidth of 4 GHz. That frequency range, the corresponding wavelength of about 4 mm, is capable of detecting movements that are as small as a fraction of a millimeter [105].

3.1.1 Processing Chain for FMCW Radar

FMCW is one of the typical modulation techniques commonly used in radar systems. Compared to pulsed radar systems, it offers advantages in range resolution, robustness to noise and lower peak power consumption [129, 130]. Unlike pulse radars, which send discrete pulses, the signal transmitted by FMCW radars consists of a continuous wave with its frequency varying linearly over time. A key benefit of this approach is that it encodes process variable information in the frequency domain rather than relying on amplitude modulation or time differences, enabling more precise signal conversion [130].

FMCW modulation technique involves transmitting a signal (known as chirp) with a frequency that ramps up or down linearly within a defined frequency range and time period. In the case of the IWR1843, illustrated in Figure 3.1, it emits a chirp with a frequency f_c ranging from 77 GHz to 81 GHz over a time period T_c i.e. 40 μ s. Consequently, the slope S of this chirp can be determined [105, 131].

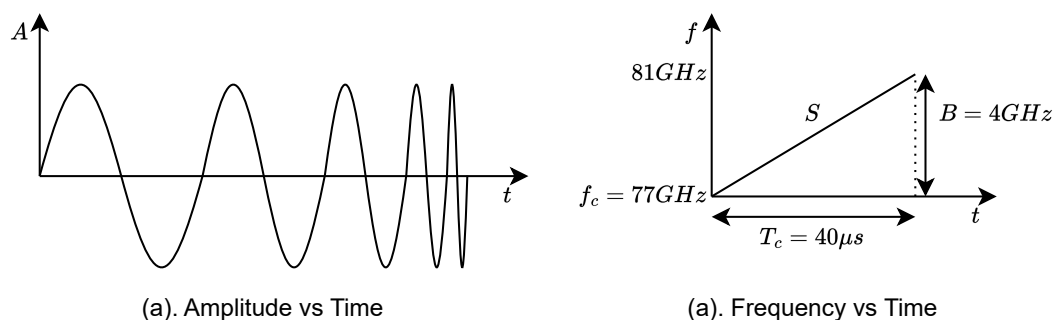


Figure 3.1: A single chirp sent from the radar in both amplitude and frequency versus time.

With the FMCW technique, mmWave radars can transmit chirp signals with a continuously varying frequency within the bandwidth. When the signal propagates through space and is bounced back by objects, the reflected signal with a time shift can be collected. These two signals are mixed together to generate an Intermediate Frequency (IF) signal, as illustrated in Figure 3.2 [105]. The frequency and phase of the IF signal correspond to the difference between the

transmitted chirp and the reflected chirp. Utilizing Cortex-R4F and C674x DSP chips, a data processing chain is then applied to the IF signal to create a Range-Doppler Map (RDM) in 2D using Fast Fourier Transforms (FFT). Subsequently, the Constant False Alarm Rate (CFAR) algorithm is applied to identify peaks by estimating the energy strength on the RDM, where each peak is considered as one object [132, 133]. After that, the Angle of Arrival (AoA) of the object can be determined by using angle FFT based on the peaks across the virtual antenna array. Finally, the Arm Cortex-R4F-based control system will provide the point cloud of the targets to a host PC through the XDS110 and USB interface. The Analog-to-Digital Converter (ADC) data is provided through the Low-Voltage Differential Signaling (LVDS) interface for further processing [134].

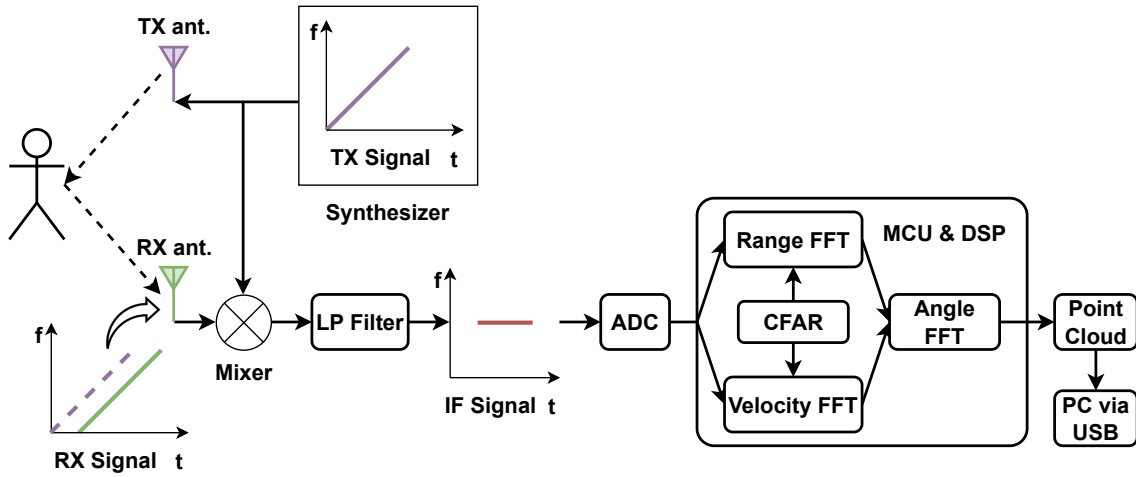


Figure 3.2: Data processing chain of the TI mmWave radar.

3.1.2 Range Measurement

To determine the range of the object, the IF signal must be identified first by measuring the frequency difference between the two chirps sent/received by the Transmitter (TX)/Receiver (RX). This is achieved by employing a mixer, as depicted in Figure 3.2, to derive the frequency of the IF signal f_{IF} (labelled as f_{IF1} in Figure 3.3). As the RX chirp is reflected by the object, the propagation time of the reflected signal τ (labelled as τ_1 in Figure 3.3) is determined by dividing the round-trip distance by the speed of light c . Subsequently, utilizing the known constant slope rate S and the measured frequency of the IF signal f_{IF} , the target range can be computed using Equation (3.1) [105, 131].

$$\begin{aligned} f_{IF} &= S\tau \\ &= \frac{S \cdot 2d}{c} \implies d = \frac{f_{IF} c}{2S} \end{aligned} \quad (3.1)$$

where:

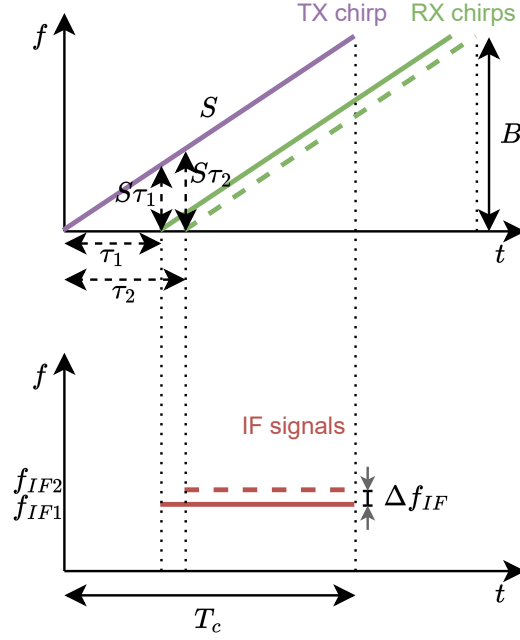


Figure 3.3: Chirps of TX (purple) and RX (green) modules in FMCW. Two reflected chirps are collected by the RX module, resulting in two corresponding IF signals.

- f_{IF} is the frequency of the IF signal, equivalent to the frequency difference between the TX and RX chirps;
- S represents the slope rate of the chirp;
- τ is the time of flight;
- d is the target distance, $2d$ is the round-trip distance;
- c is the speed of light.

3.1.2.1 Range Resolution

If two targets are present in the field, two RXs chirps will be received, as shown in Figure 3.3. In order to distinguish these two close targets, according to the Fourier Transform theory, the frequency difference between two IFs must be larger than $1/T$ Hz to be distinguished in range, where T is the observation window period [105, 131]. Therefore, it can be derived as:

$$\begin{aligned}
 \Delta f_{IF} &= f_{IF2} - f_{IF1} \\
 &= S(\tau_2 - \tau_1) \\
 &= \frac{S2(d_2 - d_1)}{c} \implies d_{res} = \frac{c}{2B} \\
 &= \frac{S2\Delta d}{c} > \frac{1}{T_c} \\
 \implies \Delta d &> \frac{c}{2B}
 \end{aligned} \tag{3.2}$$

where:

- Δf_{IF} is the frequency difference between two IF signals;
- Δd is the range difference between two targets;
- d_{res} is the range resolution which is the minimum distance at which two targets can be distinguished;
- B represents the bandwidth of the FMCW chirps.

This range resolution d_{res} is related to the radar bandwidth B as shown in the Equation (3.2). In practice, mmWave radars often use a 3–4 GHz bandwidth resulting in a range resolution of around 4 cm.

3.1.2.2 Range Maximum

As shown in Equation (3.3), the maximum detection range is proportional to the frequency of the IF signal which is limited by the sampling rate of the ADC module. For the case of IWR1843, due to the usage of a complex baseband signal, the Nyquist rate of a real signal is halved compared to the complex one [131]. Therefore, the ADC sampling rate F_s should be higher than the Nyquist rate to avoid the aliasing issue, as illustrated in Equation (3.3).

$$f_{IF} = \frac{S2d}{c} < F_s \implies d_{max} = \frac{F_s c}{2S} \quad (3.3)$$

where:

- d_{max} is the maximum range that target can be detected;
- F_s is the ADC sampling rate.

In addition to the ADC computing ability limiting the maximum detection range, this maximum range is also limited by the Signal Noise Ratio (SNR) preset for the radar system due to signal attenuation [105]. In practice, the IWR1843 radar used in this work has a maximum range of approximately 30 meters [135].

3.1.3 Velocity Measurement

To ascertain the target velocity, the radar emits two chirps separated by time T_c . Each reflected chirp undergoes FFT processing to identify the range of the target (Range-FFT). The Range-FFT corresponding to each chirp exhibits peaks in the same location but with a different phase. The observed phase difference corresponds to motion in the target of vT_c . The velocity v can be computed using Equation (3.4):

$$\begin{aligned}
 \omega &= 2\pi \frac{d'}{\lambda} \\
 &= 2\pi \frac{2vT_c}{\lambda} \implies v = \frac{\lambda\omega}{4\pi T_c} \\
 &= \frac{4\pi v T_c}{\lambda}
 \end{aligned} \tag{3.4}$$

where:

- ω represents the phase difference between two received chirps for the moving target;
- d' is the distance that the target moves within the period between two chirps;
- λ is the wavelength;
- v is the target velocity;
- T_c is the time period of a chirp.

In Equation (3.4), the presence of factor of 2 in $d' = 2vT_c$ is due to the fact that the target's movement of vT_c causes a distance difference for the chirp on both the transmit and reflected paths. For precise velocity estimation, the radar transmits multiple consecutive chirps to create a chirp frame as shown in Figure 3.4 [105]. Subsequently, it conducts a Doppler-FFT over the phases received from these chirps to determine the velocity.

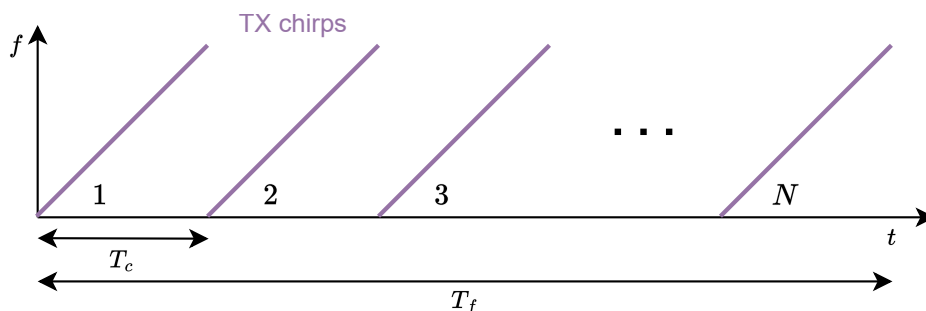


Figure 3.4: A frame of chirps in FMCW.

3.1.3.1 Velocity Resolution

For multiple objects detection, a chirp frame including N_c equal spaced chirps is applied to obtain the Doppler-FFT which reveals multiple angular frequencies ω corresponding to multiple object velocities v . Similar in Range-FFT, the minimum separable angular frequency must be larger than $2\pi/N_c$ in radians following Fourier Transform theory. As shown in Equation (3.5), increasing the total chirp length T_f results in a lower value in v_{res} which means better velocity resolution [105].

$$\begin{aligned}
 \Delta\omega &= \frac{4\pi\Delta v T_c}{\lambda} > \frac{2\pi}{N_c} \implies v_{res} = \frac{\lambda}{2T_f} \\
 \Rightarrow \Delta v &> \frac{\lambda}{2N_c T_c}
 \end{aligned} \tag{3.5}$$

where:

- $\Delta\omega$ represents the phase difference between two targets;
- Δv is the velocity difference between two targets;
- N_c is the number of chirps in one frame;
- v_{res} is the velocity resolution which is the minimum velocity for two targets that can be distinguished;
- T_f is the time period of one frame as shown in Figure 3.4.

3.1.3.2 Velocity Maximum

When determining the velocity of a target, it's crucial that the phase difference between two received chirps, denoted as ω , remains less than half a cycle (π). Otherwise, the radar system will fail to discern the direction of movement accurately. For instance, if the phase difference ω measures $\frac{3}{2}\pi$, the system will interpret it as $\frac{1}{2}\pi$, but in the opposite direction. Hence, the maximum measurable velocity can be calculated as shown in Equation (3.6). In practice, the maximum velocity for IWR1843 is around 5 m/s [135].

$$\omega = \frac{4\pi v T_c}{\lambda} < \pi \implies v_{max} = \frac{\lambda}{4T_c} \quad (3.6)$$

where:

- v_{max} is the maximum velocity at which the target can be correctly detected.

3.1.4 Angle Measurement

The configuration of multiple TXs and RXs introduces a distance variance between the target and each antenna, causing a phase shift in the peak of the Range-FFT or Doppler-FFT. The FMCW radar system leverages this phase difference between two chirps received by RX modules to calculate the AoA for the target, as illustrated in Figure 3.5.

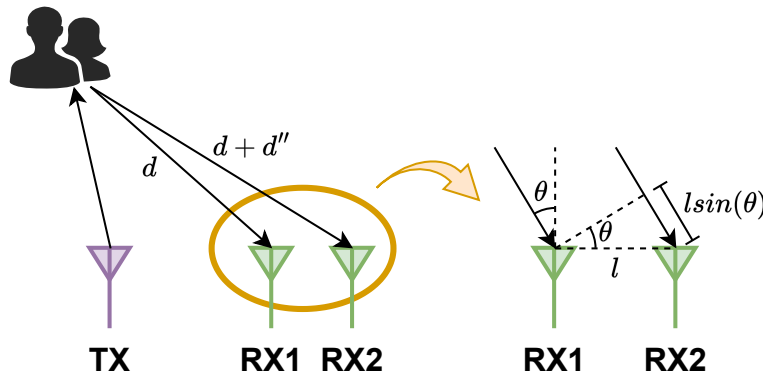


Figure 3.5: AoA estimation through the utilization of multiple antennas.

Assuming a known RX antenna with a spacing of $l = \lambda/2$, the AoA (θ) from the measured ω' can be determined using Equation (3.7):

$$\begin{aligned} \omega' &= 2\pi \frac{d''}{\lambda} \\ &= 2\pi \frac{l \sin(\theta)}{\lambda} \end{aligned} \quad \Rightarrow \quad \theta = \sin^{-1}\left(\frac{\lambda \omega'}{2\pi l}\right) \quad (3.7)$$

where:

- ω' represents the phase difference between the same chirp received by two RXs for one target;
- d'' represents the distance difference travelled by the chirp to reach two RXs, as illustrated in Figure 3.5;
- l denotes the interval between RXs, and typically equals to $\lambda/2$;
- θ is the target angle, known as the AoA.

It is worth noting that the relationship between ω' and θ is non-linear, which translates to an insensitive angle detection as θ approaches 90° .

3.1.4.1 Angle Resolution

For multiple object detection, an array of N_a virtual antennas is applied to separate objects with the same range and velocity but differing angles. Once the 2D-FFTs are obtained by each RX, the phase difference can be used to calculate the object angle θ by applying the Fourier Transform theory. The more RX antennas are used, the higher the object angle solution is. The small angle approximation states that $\sin(\alpha + \beta) \approx \sin \alpha + \beta \cos \alpha$. Therefore, $\sin(\theta + \Delta\theta) - \sin \theta \approx \Delta\theta \cos \theta$ can be derived and used in Equation (3.8). The minimum separable angular frequency must be larger than $2\pi/N_a$ in radians, as discussed in Section 3.1.3.1. Furthermore, the angle resolution is often quoted assuming $l = \lambda/2$ and $\theta = 0$ and can be simplified in Equation (3.8) [105].

$$\begin{aligned} \Delta\omega' &= \frac{2\pi l \sin(\theta + \Delta\theta)}{\lambda} - \frac{2\pi l \sin \theta}{\lambda} \\ &= \frac{2\pi l}{\lambda} (\sin(\theta + \Delta\theta) - \sin \theta) \\ &\approx \frac{2\pi l}{\lambda} \Delta\theta \cos \theta > \frac{2\pi}{N_a} \end{aligned} \quad \Rightarrow \quad \begin{aligned} \theta_{res} &= \frac{\lambda}{N_a l \cos \theta} \\ &\approx \frac{2}{N_a} \end{aligned} \quad (3.8)$$

$$\Rightarrow \quad \Delta\theta > \frac{\lambda}{N_a l \cos \theta}$$

where:

- $\Delta\omega'$ represents the phase difference between two targets;
- $\Delta\theta$ is the angle difference between two targets;
- N_a is the number of antennas;
- θ_{res} is the angle resolution which is the minimum angle at which two targets can be distinguished.

3.1.4.2 Angle Maximum

For the maximum angle detection, similarly, the phase change between two RXs must be lower than half cycle π in radians to guarantee an unambiguous value. Typically, in Equation (3.9), the largest field of view ($\pm 90^\circ$) can be achieved when $l = \lambda/2$ [105].

$$\omega' = \frac{2\pi l \sin(\theta)}{\lambda} < \pi \implies \theta_{max} = \sin^{-1}\left(\frac{\lambda}{2l}\right) \quad (3.9)$$

where:

- θ_{max} is the maximum angle at which the target can be correctly detected.

Hence, the theoretical Angle of View (AoV) for mmWave radars extends to $\pm 90^\circ$ in both horizontal and vertical directions. However, the effective AoV drops in practical experiments. According to the radar perspective analysis report in [109], the effective AoV of IWR1443 radar from TI decreases to approximately $\pm 50^\circ$ horizontally and $\pm 30^\circ$ vertically due to antenna characteristics and signal attenuation.

3.1.4.3 Virtual Antennas

MmWave radar is designed as a Multiple-Input and Multiple-Output (MIMO) system to measure angles typically, featuring multiple TXs and RXs. As an example, the IWR1843 radar development board used in the experiments is equipped with three TXs and four RXs, resulting in twelve virtual antennas operating simultaneously [134]. Illustrated in Figure 3.6, three TXs labelled in purple are spatially separated in both horizontal and vertical directions, facilitating the estimation of distance and elevation angle for objects within a three-dimensional plane. Meanwhile, four RXs labelled in green are distributed at intervals of half of the wavelength $\lambda/2$.

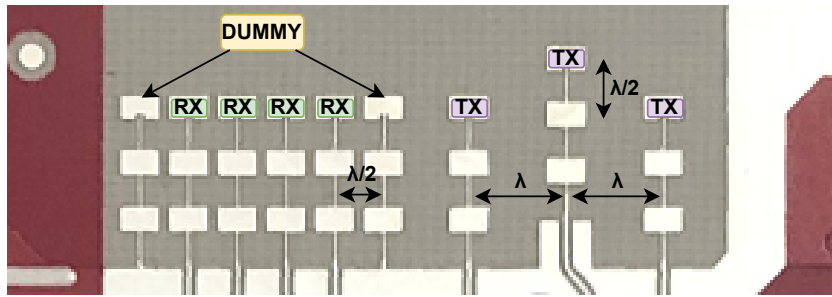


Figure 3.6: The antenna layout of TI IWR1843 mmWave radar.

The arrangement of TXs and RXs described above yields twelve virtual antennas separated by $\lambda/2$ in both horizontal and vertical directions, as shown in Figure 3.7. By spacing the antenna elements at half-wavelength intervals, the antenna array can cover the desired angular range effectively and optimize the elimination of grating lobes, which can affect the radar performance [136].

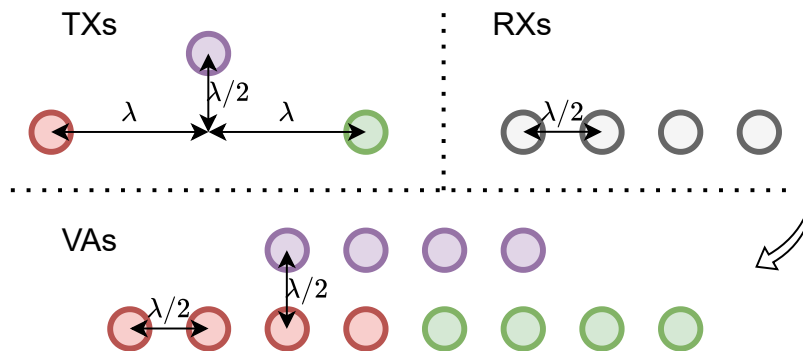


Figure 3.7: The antenna distribution including TXs, RXs and Virtual Antennas (VAs) for the IWR1843 mmWave radar.

Virtual antennas improve the radar’s ability to detect and localize targets accurately, especially in complex and dynamic scenarios. The finer spatial resolution provided by virtual antenna beams allows for better discrimination between multiple objects, even in challenging environments with high levels of noise and clutter [137].

3.1.5 TI Radar Comparison

This section delves into the hardware specifications of TI mmWave radars. These radars encompass two primary series: the IWR series for industry and the AWR series for the automotive field, each featuring various models. These two series share most specifications, with the exception of an additional automobile interface and a wider temperature range for the AWR series [138–140]. Table 3.1 presents a hardware comparison of the common radar models.

The models primarily differ in the following aspects:

- While all models incorporate Radio Frequency (RF) components and an ARM Cortex MCU for system control and communication, certain models (e.g., xWR1443) feature hardware accelerators specifically tailored for FFT operations, whereas others (e.g., xWR1642) are equipped with C674x DSPs for post-processing tasks.
- All models operate within a frequency range of either 76 GHz to 81 GHz or 60 GHz to 64 GHz.
- The majority of models (i.e., the xWR1443, xWR1843, xWR6443 and xWR6843) are equipped with three TX antennas to cover both azimuth and elevation channels, whereas xWR1642 is equipped with two TX antennas to cover the azimuth channel only. Additionally, all models listed in Table 3.1 incorporate four RX antennas.
- Most models utilize ADCs operating at a rate of 25 mega-samples per second (MSPS), while xWR1443 operates at 37.5 MSPS.

Table 3.1: TI mmWave radar hardware comparison.

Platform	xWR1443 [141]	xWR1642 [142]	xWR1843 [143]	xWR6443 [144]	xWR6843 [145]
EVM	xWR1443BOOST	xWR1642BOOST	xWR1843BOOST	xWR68xx ISK	xWR68xx ISK
Architecture	RF + MCU + HWA	RF + MCU + DSP	RF + MCU + HWA + DSP	RF + MCU + HWA	RF + MCU + HWA + DSP
Frequency (GHz)	76-81	76-81	76-81	60-64	60-64
Antennas (Rx*Tx)	4*3	4*2	4*3	4*3	4*3
TX power (dBm)	12	12	12	12	12
ADC sampling rate (MSPS)	37.5	12.5	25	25	25
Arm CPU	ARM Cortex-R4F @200MHz	ARM Cortex-R4F @200MHz	ARM Cortex-R4F @200MHz	ARM Cortex-R4F @200MHz	ARM Cortex-R4F @200MHz
DSP	—	C674x DSP @600MHz	C674x DSP @600MHz	—	C674x DSP @600MHz
Co-processor	Radar hardware accelerator	—	Radar hardware accelerator	Radar hardware accelerator	Radar hardware accelerator
RAM (KB)	576	1536	2048	1472	1792

For more comprehensive comparisons, additional detailed information can be found in the TI radar datasheets [141–145]. In this research, the IWR1843 radars are chosen for the experiments in Chapters 5 and 6 based on their relatively superior hardware specifications. However, there is no significant performance difference observed, as the majority of the data processing in this study is conducted within a host environment.

3.1.6 Radar Configuration

This section introduces how to use the mmWave Software Development Kit (SDK) provided by TI [146] to configure the IWR1843 radar board. This radar board features two USB COM ports: one user COM port for receiving configuration from users and one data COM port for transmitting point cloud data to the host PC. To enable the IWR1843 to produce point clouds as described in Section 3.1.1, the device is initially flashed with the demo firmware (version 3.5, the most recent version available at the time of system design) provided by the TI SDK [146]. This firmware outputs point clouds in structured packets, with each packet representing one frame, and sends them continuously once the start command is received.

Based on the demo visualizer from TI [135, 147], a configuration file is prepared that specifies the properties of TX and RX antennas, chirp and frame settings, ADC sampling rate and duration, among other parameters, as detailed in Table 3.2. This configuration is designed to achieve a maximum range detection of 5 m with a range resolution of 0.044 m and a maximum velocity of 2 m/s with a resolution of 0.26 m/s, meeting the requirements of the experiments. This configuration is sent through the user COM port to start the radar (details are provided in Appendix B).

Table 3.2: Radar configuration and parameters.

Parameter	Value
Created for SDK version	3.5
Platform	xWR18xx
Number of transmitters	3
Number of receivers	4
Working frequency	77–81 GHz
Frequency slope	70 MHz/ μ s
Idle time	105 μ s
Ramp time	57.14 μ s
Chirp time	162.14 μ s
Chirp loop per frame	32
Number of ADC samples	128
Frame duration	50 ms
Frames per second (FPS)	20
Maximum range	5 m
Range resolution	0.044 m
Maximum velocity	2 m/s
Velocity resolution	0.26 m/s

The choice to use only the 77–81 GHz bandwidth, rather than the 76–81 GHz range specified in the datasheet [143], is due to the radar module being equipped with two Voltage-Controlled Oscillators (VCOs) [148]: VCO1 for the 76–78 GHz narrow band and VCO2 for the 77–81 GHz wide band. There is an overlap region of 77–78 GHz where either VCO can be used, but for other regions, only the VCO capable of operating in that specific range can be used. It is hard to use both VCOs and switch between them, so VCO2 is selected to cover the 77–81 GHz bandwidth exclusively.

The chirp setting regulates how the chirps are transmitted by the TX. The frequency slope indicates how quickly the chirp sweeps across the bandwidth. Idle time represents the necessary reset period between two chirps, while ramp time specifies the duration of the chirp ramp, as illustrated in Figure 3.8 (b) [149, 150]. These two parameters collectively determine the total chirp time for each chirp. Based on the frequency slope and ramp time, the effective bandwidth (4 GHz) can be calculated. Chirp loop per frame refers to the number of chirp cycles, where each cycle includes three chirps if all three TX antennas are used in one frame, as depicted in Figure 3.8 (a). The systems in the following chapters operate at 20 FPS, translating to a frame duration of 50 ms.

When multiple TXs are utilized simultaneously, a modulation scheme is necessary to distinguish signals from each TX. Figure 3.8 employs Time-Division Multiplexing (TDM) to separate chirps: only one transmitter is active at a time, and the chirps are interleaved in the time domain. In contrast, in Binary Phase Modulation (BPM) mode, all transmitters operate simultaneously but transmit signals with specific phase shifts [151]. However, this radar module does not fully support BPM mode, so TDM is enabled by default.

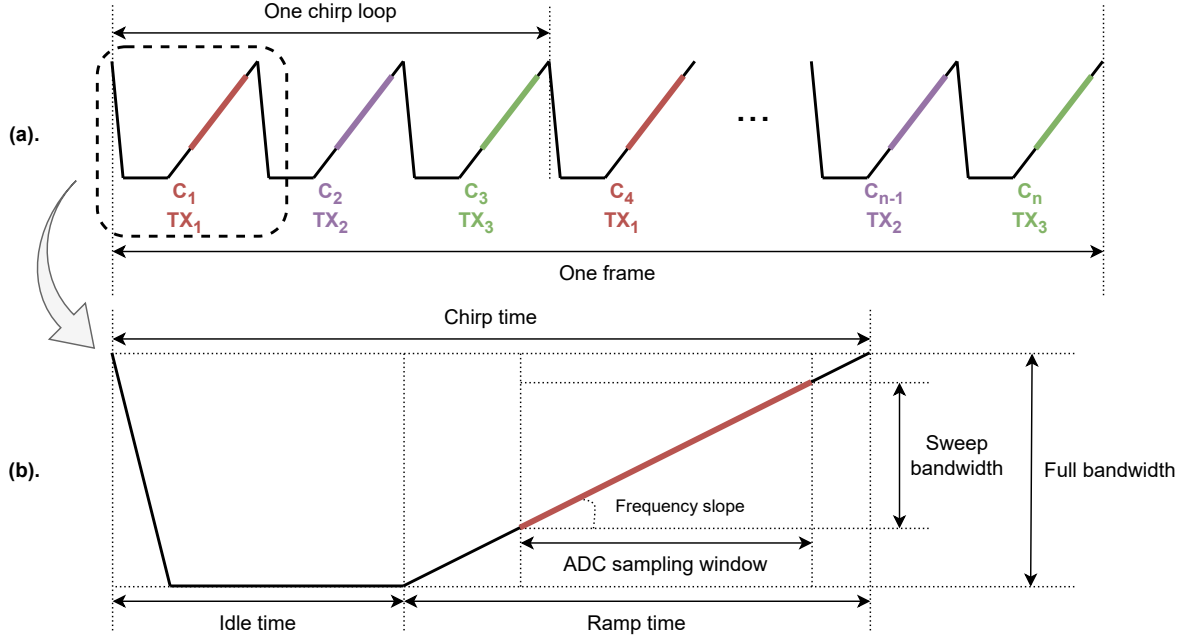


Figure 3.8: The structure of chirps within a frame (a) and the structure of an individual chirp (b).

3.1.7 Point Cloud Packet Parse

Once everything is set up and the configuration file is sent to the IWR1843, the radar begins measuring potential targets in the field and continuously produces point clouds in packets via the data COM port until a stop command is received. These packets are structured in binary and must be parsed into a usable format before applying complex algorithms.

According to the TI SDK documentation [146], a single packet contains the information for one frame. Each packet consists of several Type-Length-Value (TLV) blocks that contain various data types, such as points, side information, and statistical information. Figure 3.9 illustrates the basic structure of an example packet. The packet begins with a header that includes the magic word (a flag indicating the start), firmware version, total packet length, number of TLVs, and more. The most important TLVs for the experiments are the points TLV and side information TLV, which include all the detected points with x-y-z coordinates, velocity, and SNR for the current frame. Since the packet length varies depending on the number of detected objects, a padding block is added at the end to ensure the total packet length is always a multiple of 32 bytes.

Python (version 3.8.6) and the PySerial library (version 3.5) are employed to establish a connection with the radar COM ports. After transmitting the configuration file through the user COM port, the system listens to the data COM port for a specific magic word. Upon detecting this magic word, it waits for a second magic word to confirm that a complete packet has been successfully received. The system then parses the packet according to the structure shown in Figure 3.9, extracts the data points, and forwards them to the back-end modules for further processing. The subsequent processes for point clouds are discussed in Chapters 5 and 6.

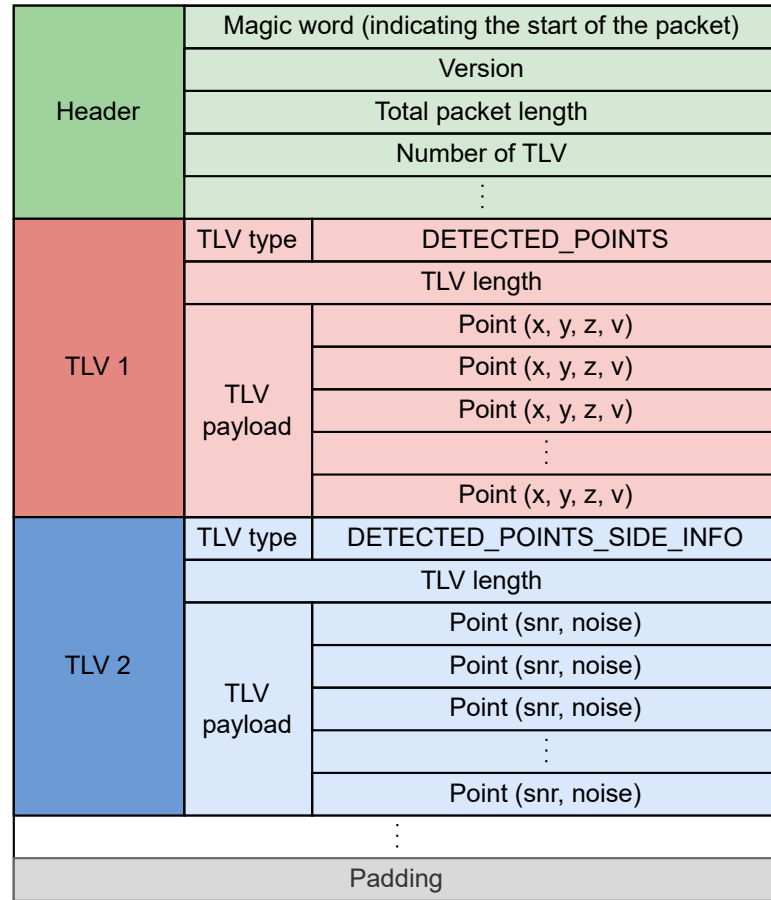


Figure 3.9: An example of the packet structure.

3.2 Deep Learning Fundamentals

Machine learning can be mainly divided into supervised learning, unsupervised learning, and reinforcement learning. Among these, deep learning, a branch of supervised learning, has gained widespread adoption in modern applications. Various deep learning network architectures, such as MLPs and CNNs, have been developed and validated, as evidenced by studies in [152] and [153]. The evolution of deep learning techniques has propelled advancements in HAR applications. This section aims to provide a concise overview of the fundamental concepts in deep learning and cluster algorithms, both of which are used in the following chapters.

3.2.1 Multilayer Perceptron (MLP)

Inspired by biological neuroscience, the MLP is developed with multiple layers of interconnected artificial neurons, providing a flexible framework for learning complex patterns from data [154]. Figure 3.10 illustrates a typical MLP structure with two hidden layers, each containing multiple artificial neurons. These fully connected layers (also referred to as a dense layer) mean every

neuron in one layer is linked to all neurons in the preceding and succeeding layers. Input values pass through the hidden layers, where they are multiplied by weights to produce outputs, eventually reaching the final output layer to generate predictions.

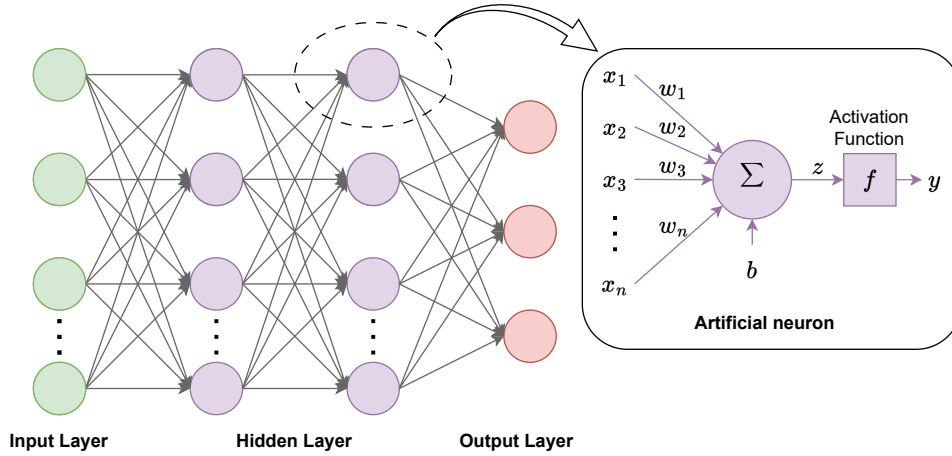


Figure 3.10: An example of an MLP with an input layer, two hidden layers, and an output layer. A zoomed-in view of a neuron is shown on the right.

3.2.1.1 Activation Functions

The activation function in Figure 3.10 plays a critical role, enabling neural networks to effectively learn and fit complex nonlinear data by introducing nonlinearity, enhancing the expressiveness of the model, and helping with gradient descent. Notable activation functions include Sigmoid, Hyperbolic Tangent (Tanh), and Rectified Linear Unit (ReLU) (Figure 3.11). Sigmoid and Tanh functions compress input values into the ranges $[0, 1]$ and $[-1, 1]$ respectively, as shown in Equation (3.10), (3.11), where z is the value before activation. ReLU, known for its computational efficiency, returns the input if it is positive, otherwise zero, as shown in Equation (3.12). ReLU is widely used in deep learning models for its simplicity and effectiveness in training deep networks [155, 156], and this activation function is applied in neural networks in the following chapters.

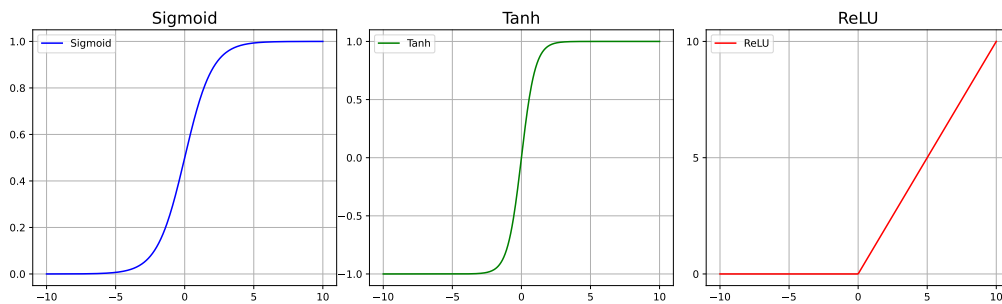


Figure 3.11: Comparison between Sigmoid, Tanh, and ReLU activation functions.

Sigmoid Activation Function:

$$f(z) = \frac{1}{1 + e^{-z}} \quad (3.10)$$

Tanh Activation Function:

$$f(z) = \frac{2}{1 + e^{-2z}} - 1 \quad (3.11)$$

ReLU Activation Function:

$$f(z) = \max(0, z) \quad (3.12)$$

3.2.1.2 Softmax

Unlike the Sigmoid, Tanh, and ReLU functions introduced in Section 3.2.1.1, the Softmax activation function is commonly applied to the entire output layer rather than individual neurons. Specifically designed for multi-class classification problems, Softmax produces a vector representing probability distributions of the neuron outputs, thereby indicating the likelihood of various potential outcomes [155, 157]. In Equation (3.13), z_i represents the value for neuron i before activation, $\sum_j e^{z_j}$ is the normalization factor, and j is the total number of neurons in the final layer. This ensures that the sum of these probabilities equals 1. Figure 3.12 effectively illustrates the computation process involved in applying the Softmax activation function within the final layer of an MLP.

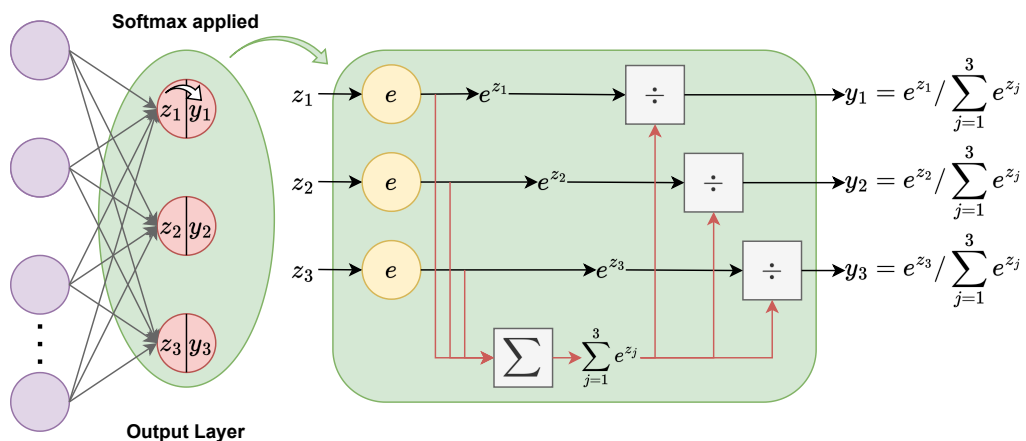


Figure 3.12: The calculation flow diagram for Softmax activation function.

Softmax Activation Function:

$$f(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (3.13)$$

3.2.2 Convolutional Neural Network (CNN)

The prototype of the CNN can be traced back to the year 1998, when LeCun et al. [158] proposed their CNN structure called LeNet to address the handwriting character recognition problem. This

milestone achievement resulted in achieving a less than 1% error rate on the MNIST dataset. Figure 3.13 illustrates the CNN structure proposed in [158], featuring two convolutional layers stacked with pooling layers inserted in between, resulting in a 10-class output.

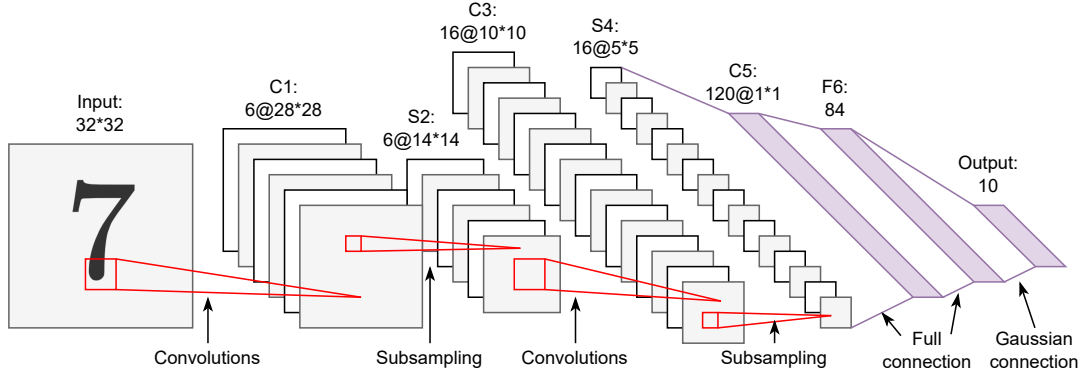


Figure 3.13: The CNN structure with convolutional layers, pooling layers and dense layers.

Since the inception of Graphics Processing Unit (GPU) acceleration for neural networks around 2000, the challenge of inadequate computation during network training has been alleviated. CNNs gained momentum with the introduction of AlexNet [159], a CNN comprising 12 layers designed for image classification. AlexNet showcased notable advancements, achieving top-1 and top-5 error rates of 37.5% and 17.0%, respectively, on the ILSVRC-2010 ImageNet test dataset, surpassing traditional machine learning techniques in image classification.

After that, The VGGNet architecture [160] explored variations with different layer depths, from 11 weight layers to 19 weight layers, highlighting the benefits of increased representation depth for classification accuracy. However, blindly increasing the number of layers can lead to model degradation due to vanishing and exploding gradients [161]. To solve this, He et al. [161] proposed the residual block structure for deep networks (as depicted in Figure 3.14) to

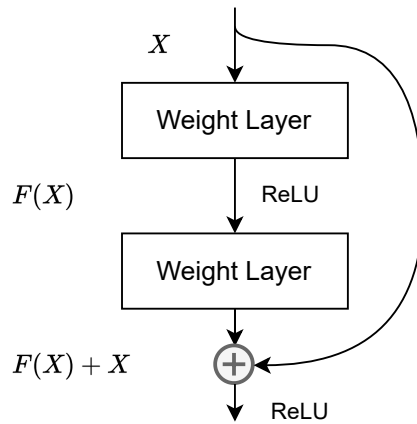


Figure 3.14: The residual block structure.

mitigate model degradation. Building upon this structure, they extended the network to 152 layers, achieving an error rate of 3.57% on the ILSVRC-2015 ImageNet test set. The authors also analyzed CIFAR-10 datasets with networks of 100 and 1000 layers, demonstrating that deep networks with 1000 layers can still maintain a relatively high but reasonable test error. Although the authors claimed that network performance scarcely improves beyond 110 layers, an extension of their work by Huang et al. [162] revealed meaningful enhancements even with a 1200-layer network. While traditional machine learning methods typically rely on effective feature extraction from raw data, deep neural networks can directly process raw data and extract features as part of their learning process.

3.2.2.1 Convolutional Kernel and Operation

In a typical convolutional layer, multiple convolutional kernels (or filters) generate several feature maps from the input image. These small kernel matrices, usually 3×3 or 5×5 , slide over the input data to extract features. At each position, the kernel performs pointwise multiplication with the corresponding region (receptive field) of the input, and the products are summed to produce a single output value [152, 163, 164].

A single-channel image is used as input in the work of [158]. However, for the issues of multiple inputs, like three-channel RGB images, the kernels also become three channels deep [159]. For instance, with a $10 \times 10 \times 3$ input image and 3×3 kernels, using 5 kernels results in dimensions of $3 \times 3 \times 3 \times 5$ for the kernels. After applying convolution with no padding and a stride of 1, each kernel produces an 8×8 feature map, totalling 5 maps as shown in Figure 3.15 [165].

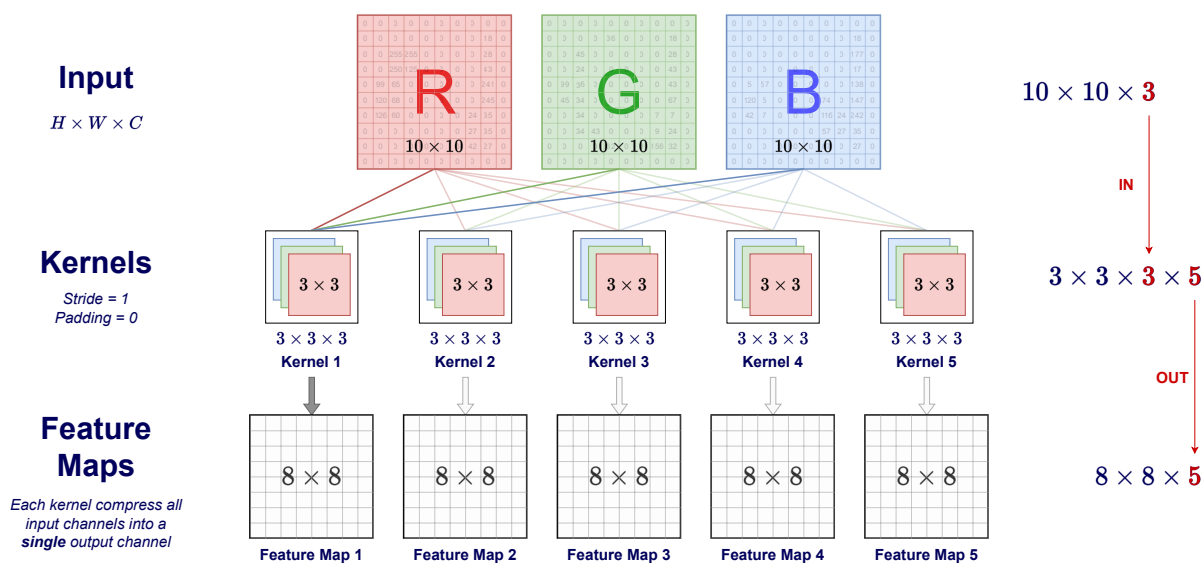


Figure 3.15: The overview of the convolutional layer propagation process with dimensions labelled.

3.2.2.2 Pooling Layer

In classic CNN architectures, a pooling layer is typically appended after a convolutional layer to reduce the spatial scale of the feature maps while retaining crucial information [158–160]. This reduction serves to ease the computational load in subsequent layers and enhance network efficiency. Common pooling methods, as depicted in Figure 3.16, include max pooling and average pooling, both using a pooling size of 2×2 in the given examples. Max pooling selects the maximum value within the local region, whereas average pooling computes the average value. Max pooling is more prevalent as it aids in preserving prominent features in the feature maps [166].

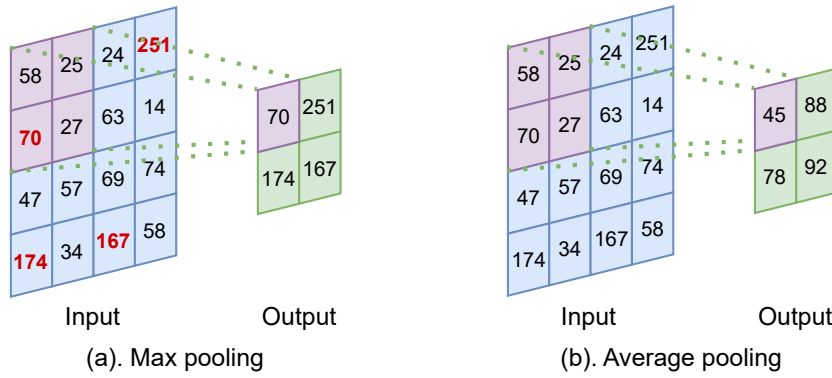


Figure 3.16: Comparison for max pooling and average pooling approaches.

3.2.3 Long Short-Term Memory Network (LSTM)

RNN represents one of the most popular deep learning models designed for tasks involving inputs sequenced over time. The effectiveness of RNN in such tasks stems from their ability to leverage both previous inputs and the current input to compute the current output, in contrast to other architectures like MLPs and CNNs, which solely rely on the current input [167].

However, a limitation of the traditional RNN is their inability to precisely predict based on previous inputs when the input sequence is excessively long, a challenge known as the long-term dependencies issue. This issue arises due to the problem of vanishing or exploding gradients [168, 169]. To address this issue, the LSTM architecture was introduced. The concept of LSTM was first proposed in 1997 by Hochreiter and Schmidhuber [170], and it has since been further developed in subsequent decades [171–174]. LSTM tackles the long-term dependencies issue by incorporating gates that regulate the flow of information from previous steps to the next cell, as illustrated in Figure 3.17 [169].

The top line, denoted as C_t , represents the cell state that traverses horizontally throughout the entire chain, carrying the information for the current cell. The forget gate, labelled as f_t , is designed to regulate the extent to which information from the previous cell state is retained. Meanwhile, the input gate i_t , controls how much current input x_t will be added to the current

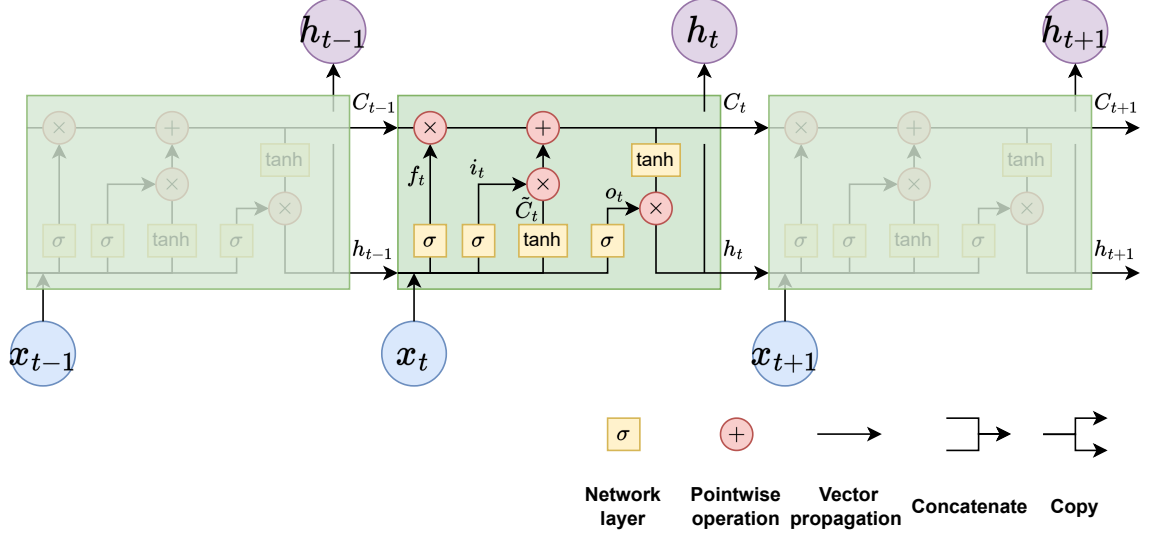


Figure 3.17: The cell view of the LSTM structure.

cell state C_t . Once the current cell state C_t is updated, the output can be generated using a filter controlled by the output gate o_t , which determines which parts of the cell state C_t are utilized as the output h_t [169].

In Figure 3.17, the yellow blocks symbolize a layer comprising weights, bias, and an activation function. The red dots represent pointwise operations. Arrows illustrate the flow of vectors; intersecting arrows denote vector concatenation, while bifurcated arrows indicate vector copying.

3.2.4 PointNet

While deep neural networks excel in processing inputs such as images, challenges arise when dealing with point clouds in 3D. Distinct from image matrices, point clouds, which serve as sparse geometric data structures, are sets of points representing 3D objects or scenes. Given their irregular format, many researchers, including Zhao et al. [48, 49], often resort to transforming this data into regular 3D voxel grids and employing CNNs or LSTMs. However, such transformations result in unnecessarily voluminous sparse data and introduce associated complications.

In order to circumvent the need for mapping the point cloud to a 3D voxel representation, Qi et al. [128] introduced PointNet, a groundbreaking deep learning model tailored for processing and analyzing point clouds, consisting of two parts: classification and segmentation. PointNet presents a novel architecture that directly accepts raw point clouds as input, eliminating the necessity for intermediate voxelization or mesh construction procedures. It adopts an MLP-based network structure, featuring a joint alignment network employing a T-net to ensure invariance under rigid transformations (labelled in blue in Figure 3.18), and a symmetry function approach (Global Max Pooling) to address the permutation invariance of input points (labelled in purple in Figure 3.18). By directly utilizing the point cloud, Qi et al. [128] achieved impressive performance, attaining

89.2% accuracy for object classification on ModelNet40 and 83.7% for object segmentation on ShapeNet. Figure 3.18 exclusively illustrates the classification part of the PointNet architecture, as the segmentation part is not pertinent to the thesis topic.

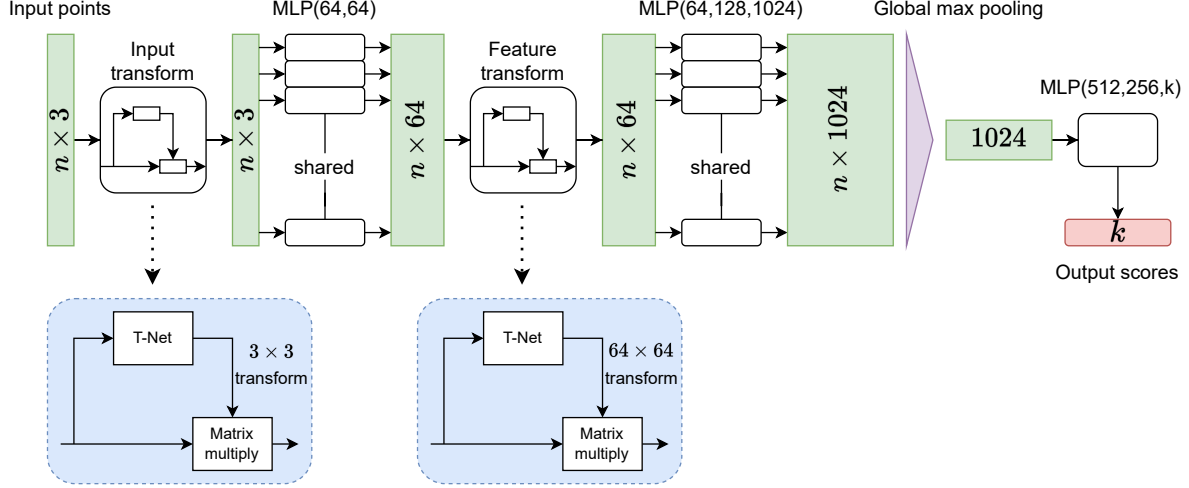


Figure 3.18: The classification part of PointNet architecture.

3.2.4.1 Invariance under Rigid Transformations

Applying rigid transformations such as translation and rotation on all points together does not lead to information loss or substantial alterations [128]. Therefore, the system should maintain its robustness on learned features when processing a transformed 3D point cloud input. However, the system will naturally treat two point clouds with identical shapes (one before and the other after a rigid transformation) as different inputs, leading to different outputs.

A natural approach to overcome this issue is aligning all input sets to a canonical space before feature extraction, as suggested in [175]. PointNet applies a T-Net that aims to learn a transformation matrix that can be applied to the input point cloud to align it into a canonical space, making the subsequent feature extraction process more robust to rigid transformations. Two T-Nets are embedded after both raw points input and extracted features as shown in Figure 3.18. Once the transformation matrix is learned, it is applied to the input point cloud before passing it through the rest of the PointNet architecture [128].

3.2.4.2 Permutation Invariance

From a data structure perspective, a point cloud is a collection of vectors without a specific order, distinguishing it from pixel arrays in images or voxel arrays in volumetric grids as utilized in [48, 49]. Essentially, when a network processes 3D point sets, it needs to maintain invariance to permutations in the input set's data feeding order. Specifically, two identical point cloud sets

exhibit the same geometric distribution but can possess different orders in table representation. Therefore, the system must treat these distinct table representation inputs as identical, as they represent the same point cloud set.

Sorting seems straightforward, but in high-dimensional space, stable ordering for point perturbations is unattainable. If such ordering existed, it implies a bijection map between high-dimensional space and a 1D real line. Requiring stability for point perturbations equates to demanding preservation of spatial proximity for this map during dimension reduction, a generally unachievable task [128]. Thus, sorting incompletely addresses the issue, making it challenging for a network to learn a consistent input-to-output mapping. Experiments by Qi et al. [128] revealed that directly applying an MLP to a sorted point set yields only 45% accuracy, though slightly better than processing an unsorted input.

To instill input permutation, PointNet employs a global max pooling layer after feature extraction, as depicted in Figure 3.18. This pooling layer acts as a symmetric function, taking the maximum value across all dimensions (channels) for each feature. Consequently, the extracted features are invariant to the order of the input points. This is due to the fact that the maximum value for each feature is determined independently of the order in which points are presented, resulting in a consistent representation regardless of the point cloud permutation. As a result, PointNet can efficiently process unordered point clouds without the need for expensive preprocessing steps or point-ordering algorithms.

3.2.5 Network Training and Hyperparameters

Upon constructing the neural network, it becomes essential to acquire a dataset comprising both data and labels. This dataset is employed to train the network, enabling it to optimize its output and acquire classification or fitting capabilities for various tasks. This learning approach is commonly referred to as supervised learning [176]. Typically, the dataset is divided into training, validation, and test sets. The training set is used to train the network and update the weights and biases by minimizing the loss function. The validation set is employed to fine-tune hyperparameters such as the learning rate, number of epochs, and neuron numbers in the layers. Finally, the test set evaluates the model performance in terms of accuracy and loss.

3.2.5.1 Normalization

Normalization is a technique to standardize datasets, thereby enhancing the efficiency of neural network learning [177, 178]. Typically, there are two types of normalization: batch normalization, which operates across the feature dimension, and layer normalization, which operates across samples, as illustrated in Figure 3.19. Due to its inherent characteristics, layer normalization finds extensive application in Natural Language Processing (NLP) tasks characterized by variable-length sequences [179]. The objective of batch normalization is to ensure that features are scaled to a comparable range to prevent dominance during the training process. In this thesis, batch

normalization is applied after the CNN layer within the network structures to expedite network learning convergence and mitigate the risk of overfitting.

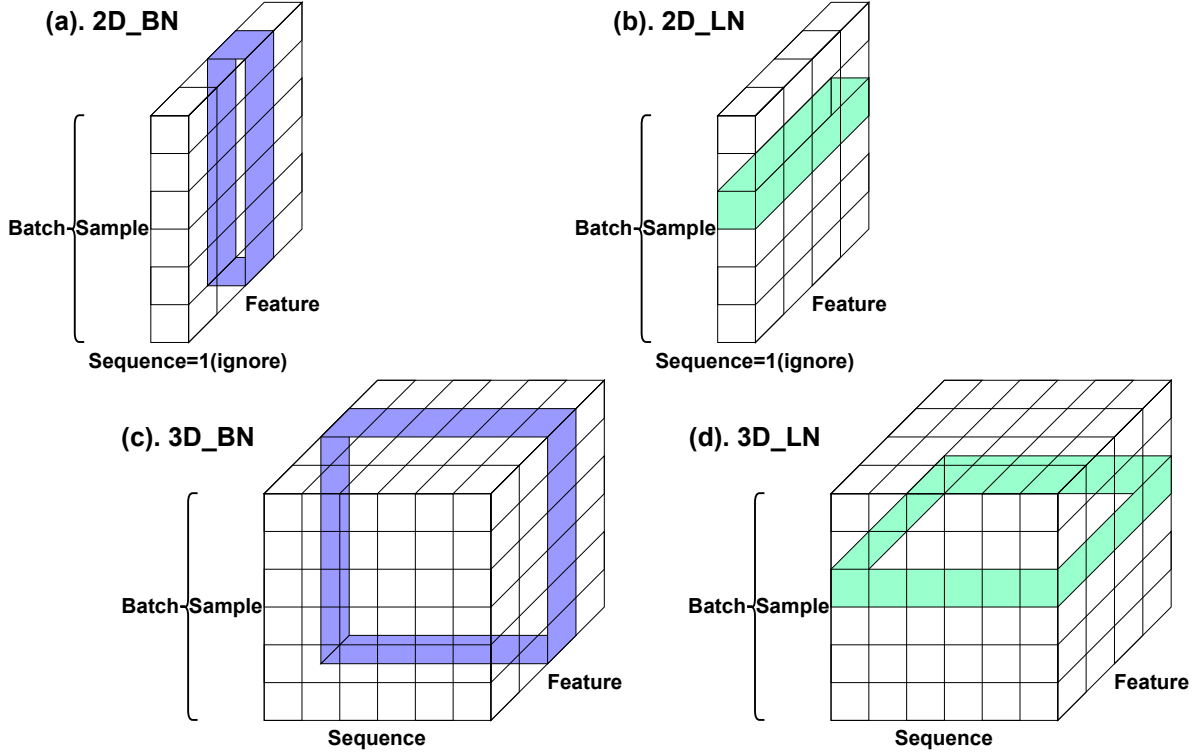


Figure 3.19: Comparison between Batch Normalization (BN) and Layer Normalization (LN) in 2D and 3D.

3.2.5.2 Overfitting

Overfitting occurs when a model excels on the training data but performs poorly when presented with new, unseen data. This phenomenon arises because overfitted models struggle to adapt to the variations present in the testing set, which may diverge from those in the training set. Moreover, overfitted models tend to memorize all the data, including unavoidable noise inherent in the training set, rather than learning the potential patterns concealed within the data [163, 180].

In Figure 3.20, an example of overfitting is shown. The black line represents a well-trained model that maintains robustness on new, unseen data. Conversely, the green line closely adheres to the training data, but its excessive reliance on this data may result in a higher error rate on new, unseen data, as evidenced by the black-centred dots, in comparison to the black line model. In this work, strategies such as data augmentation, batch normalization, and dropout layers are employed to prevent overfitting in the networks.

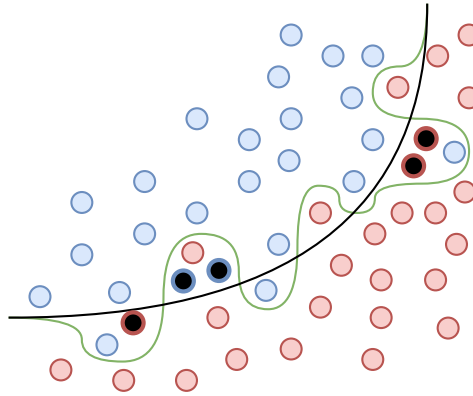


Figure 3.20: An example for the overfitting issue. The green line represents an overfitted model, while the black line represents a regularized model.

3.2.6 Deep Learning Framework

To implement deep learning network strategies mentioned in previous sections in the experiments, a suitable deep learning training framework should be selected to streamline and accelerate the process. Popular frameworks such as Keras, TensorFlow, and PyTorch offer packaged network layers and training methods, including loss functions and epoch iterations. Keras, initially developed by an independent developer, has been adopted by TensorFlow as an advanced API. The TensorFlow framework, developed by Google’s research team, supports network training and deployment on edge devices through TensorFlow Lite, making it highly suitable for HAR tasks compared to PyTorch. Therefore, TensorFlow is chosen for network implementation in the following chapters.

Listing 3.1 provides an example of a CNN for classification problems using the Keras/TensorFlow (version 2.6.5) framework in Python (version 3.8.6). The code first imports the necessary packages to establish the network. In this example, a 2D convolutional layer is used, followed by a batch normalization layer, an activation layer, and a max pooling 2D layer. The convolutional layer uses 256 convolutional kernels, each with a size of 3 by 3. The dense layer at the end is used to output the classification categories. Once the model is established, an appropriate loss function and optimizer are set to prepare for the training process. Finally, the data can be fed into the network with a batch size of 512 and trained for 200 epochs. For more examples of network code implementations of other types, such as MLP and LSTM, please refer to Appendix A.

Listing 3.1: An example of CNN layers training code using Keras/TensorFlow.

```

1  from tensorflow.keras import Model
2  from tensorflow.keras.layers import *
3
4  model_input = Input(shape=input_shape)
5

```

```

6  layer = Conv2D(256, (3, 3), strides=1, padding="same")(model_input)
7  layer = BatchNormalization()(layer)
8  layer = Activation("relu")(layer)
9  layer = MaxPooling2D(pool_size=(2, 1))(layer)
10
11 layer = Flatten()(layer)
12 layer = Dense(output_shape, activation="softmax")(layer)
13
14 model = Model(model_input, layer)
15 model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(),
16               optimizer="adam",
17               metrics=["accuracy"])
18 history = model.fit(train_data, train_labels,
19                    validation_data=(val_data, val_labels),
20                    shuffle=True,
21                    batch_size=512,
22                    callbacks=[learning_rate_callback],
23                    epochs=200)

```

3.2.7 DBSCAN Clustering

All the deep learning approaches mentioned above are supervised learning methods which require data and labels to train the network and provide predictions. Besides supervised learning, unsupervised learning method such as the DBSCAN algorithm is also used to find hidden structures for the data without labels in this thesis.

DBSCAN, a data clustering algorithm proposed by Ester et al. [181], serves to group points that are close together and identify outliers based on the point density in space. DBSCAN algorithm relies on two parameters: the radius of a neighbourhood ϵ and the minimum number of points $minPts$. These parameters collectively determine whether a set of points should be considered as a cluster or not. For example, when $minPts = 4$, as depicted in Figure 3.21, the red point C_1 is considered as a Core point due to the presence of at least 4 neighbouring points (including C_1) within the radius ϵ . Points C_2 through C_5 satisfy the aforementioned criteria and are thus deemed directly reachable, forming a single cluster. Points E_1 and E_2 are reachable from Core points but lack sufficient neighbouring points ($< minPts$). So, they are considered as Edge points instead of Core points. Conversely, Outliers, represented by O , are isolated points not reachable from any Core point and are thus classified as noise. Therefore, a cluster is formed by C_1 to C_5 , E_1 , and E_2 for the example in Figure 3.21. The full DBSCAN algorithm is shown in Algorithm 1.

By utilizing Scikit-learn (version 1.2.1), the packaged DBSCAN function can be easily invoked to cluster the point cloud data into subgroups. Listing 3.2 demonstrates the code for calling DBSCAN in Python. After importing the Scikit-learn package, the two parameters eps (ϵ) and

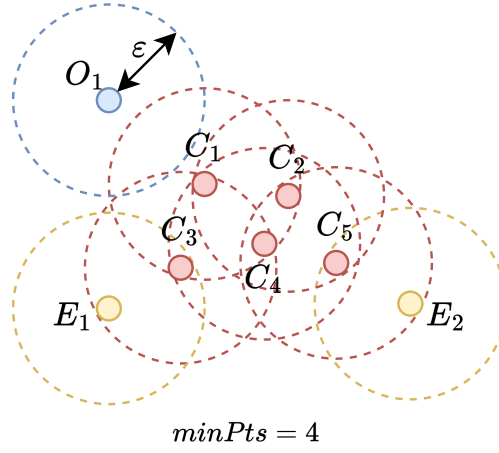


Figure 3.21: An example for DBSCAN algorithm, where C is Core points (red), E is Edge (non-Core) points (yellow) and O is Outliers (blue)

Algorithm 1 DBSCAN Algorithm

Input: Dataset D , ϵ , $minPts$

Output: Cluster labels L

```

1: ClusterCounter  $C \leftarrow 0$                                 ▷ Initialize cluster counter
2:  $L \leftarrow [-1, \dots, -1]$                               ▷ Initialize all points as unvisited
3: for point  $p$  in  $D$  do                                     ▷ Iterate all points
4:   if  $L[p] \neq -1$  then continue                          ▷ If point  $p$  is visited already, skip the loop
5:   end if
6:   Neighbors  $N \leftarrow \text{RangeQuery}(D, p, \epsilon, \text{distFunc})$   ▷ Find Neighbor points  $N$  around  $p$ 
7:   if  $|N| < minPts$  then                                    ▷ Check density
8:      $L[p] \leftarrow 0$                                        ▷ Mark point  $p$  as noise
9:     continue                                              ▷ Skip the loop
10:  end if
11:   $C \leftarrow C + 1$                                        ▷ A new cluster found
12:   $L[p] \leftarrow C$                                        ▷ Mark point  $p$  with cluster number
13:  SeedSet  $S \leftarrow N$                                     ▷ Create a potential seed set for the current cluster
14:  for point  $q$  in  $S$  do                                    ▷ Iterate points in SeedSet  $S$  and remove  $q$  from  $S$ 
15:    if  $L[q] == 0$  then  $L[q] \leftarrow C$                 ▷ If point  $q$  is noise, allocate  $q$  to the current cluster  $C$ 
16:    end if
17:    if  $L[q] \neq -1$  then continue                        ▷ If point  $q$  is visited, skip the loop
18:    end if
19:     $L[q] \leftarrow C$                                        ▷ Allocate  $q$  to the current cluster  $C$ 
20:    Neighbors  $N \leftarrow \text{RangeQuery}(D, q, \epsilon, \text{distFunc})$   ▷ Find Neighbor points  $N$  around  $q$ 
21:    if  $|N| \geq minPts$  then                                ▷ Check density (if  $q$  is a Core point)
22:       $S \leftarrow S \cup N$                                   ▷ Update SeedSet  $S$ 
23:    end if
24:  end for
25: end for
  
```

min_samples (*minPts*) can be set accordingly. This DBSCAN algorithm is applied on the point clouds of mmWave radar projects to isolate the noise and identify the targets in Chapter 5 and Chapter 6.

Listing 3.2: An example of DBSCAN code using Scikit-learn.

```
1  from sklearn.cluster import DBSCAN
2
3  labels = DBSCAN(eps=0.5, min_samples=6).fit_predict(data_points)
4  noise = data_points[labels == -1]
5  valid_points = data_points[labels != -1]
6  valid_labels = labels[labels != -1]
```


HUMAN ACTIVITY RECOGNITION WITH LOW POWER PROCESSORS

This chapter explores the energy savings achievable with near-subthreshold processors in HAR applications and proposes strategies to enhance the performance of such edge devices. The selected processors utilize adaptive voltage scaling techniques, adjusting the processor core's frequency and voltage levels at runtime. These systems typically limit embedded RAM and flash memory to less than 1 MB to conserve power, which restricts the complexity of neural network models that can be implemented and necessitates trade-offs between accuracy and efficiency. To address these challenges, alternative "Big-Little" neural network strategies are proposed and evaluated to optimize efficiency and battery life while maintaining prediction accuracy. When applied to a HAR application, these strategies achieve up to an 80% energy reduction compared to the original network model, while retaining the inference accuracy of above 90%. The content of this chapter has been published in the MDPI Journal of Low Power Electronics and Applications [50]¹.

This chapter is organized as follows: Section 4.1 provides a brief overview and key contribution points for this chapter. An initial evaluation in terms of performance and energy consumption in near-threshold MCUs and a standard MCU is carried out in Section 4.2 before the main experiment. Section 4.3 proposes and discusses three different configurations of adaptive neural network systems with different features and performance characteristics. Section 4.4 describes and demonstrates the implementation and deployment steps of neural network models needed to target the selected low-power MCUs. The results obtained in terms of speed, accuracy and energy are presented in Section 4.5. Finally, the conclusions and future work are discussed in Section 4.6.

¹©2022 MDPI. Reprinted, with permission, from Z. Shen, N. Howard, and J. Nunez-Yanez, "Big-little adaptive neural networks on low-power near-subthreshold processors," Journal of Low Power Electronics and Applications, vol. 12, no. 2, p. 28, 2022.

4.1 Overview

Over the past few decades, the rapid development of the IoT and deep learning has increased the demand for deploying deep neural networks to low-power edge devices [33]. Due to high latency and privacy issues, cloud computing tasks are gradually being transferred to the edge in areas such as HAR tasks and natural language processing [34]. However, to save costs and reduce power consumption, memory usage is minimized, as these devices do not support power-gating for unused memory blocks. The limited memory and computing power of edge devices, such as MCUs with often less than 1 MB of flash memory, pose challenges for deploying large neural networks with millions of parameters [33, 34].

To address this issue, maximizing device efficiency in terms of memory usage and energy consumption becomes a key objective. An adaptive "Big-Little" neural network system is explored. This system comprises a big network and several little networks, designed to save energy by minimizing the number of big network executions without sacrificing accuracy. The term "Big-Little" is inspired by ARM's "Big-Little" technology, which combines powerful and lightweight processors in a single SoC [182]. In this system, the big network offers higher accuracy but requires more time for inference, while the little networks are faster. The big network typically remains in sleep mode, only activating when the little network determines it cannot achieve the required confidence level. This "Big-Little" system configuration is particularly well-suited for HAR tasks because individuals typically engage in the same activity for a long time, usually lasting several seconds [79]. During these periods, the little network can take over the prediction process for lower computational costs.

This research focuses on establishing and deploying the adaptive neural network system on edge devices. It explores strategies for managing primary and secondary networks to enhance speed, accuracy, and energy efficiency in HAR applications. The key contributions of this research are summarized below:

- This chapter evaluates state-of-the-art near-threshold processors with adaptive voltage scaling and compares them to a standard edge processor.
- This chapter optimizes a popular HAR task based on TensorFlow for MCU deployment using different vendor toolchains and compilers.
- Novel "Big-Little" adaptive neural network strategies are proposed and evaluated to achieve fast inference and energy savings.
- The work is open source at https://github.com/DarkSZChao/Big-Little_NN_Strategies (accessed on 9 May 2024) to further promote work in this field.

4.2 Low-Power Microcontroller Evaluation

Before proposing the strategies and applying them to near-threshold MCUs, it is essential to conduct an initial performance evaluation, comparing the speed and energy consumption of near-threshold MCUs with a standard MCU. Four commercially available MCU devices designed for energy-efficient applications from STMicroelectronics, Ambiq and Eta Compute are considered in this comparison. Table 4.1 shows the technical details of these four MCUs. Three of them (STM32L4R5ZI, Apollo2 Blue and SparkFun Edge (Apollo3 Blue)) are based on the Cortex-M4 microarchitecture with Floating-Point Units (FPU) [62, 183, 184], while the ECM3532 is based on the Cortex-M3 microarchitecture with a "CoolFlux" 16-bit DSP [72]. The 32-bit ARM Cortex-M3 and M4 are comparable microarchitectures, both featuring a three-stage pipeline and implementing the Thumb-2 instruction set with some differences in the number of instructions available.

Table 4.1: Low-power MCU comparison.

Device	Manufacturer	Architecture	Memory (flash/SRAM /cache)	Clock Frequency	Core Voltage	Sleeping Mode Current	Work Mode Current
STM32L4R5ZI	STMicro- electronics	32-bit Cortex-M4 CPU with FPU	2 MB/640 KB	up to 120 MHz	1.05 V	2.8 μ A with RTC	110 μ A/MHz
Apollo2 Blue	Ambiq Micro	32-bit Cortex-M4 CPU with FPU	1 MB/256 KB /16 KB	up to 48 MHz	0.5 V	< 3 μ A with RTC	10 μ A/MHz
Apollo3 Blue (SparkFun Edge)	Ambiq Micro	32-bit Cortex-M4 CPU with FPU	1 MB/384 KB /16 KB	up to 96 MHz (with Turbo- SPOT Mode)	0.5 V	1 μ A with RTC	6 μ A/MHz
ECM3532	Eta Compute	32-bit Cortex-M3 CPU with 16-bit 'CoolFlux' DSP	512 KB/352 KB	up to 100 MHz	0.55 V	1 μ A with RTC	5 μ A/MHz

The STM32 Nucleo-144 development board with the STM32L4R5ZI MCU is used as a comparison point. The choice of this device is based on its low power consumption and similar architecture to the near-threshold devices. However, it operates at a relatively higher core supply voltage of 1 V, compared to the near-threshold devices from Ambiq and Eta Compute, which have core voltages of approximately 0.5 V [78]. Theoretically, the sub-threshold core supply voltage can be as low as 0.3 V which should be more power-efficient. However, at 0.3 V, the transistor switching time will be longer, which leads to a higher leakage current. The leakage can exceed 50% of the total power consumption for a threshold voltage level of around 0.2 V [185]. Therefore, in practice, selecting near-threshold voltage points instead of sub-threshold voltage points has been shown to be a more energy-efficient solution [185].

Comparing the datasheets, the STM32L4 has the highest clock frequency which results in an

advantage in processing speed. Ambiq and Eta Compute's near-threshold devices only require about half of the core supply voltage of STM32L4. All considered processors are equipped with limited flash sizes from 0.5 MB to 2 MB and a size of around 300 KB SRAM. This requires deploying a neural network model small enough to fit within these memory constraints. To address this, the TensorFlow framework and TensorFlow Lite converter are used to create a simple pre-trained CNN model designed for a HAR task using UCI-HAR dataset [54] (as shown in Figure 4.1) to perform the initial energy evaluation of the four MCU devices.

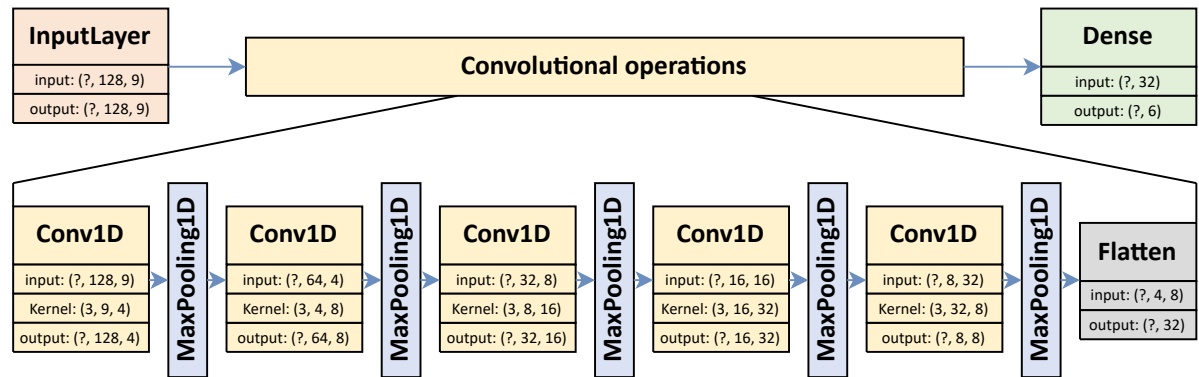


Figure 4.1: CNN structure for the initial performance and energy evaluation of the MCU.

The energy measurement board X-NUCLEO-LPM01A from STMicroelectronics is used to evaluate the performance and energy consumption, measuring the current used by the target board under a given supply voltage of 3.3 V (lower core voltages are regulated internally in the device). The power consumption of the four tested boards is shown in Figure 4.2. STM32L4

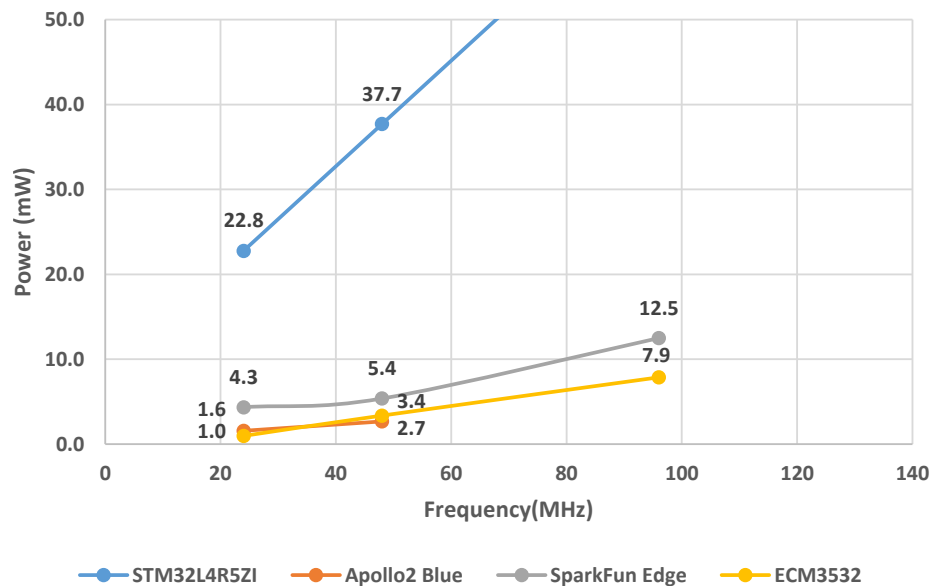


Figure 4.2: MCU initial evaluation in terms of power consumption.

operates at a much higher power level which is around six times that of the near-threshold processors. In contrast, the near-threshold processors Apollo2, Apollo3 and ECM offer significantly lower power, consuming less than 5 mW at the normal frequency of 48MHz and around 10 mW in the burst mode of 96 MHz. Since the Apollo3 core is tightly integrated into the SparkFun Edge board with peripheral sensors and ports which cannot be disabled during the power evaluation, the peripheral devices are responsible for a component of the power consumption, leading to a slightly higher power than Apollo2 at each frequency level. Apollo2 and ECM3532 exhibit similar power consumption levels at 24 and 48 MHz. Since Apollo2 does not support frequencies higher than 48 MHz, there is no data available for Apollo2 at the 96 MHz frequency point.

Figure 4.3 presents the execution times for the four processors during one inference of the pre-trained CNN model shown in Figure 4.1. Apollo2 is the slowest, taking over 100 ms at 24 MHz and around 50 ms at 48 MHz. The SparkFun Edge board (Apollo3) improves on this by reducing execution time by approximately 40% compared to Apollo2, with times dropping below 20 ms in burst mode (96 MHz). STM32L4 ranks as the second fastest, benefiting from a higher core supply voltage (as shown in Table 4.1), which allows for quicker transistor switching and faster processing speeds. ECM3532 delivers the fastest execution times, with 28 ms at 24 MHz, 15 ms at 48 MHz, and 8 ms at 96 MHz, largely due to the significant optimizations provided by the TENSAI Flow compiler from Eta Compute.

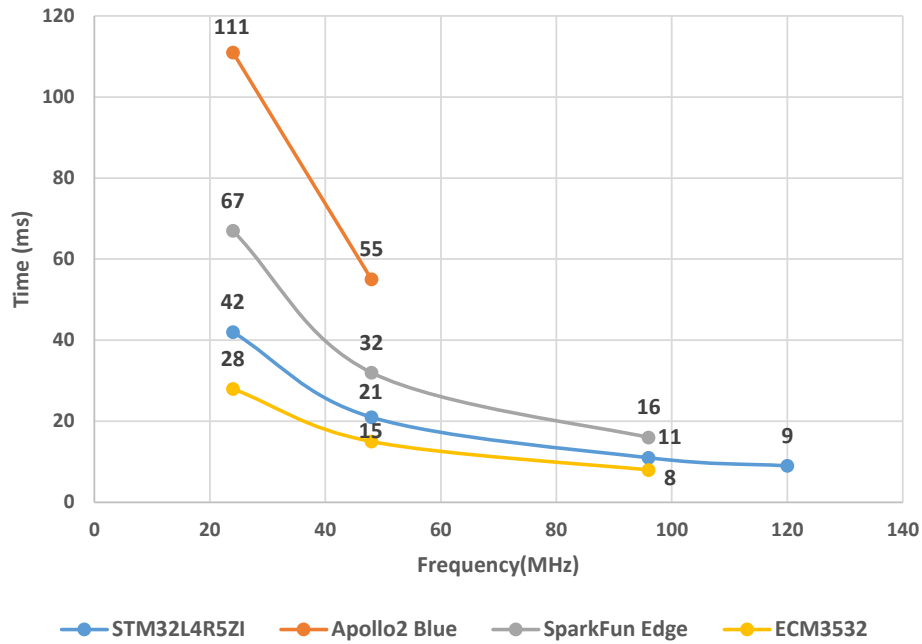


Figure 4.3: MCU initial evaluation in terms of time cost.

Figure 4.4 indicates the energy consumption values observed using the X-NUCLEO-LPM01A energy measurement board. Since the power consumption of the standard MCU STM32L4 in Figure 4.2 is six times higher compared to the near-threshold MCUs and there is no obvious

advantage in processing speed at the same frequency, STM32L4 is the worst device in terms of energy consumption for all operating frequencies from 24 to 96 MHz. SparkFun Edge (Apollo3) is slightly higher than Apollo2 at 24 and 48MHz due to the energy consumed by the peripheral equipment on board. ECM3532 achieves the minimum energy consumption at normal frequency points (24 and 48 MHz) in the energy test because it has better results in both power and time evaluations. However, when operating in the 96 MHz burst mode, ECM3532 requires more power to obtain a higher processing speed, resulting in a slight increase in energy consumption, and the same situation can be seen for the SparkFun Edge board.

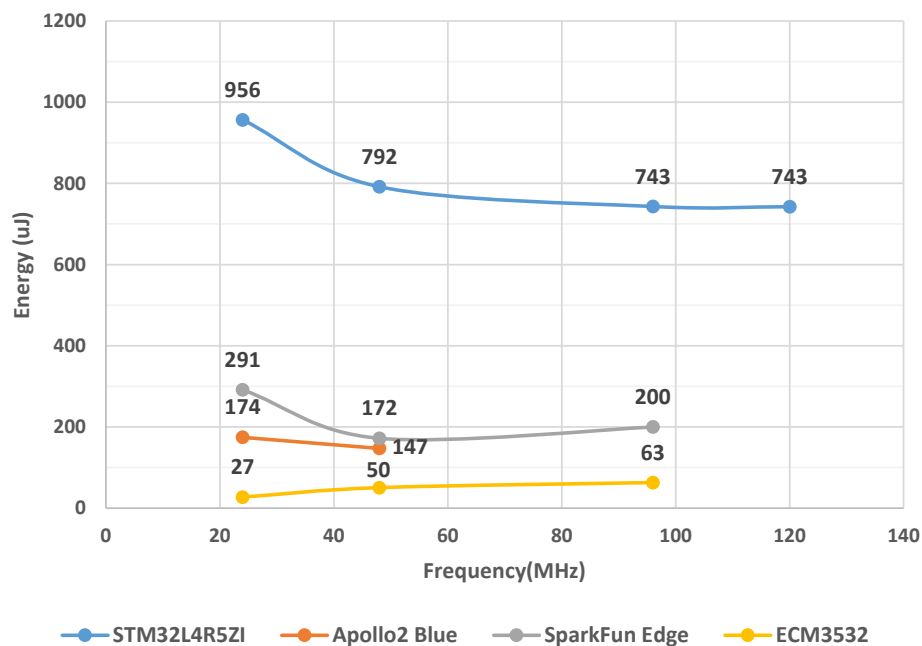


Figure 4.4: MCU initial evaluation in terms of energy consumption.

Overall, compared to the STM32L4 reference point all three near-threshold MCUs have a significant advantage in power and energy consumption which is around 80% to 90% lower. Although the near-threshold MCUs are comparable with the standard MCU STM32L4 in terms of inference time, their lower core voltage supplies (Table 4.1) result in lower power (Figure 4.2) at the same frequency level. Therefore, in this model inference evaluation, the near-threshold MCU devices can achieve better results in energy consumption compared to STM32L4 at 24, 48 and 96 MHz. Thanks to the additional model optimization obtained with the TENSAT Flow compiler provided by Eta Compute, ECM3532 offers a good balance between performance and energy efficiency to reach a lower execution time, enabling the lowest energy consumption for model inference from 24 to 96 MHz. In contrast, Apollo2, with a relatively slow processing speed, needs more time for model inference, which leads to higher values in energy consumption at 24 and 48 MHz. Due to the energy consumed by the inaccessible peripheral equipment on SparkFun Edge (Apollo3), this device consumes higher energy than Apollo2 (Figure 4.4).

4.3 Adaptive Neural Network Methodology

To save energy by reducing execution time, an adaptive neural network system including primary and secondary modules is proposed. This adaptive system is ideal for applications like HAR tasks, where the system state typically remains constant for a short period before transitioning to the next state. It deactivates the primary network temporarily and activates a secondary network when the system state remains unchanged. Therefore, this system is applied to a HAR task involving six activities using the UCI dataset [54] on edge devices. This approach aims to reduce both latency and energy consumption without compromising prediction accuracy.

To create this adaptive neural network system, Python (version 3.6.8) and TensorFlow (version 1.15) are used, with its dependencies installed on a desktop PC with Intel(R) Core (TM) i7-10850H CPU @ 2.70 GHz, NVIDIA GeForce MX250 GPU, and 16 GB RAM. Although alternatives like PyTorch and Caffe are available for training neural networks, TensorFlow is selected due to its compatibility and stability with MCUs. The primary and secondary network models are initially trained on the PC using TensorFlow. For deployment on MCUs, these models are converted from single-precision floating-point (FP32) format to unsigned 8-bit integer (UINT8) format using TensorFlow Lite and the NNoM Converter.

4.3.1 Dataset

To ensure consistency across different platforms and due to the inability to collect data during the COVID-19 pandemic, the publicly available UCI-HAR dataset is selected for testing in this work instead of conducting real-time data collection.

The UCI-HAR dataset utilizes a body accelerometer, a body gyroscope, and a total accelerometer, each with three axes, to capture data for six activities (WALKING, WALKING_UPSTAIRS, and WALKING_DOWNSTAIRS, SITTING, STANDING, LYING) performed by 30 volunteers. Data is sampled in fixed-width sliding windows of 128 points, resulting in a dataset structured as $N \times T \times D \times S$, where N is the number of samples, T is the sliding window length of 128, D represents the 3 axes, and S denotes the 3 sensors. The dataset is randomly partitioned into two subsets, with around 70% of data samples used for training ($7352 \times 128 \times 3 \times 3$) and 30% used for testing ($2947 \times 128 \times 3 \times 3$). The training and testing data sets from UCI-HAR use floating-point values with a wide range so that before training the networks, all the data is rescaled to quantized integer values with a range of $[-128, 127]$. In the following sections, these actions are referred to as follows:

- Activity I = WALKING;
- Activity II = WALKING_UPSTAIRS;
- Activity III = WALKING_DOWNSTAIRS;

- Activity IV = SITTING;
- Activity V = STANDING;
- Activity VI = LYING.

In principle, using the "big"-only configuration would yield higher accuracy. The challenge is to minimize the activation frequency of the "big" network without sacrificing accuracy. To address this, the following three alternative configurations are proposed:

- "Big"-only (base method);
- "Big" + six "little";
- "Big" + "dual";
- "Big" + distance.

Since the secondary network is designed using data from a single sensor rather than three, the accuracy of each sensor is assessed individually. Figure 4.5 shows the results of applying test data from all three sensors to the secondary network. The total accelerometer sensor demonstrated the highest overall accuracy, making it the chosen sensor for the secondary network's inference in the following sections.

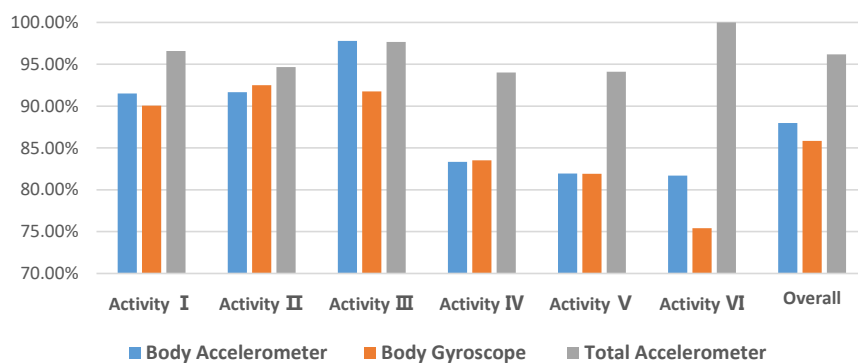


Figure 4.5: The test accuracy of three sensors on the secondary network. The total accelerometer sensor which achieves the highest overall accuracy is chosen as input for the secondary network.

4.3.2 "Big" + Six "Little" Configuration

The main idea is to deploy an adaptive system that consists of one "big" (output of six classes) and six "little" (output of two classes for each) network models in the MCU devices. As shown in Figures 4.6, the "big" network with higher accuracy is engaged to perform classification when the system starts. Once the "big" network provides a category and stores it in the register, the corresponding "little" network will be triggered to infer the next incoming activity. The "little"

network is responsible for detecting whether this current activity belongs to the previous category provided by the "big" network.

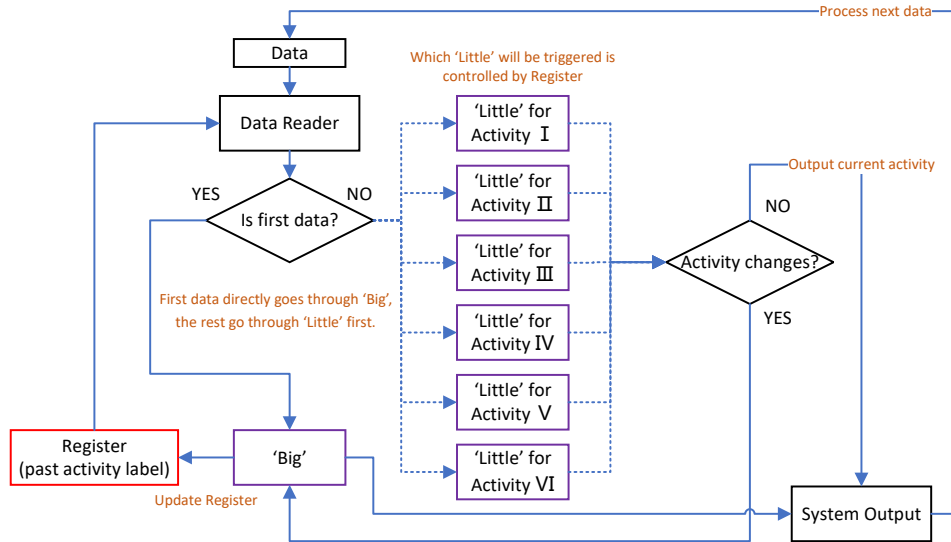


Figure 4.6: The flowchart for the "big" + six "little" configuration of the adaptive neural network system. The dotted line means only one "little" network model of six is invoked at each loop.

For instance, as shown in Figure 4.7, after the "big" network initially detects the activity as WALKING (Activity I), it is temporarily disabled. The subsequent activity data sample is then processed by the "little" network, which determines whether the activity is still WALKING. If the next activity is different, the "big" network is reactivated. Otherwise, the "little" network continues to operate, conserving power and reducing inference time. Accordingly, there are six "little" networks, each corresponding to one of the activity categories.

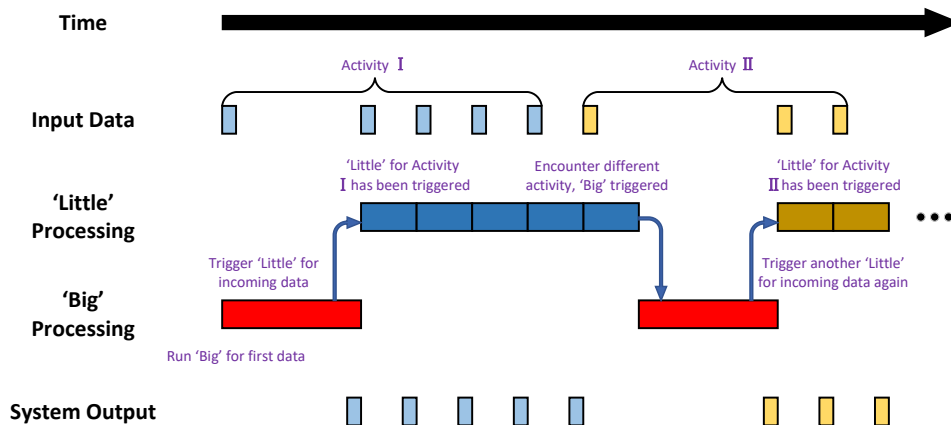


Figure 4.7: The processing steps for the "big" + six "little" configuration of the adaptive neural network system. Dark blue and brown represent two "little" network models corresponding to the input activities.

Algorithm 2 displays the code implementation for transitioning between network models during the HAR task with continuous data samples, where each sample is $128 \times 3 \times 3$. For "big" model inference, the entire data sample is used, while for "little" model inference, only the data from the third channel in the S dimension (shape of $128 \times 3 \times 1$) is processed.

Algorithm 2 Big + Six "Little" Switching Algorithm

Input: Data $D[none, 128, 3, 3]$

Output: Activity label A

```

1: Initialize a boolean for flag of Big model  $F = True$ , true means enable Big
2: Initialize an integer for Big model output label  $B = 0$ 
3: Initialize a boolean for Little model output  $L = False$ , false means activity changes
4: for each data  $d[128, 3, 3]$  in  $D$  do
5:     if not  $F$  then                                     ▷ If Big model is disabled
6:         switch ( $B$ ):
7:             case 0:                                       ▷ If last label is 0
8:                  $L = \text{invoke\_little\_model0}(d[:, :, 2])$  ▷ Invoke Little model 0 with sensor 3 data
9:             case 1:                                       ▷ If last label is 1
10:                 $L = \text{invoke\_little\_model1}(d[:, :, 2])$  ▷ Invoke Little model 1 with sensor 3 data
11:            case 2:                                       ▷ If last label is 2
12:                 $L = \text{invoke\_little\_model2}(d[:, :, 2])$  ▷ Invoke Little model 2 with sensor 3 data
13:            case 3:                                       ▷ If last label is 3
14:                 $L = \text{invoke\_little\_model3}(d[:, :, 2])$  ▷ Invoke Little model 3 with sensor 3 data
15:            case 4:                                       ▷ If last label is 4
16:                 $L = \text{invoke\_little\_model4}(d[:, :, 2])$  ▷ Invoke Little model 4 with sensor 3 data
17:            case 5:                                       ▷ If last label is 5
18:                 $L = \text{invoke\_little\_model5}(d[:, :, 2])$  ▷ Invoke Little model 5 with sensor 3 data
19:            default:
20:                 $F = True$ 
21:        end switch
22:        if not  $L$  then                                     ▷ If activity changes
23:             $F = True$                                      ▷ Enable Big model
24:        end if
25:    end if
26:    if  $F$  then                                           ▷ If Big model enabled
27:         $B = \text{invoke\_big\_model}(d[:, :, :])$              ▷ Invoke Big model with data from all sensors
28:         $F = False$                                        ▷ Disable Big model
29:    end if
30:     $A = B$                                                ▷ Update the activity label to output  $A$ 
31: end for
    
```

The network architecture, shown in Figure 4.8, consists of three branches formed by stacking convolutional 1D layers and max pooling layers from Keras. These branches are then merged using a concatenate layer, followed by a dense layer with six neurons corresponding to the six activity categories. The activities in the dataset are labelled from 0 to 5 to represent each activity during the training and testing phases of the "big" network. Table 4.2 details the network layer

parameters. Since the "big" network uses data from all three sensors in the UCI-HAR dataset to classify six activities, it has three inputs, resulting in approximately 9,000 parameters.

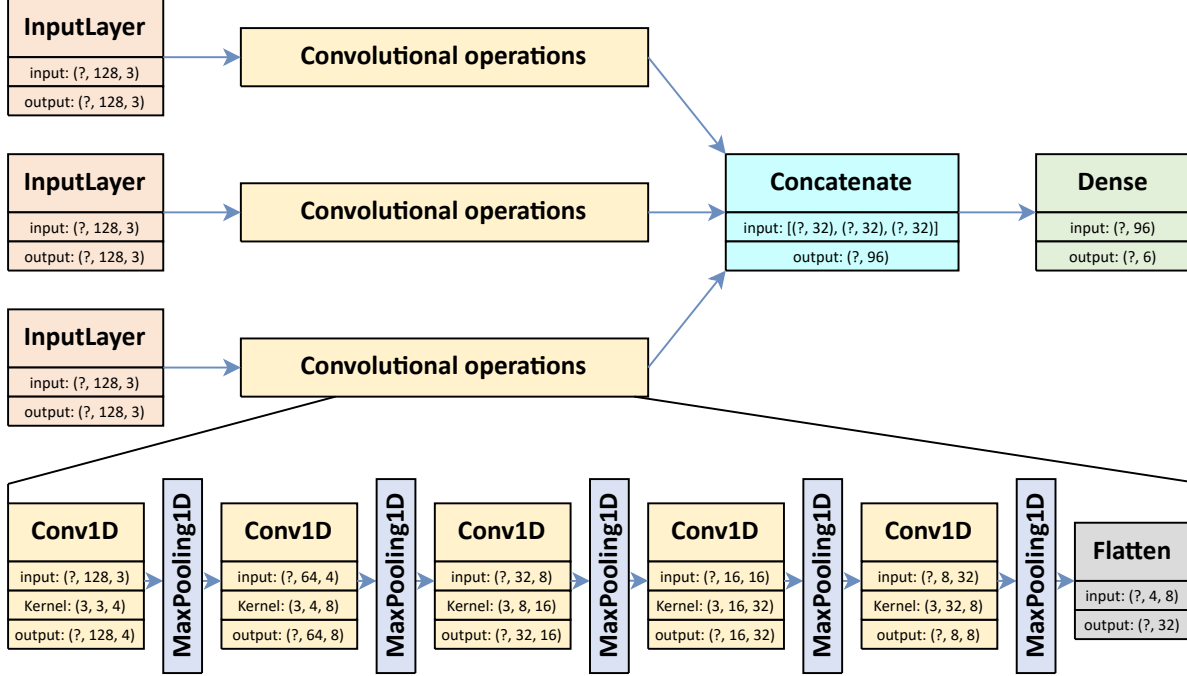


Figure 4.8: "Big" network model structure.

Based on the results in Figure 4.5, the total accelerometer sensor is selected as the input for the "little" network. Each "little" network consists of several convolutional 1D layers and max pooling layers, totalling 184 parameters, as shown in Table 4.2. The network's output is generated by a dense layer with two neurons representing two categories: "corresponding activity" and "not corresponding activity," as illustrated in Figure 4.9. For instance, if the "little" network associated with Activity I outputs a value of 1, it indicates the current activity is WALKING; if it outputs a 0, it signifies another activity, prompting the "big" network to intervene. Given the limited size of the UCI-HAR dataset [54], with fewer than 2,000 samples per activity, all available data are used, and the labels are converted from six categories to two for training the "little" networks.

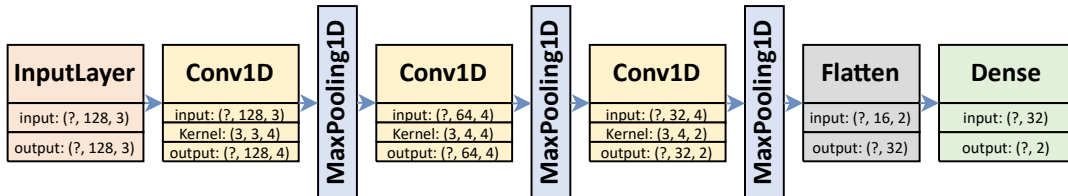


Figure 4.9: "Little" network model structure.

Table 4.2: "Big" (left) and "little" (right) model parameter details. The pooling layers are hidden.

Model: "Big"			Model: "Little"		
Layer	Output Shape	Param#	Layer	Output Shape	Param#
model_input1	[(None, 128, 3)]	0	model_input	[(None, 128, 3)]	0
model_input2	[(None, 128, 3)]	0	conv1d	(None, 128, 4)	40
model_input3	[(None, 128, 3)]	0	conv1d_1	(None, 64, 4)	52
conv1d	(None, 128, 4)	40	conv1d_2	(None, 32, 2)	26
conv1d_5	(None, 128, 4)	40	model_output	(None, 2)	66
conv1d_10	(None, 128, 4)	40			
conv1d_1	(None, 64, 8)	104			
conv1d_6	(None, 64, 8)	104			
conv1d_11	(None, 64, 8)	104			
conv1d_2	(None, 32, 16)	400			
conv1d_7	(None, 32, 16)	400			
conv1d_12	(None, 32, 16)	400			
conv1d_3	(None, 16, 32)	1568			
conv1d_8	(None, 16, 32)	1568			
conv1d_13	(None, 16, 32)	1568			
conv1d_4	(None, 8, 8)	776			
conv1d_9	(None, 8, 8)	776			
conv1d_14	(None, 8, 8)	776			
concatenate	(None, 96)	0			
model_output	(None, 6)	582			
Total params: 9,246			Total params: 184		

To prevent the model from getting trapped in a local optimum, the Stochastic Gradient Descent with Restarts (SGDR) learning rate strategy [186] is implemented during training. This approach periodically resets the learning rate, allowing the model to escape potential local minima. In parallel, the checkpoints of the model weights after each training epoch are saved. This ensures that the best-performing model can be captured by reloading the optimal weights. Finally, the models are generated in Keras format.

4.3.3 "Big" + "Dual" Configuration

The "big" + "dual" configuration is an alternative approach for the adaptive neural network system. Instead of using six "little" networks, this method incorporates a single smaller neural network, referred to as "dual". Compared to the "big" + six "little" configuration, this configuration only involves one "big" network and one "dual" network, rather than one + six. To replace the six "little" networks, the "dual" network stores the data sample of the previous activity and compares it with the current activity sample, as shown in Figure 4.10. This comparison allows the "dual" network to recognize patterns and determine if the current activity has changed. For example, if the "big" network classifies the first activity as WALKING (Activity I) and stores the data sample,

the "dual" network will compare the second data sample with the previously stored sample. If the "dual" network detects a change between the activities, it triggers the "big" network for further inference. Otherwise, the "dual" network remains active to conserve time and energy, as depicted in Figure 4.11.

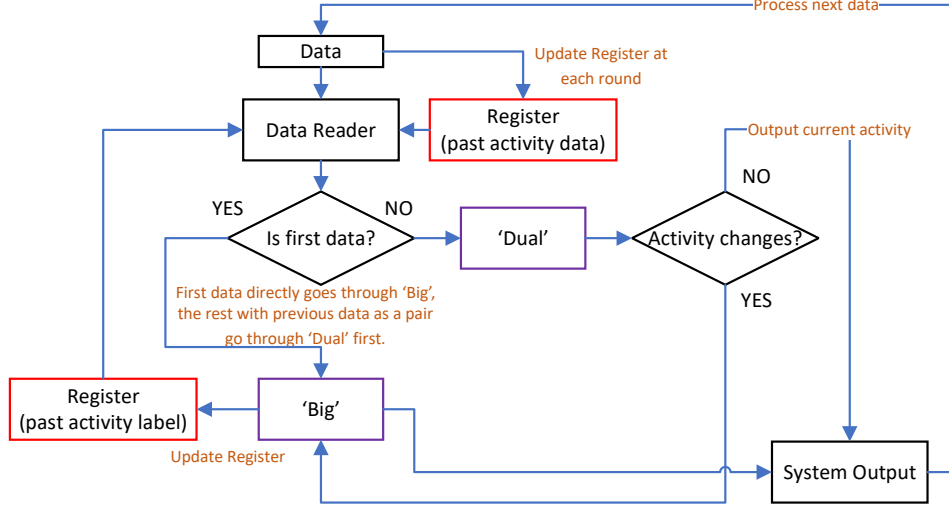


Figure 4.10: The flow chart for the "big" + "dual" configuration of the adaptive neural network system. Registers store the previous data and label, ready to be used in the processing within the "dual" network.

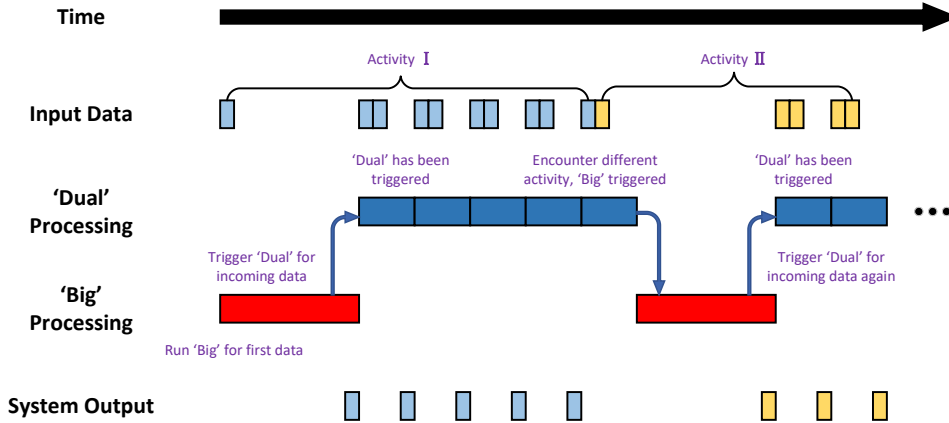


Figure 4.11: The processing steps for the "big" + "dual" configuration of the adaptive neural network system. The two input data blocks represent a pair of adjacent data samples required by the "dual" network.

The "big" network remains the same as the one described in the previous configuration, while the secondary network ("dual") is restructured, as shown in Figure 4.12 and detailed in Table 4.3. Similar to the "little" network, the "dual" network also uses input data from the total accelerometer sensor. Consequently, the input data shape for the "dual" network is $128 \times 3 \times 2$,

which includes two consecutive input data samples. Due to the increased input data size, the number of parameters rises from 184 in the "little" network to 300 in the "dual" network.

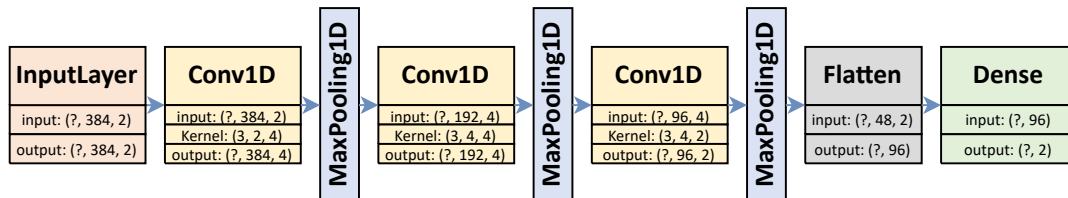


Figure 4.12: "Dual" model structure.

Table 4.3: "Dual" model parameter details. The pooling layers are hidden.

Model: "Dual"		
Layer	Output Shape	Param#
model_input	[(None, 384, 2)]	0
conv1d	(None, 384, 4)	28
conv1d_1	(None, 192, 4)	52
conv1d_2	(None, 96, 2)	26
model_output	(None, 2)	194
Total params: 300		

4.3.4 "Big" + Distance Configuration

Finally, this section explores the possibility of replacing the wake-up module (secondary module) in the adaptive system with a even simpler algorithm to speed up, instead of using neural networks like the "little" and "dual" networks. This alternative configuration, similar to the second setup, substitutes the "dual" network with a distance calculator that measures the difference between two consecutive input samples. When activity changes occur, the distance difference between consecutive samples becomes significant, triggering the "big" network. To detect such changes, a distance calculator utilizing Minkowski and Mahalanobis distance methods is employed to activate the "big" network when the difference exceeds a predefined threshold, as shown in Figures 4.13 and 4.14. The selection of these two distance methods is based on their simplicity and efficiency.

As shown in Equation (4.1), Minkowski distance is a generalized form of Manhattan distance and Euclidean distance. It is equivalent to the Manhattan distance when $p = 1$ and to the Euclidean distance when $p = 2$. The Euclidean distance is a common metric that measures the actual distance between two points in N-dimensional space. Additionally, Mahalanobis distance, as described in Equation (4.2), measures the distance between two points, x and y , within a distribution D , taking into account the distribution's scale. This distance increases if x is farther

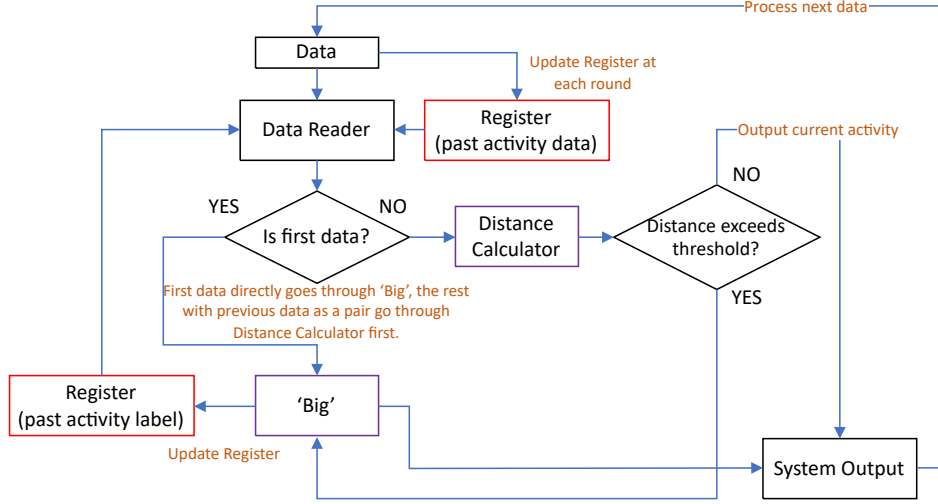


Figure 4.13: The flow chart for the "big" + distance configuration of the adaptive neural network system. Registers store the previous data and label, ready to be used in the processing within the distance calculator.

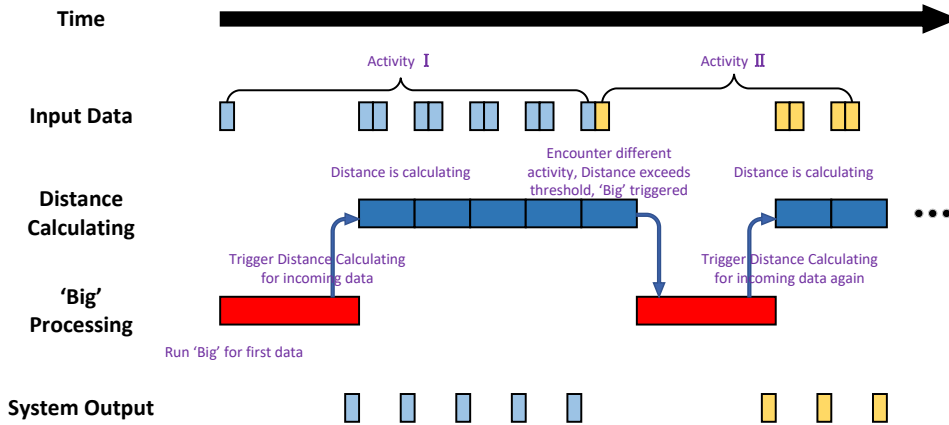


Figure 4.14: The processing steps for the "big" + distance configuration of the adaptive neural network system. The two input data blocks represent a pair of adjacent data samples required by the distance calculator.

from y along each principal component axis of D . Mahalanobis distance simplifies to Euclidean distance when these axes are scaled to have unit variance [187, 188].

$$D_{\text{Minkowski}}(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (4.1)$$

$$D_{\text{Mahalanobis}}(x, y) = \sqrt{(x - y)^\top S^{-1}(x - y)} \quad (4.2)$$

In order to fit the equations, the input data shape of $2 \times 128 \times 3$ for the "dual" network should be reshaped into 2×384 , where the value of 2 indicates that two adjacent data samples

are required by the distance calculator. The calculator then measures the Minkowski distance between these two adjacent data samples using Equation (4.1) for both $p = 1$ and $p = 2$ cases. For the Mahalanobis distance, it requires the covariance matrix of the dataset distribution. Therefore, the covariance matrix of the training set is first calculated, and based on this matrix, the Mahalanobis distance between two adjacent data samples is computed. To wake up the "big" network, multiple thresholds can be set to achieve different sensitivities. The "big" network is only triggered when the distance between the previous data sample and the current one exceeds the pre-set threshold. A lower threshold value results in higher inference accuracy because the "big" network will be invoked more frequently. Conversely, a higher threshold value means the "big" network is invoked fewer times, leading to a shorter inference time. The evaluations and discussions of these three distance measurements are in Section 4.5.4.

4.4 Neural Network Microcontroller Deployment

To deploy network models on resource-constrained devices, the model parameters, such as weights and activations, must be quantized to UINT8 format to minimize memory usage. The input data samples are also quantized to match this format. According to Equation (4.3) [189], the real value of the input during training falls within the range $[-128, 127]$, while the target quantized value is in the UINT8 range of $[0, 255]$, consistent with the model. The mean and standard deviation are calculated as 128 and 1, respectively. This process results in a fully quantized model and input dataset.

$$\text{real_value} = (\text{quantized_value} - \text{mean_value}) / \text{std_dev_value} \quad (4.3)$$

The available UCI-HAR data samples [54] are used instead of real-time data to ensure a fair comparison across different platforms. This approach allows the MCU to correctly access stored data and network models when running the application. Additionally, the model-switching algorithm for the adaptive system, discussed in Section 4.3, is implemented at the C code level rather than within the network model layer. This enables the "big" and "little" models to be invoked independently, making the adaptive system more flexible and efficient in balancing performance and energy consumption. Finally, before deploying the application on the target boards, it must be compiled into an executable binary using cross-compilation tools such as GCC [190], ARM Compiler [191], and the TENSAT Flow compiler from Eta Compute [85]. The model deployment process is illustrated in Figures 4.15 and 4.16.

4.4.1 STM32L4R5ZI

STM32Cube.AI from STMicroelectronics [84] is a framework specifically designed to optimize STM devices like the STM32L4. However, because it operates within a proprietary environment, it does not support deploying the switching algorithm between primary and secondary networks

Device	NN Training	Conversion of NN Model	NN Library in C	SDK	Debugger
STM32L4R5ZI	Keras (TensorFlow)	NNoM Converter	NNoM lib	STM32Cube SDK	ST-Link
Apollo2 Blue	Keras (TensorFlow)	NNoM Converter	NNoM lib	AmbiqSuite SDK	SEGGER J-Link
SparkFun Edge (Apollo3 Blue)	Keras (TensorFlow)	TensorFlow Lite Converter	TensorFlow lib	AmbiqSuite SDK	SEGGER J-Link
ECM3532	Keras (TensorFlow)	TensorFlow Lite Converter	TENSAI Flow lib	TENSAI Flow SDK	SEGGER J-Link

Figure 4.15: Comparison of the software tools used in each deployment phase for different MCUs. The colours indicate different sources.

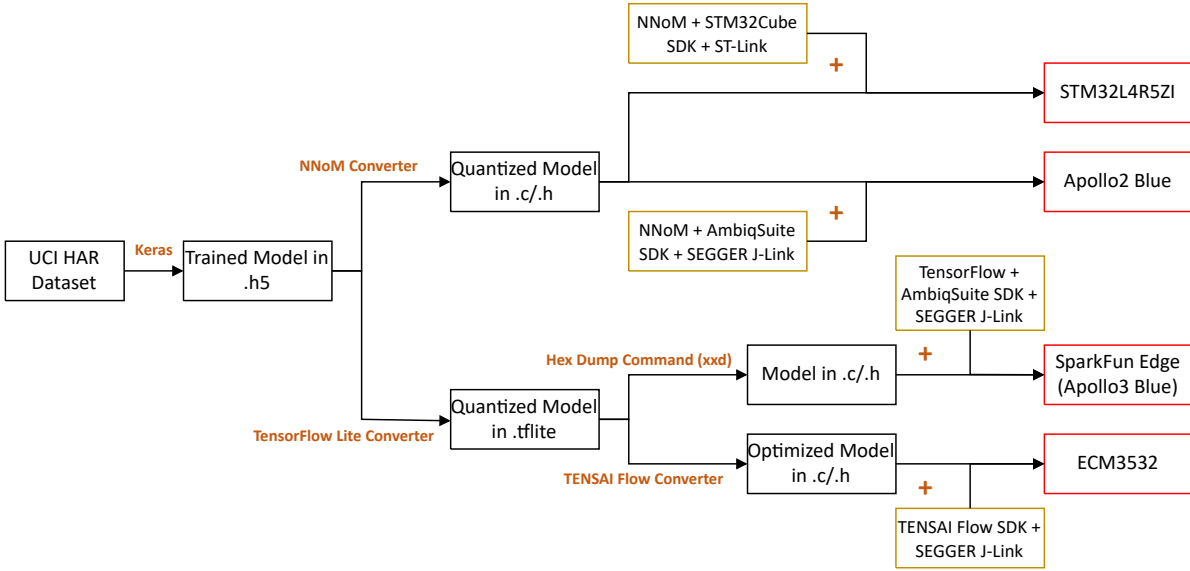


Figure 4.16: The steps for deploying neural network models on target MCU boards. Black boxes denote trained models, brown boxes denote the library source codes used, and red boxes denote the MCU boards.

at the C code level. In contrast, the proposed system is designed for general-purpose and flexible deployment across various MCU boards. To align with this goal, the NNoM converter (version 0.4.3) [82], which is capable of converting Keras models for multiple platforms, is selected for deployment on the STM32L4 instead of using STM32Cube.AI (see Figure 4.15). The NNoM framework converts the pre-trained Keras model into C code, allowing it to be invoked using its neural network library.

To operate the STM32L4R5ZI MCU board, the STM32Cube SDK (version 1.17.0) from STMicroelectronics is required, which includes utility tools and example projects. Keil uVision Integrated Development Environment (IDE) from ARM is used to establish a coding environment compatible with the STM32L4. The necessary driver packs for STM32L4 must be installed via Keil's pack installer. The STM32L4 board is connected to a desktop PC using a micro-USB cable

through the CN1 port. The ST-Link debugger can then be selected in the target debug settings, allowing the PC to connect to and detect the STM32L4 device. If the connection fails, the STM32 ST-LINK Utility software from STMicroelectronics can be used to erase the board and resolve any software conflicts.

After training the neural network models in Keras/TensorFlow (version 1.15), they must be quantized using the NNoM converter command, as shown in Listing 4.1. This process generates a header file containing the model weights. Before building the project, the weight header file, input data file, and files from the NNoM library need to be integrated into a single application using Keil's project manager. Once these components are in place, the project can be built and flashed onto the development board. To view the output in the debug viewer, ensure that the core clock setting under the trace page of the target debug configuration matches the device's operating clock.

Listing 4.1: The Python function of NNoM converter for quantized model generation.

```
1 generate_model(model, x_test, name="weight.h")
```

4.4.2 Apollo2 Blue

The AmbiqSuite SDK (version 2.2.0) from Ambiq is used to support model deployment on the Apollo2 Blue platform. Keil uVision IDE from ARM is utilized to set up the coding environment. After installing the driver pack for Apollo2, the board is connected to the PC via a micro-USB cable, with SEGGER J-Link selected under Keil's target debug settings, as illustrated in Figure 4.16. Similar to the STM32L4 board, Apollo2 is not supported by TensorFlow Lite or TENSAL Flow. Therefore, the pre-trained models in Keras format are converted into a quantized format using the NNoM converter command, as shown in Listing 4.1. The model weights, data header files, and NNoM library are then integrated into the project using the Keil project manager. After building and flashing the project onto the target board, the Keil debug viewer can be used to observe the model outputs.

4.4.3 SparkFun Edge (Apollo3 Blue)

Similar to Apollo2 Blue, the SparkFun Edge (Apollo3 Blue) also utilizes AmbiqSuite SDK (version 2.2.0), which includes the necessary utility tools and drivers to manage and flash the board. However, unlike Apollo2 Blue, SparkFun Edge (Apollo3 Blue) is fully compatible with TensorFlow and TensorFlow Lite. TensorFlow Lite, a sub-module of the TensorFlow framework developed by Google, converts trained neural network models saved in Keras format into a quantized TFLite format, making them suitable for MCU inference. Therefore, TensorFlow Lite (version 1.15) is employed to deploy models on the SparkFun Edge (Apollo3) MCU development board.

The commands for converting the Keras models to TFLite format are detailed in Listings 4.2 and 4.3, corresponding to the "big" and "little" networks, respectively. These commands transform

the floating-point Keras models into UINT8 TFLite models. The resulting TFLite model is then converted into a hexadecimal file using the 'xxd' hex dump command, which is compatible with the TensorFlow Lite library. Based on the C library source code and board SDK files, the binary file can be compiled. Finally, the Apollo3 board is connected to the PC via a micro-USB cable, and the AmbiqSuite SDK is used to flash the binary file onto the target board.

Listing 4.2: TensorFlow Lite converter command lines for "big" model quantization.

```
1  tflite_convert \
2  --keras_model_file=./Output_Models/${MODELNAME}.h5 \
3  --output_file=./Output_Models/${MODELNAME}.tflite \
4  --inference_type=QUANTIZED_UINT8 \
5  --input_shapes=1,128,3:1,128,3:1,128,3 \
6  --input_arrays=model_input1,model_input2,model_input3 \
7  --output_arrays=model_output/BiasAdd \
8  --default_ranges_min=0 --default_ranges_max=255 \
9  --mean_values=128,128,128 --std_dev_values=1,1,1 \
10 --change_concat_input_ranges=false \
11 --allow_nudging_weights_to_use_fast_gemm_kernel=true \
12 --allow_custom_ops
```

Listing 4.3: TensorFlow Lite converter command lines for "little" model quantization.

```
1  tflite_convert \
2  --keras_model_file=./Output_Models/${MODELNAME}.h5 \
3  --output_file=./Output_Models/${MODELNAME}.tflite \
4  --inference_type=QUANTIZED_UINT8 \
5  --input_shapes=1,128,3 \
6  --input_arrays=model_input \
7  --output_arrays=model_output/BiasAdd \
8  --default_ranges_min=0 --default_ranges_max=255 \
9  --mean_values=128 --std_dev_values=1 \
10 --change_concat_input_ranges=false \
11 --allow_nudging_weights_to_use_fast_gemm_kernel=true \
12 --allow_custom_ops
```

4.4.4 ECM3532

TENSAI Flow, developed by Eta Compute, is a framework specifically designed for deploying pre-trained network models on Eta products such as ECM3531 and ECM3532 [85]. Optimized for Eta Compute's hardware, TENSAI Flow focuses on achieving an optimal balance between performance and efficiency. Unlike TensorFlow, TENSAI Flow does not support the training of neural network models; it is used exclusively for model conversion and deployment after training. Since ECM3532 is not compatible with NNoM or TensorFlow Lite directly, TENSAI Flow SDK

(version 2.0.2), which includes the TENSAT Flow converter and neural network C code library, is used to support the model deployment on ECM3532.

To deploy a model on ECM3532, as illustrated in Figure 4.16, the pre-trained Keras model is first converted into a quantized TFLite format using TensorFlow Lite (as detailed in Listings 4.2 and 4.3). Subsequently, TENSAT Flow converts this TFLite model into C code (Listing 4.4), which can be executed using its library source code. Finally, the project is built and flashed to the ECM3532 development board.

Listing 4.4: TENSAT Flow converter command lines for model conversion.

```
1 ./tensaiflow_compile \
2   --tflite_file ../model_zoo/${MODELNAME}.tflite \
3   --out_path ../../Applications/${PROJECTNAME}/src/ \
4   --weights_path ../../Applications/${PROJECTNAME}/include/
```

4.5 Results and Discussion

This section starts by evaluating the accuracy of the different configurations using the full UCI-HAR test dataset, as illustrated in Figure 4.17. Due to memory limitations on MCU platforms, storing the entire test dataset is not feasible. To maintain consistency, this accuracy evaluation is conducted on a PC, using the quantized model in TFLite format which is prepared for MCUs.

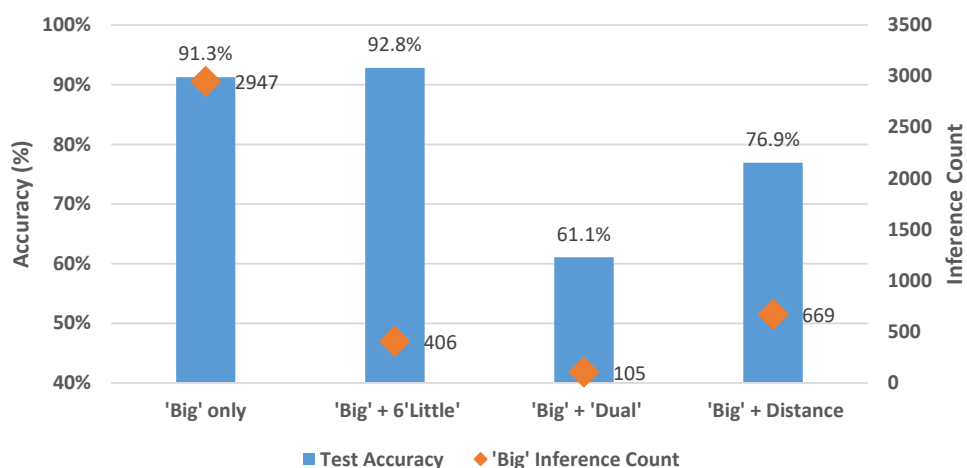


Figure 4.17: The accuracy and the "big" inference counts for four configurations (quantized TFLite format) on the test data set (2947 samples) are evaluated on a PC.

In Figure 4.17, the "big"-only configuration achieves an accuracy of 91.3%, but this comes at the cost of a high inference count, leading to significant latency. The "big" + six "little" configuration attains a comparable level of accuracy while reducing the number of inference times of the "big" network from 2,947 to 406, effectively cutting the total inference time. Although the configurations of "big" + "dual" and "big" + distance exhibit slightly lower accuracy, they effectively

manage tasks as the number of categories increases while simplifying the system and improving processing speed. Overall, the "big" + six "little" configuration meets expectations, delivering both the desired accuracy and a significantly reduced number of "big" network inferences.

After completing the accuracy evaluation, fair comparisons of power, time, and energy consumption across different MCUs are conducted using the same trained network. To ensure consistent testing environments and provide identical test samples for all MCU boards, 60 data samples from the UCI-HAR test dataset are loaded onto the MCU boards due to memory constraints, instead of real-time signals from the board sensors. Ten data samples are selected for each of the six activities and organized into a specific sequence ranging from Activity I to Activity VI, resulting in five activity transitions within the test sequence. The classification results for these 60 samples closely align with those obtained from the entire dataset, with only minor variations across devices caused by different toolchains. The following evaluations are performed at a working frequency of 48 MHz without debugging, assessing four configurations of the adaptive neural network.

4.5.1 "Big"-Only

Although the "big" network achieves an accuracy of around 90% on the activity classification task, as shown in Figure 4.17, this "big"-only approach performs 2947 inferences across all test data samples. Due to the network's large topology and the high number of inferences, this "big"-only configuration results in the longest execution time among all configurations on all four MCUs operating at the same frequency of 48 MHz, as shown in Figure 4.18. As previously mentioned, the more frequently the "big" network is invoked, the greater the latency.

Since the MCU boards apply the same power strategy when running different configurations, the power consumption of these configurations is similar to the MCU shown in Figure 4.19. Due to the negligible differences between network configurations in terms of power, the distribution of the energy consumption of the configurations for each MCU follows the time cost distribution in Figure 4.18. However, it is evident that the STM32L4 board typically consumes more power than the others with near-threshold design, which aligns with expectations. Consequently, energy consumption for each configuration is mainly determined by the inference time and the power profile of the boards. Therefore, as shown in Figure 4.19, the "big"-only configuration results in the highest energy consumption across all MCU boards.

4.5.2 "Big" + Six "Little"

Calculating the average time cost for both the "big" and "little" networks reveals that a single "big" inference takes approximately 10 to 12 times longer than a "little" inference. In this configuration, six "little" networks are utilized to save time by reducing the "big" inference count to around 10 instead of 60, as shown in Figure 4.18. In other words, substituting around 50 "big" inferences with 60 "little" inferences, which is equivalent to over 500 "little" inferences in terms of efficiency,

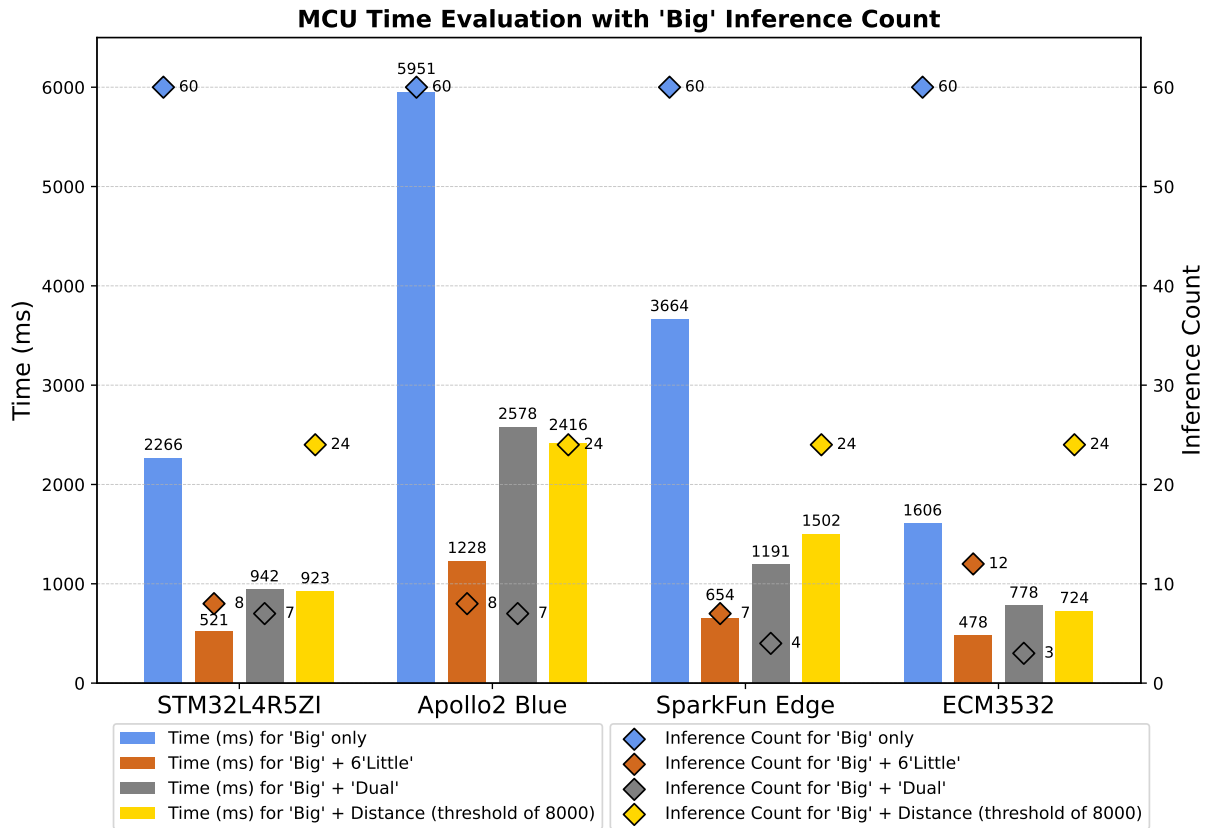


Figure 4.18: The time evaluation of four adaptive configurations on MCU boards with the "big" inference counts. A total of 60 data samples extracted from the UCI-HAR test data set are tested to form the evaluation.

proves worthwhile in the experiment. As a result, across all MCU evaluations, the "big" + six "little" configuration achieves the lowest execution time, reducing the time by approximately 80% compared to the original "big"-only configuration and about 50% compared to other configurations, while still maintaining overall system accuracy (Figure 4.17).

Given that power usage remains largely consistent across different configurations on each board, the energy consumption for this configuration achieves savings of approximately 80% compared to the original "big" method on all MCUs, thanks to its significant reduction in execution time. By leveraging near-threshold MCUs, energy consumption can drop to as low as 1.4 mJ, as shown in Figure 4.19, representing a 98.4% reduction compared to the original "big"-only configuration on the STM32L4. This improvement could potentially extend battery life by up to 62 times for HAR applications.

However, despite its advantages in inference speed and energy efficiency, the number of "little" networks in this configuration is tied to the number of categories, making model deployment challenging when dealing with a large number of categories (e.g., 100 categories in the CIFAR-100 dataset). So, this configuration is best suited for applications with a small number of categories.

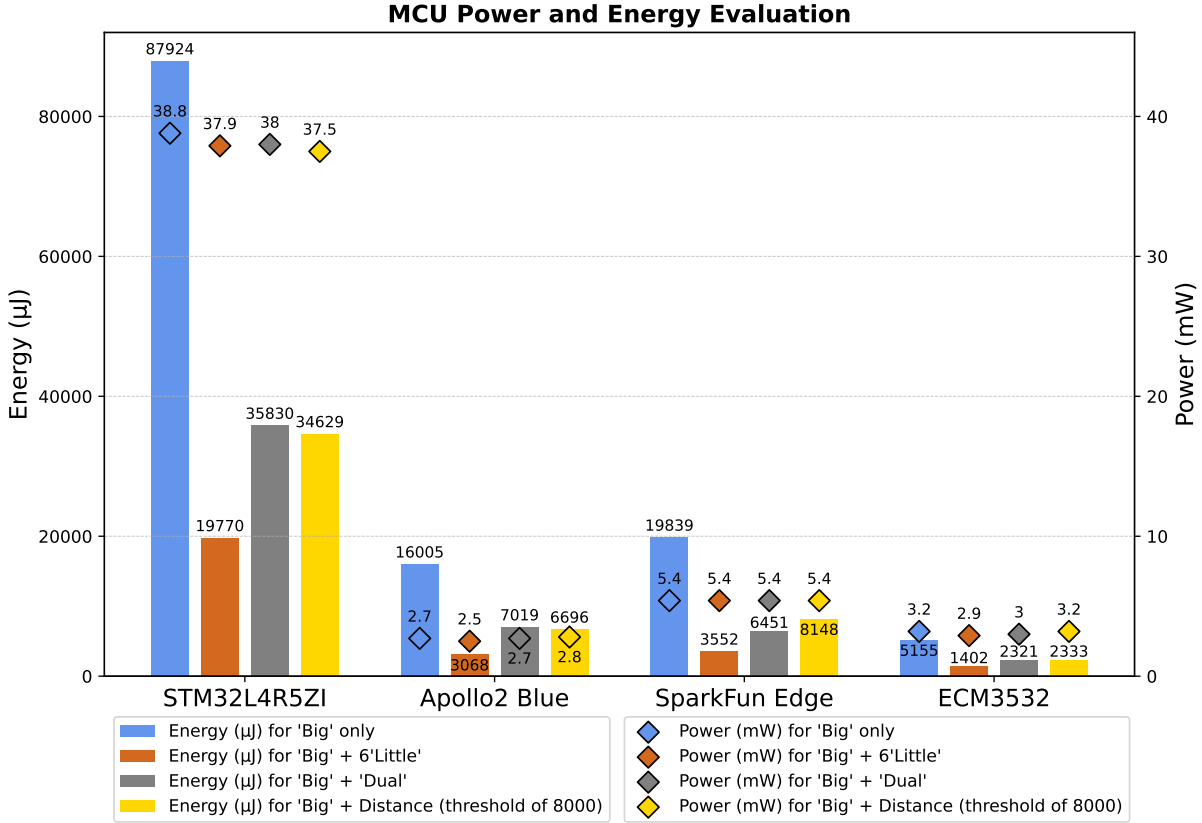


Figure 4.19: The power and energy evaluation of four adaptive configurations on MCU boards. A total of 60 data samples extracted from the UCI-HAR test data set are tested to form the evaluation.

4.5.3 "Big" + "Dual"

In contrast to the "big" + six "little" configuration, the "big" + "dual" configuration is not limited by the number of categories that need to be classified. By utilizing a single secondary network that focuses on detecting category changes, the "big" + "dual" configuration can identify activity changes by comparing the current activity with the previous one. However, this approach has two key drawbacks. First, among the 7,352 training data samples, only 280 instances involve activity transitions. To train the "dual" network, a total of 280 samples labelled as "activity changes" and 7,072 samples labelled as "activity remains" are used, resulting in an imbalanced dataset. This can be resolved by expanding the dataset in the future. Second, the configuration is susceptible to error propagation if the "dual" network incorrectly classifies an activity change. In Figure 4.20, the "dual" network incorrectly classifies sample 7, missing the transition from activity I to III. This error causes the system to skip the "big" network inference and incorrectly label the activity as I instead of III. Although no further errors occur, the system continues to propagate this initial mistake, maintaining the wrong output. In contrast, the "big" + six "little" configuration does not

encounter this issue, as it would identify and correct the error in subsequent samples.

Data Number		1	2	3	4	5	6	7	8	9	10	
Real Activity Label		II	II	II	I	I	I	III	III	III	III	
'Little' Output		II	II	II	Not II	I	I	I	Not I	III	III	
'BIG' Output	...	Not Invoked	Not Invoked	Not Invoked	I	Not Invoked	Not Invoked	Not Invoked	III	Not Invoked	Not Invoked	...
'Big' + 6 'Little' Output		II	II	II	I	I	I	I	III	III	III	
'Dual' Output		Remains	Remains	Remains	Changes	Remains	Remains	Remains	Remains	Remains	Remains	
'BIG' Output		Not Invoked	Not Invoked	Not Invoked	I	Not Invoked	Not Invoked	Not Invoked	Not Invoked	Not Invoked	Not Invoked	
'Big' + 'Dual' Output		II	II	II	I	I	I	I	I	I	I	

Figure 4.20: The output comparison of two configurations when an error occurs at the moment of an "activity changes". Errors are marked in red.

Although the "dual" network can handle a large number of categories, its performance is hampered by the unbalanced training dataset. Due to the poor accuracy of the "dual" network, the error propagation mentioned in Figure 4.20 occurs and fails to switch on the "big" network when detecting an activity change for further inference. This results in a minimal "big" inference count but a relatively poor performance in terms of accuracy, around 60% as shown in Figure 4.17). Additionally, the "dual" network's higher complexity and continuous activation result in latency and energy consumption that is twice as high compared to the "big" + six "little" network configuration, as depicted in Figures 4.18 and 4.19.

4.5.4 "Big" + Distance

This "big" + distance configuration even simplifies the secondary module by applying a distance calculator. Three distance measurement methods are evaluated for accuracy and efficiency, and one is selected for deployment based on the results. Figure 4.21 shows that the Mahalanobis distance struggles to differentiate activities I to III (WALKING, WALKING_UPSTAIRS, WALK-

Data Number	1	2	3	4	5	6	7	8	9	10	11	12	
Activity Label	V	V	IV	IV	VI	VI	I	I	III	III	II	II	Activity remains
Manhattan Distance (p = 1)		154	10,230	230	19,717	4,153	28,782	8,642	7,300	8,518	9,174	5,376	Activity changes
Euclidean Distance (p = 2)	NULL	11	582	20	1,090	124	1,693	565	485	756	604	389	Distinguishable Values
Mahalanobis Distance		201,532	1,609,852	263,481	1,349,489	746,925	418,438	511,839	653,612	547,821	495,100	586,691	Indistinguishable Values

Figure 4.21: Distance results for different pairs of adjacent data samples. Values in one distance measurement (one row) are comparable.

ING_DOWNSTAIRS), as distances within the same activity sometimes exceed those between different activities. This overlap complicates setting a clear threshold for distinguishing "activity changes" from "activity remains". Similarly, Euclidean distance, due to its square root calculation, offers less distinct separation compared to Manhattan distance.

To further investigate, seven different threshold levels are evaluated for these distance measurements on a PC to cover a wide range of scenarios. Figures 4.22, 4.23, and 4.24 show the trends in accuracy and time cost, starting with a threshold of 0, which triggers the maximum 2947 "Big" inferences (no skipping). As the threshold value increases, the number of "Big" inferences drops, leading to reduced time costs but also a decline in accuracy.

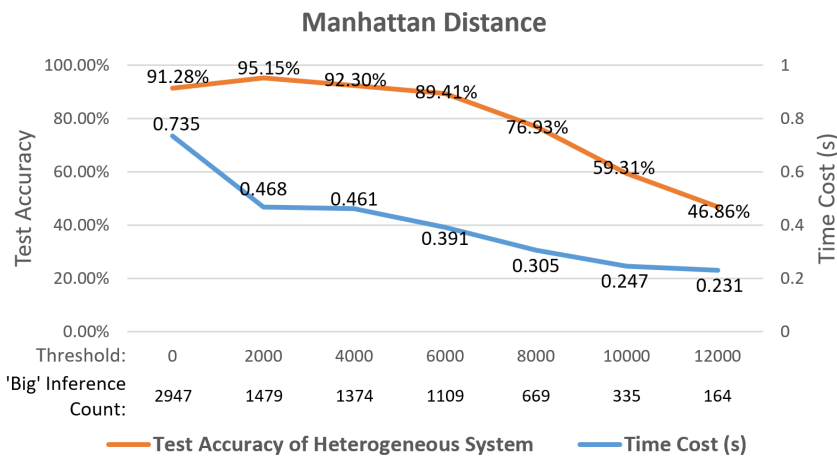


Figure 4.22: The overall accuracy and time cost (on PC) against seven levels of Manhattan distance, with the number of "Big" inferences labelled.

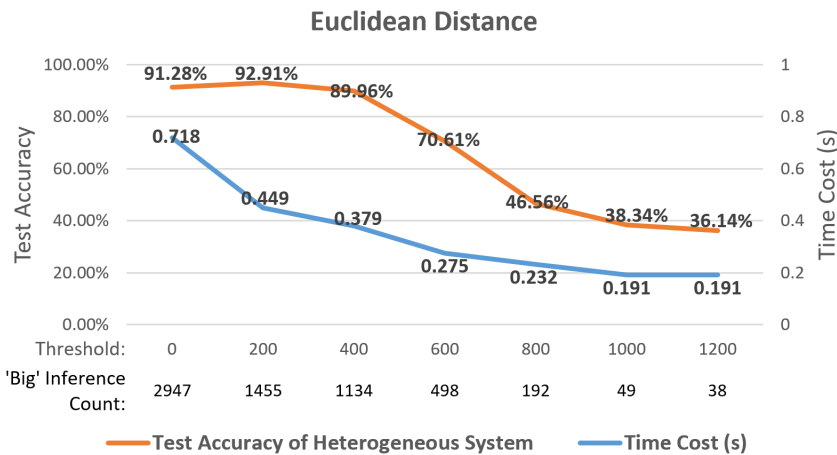


Figure 4.23: The overall accuracy and time cost (on PC) against seven levels of Euclidean distance, with the number of "Big" inferences labelled.

Given the relatively poor performance of the Euclidean and Mahalanobis distances, the Manhattan distance is deployed as the secondary module. As the threshold directly influences the

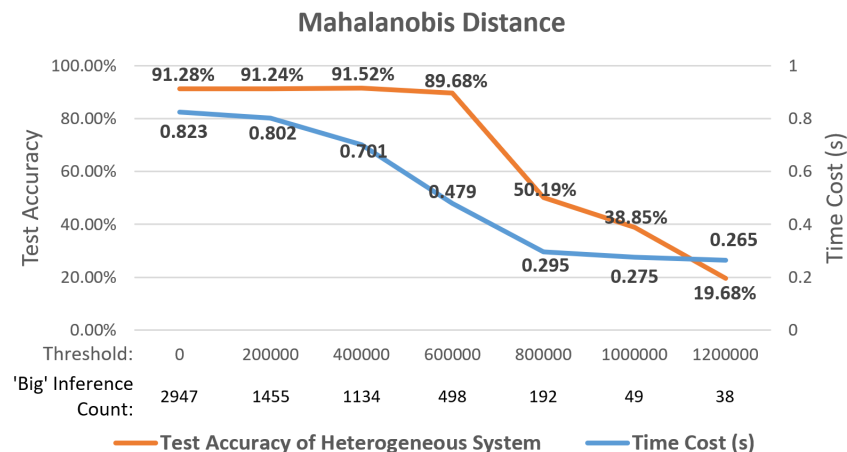


Figure 4.24: The overall accuracy and time cost (on PC) against seven levels of Mahalanobis distance, with the number of "Big" inferences labelled.

number of "Big" inferences, the goal is to maintain this number below 1,000 to ensure efficiency. Therefore, a Manhattan distance threshold of 8,000 is selected for the evaluations presented in Figures 4.17, 4.18, and 4.19.

Similar to the "big" + "dual" configuration, the "big" + distance configuration also experiences issues with error propagation, which significantly impacts overall accuracy. Compared to the "big" + six "little" configuration, this configuration achieves a lower accuracy of about 76% while requiring more frequent activations of the "big" network, as shown in Figure 4.17. Additionally, it incurs substantial latency and energy costs, roughly double those of the "big" + six "little" configuration, while comparable to the "big" + "dual" configuration, as depicted in Figures 4.18 and 4.19.

One area for future research could involve enhancing the performance of the Mahalanobis distance. Typically, the Mahalanobis distance measures the distance between a target point x and the mean of a distribution D , rather than between two distinct points. Based on this, six separate covariance matrices can be applied for each activity subset D_i instead of a single covariance matrix for the entire dataset D . By doing so, the Mahalanobis distance increases if point x deviates significantly from the activity subset D_i , indicating that it does not belong to this activity and thereby triggering the "big" network.

4.6 Conclusions

This study compares commercially available near-threshold and standard MCUs in terms of performance and energy consumption. At the same operating frequency, near-threshold MCUs demonstrate a significant power consumption advantage, being approximately 85% lower than standard MCUs. With comparable processing speeds, these low-power near-threshold MCUs achieve energy savings of about 80% to 85% compared to the standard MCU, STM32L4.

Moreover, three adaptive neural network configurations for HAR tasks are developed and evaluated, focusing on how MCU deployments can leverage these algorithms to reduce energy consumption while preserving prediction accuracy. It is demonstrated that, despite the limited memory, "Big-Little" configurations can be deployed to significantly enhance energy efficiency and performance on MCU boards including STM32L4R5ZI, Apollo2 Blue, SparkFun Edge (Apollo3 Blue), and ECM3532. The most effective adaptive configuration, the "big" + six "little" setup, reduces model inference time by approximately 80% by cutting the "big" inference count by around 85% compared to the base method across all MCU platforms. This reduction in inference time leads to an 80% decrease in energy consumption while maintaining a similar prediction accuracy of about 90% for the HAR task. Furthermore, coupled with the state-of-the-art near-threshold MCUs, this best configuration is able to achieve a reduction in energy of up to 98% that will translate into a $62\times$ increase in the operating lifetime for HAR applications.

However, the "big" + six "little" configuration is constrained by the number of categories, as it requires a corresponding "little" network for each category. Managing a task with a high number of categories becomes challenging due to the need for numerous "little" networks, resulting in significant implementation complexity and storage difficulties. The other two proposed configurations can address this issue, but they suffer other limitations. The results indicate that these two proposed methods deliver relatively poorer performance in terms of time-energy reduction and accuracy on MCU devices compared to the best one.

Future efforts will focus on integrating IMU sensors with human subjects to collect real data and facilitate real-time activity prediction using battery power. This integration will help in creating a sufficient dataset for model training, overcoming the overfitting issues caused by the limited dataset and enhancing the performance of "big" + "dual" and "big" + distance configurations. Additionally, this research has the potential to be extended to other applications, including human presence detection and health monitoring. More discussion can be found in Section 7.2.

HUMAN TRACKING AND FALL DETECTION USING MMWAVE RADAR

This chapter explores an indoor system designed for tracking multiple human targets and detecting falls, employing three mmWave radars from TI. Compared to wearables and camera-based methods, mmWave radar overcomes challenges related to mobility, lighting conditions, and privacy. An initial evaluation assesses radar characteristics, including resolution, interference, and coverage area. In contrast to the use of the published dataset (i.e., UCI-HAR) in the previous chapter, a real-time framework is established to integrate signals from these radars, facilitating non-intrusive tracking of human positions and body states. Innovative strategies introduced include dynamic DBSCAN clustering based on signal SNR levels, a probability matrix for enhanced target tracking, state prediction for fall detection, and a feedback loop for noise reduction. Extensive evaluation is conducted with over 300 minutes of data, equating to approximately 360,000 frames. The prototype system exhibits remarkable performance, achieving an F1 score of 98.4% for tracking a single target, and 97.3% and 95.9% for two and three targets in human tracking, respectively. Moreover, in the field of human fall detection, the system demonstrates a high accuracy rate of 96.3% and a 99.0% sensitivity for the fall category, underscoring its effectiveness in distinguishing falls from other states. The content of this chapter has been published in the MECO conference 2023 [51]¹ and MDPI Sensors journal [52]².

The sections are organized as follows: Section 5.1 provides a brief overview and lists key contributions of this chapter. Section 5.2 conducts an evaluation of the mmWave radar system, covering aspects such as resolution, angle of view compensation, and the relationship between

¹©2023 IEEE. Reprinted, with permission, from Z. Shen, J. Nunez-Yanez, and N. Dahnoun, “Multiple human tracking and fall detection real-time system using millimeter-wave radar and data fusion,” in 2023 12th Mediterranean Conference on Embedded Computing (MECO), pp. 1–6, IEEE, 2023.

²©2024 MDPI. Reprinted, with permission, from Z. Shen, J. Nunez-Yanez, and N. Dahnoun, “Advanced millimeter-wave radar system for real-time multiple-human tracking and fall detection,” *Sensors*, vol. 24, no. 11, p. 3660, 2024.

radar placement and coverage. Section 5.3 illustrates the mmWave radar setup in the test environment and data collection. Subsequently, Section 5.4 delves into the details of the software framework’s workflow and its utilization for human tracking and fall detection. A performance evaluation of the designed real-time system in scenarios involving multiple people is presented in Section 5.5. Finally, Section 5.6 outlines conclusions and discusses future work.

5.1 Overview

HAR systems have garnered significant attention in the industry, particularly in the field of camera-based systems leveraging machine learning techniques [8, 64, 65, 101]. However, these camera-based systems come with drawbacks, including privacy invasion, dependency on specific lighting conditions, and reduced performance in the presence of smoke or fog. As a solution to these challenges, many researchers have turned to mmWave radar technology employing the FMCW technique.

The mmWave radar used in this thesis operates at a high-frequency range (from 76 to 81 GHz), providing several advantages such as high resolution and improved anti-interference capabilities. Therefore, FMCW mmWave radar technology has demonstrated significant potential in various indoor HAR applications, including human tracking [19, 49, 192]. Furthermore, human posture detection and fall detection represent popular applications for mmWave radar, addressing critical safety concerns in various settings such as the healthcare system for the elderly [57, 60, 109].

This chapter explores the use of mmWave radar for human tracking and fall detection, discussing the operational principles, ongoing research, and development efforts in this area. A real-time system is introduced, detailing its successful implementation. Using three FMCW radar IWR1843 development boards from Texas Instruments (TI), the system achieves high precision for tracking and fall detection (as illustrated in Figure 5.1). This enables accurate real-time monitoring of multiple human targets in indoor environments. The key contributions of this work are as follows:

- Three radars are deployed to expand the coverage area, and a real-time system is designed to collaborate with all sensors, capturing point clouds at 20 FPS from the scene.
- Innovative strategies are introduced, including dynamic DBSCAN for improved target detection when the human target is static, a probability matrix for multi-target tracking, and target state prediction for fall detection.
- The system is evaluated over 300 minutes of experiments, encompassing single- and multi-person scenarios involving walking, sitting, and fall states, demonstrating its effectiveness in human target tracking and fall detection.
- The work is available at https://github.com/DarkSZChao/MMWave_Radar_Human_Tracking_and_Fall_detection to further promote work in this field.

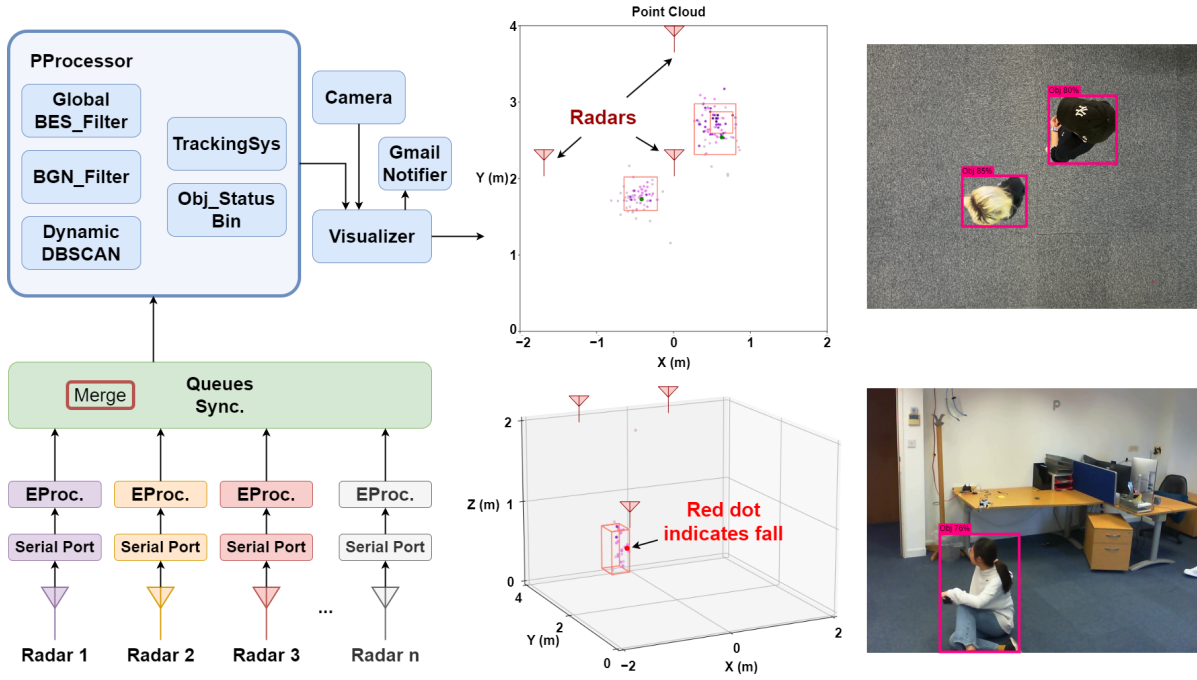


Figure 5.1: The system overview with modules (left), 2D/3D reconstruction (middle), and real images (right).

5.2 Radar System Evaluation

Before integrating the IWR1843 mmWave radars into the experiments, the preliminary assessments are conducted covering various aspects such as point cloud resolution (Section 5.2.1), potential interference from multiple radars (Section 5.2.2), and evaluation of radar placement and coverage (Section 5.2.3). The resolution evaluation provides insights into the point cloud density reflected by human bodies. Given the deployment of three radars in the experimental field, assessing potential interference becomes crucial. Furthermore, the radar placement evaluation highlights any coverage discrepancies associated with different angles of placement.

5.2.1 Radar Resolution

As discussed in Section 3.1.2.1 of Chapter 3, based on [105, 131] and Fourier Transform theory, to separate two adjacent targets, the difference between their corresponding IF signals must exceed $1/T$ as indicated in Equation (3.2). This leads to Equation (5.1) as follows. In this equation, B equates to the 4 GHz bandwidth of the IWR1843 radar. Then, the theoretical resolution d_{res} is calculated to be 3.75 cm.

$$d_{res} = \frac{c}{2B} \quad (5.1)$$

To validate the theoretical calculations, an experiment is conducted in a long corridor, as illustrated in Figure 5.2. A corner reflector is positioned as the sole target within the specified area. This corner reflector, designed to reflect signals in the opposite direction, essentially represents a single reflection point in practical scenarios. To enhance the intensity of the reflected signal, the corner reflector is covered with multiple layers of tin foil. The mmWave radar is positioned at a height of 1.2 meters, matching the corner reflector's height. The corner reflector is initially placed 1 meter in front of the mmWave radar, with the distance gradually increased by 1 centimeter until it reaches 2 meters.

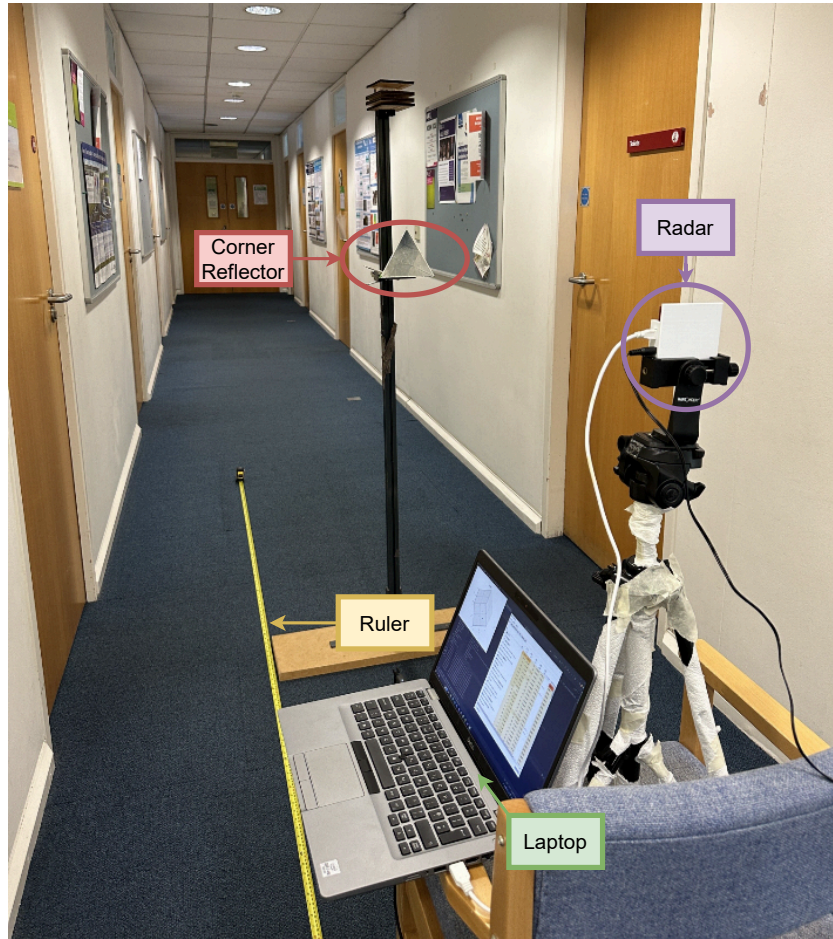


Figure 5.2: The image of the corner reflection experiment for radar range resolution.

After filtering out the noise, the resolution results are depicted in Figure 5.3. It is observed that the practical resolution achieved is approximately 4 cm, aligning closely with the theoretical resolution above. Such a resolution level is adequate for maintaining user privacy (as mentioned in Section 2.2.2.3) and simultaneously yields a rich point cloud for comprehensive analysis.

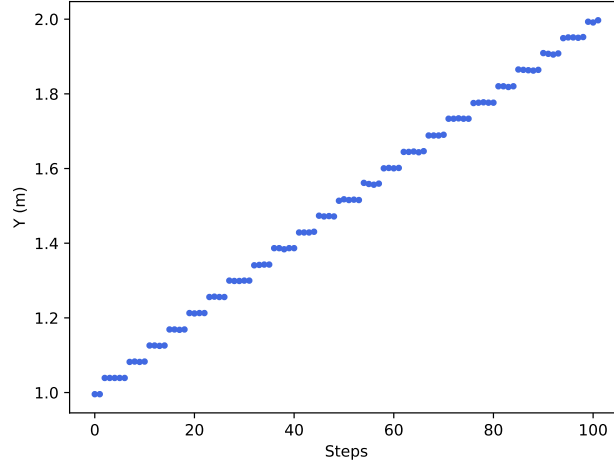


Figure 5.3: The distance of points reflected by the corner reflector in y-axis versus steps. The corner reflector is moved 1 cm per step away from the radar.

5.2.2 Multiple Radar Arrangement

The theoretical Angle of View (AoV) for mmWave radars is $\pm 90^\circ$ in both horizontal and vertical directions. However, according to the radar evaluation manual [134], the effective AoV for 6dB-beamwidth is reduced to around $\pm 50^\circ$ horizontally and $\pm 20^\circ$ vertically due to antenna characteristics and signal attenuation. This reduced vertical AoV means that a radar placed at a height of 1 meter can only capture signals reflected from the human chest to the knee, limiting its ability for complete human body detection.

To solve this limitation, a radar placed on the ceiling can capture data points representing the human head. Additionally, employing multiple radars facing different directions enables the distinction of human targets even when one is positioned behind another, ensuring accurate tracking of multiple human subjects. Therefore, three identical TI IWR1843 mmWave radars are deployed on both the wall and ceiling. This setup enables the capture of strong signals not only from the human main body but also from the human head. For detailed information, please refer to Section 5.3.

5.2.2.1 Radar Interference

When employing multiple radars, it is crucial to ensure that they do not interfere with each other. Given that the chirp ramp time is set to $57.14 \mu\text{s}$ in Table 3.2 in Section 3.1.6, and the full bandwidth for the IWR1843 is 4 GHz, a slope rate of $70 \text{ MHz}/\mu\text{s}$ can be derived. For instance, when considering a maximum measurement distance of 4 meters, the round-trip time-of-flight is calculated as $0.027 \mu\text{s}$, based on the speed of light. This time delay corresponds to a frequency shift of approximately 1.87 MHz, as depicted in Figure 5.4.

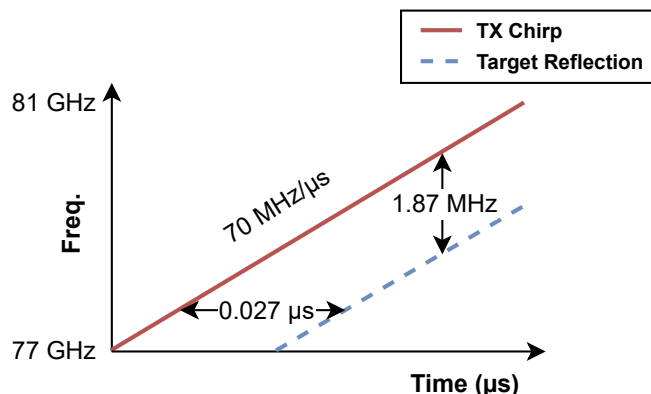


Figure 5.4: The sending chirp and the reflection chirp from the target.

Referring to the interference probability equation (Equation (5.2)) for N radars presented in [19], the interference probability for the scenario involving three radars can be computed. In this equation, B_{total} represents the 4 GHz chirp bandwidth of the TI radar, and B_{inter} denotes the interference bandwidth. This implies that radars will only interfere if the frequency difference between any two radars falls within the interference bandwidth of 5.61 MHz in this experiment [19]. Ultimately, the interference probability for three radars is 0.4%, assuming that the radars are activated at random times.

$$P(N) = 1 - \prod_{i=1}^N \frac{B_{total} - B_{inter} \cdot (i - 1)}{B_{total}} \quad (5.2)$$

Additionally, Cui and Dahnoun [19] demonstrated that the average variances of radar detection with and without interference from a second radar are quite similar, even when the two radars are within a 3 m range. Based on these findings and the experiment setup (in Section 5.3.1), the interference between radars in this work is minimal and can be considered negligible.

5.2.3 Radar Placement and Coverage Evaluation

In the previous study [51], a significant effort was made to enhance the resolution of point clouds representing the human body. Initially, the radars were positioned at a height of 1 meter on the wall and placed perpendicular to the ground, thereby ensuring comprehensive coverage from knee to neck level. However, this arrangement yielded suboptimal resolution and restricted recognition capabilities when human subjects were approaching the radar closely. Specifically, as depicted in Figure 5.5, this setup led to a diminished coverage area of approximately 50% when the human target approached within 1 meter of the radar. This limitation arose due to the inadequacy of the mmWave signal emitted from the radar (green) positioned at a height of 1 meter, failing to sufficiently spread across the human body's main area before encountering targets.

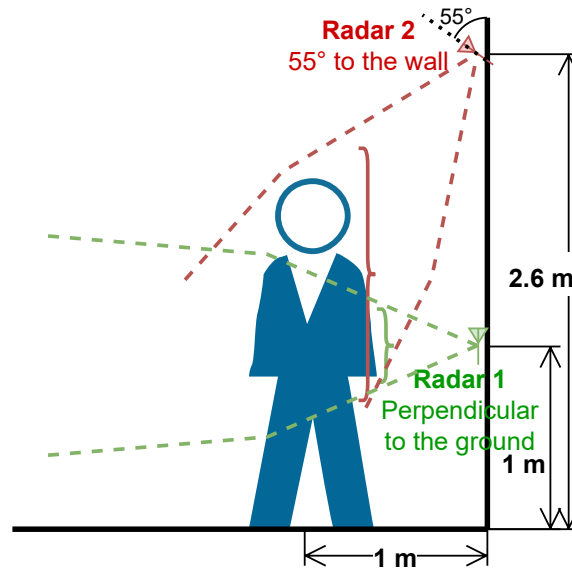


Figure 5.5: The comparison of the cover area between the radar installed at a height of 1 meter (green) and the radar positioned at 2.6 meters with a 55° angle to the wall (red).

To solve this issue, the radar is repositioned at a higher point and tilted downward to focus on the centre of the ground, as depicted by the red radar in Figure 5.5. This alteration aims at broadening the mmWave signal's coverage, allowing it to encompass a larger area of the human body before reflection. It is worth noting that the tilt angle can be adjusted according to height and distance needs. In the experiment, the radar is tilted at 55° to the wall, which results in a coverage area twice as large as that of the green radar when a human target approaches it. As shown in Figure 5.6, a custom radar case is produced using 3D printing to facilitate installation and ensure the radar maintains the precise 55° tilt.

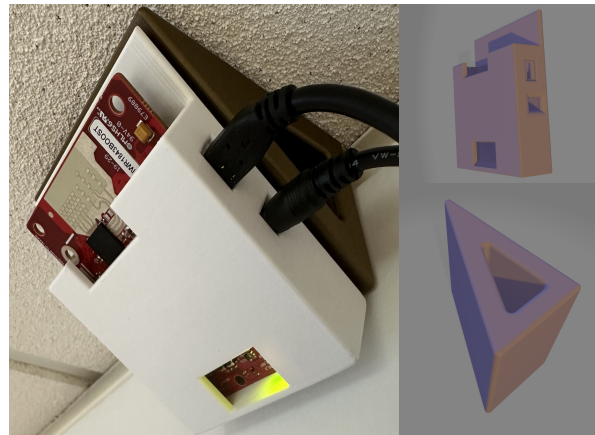


Figure 5.6: The radar placed on the ceiling at a 55° angle to the wall shown on the left side. Both the white radar case and the brown support are 3D printed, shown on the right side.

Figure 5.7 accumulates 200 frames over a span of 10 seconds, depicting a scenario where a person stands stationary approximately 1 meter away from the wall. In the left column, the point cloud received and the corresponding histogram for the radar positioned at a height of 1 meter are displayed. Most data points cluster tightly in the middle of the human target due to the radar's limited field of vision, leading to the loss of information regarding the head and legs. So, in that case, the system might interpret this as a seated human due to the absence of data on the head and chest [51].

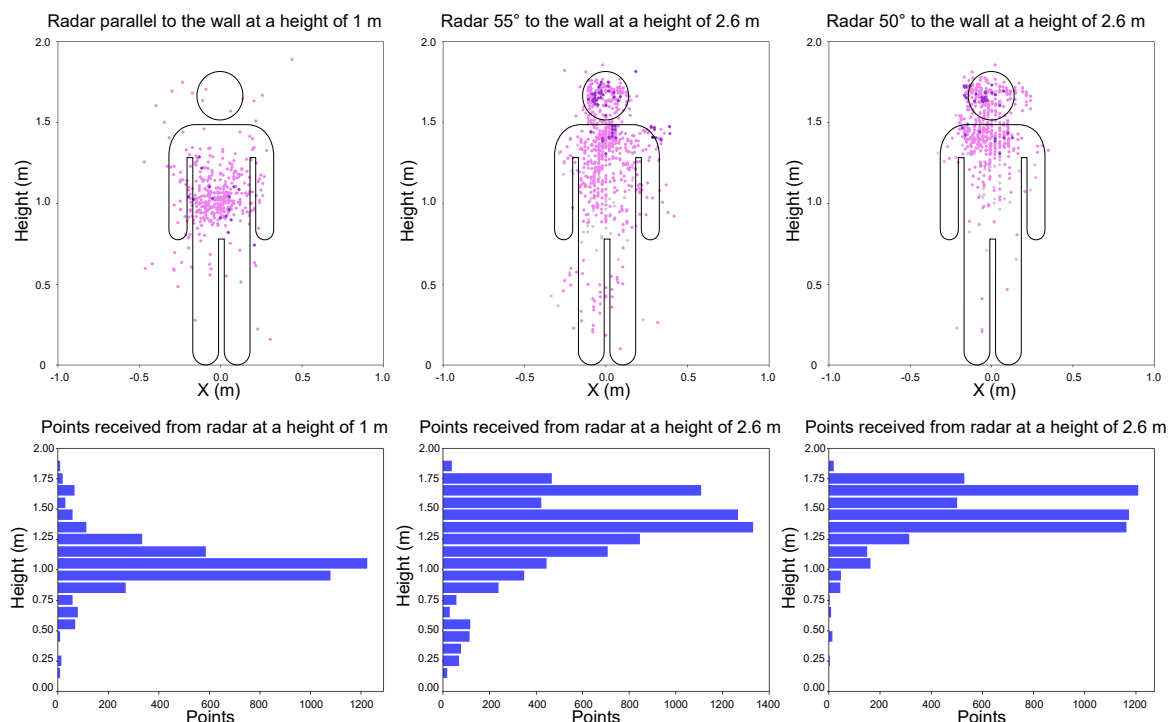


Figure 5.7: The point clouds acquired from the 1 m height radar (left), the 2.6 m height radar with a tilt of 55° (middle), and the 2.6 m height radar with a tilt of 50° (right), along with the corresponding histograms depicting the number of points recorded over a duration of 10 s when a target is positioned approximately 1 m in front of the radars.

However, the point clouds and histograms shown in the middle and right columns, captured by the radars placed at a height of 2.6 m (tilt of 55° and 50°), reveal a significant concentration of points representing the human head, chest, abdomen, and legs. These comprehensive data ensure the accurate classification of the human's states, even when the person is close to the wall, eliminating any potential blind spots. In the experiment, the coverage provided by the 50° tilt angle is relatively poor compared to the 55° tilt, as it less effectively covers the human body parts below the chest. Meanwhile, a larger tilt angle causes the radar to face more toward the ground, leading to decreased detection ability when the person is relatively far from the wall. Therefore, the 55° angle is chosen for the experiment setup. In order to compensate for the 55° tilt applied

to the radar device, a point cloud rotation adjustment is implemented to map the data points into global coordinates, details can be found in Section 5.4.1.

5.3 Experimental Setup

5.3.1 Radar Characterisation

To generate 3D point cloud data, three IWR1843 radars are configured to utilize all three TXs and four RXs following the instructions of the TI mmWave visualizer [135]. The start frequency and end frequency are set to 77 GHz and 81 GHz, respectively, resulting in a bandwidth (B) of 4 GHz. The Chirp Cycle Time (T_c), defined as the duration of each sweep cycle, is set to 162.14 μ s, while the Frequency Slope (S), representing the sweep rate of the FMCW signal, is set to 70 MHz/ μ s. With this setup, the sensor achieves a range resolution of 4.4 cm and a maximum unambiguous range of 5 m. Regarding velocity, it can measure a maximum radial velocity of 2 m/s with a resolution of 0.26 m/s [135]. This level of measurement accuracy is sufficient for capturing human movements within the room in Figure 5.8, supporting the HAR applications. To balance the workload between the three radars and the PC, each radar is configured to operate at 20 FPS, with 32 chirp loops for each frame.

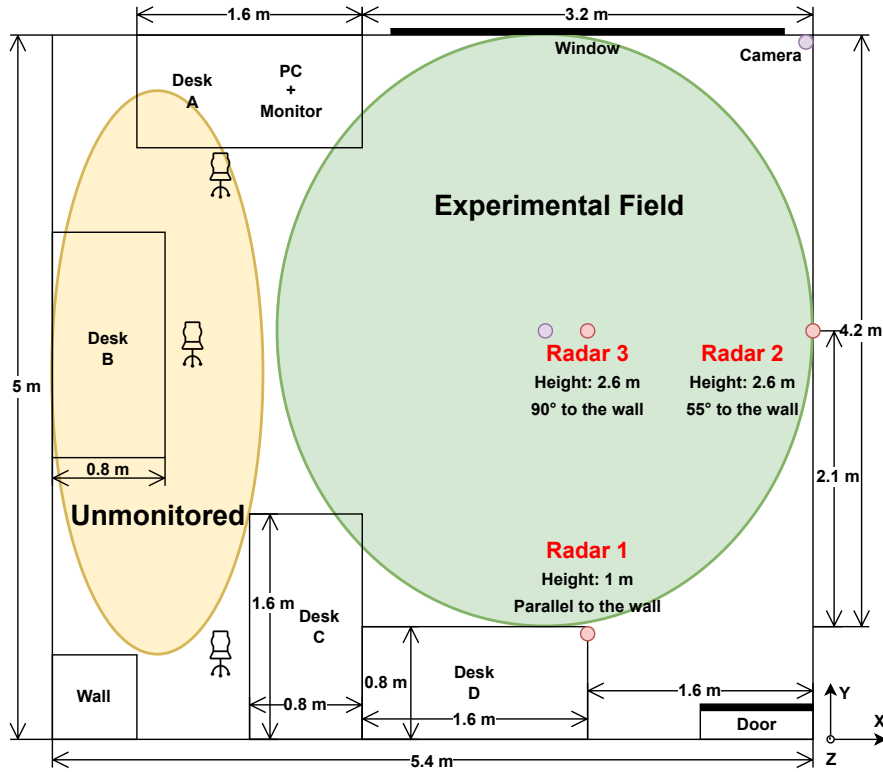


Figure 5.8: The 2D comprehensive layout for the radar room.

The experimental space occupies dimensions of 4 meters in width, 4.2 meters in length, and 2.6 meters in height. Within this space, *Radar 1* is vertically positioned on the desk adjacent to the wall, *Radar 2* is installed on the wall with a 55° tilt, and *Radar 3* is mounted on the ceiling, directed towards the ground, as depicted in Figures 5.8 and 5.9. All three radars are connected to the PC through USB cables and extensions, powered by a 5 V DC power supply. Additionally, two cameras are strategically placed at the upper corner and the ceiling of the room to serve as visual references. Human targets have the potential to enter the monitored area either through the door or the unmonitored region on the left of Figure 5.8.

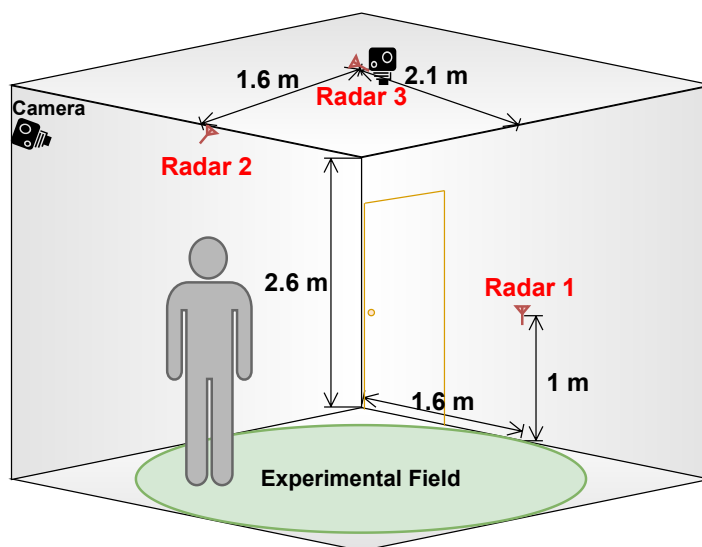


Figure 5.9: The 3D view simulation with measurement details.

5.3.2 Data Collection

Point clouds are collected from 15 randomly selected individuals, resulting in a total duration of approximately 300 minutes (~360,000 frames). Initially, each participant is instructed to walk around the radar room for 10 minutes, constituting 51.5% of the total duration (156 minutes) where only one person is present in the scene. Subsequently, participants form randomly assigned pairs and enter the scene for 10-minute intervals, for a total of 10 rounds, accounting for 33.1% of the total duration (100.4 minutes) where two people are present. Similarly, for scenarios involving three individuals, groups of three enter the scene for around 5 minutes per round, totalling 10 rounds, which constitute 15.4% of the total duration (46.7 minutes). In all scenarios, participants are instructed to sit on a chair or fall to the ground randomly throughout the duration of the experiment. The data collection composition is shown in Table 5.1. Since the data is evaluated target by target, the duration is multiplied by 2 and 3 for two-target and three-target scenarios, respectively, to obtain the total duration for each target.

Table 5.1: Data collection composition in minutes.

Scenario	Duration	Total duration	Walking	Sitting	Fall
1 target	156	156×1	75.4	32.1	48.5
2 targets	100.4	100.4×2	103.3	58.3	39.2
3 targets	46.7	46.7×3	77.3	35.7	27.1
Total	303.1	496.9	256	126.1	114.8

The participants, aged 20-30, vary in height (160-185 cm) and weight (50-80 kg), encompassing both male and female representations. The video footage is only used to verify the system’s performance. This data collection received approval from the Faculty of Engineering Research Ethics Committee at the University of Bristol (ethics approval reference code 17605).

To maintain consistency, all point clouds are represented in the coordinate format (x, y, z, v, snr) , where (x, y, z) denote the coordinates of the point, and (v, snr) signify the velocity and SNR level, respectively. A global coordinate system is established to facilitate the alignment of points gathered from different radars through spatial rotation and positional adjustment. Since radars output point clouds of arbitrary sizes, the resulting dataset is structured as $N \times P \times C$, where N represents the number of samples (frames), P indicates the number of points in each frame (of arbitrary size), and C denotes the number of feature channels ($C = 5$). The elements of this tensor can be represented as $X_{n,p,c}$, where n, p, c correspond to the respective indices (Equation (5.3)).

$$X \in \mathbb{R}^{N \times P \times C} \quad (5.3)$$

5.4 System Design

This system is developed using Python (version 3.8.6) and relies on essential libraries like NumPy, Sklearn, and Matplotlib. It operates on a Desktop PC featuring an Intel(R) Core(TM) i7-10850H CPU @ 2.70 GHz and 16 GB of RAM. The system effectively utilizes point cloud data from radars to implement precise tracking and fall detection. It is worth noting that, thanks to the use of a high-end CPU, the system’s performance bottleneck is primarily attributed to the radar speed due to the complexity of the 3D point cloud data. Therefore, this system is set to run at 20 FPS to match the radar boards.

Importantly, in this real-time system, a configuration file feature is provided. This file allows users to specify parameters, enabling seamless integration of data from additional radars, thereby expanding the system’s potential applications. Moreover, the configuration file is instrumental in post-processing tasks such as data merging, clustering, and human tracking. It ensures a customized and efficient approach to handling the collected data, making the system adaptable to diverse scenarios and requirements.

Operating within a multiprocessor environment, the system can leverage multiple CPU cores to concurrently execute multiple modules such as *Radar_Process* and *Vis_Process* depicted

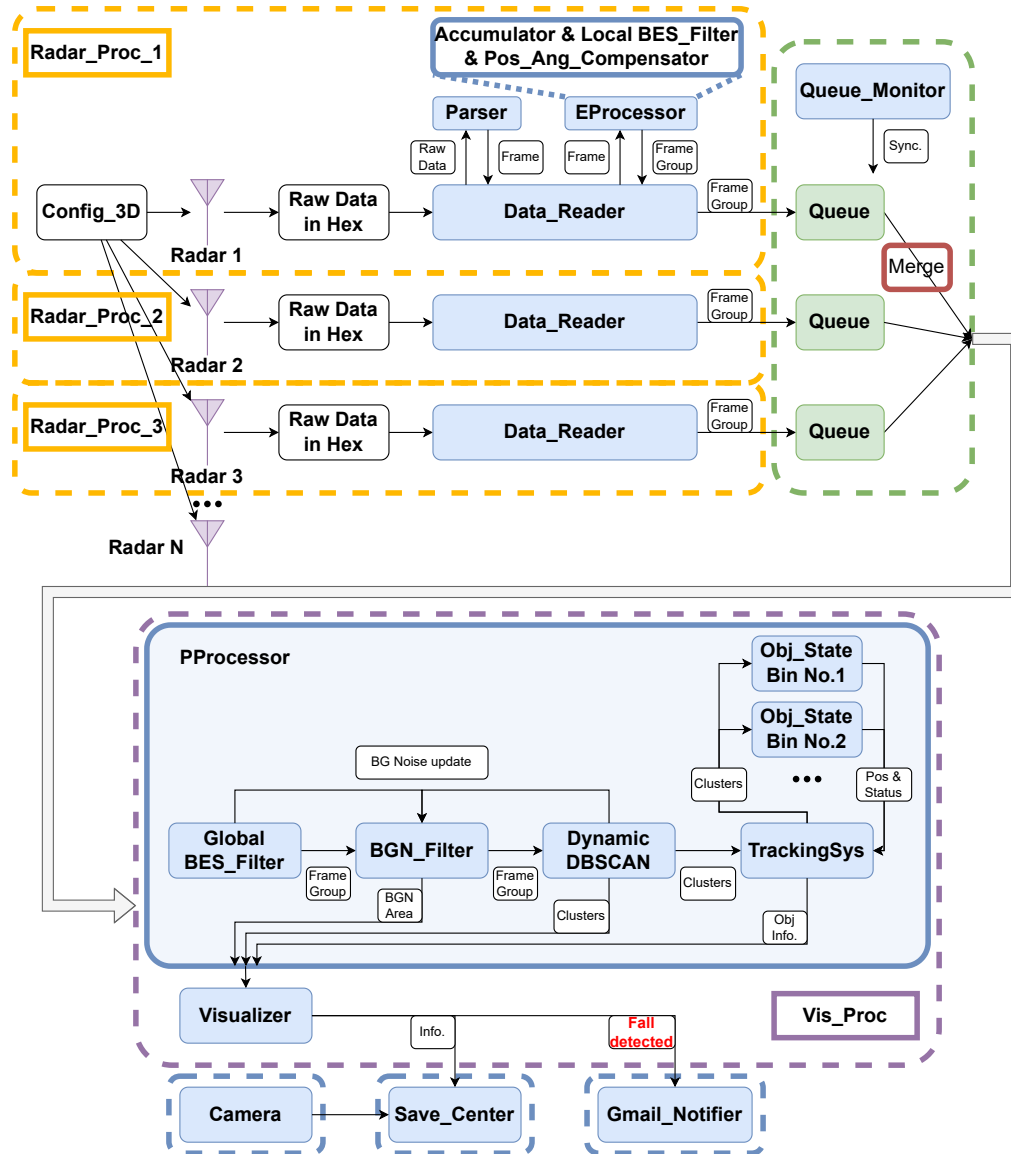


Figure 5.10: The working flowchart of the software system for real-time human tracking and fall detection. The blue boxes represent each module, while the arrows represent data transfer between modules. Each dotted box denotes a concurrent working process.

in Figure 5.10. This concurrent execution optimizes workload distribution, leading to efficient utilization of system resources. Figure 5.10 illustrates the system's architecture, which comprises the following modules:

- *Data_Reader*: Data Readers parse the data packages sent by the radars.
- *EProcessor*: The Early Processor provides data rotation and position compensation based on the radar placement.

- *PProcessor*: The Post Processor provides data filtering, clustering and target tracking.
- *Visualizer*: The Visualizer provides 3D demonstrations for the human tracking and state.
- *Queue_Monitor*: The Queue Monitor monitors the frame traffic and provides synchronization.
- *Camera*: The Camera module provides video footage during the experiment as ground truth.
- *Gmail_Notifier*: The Gmail Notifier provides an email-sending service based on the Gmail API [193] when a human fall is detected.

5.4.1 Radar Raw Data Preprocessing

Prior to data collection using the mmWave radar, it is essential to prepare a configuration file adapted to the TI firmware. This file involves many properties mentioned in Section 3.1.6 and Section 5.3.1 to ensure the reception of sufficient 3D data. As illustrated in Figure 5.10, after sending the *config_3D* file, the three radars collect point cloud data in 3D format, including SNR and velocity information, transmitted via packets through the USB ports.

Upon data reception, the initial step involves parsing the data into a two-dimensional format. The first dimension represents the number of points, while the second dimension includes (x-pos, y-pos, z-pos, velocity, SNR) information for each point. Due to the limited number of data points collected in each frame, 10 frames are aggregated to create a frame group for processing at that moment. A sliding window approach, spanning 10 frames (equivalent to 0.5 seconds) with the step of 1 frame, is employed to shift the frames forward, ensuring continuous processing.

To account for the utilization of multiple radars positioned at varying locations and angles, compensation methods for data point rotation and repositioning are applied.

5.4.1.1 Rotation Compensation

This method is based on equations derived from sources [194–196]. The three 3D rotation Equations (5.4), (5.5), (5.6) are presented below:

$$RM_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & rp_y(1 - \cos \alpha) + rp_z \sin \alpha \\ 0 & \sin \alpha & \cos \alpha & rp_z(1 - \cos \alpha) - rp_y \sin \alpha \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.4)$$

$$RM_y = \begin{bmatrix} \cos \beta & 0 & \sin \beta & rp_x(1 - \cos \beta) - rp_z \sin \beta \\ 0 & 1 & 0 & 0 \\ -\sin \beta & 0 & \cos \beta & rp_z(1 - \cos \beta) + rp_x \sin \beta \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.5)$$

$$RM_z = \begin{bmatrix} \cos \gamma & -\sin \gamma & 0 & rp_x(1 - \cos \gamma) + rp_y \sin \gamma \\ \sin \gamma & \cos \gamma & 0 & rp_y(1 - \cos \gamma) - rp_x \sin \gamma \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5.6)$$

$$P = \begin{bmatrix} P1_x & Pn_x \\ P1_y & \dots & Pn_y \\ P1_z & Pn_z \\ 1 & 1 \end{bmatrix} \quad P' = \begin{bmatrix} P1'_x & Pn'_x \\ P1'_y & \dots & Pn'_y \\ P1'_z & Pn'_z \\ 1 & 1 \end{bmatrix} \quad (5.7)$$

In the equations provided, RM_x , RM_y , and RM_z represent rotation transformation matrices along the x, y, and z axes, respectively. P and P' (Equation (5.7)) denote the point matrices before and after transformation. The rotation angles for the x, y, and z axes, denoted by α , β , and γ respectively, are determined by the facing angle of the radars. The reference point coordinates in 3D, represented as rp_x , rp_y , and rp_z , are set to (0,0,0), indicating the origin in the experiment. Using Equation (5.8), the data points obtained from radars can be transformed and mapped into a unified global coordinate system. It is important to mention that the order of applying RM_x , RM_y , and RM_z in Equation (5.8) dictates the order of rotation axes, leading to different outcomes based on the chosen sequence.

$$P' = RM_x RM_y RM_z P \quad (5.8)$$

5.4.1.2 Position Compensation

For the position compensation, the Equation (5.9) is applied to correct the radar position offset in the global coordinate system.

$$P' = \begin{bmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & \Delta y \\ 0 & 0 & 1 & \Delta z \\ 0 & 0 & 0 & 1 \end{bmatrix} P \quad (5.9)$$

After applying point repositioning and rotation algorithms based on the radars' positions and facing directions, the resulting frame groups are then placed into *Queues*, where they await further processing and analysis.

5.4.2 Multiple Radar Data Lines Synchronization

Data synchronization becomes a challenge when attempting to communicate simultaneously with multiple devices. In the experimental setup, which incorporates three IWR1843 radars, the firmware flashed in these radar boards enables them to transmit data packets continuously at a

predetermined rate, as configured in Section 5.3.1, upon initiation and until a reset command is issued. However, a synchronization problem arises due to the occurrence of error packets, which are non-usable but have the potential to disrupt the timing sequence, sent by the radars. To be more specific, all raw data sent by TI mmWave radar comprising coordinates and velocity information is well packaged in frames and follows the prescribed TI output packet structure. Occasionally, the hardware may send incomplete packets or corrupted packets that cannot be parsed into usable data for the following process. Therefore, these error packets highlighted in red as shown in Figure 5.11 will be discarded by the *Data_Reader* upon reception. During the identical time frame, *Radar 3* transmits 10 viable packets, all of which are captured by *Data_Reader*. However, for *Radar 1* and *Radar 2*, the count of successfully retrieved packets by *Data_Reader* diminishes to 8 and 9, respectively.

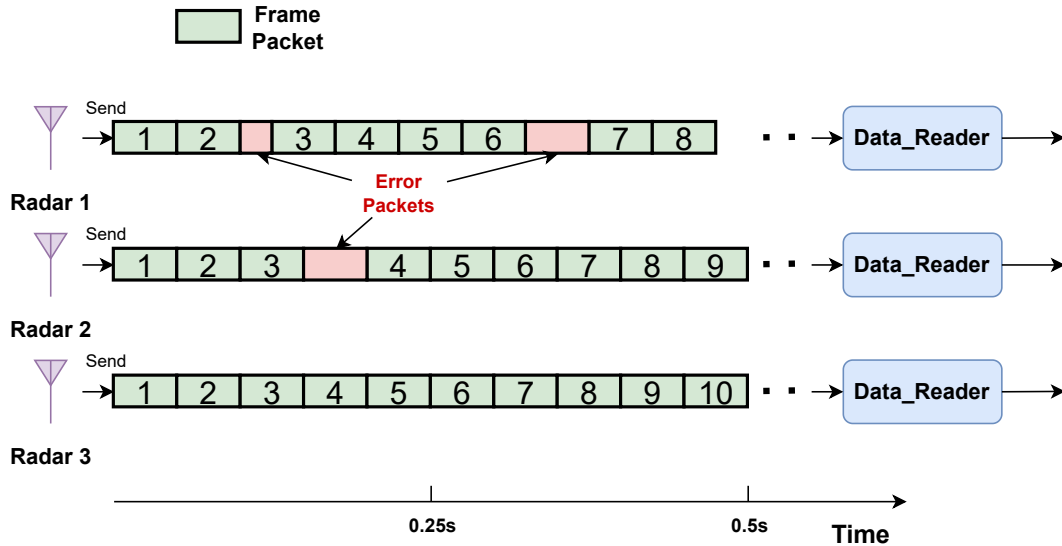


Figure 5.11: The demonstration of the synchronization challenge over time. Error packets highlighted in red are incomplete packets or corrupted packets which are discarded by *Data_Reader*. The system is expected to work at 20 FPS.

Normally, the *PProcessor* in Figure 5.10 systematically retrieves one packet from each *Queue* sequentially to ensure sufficient data. However, if errors occur causing packet loss in one or two radars, the *Queue* associated with the radar(s) unaffected by errors will start accumulating packets. Because *PProcessor* will wait for the next good packet from the radar line with errors, while the radar line without errors continues to send good packets, causing a disparity in the number of packets in this error-free *Queue* compared to the other *Queues*. While this accumulation is manageable for small numbers of packets, it becomes problematic when it extends over several hours or even longer, resulting in hundreds of accumulated packets and causing significant delays for that radar. To elaborate further, the use of a First-In-First-Out (FIFO) queue strategy means that when the *PProcessor* extracts the packets, the frame packet it receives from the radar with

an accumulation issue is from seconds ago. This approach results in the emergence of a ghost shadow issue, significantly impacting the subsequent human detection process.

Two approaches can address this issue. The first method (Section 5.4.2.1) involves discarding accumulated packets if the disparity exceeds a predefined threshold. Although straightforward to implement and computationally inexpensive, this method may result in packet loss (information loss). The second approach (Section 5.4.2.2) utilizes timestamps to label each packet, allowing for their retrieval and merging if they fall within the same time interval. While this method can prevent information loss, it requires more computational resources.

5.4.2.1 Synchronization via Accumulation Removal

To synchronize data lines of the radars, a *Queue_Monitor* module is designed to oversee the packet accumulation within the *Queues*. A criterion is established: if the disparity in packet numbers between each *Queue* is limited to one packet (0.05 seconds under 20 FPS), it signifies that the radar data lines are synchronized. However, if this difference exceeds one packet, indicating asynchronicity, the *Queue_Monitor* module intervenes by dropping the surplus packet before the *PProcessor* extracts them as displayed in Figure 5.12. In this scenario, the *PProcessor* retrieves the latest packets from each radar to achieve data synchronization and merges them for further processing.

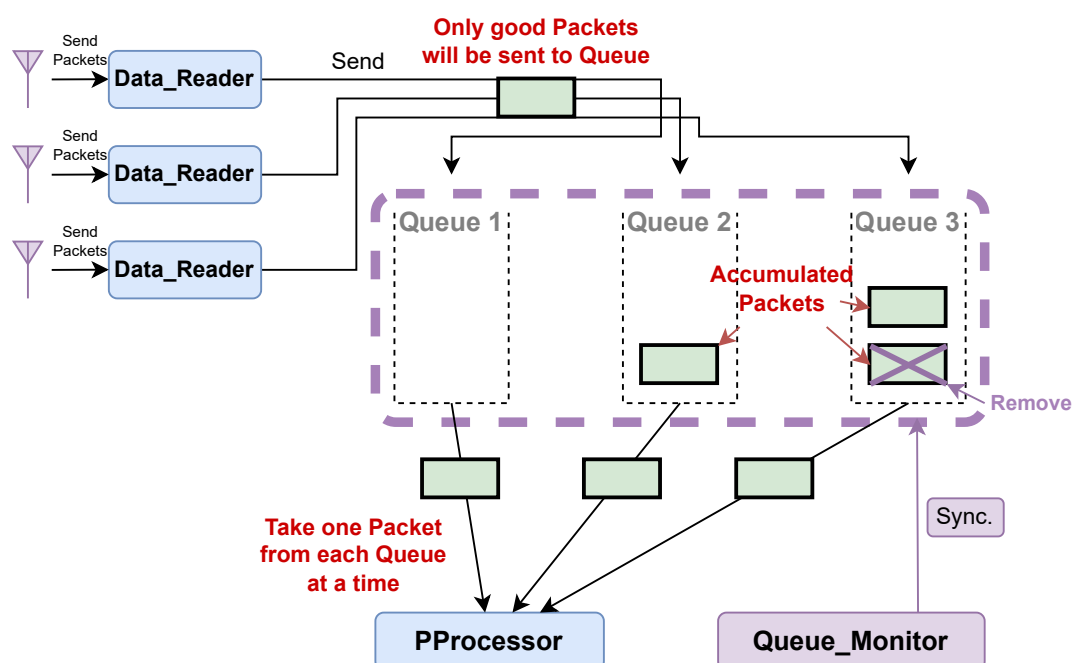


Figure 5.12: The workflow for resolving the synchronization issue through accumulation removal. The *Queue_Monitor* remove the accumulated packets to ensure the synchronization between these *Queues*.

This method effectively mitigates the ghost shadow issue and is easy to implement. However, it does lead to packet loss, causing missed frames within the system. While some information loss is inevitable with this approach, it remains acceptable for the intended application. To assess the extent of information loss, tests are conducted by continuously running the system for 2 hours (144,000 frames) as one trial, with a total of 10 trials. The average number of dropped packets across these trials is approximately 431 as shown in Figure 5.13, resulting in 1 dropped accumulated frame packet for every 334 frames (approximately 17 seconds). Considering that each recording sessions typically last around 2 minutes, this equates to only 7 dropped frames per recording session, representing a drop rate of 0.29%, which is deemed acceptable.

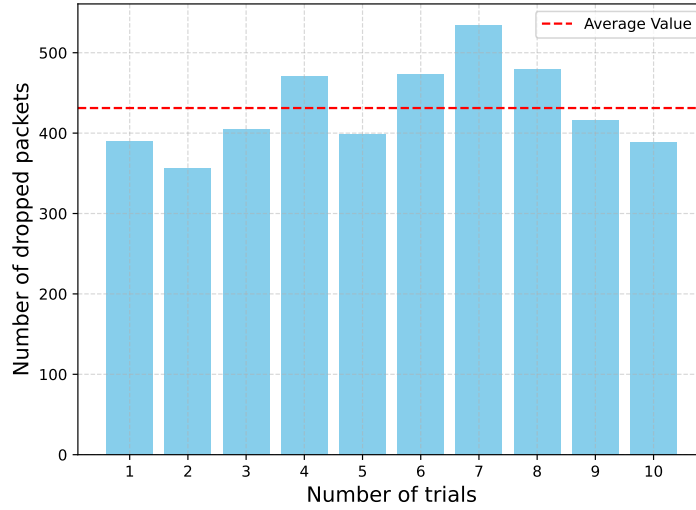


Figure 5.13: The histogram for the number of dropped packets over 2 hours across 10 trials.

5.4.2.2 Synchronization via Timestamp

To prevent information loss, a second method involving timestamps for all frame packets is implemented. A criterion is established: frame packets received from the three radars within a 0.05-second window (equivalent to 20 FPS) are considered synchronized. Conversely, if two packets are separated by an interval greater than 0.05 seconds, they are treated as asynchronous packets. The synchronized packets are merged before being sent to the *PProcessor*. Subsequent packets are aggregated within the next 0.05-second time period, as illustrated in Figure 5.14. In certain 0.05-second intervals, there may be only 2 frame packets due to the error packets, which is deemed acceptable in this setup.

This approach prevents packet disparity accumulation and effectively mitigates the ghost shadow problem. Integrating timestamps for each packet avoids the need for packet removal, but it comes at the cost of higher computational requirements.

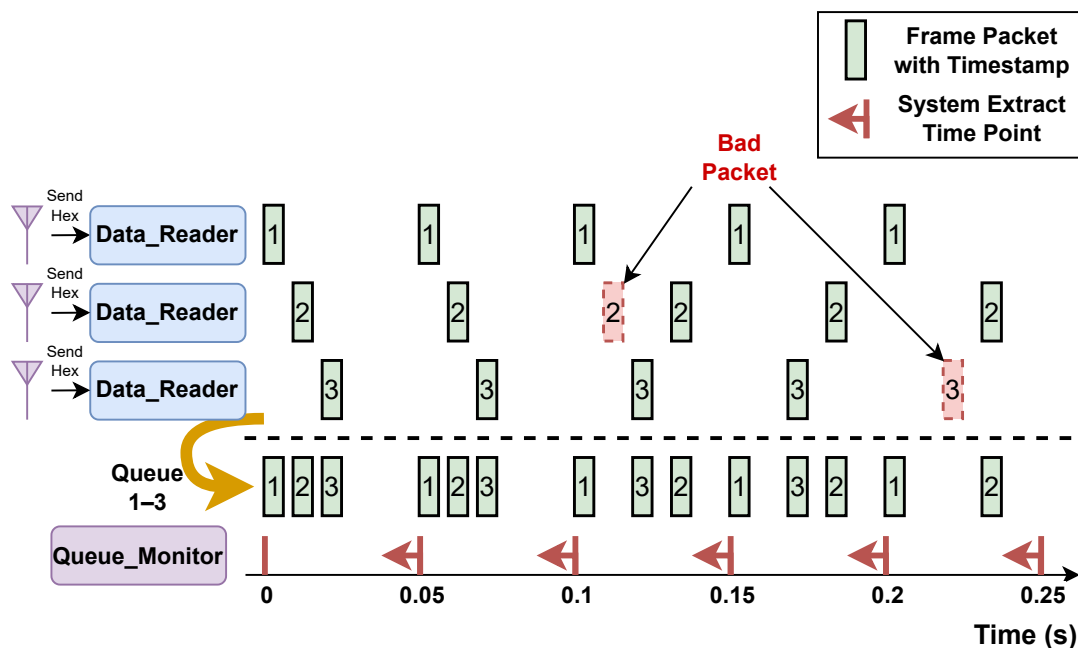


Figure 5.14: The workflow for resolving the synchronization issue through using timestamps. All packets are marked with timestamps by *Data_Reader* before sending to *Queues*. Error packets highlighted in red are discarded. *Queue_Monitor* merges all packets for each time interval.

5.4.2.3 First-In-First-Out Strategy

In simple terms, the FIFO strategy prioritizes processing the earliest data in a data queue. In this experiment, the implementation of the FIFO queue strategy enhances the system's ability to process data in real time. In situations where there is data congestion within the *Queues*, it ensures that *PProcessor* retrieves the earliest data according to the established timeline. This approach guarantees sequential processing, even in worst-case scenarios, thereby ensuring the system's responsiveness.

5.4.3 Background Noise Reduction

After data retrieval from each radar by the *PProcessor* in Figure 5.10, the data points with low SNR are identified and stored by the *Global BES_Filter* module for background recognition. Additionally, noise points identified by *Dynamic DBSCAN* module are also collected for background recognition in the subsequent process. Using data from both the *Global BES_Filter* and *Dynamic DBSCAN*, the *BGN_Filter* module tracks noise points by generating noise cube areas with corresponding counters. If noise points in the following frames fall within these areas, indicating persistent noise, the counter increments by one, as outlined in Algorithm 3. If the counter exceeds a certain threshold of 100 frames, for example, indicating the noise has persisted for over 5 seconds, it will be filtered out in future frames. This module effectively discerns background

noise and isolates areas where data points marked as noise persist for an extended duration. This is especially applicable to cases where static targets such as chairs and tables are present in the field. This feedback loop approach for noise reduction results in a clean frame comprising data points with reduced noise being generated and utilized for DBSCAN clustering.

Algorithm 3 Background Noise Filter Algorithm

Input: Noise point set P , Threshold T_{frame}

Output: Updated filter areas A

```

1: Initialize an empty list for filter areas  $A = \emptyset$  when system starts
2: Initialize an empty list for potential cubes  $C = \emptyset$  when system starts
3: Initialize an empty list for cube counters  $Counter = \emptyset$  when system starts
4: for each cube  $c$  in  $C$  do                                     ▷ For existing cubes
5:   for each point  $p = (x_p, y_p, z_p)$  in  $P$  do
6:     if  $p$  locates in cube  $c$  then
7:       remove  $p$  from  $P$                                        ▷ Remove this point if it locates in any cube
8:       if  $Counter[c]$  has not been added then
9:          $Counter[c] = Counter[c] + 1$                          ▷ Counter for that cube + 1
10:      end if
11:    end if
12:  end for
13:  if  $Counter[c] > T_{\text{frame}}$  then                               ▷ If counter exceeds the frame threshold
14:     $A.append(c)$                                              ▷ Update the cube to filter area list  $A$ 
15:  end if
16: end for
17: for each point  $p$  in rest point set  $P$  do
18:    $c = \text{Cube\_generator}(p)$                                ▷ Generate an expanded cube area for each point
19:    $counter = 1$                                              ▷ set counter for this cube as 1
20:    $C.append(c)$                                              ▷ Update cube to  $C$ 
21:    $Counter.append(counter)$                                ▷ Update counter to  $Counter$ 
22: end for

```

5.4.4 Human Target Detection

In the process of identifying clusters representing human beings instead of the noise, the *Dynamic DBSCAN* module is employed for the DBSCAN algorithm. DBSCAN stands out due to its ability to function without the need for specifying the number of clusters in advance, making it particularly suitable for situations where the exact number of human targets is unknown [181]. This algorithm groups closely packed data points into dense regions, differentiating them from sparser areas. Such an approach effectively addresses scenarios where dynamic targets are encountered in the experiment.

However, it is observed that when a human target is stable, fewer data points with a high SNR level are collected due to the mmWave radar's limited sensitivity to stationary targets, a finding supported by Cui and Dahnoun [19]. Because stationary targets cannot be effectively

distinguished from background noise in the RDM [197], and consequently, they can possibly be treated as noise and removed from the analysis. Although the challenge of distinguishing stationary targets, an inherent limitation of mmWave radar, cannot be directly overcome, accuracy is improved by employing a novel dynamic DBSCAN approach. This multi-level clustering technique first categorizes data points based on their SNR values.

As outlined in Algorithm 4 and Figure 5.15, more lenient DBSCAN parameters are applied to points with higher SNR, while stricter parameters are used for lower SNR points. When clustering low SNR points, high SNR points are also included by concatenating them into the process.

Algorithm 4 Dynamic DBSCAN Algorithm

Input: Point cloud for current frame (x, y, z, v, snr) P , DBSCAN parameters D

Output: Cluster set C

```

1: Define a list for SNR level  $L = [0, 100, 200, 300, 400]$ 
2: Initialize an integer for the number of points  $Pre\_No = 0$ , reset every frame
3: for each level  $l$  in  $L$  do                                     ▷ Iterate each SNR level
4:    $Level\_points = \text{Filter}(P, \text{dim} = \text{snr}, \text{range} = (l, \text{Inf}))$   ▷ Filter points in SNR dimension
5:    $Curr\_No = \text{len}(Level\_points)$                                 ▷ Get number of points
6:   if  $Curr\_No > Pre\_No$  then                                    ▷ If new points found in current level
7:      $Pre\_No = Curr\_No$                                          ▷ Update the number
8:      $cluster = \text{DBSCAN}(Level\_points, D[l])$                   ▷ Do the DBSCAN for points of this level
9:      $C.append(cluster)$                                          ▷ Add the cluster to set  $C$ 
10:  end if
11: end for
    
```

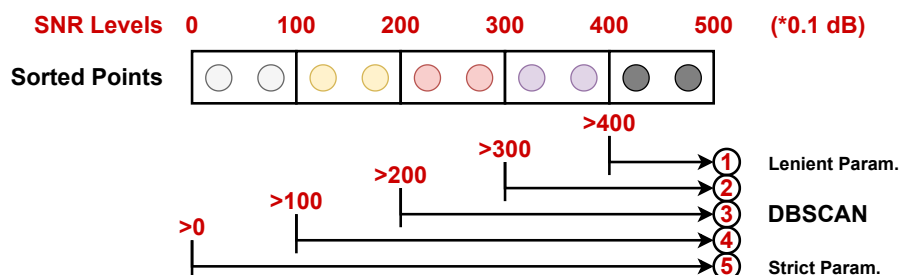


Figure 5.15: The proposed dynamic DBSCAN strategy based on the signal SNR level (unit 0.1dB) performs multiple DBSCAN clustering for each data frame.

Rather than inputting all data points directly into the DBSCAN algorithm, the points are first categorized based on their SNR values. Subsequently, as shown in Algorithm 4 and Figure 5.15, more lenient DBSCAN parameters are applied to points with higher SNR and stricter parameters to the low SNR points. When clustering the points with lower SNR, higher SNR points are also incorporated by concatenating them together. The primary advantage of this approach is the prevention of missing valuable data points with high SNR levels, which are

highly likely to represent human targets instead of noise, especially when a stationary target is present.

For instance, if all points are treated equally and apply default parameter settings of $\epsilon = 0.5$ and $minPts = 10$, where ϵ denotes point distance and $minPts$ is the minimum number of points used by DBSCAN to define a cluster, the cluster will not form due to an insufficient number of points, as shown in the left image of Figure 5.16. In this case, the high-energy points representing the human chest, which should be identified as a cluster, are missed. To not miss those points, as an example, more lenient parameters are employed: $\epsilon = 1$ and $minPts = 2$ for points with higher SNR levels above 300, and $\epsilon = 0.7$ and $minPts = 3$ for points with SNR levels above 200. These settings enable the successful formation of a cluster that represents a stationary human target (see the right side of Figure 5.16). It is worth noting that these parameter settings are selected based on the point cloud density observed in the experiments. These settings can be adjusted if a different number of radars are used or for rooms of varying sizes, as the radars will produce point clouds with different point densities.

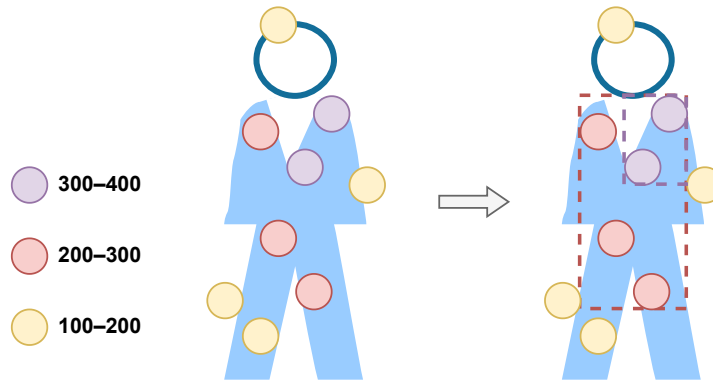


Figure 5.16: The demonstration of the dynamic DBSCAN method. On the left side, the DBSCAN algorithm is implemented using default parameters, whereas on the right, the dynamic DBSCAN strategy is applied, with dashed lines indicating the clusters.

Employing dynamic DBSCAN necessitates five times the computation for each frame clustering in the worst-case scenario, where data points in a single frame span all predefined SNR regions in Figure 5.15. However, such a scenario is infrequent. Typically, most data points fall within a single SNR region when no human presence is detected, and only two to three regions are occupied when human targets are present. To optimize performance, if an SNR region contains no data points, the corresponding DBSCAN process is skipped, as shown in Algorithm 4. This strategy aims to conserve computational resources and enhance processing speed by avoiding unnecessary computations.

5.4.5 Human Target Tracking

To achieve real-time tracking of the position, state, and moving trajectory of detected human targets, the *Obj_State Bin* modules are designed specifically to store this information. With multiple clusters generated by the *Dynamic DBSCAN* module, the issue lies in accurately assessing these clusters and assigning them to the corresponding *Obj_State Bin* modules representing human targets for continuous tracking.

To allocate clusters precisely to the right *Obj_State Bin*, this section introduces the *TrackingSys* module, outlined in Figure 5.10. This module serves the crucial role of determining which clusters from the *Dynamic DBSCAN* module should be allocated to which *Obj_State Bin* modules based on the probability matrix. The strategy of a probability matrix is generated by evaluating the correlation between each potential cluster and the previous information stored in *Obj_State Bin* modules, i.e., previous cluster position and shape. The elements P_{ele} of the probability matrix are determined using the following equation, comprising four components:

$$P_{ele} = aC_{pos} + bC_{shape} + cE_{pos} + dE_{shape} \quad (5.10)$$

In this equation, C_{pos} represents the correlation factor between the position of the potential cluster and the previously stored positions in the *Obj_State Bin* modules, while C_{shape} indicates the correlation for the cluster shape. Specifically, Z-Score is applied to evaluate those relationships and identify outliers that should be disregarded. As demonstrated in Algorithm 5, E_{pos} and E_{shape} denote the position and shape difference between the potential cluster and the predefined expected values. For instance, clusters located at the ceiling are not expected to be identified as

Algorithm 5 Cluster Probability Algorithm

Input: Cluster central point $cp(x, y, z)$, Size $si(x, y, z)$, Expected position cp_exp , shape si_exp

Output: Probability P_{ele}

- 1: Define a history list for the central point $CP = \emptyset$ when system starts
 - 2: Define a history list for the size $SI = \emptyset$ when system starts
 - 3: Define proportion coefficients $CO(a, b, c, d)$
 - 4: **if** $\text{len}(CP) = 0$ **then** ▷ If no historical central point and size
 - 5: $C_{pos}, C_{shape} = 0$
 - 6: **else if** $\text{len}(CP) < 10$ **then** ▷ If the number of historical data is insufficient for z-score
 - 7: $C_{pos} = \exp(-\text{abs}(\text{hypot}(cp, CP[-1]))) \times \text{get_speed}(cp)$ ▷ Calculate component C_{pos}
 - 8: $C_{shape} = \exp(-\text{abs}(\text{diff}(si, SI[-1])))$ ▷ Calculate component C_{shape}
 - 9: **else**
 - 10: $C_{pos} = \exp(-\text{abs}(\text{z_score}(cp, CP))) \times \text{get_speed}(cp)$ ▷ Calculate component C_{pos}
 - 11: $C_{shape} = \exp(-\text{abs}(\text{z_score}(si, SI)))$ ▷ Calculate component C_{shape}
 - 12: **end if**
 - 13: $E_{pos} = \exp(-\text{hypot}(cp, cp_exp))$ ▷ Calculate Euclidean distance for component E_{pos}
 - 14: $E_{shape} = \exp(-\text{diff}(si, si_exp))$ ▷ Calculate component E_{shape}
 - 15: $P_{ele} = \text{sum}((C_{pos}, C_{shape}, E_{pos}, E_{shape}) \times CO)$ ▷ Calculate the total probability P_{ele}
-

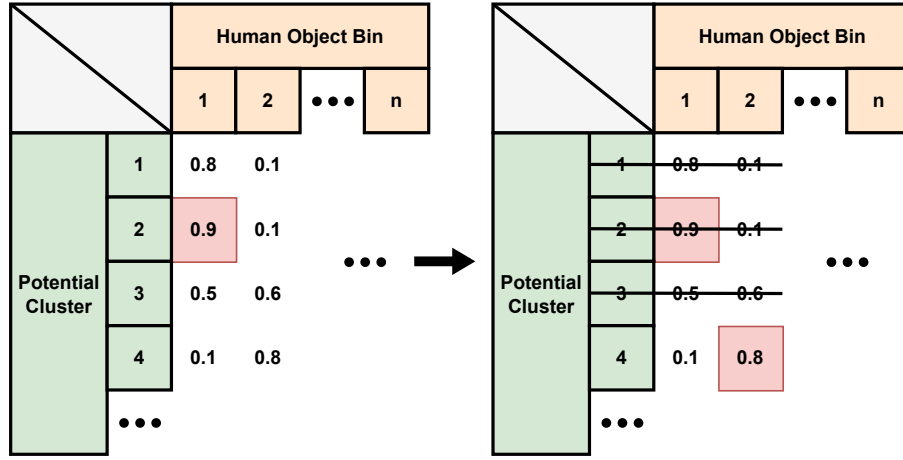


Figure 5.17: The probability matrix for *TrackingSys* module. According to the global maximum value selected from the probability matrix, cluster 2 is updated to person 1. Then cluster 2 and its neighbors 1 and 3 are removed, while cluster 4 is allocated to person 2 as shown on the right.

human targets. Additionally, proportion coefficients $[0.3, 0.3, 0.2, 0.2]$ are used in the experiment to balance these four components and calculate a more accurate P_{ele} . These coefficients can be adjusted to adapt the system to various deployment environments, providing flexibility and accuracy in the tracking process.

Algorithm 6 Probability Matrix Algorithm

Input: Cluster set C , People bins B

Output: Updated people bins B

```

1: Define a probability matrix  $M$ , reset to zeros every frame.
2:  $M = \text{zeros}(\text{len}(C), \text{len}(B))$  ▷ Initialize the matrix  $M$  with size of (C, B)
3: for each cluster  $c$  in  $C$  do ▷ Iterate each element
4:   for each bin  $b$  in  $B$  do
5:      $M[c, b] = b.\text{Cal\_prob}(c)$  ▷ Calculate the probability for the element
6:   end for
7: end for
8: while  $\text{len}(M) > 0$  and  $\text{max}(M) > 0$  do ▷ Iterate each element
9:    $c_{idx}, b_{idx} = \text{max\_index}(M)$  ▷ Get the index of the maximum element
10:   $B[b_{idx}].\text{update}(C[c_{idx}])$  ▷ Update the cluster to the bin
11:   $c_{used} = C[c_{idx}]$ 
12:  for each cluster  $c$  in  $C$  do
13:    if  $c.\text{check\_neighbor}(c_{used})$  then ▷ Determine whether the neighbor cluster
14:       $i = \text{get\_index}(c)$ 
15:       $M[i, :] = 0$  ▷ Remove the neighbour cluster row by setting to 0
16:    end if
17:  end for
18:   $M[:, b_{idx}] = 0$  ▷ Remove the bin column by setting to 0
19: end while
  
```

After applying Equation (5.10) and Algorithm 5 to each current cluster for every *Obj_State Bin* module, the *TrackingSys* module generates the probability matrix. Figure 5.17 demonstrates an example of a two-people scenario. The probabilities between each potential cluster and object bins are produced. The module utilizes the global maximum probability, highlighted in red in Figure 5.17, to update the cluster to the corresponding object bin. Subsequently, this cluster and its neighbouring clusters are regarded as the same person and removed from consideration. The process is then repeated as shown in Algorithm 6: the next maximum value from the remaining possibility elements is selected and allocated, continuing until there are no non-zero values left in the probability matrix.

Ultimately, the *Obj_State Bin* modules provide human tracking and state information to the *Visualizer*, facilitating the display of human target positions and shapes based on historical cluster data. This method ensures accurate and real-time tracking of human targets within the environment.

5.4.6 Fall Detection

This study, which lies solely on fall detection for individuals who require medical surveillance at home or hospital, excludes other postures considered in [43, 44], simplifying the task. This objective eliminates the necessity for neural networks to solve the problem. By avoiding computationally expensive algorithms like neural networks, which demand additional GPU support and entail much higher computation costs and power consumption, this system becomes easily deployable on edge devices and low-power consumption platforms with embedded processors in the future for IoT applications.

To determine the current state (walking, sitting, or lying on the ground) of a human target, the estimated portraits of position and shape for these three states are predefined. For example, if the target is walking, the centre height of the cluster representing this target is estimated at around 1 meter. Additionally, the cluster shape is modelled as a rectangle cuboid with its long side aligned with the z-axis. When the target is sitting, the centre height is estimated to be around 0.6 meters. If the target is lying on the ground, the centre height is approximately 0.2 meters, close to the ground level. In this case, the cluster shape is a rectangle cuboid with its short side on the z-axis. These metrics are decided based on the statistical analysis of the data. To calculate the state probability P_{state} , Equation (5.10) is updated to Equation (5.11), as shown below:

$$P_{state} = \alpha E_{pos} + \beta E_{shape} \quad (5.11)$$

E_{pos} and E_{shape} represent the position and shape difference between the cluster and the predefined portraits. To balance these factors, proportion coefficients [0.7, 0.3] are applied in this experiment to observe the best performance. The system determines human states primarily by analyzing the current height of the human targets, with shape analysis serving as a complemen-

tary method to improve performance. Following this calculation, the state probability for each cluster assigned to the *Obj_State Bin* in Section 5.4.5 is computed. The cluster is then labelled with the state having the highest probability.

To enhance the stability of the determined target state, a blurring process is employed. This process prevents the current state from being updated to a new state if there are not enough clusters indicating the new state. As illustrated in Figure 5.18, a sliding window of a specific length (20 frames in this experiment) is applied along the stored clusters in each *Obj_State Bin*. The target state is determined based on which state has the largest number of clusters within the sliding window.

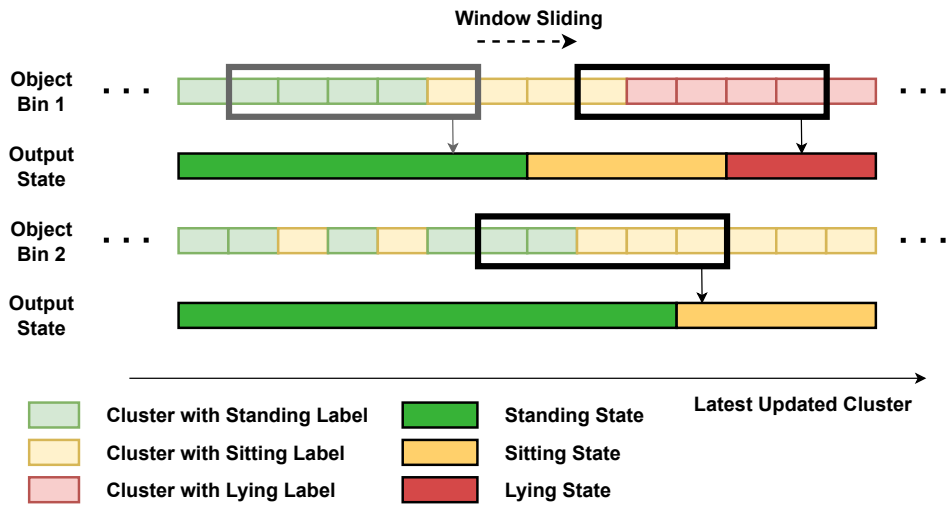


Figure 5.18: The target state blurring process with a sliding window of a certain length (5 clusters in this figure).

Finally, a notification module named *Gmail_Notifier* is developed based on the Gmail API [193]. This module is intended to send alerts in case of a detected human fall, as shown in Figure 5.10.

5.4.7 Adaptive Computing

Continuing to perform computations for human target detection when no one is present in the field results in unnecessary resource consumption and prolonged device heating for a long time. To avoid this, a dynamic FPS adjustment solution is implemented as shown in Figure 5.19. In "Working Mode", both the host system FPS and radars FPS are set to 20, but when *TrackingSys* does not detect any human targets for a period of time (e.g., 60 seconds), the host system switches to "Standby Mode" with a reduced FPS of 1. As resetting the FPS for the radars requires retransmitting the configuration file which is time-consuming (i.e. around 10 seconds), the host system minimises computations by skipping 19 out of every 20 frames on the host. Meanwhile, to conserve resources, less critical modules like *Camera* are temporarily turned off in "Standby

Mode". When *TrackingSys* detects a human target, the host system will revert to "Working Mode" with a full FPS speed.

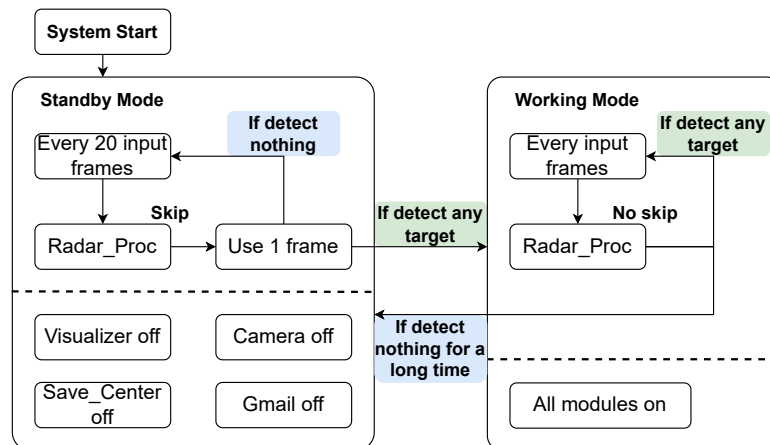


Figure 5.19: The flowchart of adaptive computing.

5.5 System Evaluation

To evaluate the real-time system's performance, frames with a total duration of 300 minutes are analyzed (details provided in Section 5.3.2). Meanwhile, video recordings are taken as ground truth by positioning two cameras at the top corner and ceiling of the experimental area. The YOLO-v3 model [198, 199] is employed to generate ground truth data, which serves as a baseline for human detection assessment.

5.5.1 Multiple Human Tracking Evaluation

The Intersection over Union (IoU) is used as the evaluation metric, measuring the ratio of the overlap area between the predicted and ground truth regions to their union area [200]. A successful detection is defined as having the centroids of the human target identified by both the system and camera detection within 0.25 meters of each other, or with a minimum overlapping frame area of 70%, as detailed in [19]. The following metrics are employed for the evaluation of the human tracking system:

- *Positives (P)*: Human targets are present in the experimental field.
- *True Positives (TP)*: Human targets are present and all identified by the radar, with their positions verified by the camera.
- *False Positives (FP)*: Human targets are absent and identified by the radar caused by noise or other objects, or positions are not verified by the camera.

- *Sensitivity* (TP/P): The ability to identify human targets with valid positions when they are present in the detection area.
- *Precision* ($TP/(TP+FP)$): The ability to identify human targets with valid positions from false detection caused by noise.
- *F1 Score* ($2*Sen*Pre/(Sen+Pre)$): The harmonic mean of *Sensitivity* and *Precision*, demonstrating the system's overall performance.

Table 5.2: Human tracking performance of the real-time system.

	Sensitivity	Precision	F1 Score
For 1 target	97.8%	98.9%	98.4%
For 2 targets	98.2%	96.5%	97.3%
For 3 targets	97.9%	94.0%	95.9%

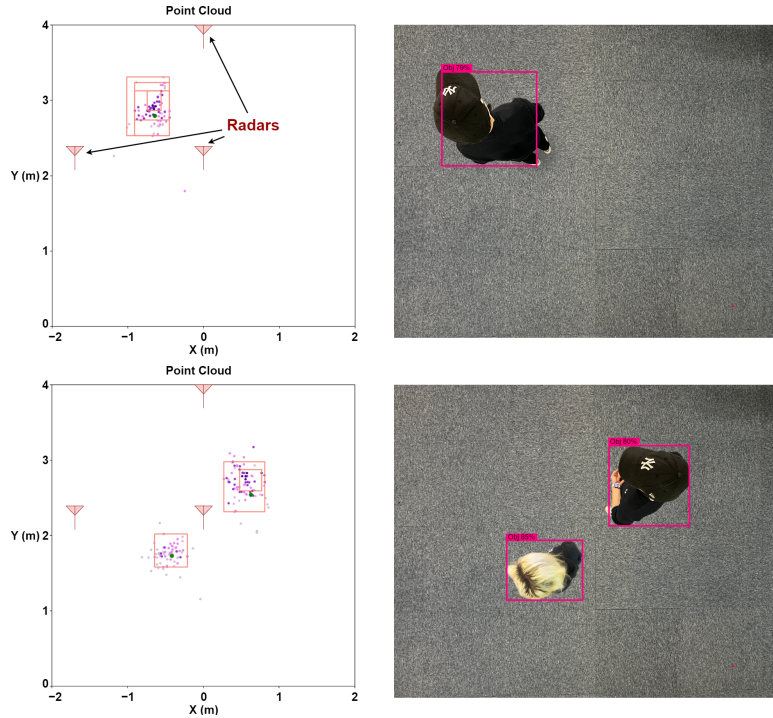


Figure 5.20: Two scenarios in the field are presented, each accompanied by the top view of the point cloud (left) and the real image from the top camera (right).

Leveraging three radars in the field and the mentioned techniques, approximately 98% sensitivity and 98.9% precision are achieved for scenarios with a single target in the field as shown in Table 5.2. This implies that the system can effectively track a human target when it appears and exhibits a strong ability to distinguish it from noise. For scenarios involving two and three people in the field, the sensitivity remains high, indicating that this system can easily

detect the presence of targets. However, the predicted positions may not be as accurate when multiple targets are present. The precision experiences slight drops of 2.4% and 4.9% for the two-person and three-person scenarios, respectively. This decline is attributed to the increased presence of moving people in the field, leading to more noise and false detections at incorrect positions. The average F1 score of the tracking system is 97.2%.

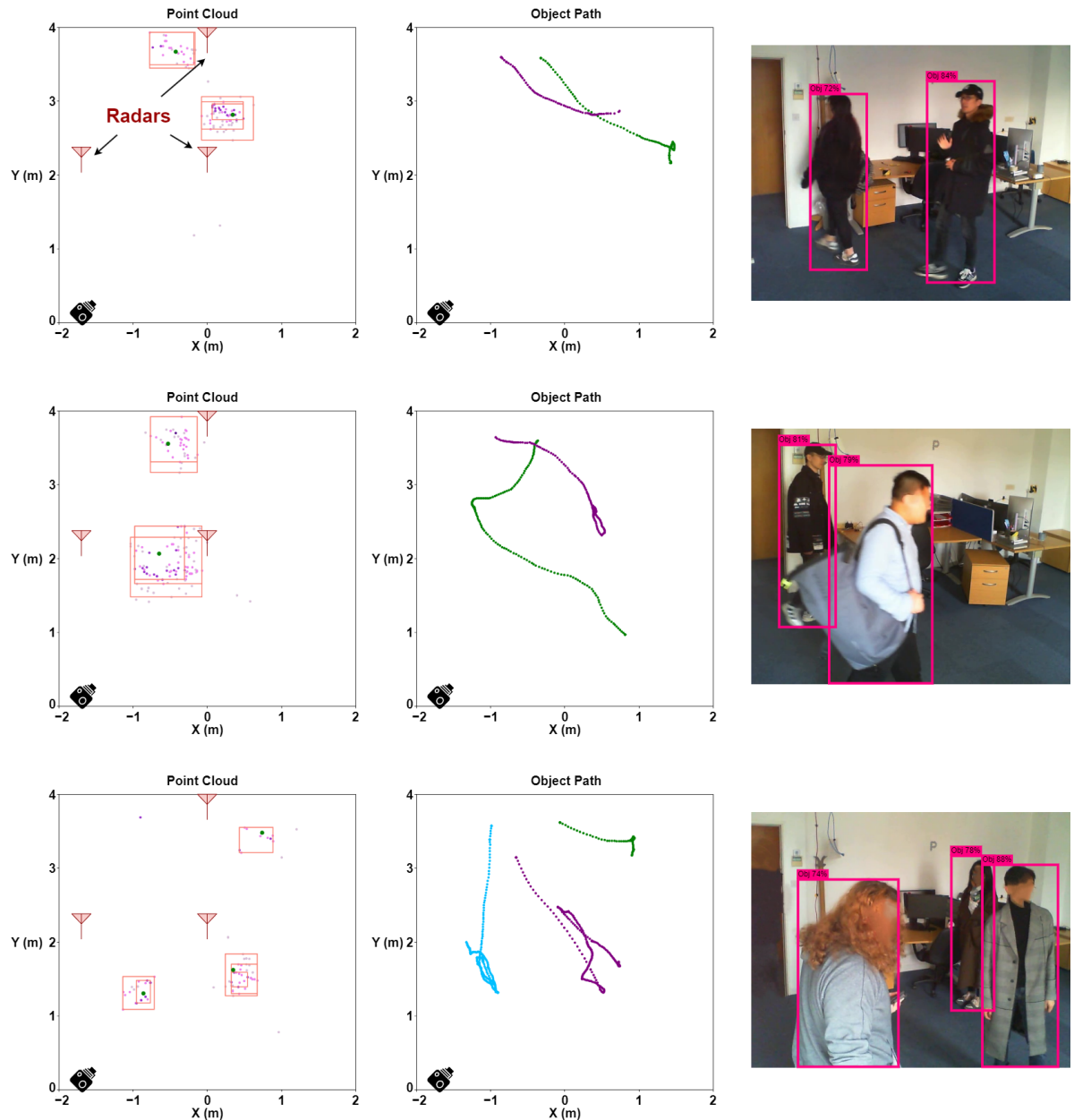


Figure 5.21: Three examples involving multiple individuals in the field are presented, each accompanied by the top view of the point cloud (left), the target trajectory of the whole period (middle), and the real image from the camera (right).

Figure 5.20 and Figure 5.21 display examples featuring one or more mobile human targets in the given environment. In Figure 5.20 on the left column, the top views of the point cloud are presented. Three radars are marked using red radar sign, while the dots with varying shades represent the point cloud with different SNR levels. The DBSCAN clusters are depicted using red rectangles. The corresponding ground truth, captured by the top camera, is displayed in the images in the right column. Figure 5.21 depicts the top views of the point cloud in the left column. The middle column displays the trajectories of multiple human targets' movements within the field, annotated in green, purple, and sky blue, respectively. On the right column, the ground truth captured by the corner camera is presented.

A limitation of the radar system is distinguishing multiple people at short distances, particularly when individuals are standing close to each other. This limitation is less pronounced under typical daily conditions when individuals are usually separated. While Cui and Dahnoun [19] have achieved success at distances of 1 meter, the proposed system demonstrates a distinguishability of 0.5 meters for moving targets and 0.3 meters for stationary targets, thanks to the radar setup and techniques employed.

5.5.2 Human Fall Detection Evaluation

This section evaluates the human states using the data classified as True Positives (TP) in the last Section 5.5.1. For this evaluation, targets are assessed individually in scenarios where multiple targets are present in the room. Figure 5.22 illustrates the confusion matrix for the three classified states: walking (or standing), sitting, and fall (or lying on the ground). The system achieves a high sensitivity level of 99.0% for fall detection and 97.7% for walking. Although the sensitivity for sitting is somewhat lower at 92.6%, it is important to note that most false

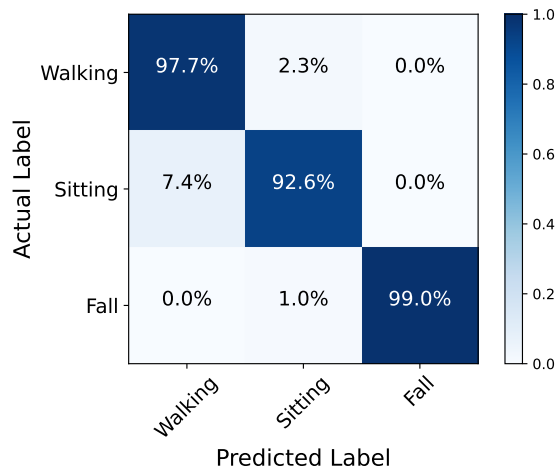


Figure 5.22: Confusion matrix of three states: walking (or standing), sitting, and fall (or lying on the ground).

classifications are misidentified as walking states rather than as falls. This indicates that, while the system has approximately a 2% to 7% chance of misjudgment between walking and sitting states, it effectively distinguishes fall detection from the other two states. This is proved by the average F1 score of 96.7% across all three categories, with a particularly high F1 score of 99.5% for fall detection. The system achieves an average precision of 97.1% and sensitivity of 96.4%, with an overall accuracy rate of 96.3%.

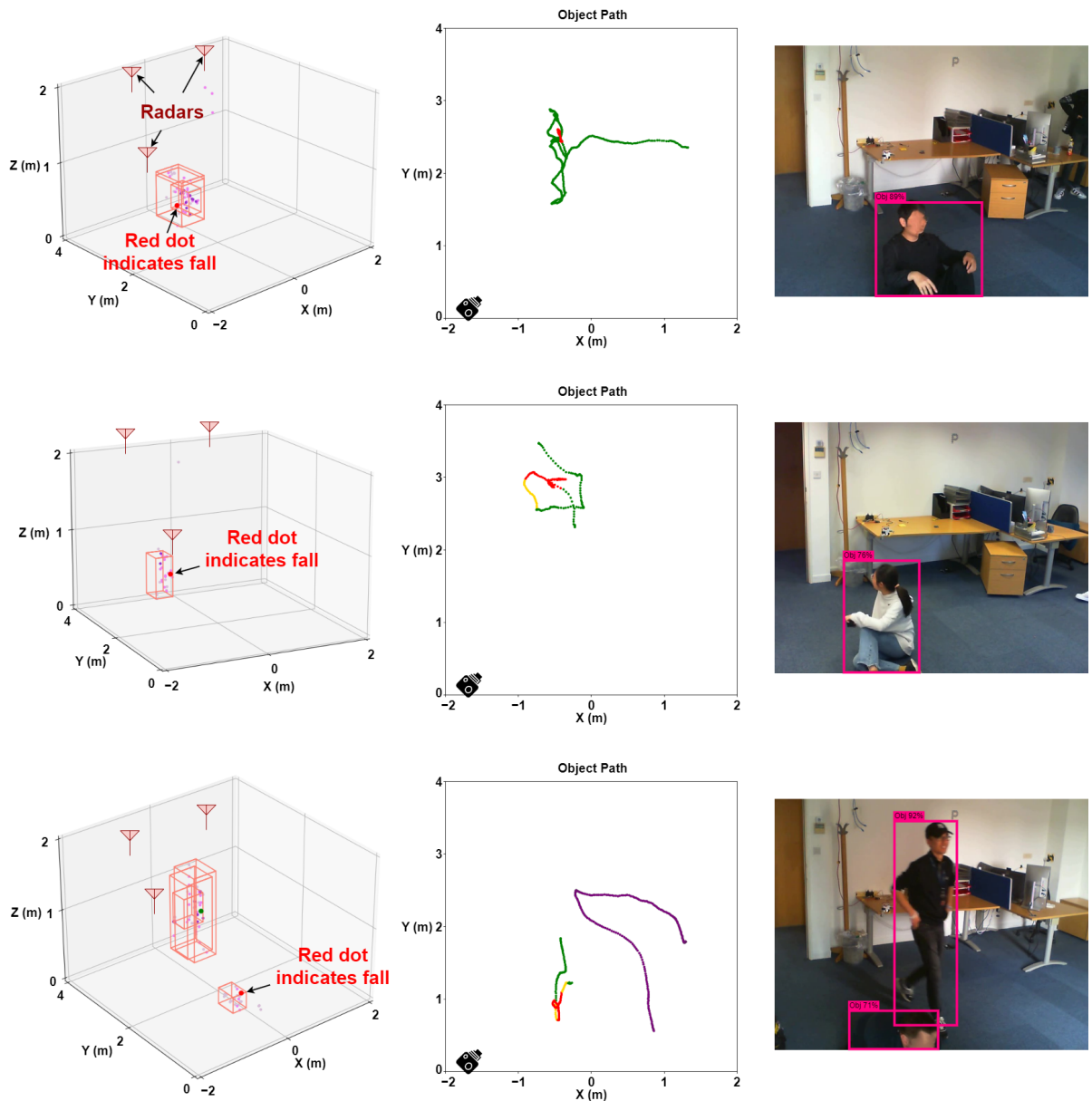


Figure 5.23: Three scenarios of human fall detection are presented, each accompanied by the 3D view of the point cloud (left), the target trajectory of the whole period (middle), and the real image from the camera (right).

Figure 5.23 displays the 3D point cloud when a human target falls in the left column. The middle column shows the moving trajectories with the target state marked by green (walking), yellow (sitting), and red (fall) dots. The camera image serves as the ground truth. When human targets fall and lie on the ground, the point clouds are concentrated at a lower height, allowing them to be identified as a fall on the ground using the method introduced in Section 5.4.6.

The presence of yellow dots in the trajectories of the second and third examples does not signify that the targets are sitting. Rather, these dots appear because the targets simulate falling to the ground instead of falling directly during the experiments. The fall is performed at an intentionally slow speed, causing the system to temporarily classify the targets as sitting. Sitting is treated as a transitional state between walking and fall (lying) on the ground. Therefore, the presence of yellow dots in the trajectories captured by the system is a normal outcome. In the third example of Figure 5.23, the system detects a fall when there are multiple people in the room. Since the second human target remains walking throughout the observation period, it is marked in purple in the visualization to distinguish it from the first target.

One of the limitations of the fall detection system is distinguishing between an actual fall and someone intentionally lying down on a mattress. If a person lies on a bed with some height, it remains relatively easy to differentiate from a fall based on the z-axis coordinates. However, differentiating between a fall and lying on a mattress requires analyzing the speed of the event. For instance, a longer duration of the event can be interpreted as going to sleep on a mattress, whereas an instant duration may indicate a fall. Further investigation into this aspect is left for future work.

5.5.3 Human Fall Posture Estimation

Fall posture estimation is a crucial step following successful fall detection, especially for elderly individuals. Knowing the posture during a fall can help healthcare providers assess the severity of the fall and provide appropriate medical intervention. To determine the posture after a fall, point clouds are accumulated over a period of 30 seconds and analyze the resulting data points. As illustrated in Figure 5.24, point clouds from both the top view and side view, as well as camera images, are compared for three different fall scenarios: lying facing up, lying facing sideways, and sitting on the ground.

In the scenario where the person is lying facing up, the top view exhibits the largest reflection area and the lowest gathering of points, occurring approximately 0.25 meters above the ground. In the second scenario, where the person is lying facing sideways, the top view shows a narrower area, and the gathering of points occurs at a higher position around 0.5 meters in the side view. Finally, in the scenario where the person is sitting on the ground, the top view displays the smallest area, and points gather around 1 meter above the ground, representing the human head.

Hence, it is viable to assess and categorize typical human fall postures by analyzing the point clouds collected by mmWave radars. Further exploration may involve the introduction of

additional posture classes and real-time estimation as part of future work.

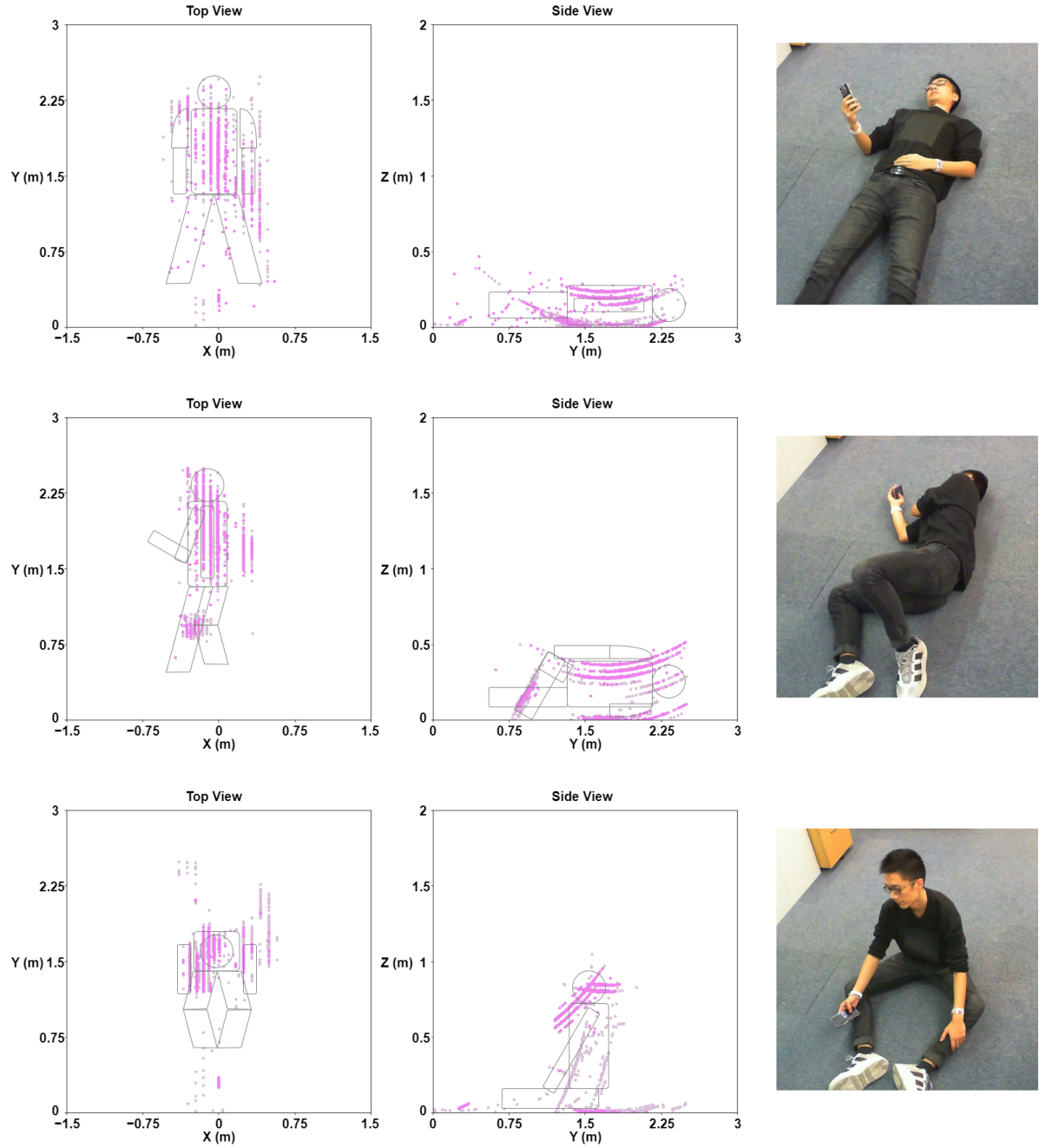


Figure 5.24: Three fall posture scenarios are presented, each accompanied by point clouds captured from both the top view (left) and side view (middle) of the point cloud, along with corresponding ground truth obtained from the camera (right).

5.5.4 System Comparison

This section presents a comparative analysis of human tracking and fall detection approaches in Table 5.3. Although wearables [23] and camera-based [8] methods demonstrate high accuracy in fall detection, they encounter challenges related to inconveniences in deployment and severe privacy issues, respectively. In contrast, mmWave radar solutions do not experience these issues. Approaches from [22, 43, 44] achieve over 92% accuracy in fall detection, but they lack support for human tracking and for scenarios involving multiple individuals. Particularly, the methods proposed in [43, 44] consider various real-life cases, but both fall short in real-time processing, a critical aspect for fall detection. Furthermore, due to the usage of neural networks, which require GPU to accelerate the computation, the approaches from [22, 43, 44] are relatively difficult to deploy compared with this system.

Table 5.3: Human tracking & fall detection system comparison.

	Wearables [23]	Camera [8]	MmWave Radar [22]	MmWave Radar [43]	MmWave Radar [44]	MmWave Radar (This system)
Fall detection	Acc. 93.0%	Acc. 96.9%	Acc. 97.6%	Prec. 97.5%	Acc.92.3%	Acc. 96.3%
Human tracking	No	Yes	No	No	No	Prec. 98.9%
Multiple people	No	No	No	No	No	Yes
Fall detection alert	Yes	Yes	No	No	No	Yes
Real-time speed	Yes	Yes, 8 FPS	Yes, < 10 FPS	No	No	Yes, 20 FPS
Privacy concerns	Low	Severe	Low	Low	Low	Low
Deployment	Inconvenient	Easy, No need GPU	Moderate, Use GPU for NN	Moderate, Use GPU for NN	Moderate, Use GPU for NN	Easy & extendable, No need GPU

In contrast, the proposed system achieves high accuracy and precision in both human tracking and fall detection. Additionally, the system also supports scenarios involving multiple individuals and provides fall detection alerts via Gmail. By avoiding deep neural networks, this real-time system can maintain at 20 FPS for runtime without any loss in accuracy and ensure ease of deployment for edge platforms. Furthermore, the system offers the flexibility to integrate additional radars for covering blind spots in complex indoor environments if required.

5.6 Conclusion

This chapter develops a real-time tracking and fall detection system designed for indoor multiple human targets by deploying three mmWave radars from TI in a multi-threaded environment. The radar characteristics evaluation delves into how the experimental field setup maximizes the radars' ability to recognize human targets. Additionally, this chapter introduces novel strategies, including dynamic DBSCAN clustering, a probability matrix for tracking updates, target state prediction, and a feedback loop for noise reduction for both the tracking system and fall detection.

The comprehensive evaluation showcases impressive results, achieving an F1 score of 98.4% for tracking a single target, and 97.3% and 95.9% for tracking two and three targets, respectively. For human fall detection, the overall accuracy reaches 96.3% with a sensitivity of 99.0% for the fall category, demonstrating the system's capability to distinguish falls from other states. Moreover, the practicality of fall posture estimation utilizing the 3D point clouds is assessed. This estimation holds the potential to offer remote medical intervention before on-site medical assistance arrives.

This research lays the groundwork for the development of advanced techniques in human tracking and fall detection using mmWave radar technology. The outcome is a non-intrusive, contactless system featuring real-time processing on a general-purpose CPU, suitable for applications in industrial and home IoT settings. Furthermore, the utilization of lightweight techniques makes it feasible to transfer the system onto low-power consumption platforms. The success achieved in human detection and tracking opens avenues for future research in more complex HAR tasks using mmWave radars. Future work may involve accommodating larger room sizes, distinguishing between sleeping on the ground and a fall, and identifying more postures (more discussion can be found in Section 7.2).

HUMAN IDENTIFICATION USING MMWAVE RADAR

This chapter introduces an innovative approach using deep learning for human identification utilizing mmWave radar technology. Unlike conventional vision methods, the approach in this work ensures privacy and accuracy in various indoor settings. Leveraging partial PointNet, MLP, CNN, and Bi-LSTM network components, this chapter proposes a unique neural network structure named MMIDNet, designed to process point cloud data from mmWave radar directly. The system achieves an impressive identification accuracy of 92.4% for 12 individuals. The research encompasses initial evaluation, data collection, system design, and system evaluation, highlighting the potential of mmWave radar combined with deep learning for secure and efficient human identification in IoT applications. The content of this chapter has been partially published in the MECO conference 2024 [53]¹.

The sections are organized as follows: Section 6.1 provides a brief overview of this chapter. Section 6.2 evaluates the human identification task using the FAUST dataset, demonstrating the feasibility of using deep learning techniques to identify human targets by using point clouds. Following this, Section 6.3 details the preparation and pre-processing of the point cloud dataset collected using mmWave radar. Section 6.4 presents the specifics of the network structure and explains its contribution to the human identification problem. Section 6.5 provides a comprehensive evaluation of the system, covering overall performance as well as the effects of different network configurations, observation durations, and numbers of people. It also includes a comparison with other researchers' works. Finally, Section 6.6 provides a conclusion for this chapter.

¹©2024 IEEE. Reprinted, with permission, from Z. Shen, J. Nunez-Yanez, and N. Dahnoun, "MMIDNet: Secure human identification using millimeter-wave radar and deep learning," in 2024 13th Mediterranean Conference on Embedded Computing (MECO), pp. 1–7, IEEE, 2024.

6.1 Overview

In the evolving landscape of sensing technologies, the precise identification of human targets within indoor environments stands as a critical endeavour with profound implications for various applications. Whether in the realms of in-house security, healthcare, or smart building management, the accurate and unobtrusive identification of individuals within indoor spaces is essential. Traditional sensing methods, including optical cameras [4, 25, 67] and infrared sensors [118], have been foundational in this pursuit. However, these methods frequently encounter challenges like limited visibility, difficulties in adapting to varying lighting conditions, and privacy concerns.

To overcome these challenges, the emergence of mmWave radar technology offers an innovative solution for indoor human target identification. Many researchers explored applications of human identification, aiming to distinguish users from each other and detect intruders from users using point clouds obtained by mmWave radar [27, 48, 49, 127, 192].

In this study, three FMCW radar IWR1843 development boards from TI are utilized to generate point clouds within the indoor environments for the training, validation, and test datasets. This chapter explores the potential of harnessing mmWave radar technology for human identification and proposes a deep neural network tailored for datasets in point cloud format to enhance accuracy. The primary contributions of this work include:

- The feasibility of employing deep learning techniques for identifying human targets through point clouds is assessed by using the FAUST dataset.
- A neural network structure, named MMIDNet, is designed to directly process point clouds from mmWave radars.
- An overall evaluation of the system is performed, demonstrating a notable identification accuracy of 92.4% for 12 individuals. The evaluation also explores the effects of various network configurations, observation durations, and the number of people on system performance.
- The work is available online at https://github.com/DarkSZChao/MMWave_Radar_Human_ID_Classification (accessed on 2 June 2024) to further promote the work in this field.

6.2 FAUST Evaluation

Human identification typically requires a significant amount of information, such as facial features, and the point cloud data from mmWave radar may be limited due to its sparse nature. Therefore, before using point clouds collected from mmWave radars for human identification, it's crucial to validate the feasibility of this approach. To accomplish this, an initial validation step is conducted using the FAUST dataset. This dataset comprises human models represented as watertight triangulated meshes, derived from a sophisticated high-resolution camera system

incorporating stereo cameras, RGB cameras, and speckle projectors [201]. Within the FAUST dataset, there are 10 subjects, each associated with 30 static postures, of which 10 postures are accompanied by aligned watertight models, resulting in a total of 100 models that can be used in this evaluation. Figure 6.1 shows some examples of the mesh models in FAUST with sampled point clouds. If accurate classification of these 10 identities can be achieved using the sparse point clouds, then the feasibility of this approach for human identification can be verified.

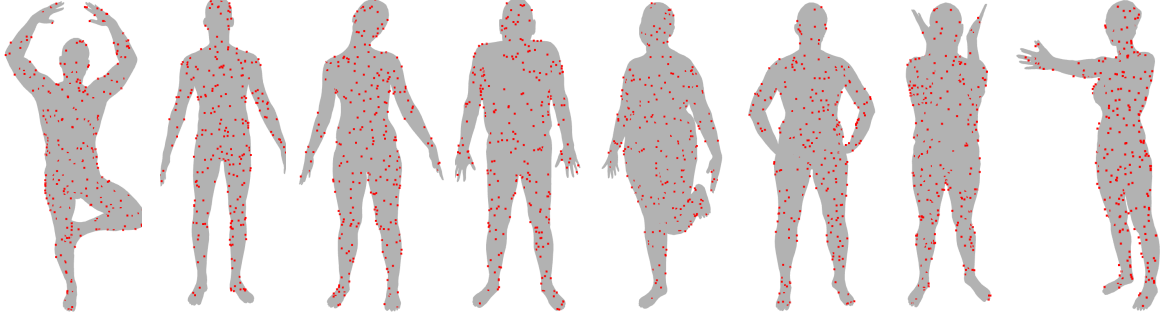


Figure 6.1: The example images for FAUST mesh models and point clouds. Red dots are points sampled on the mesh models.

6.2.1 Point Cloud Generation

Since the point clouds obtained from the mmWave radar comprise points defined by x, y, z coordinates, as elaborated in Section 5.3.2, the consistency must be maintained by extracting points with similar properties (e.g., point density) from the FAUST mesh models. Because the number of points received by mmWave radars in the experiment typically falls below 200 points per frame, randomly selecting 100 to 200 points from the surface of each mesh model as 1 sample simulates this scenario, with 100 samples taken from each model. However, neural networks struggle with data of varying lengths. Therefore, all samples are padded with zeros to maintain a consistent length of 200. The resulting dataset is structured as $N \times P \times C$ as shown in Equation (6.1), where N denotes the number of samples ($N = 10000$), P signifies the number of points in each sample ($P = 200$), and C represents the number of feature channels ($C = 3$, i.e., x, y, z coordinates). The elements of this tensor can be denoted as $X_{n,p,c}$, with n, p, c corresponding to the respective indices.

$$X \in \mathbb{R}^{N \times P \times C} \quad (6.1)$$

6.2.1.1 Data Augmentation

To address the issue of insufficient data, a coordinate field is created to simulate scenarios where subjects are present in a room. Random positional offsets are introduced to the point cloud

samples, ranging from 0 to 3 meters along the x and y axes (excluding the z-axis). Additionally, a random rotation, ranging from 0 to 360 degrees, is applied along the z-axis. To preserve the relative positions between points, which represent body shape, all points within each sample undergo the same rigid transformations for both translation and rotation.

Finally, to partition the dataset of 10,000 samples from 10 identities equally, 1,000 samples per identity are divided into training (70%), validation (10%), and testing (20%) sets. Each identity maintains the same proportion across these sets. The dataset is then shuffled in preparation for neural network training.

6.2.2 Network Design

MLP and CNN are two widely used neural network structures for spatial tasks such as image processing. They are able to capture relationships between data (pixels) efficiently to extract features. This section presents these two structures to perform the FAUST dataset evaluation. The fundamental details for MLP and CNN structures can be found in the previous Section 3.2. The networks in this section are deployed by using the Keras (version 2.6.0) deep learning framework in Python (version 3.8.6).

6.2.2.1 MLP

In this setup, two configurations of the MLP structure are prepared to evaluate the FAUST dataset comprehensively. One configuration uses five hidden dense layers with 3593k parameters, while the second uses three hidden dense layers with 1100k parameters. Figure 6.2 illustrates the configuration with five hidden dense layers. Given the input layer of size $N \times 200 \times 3$ with

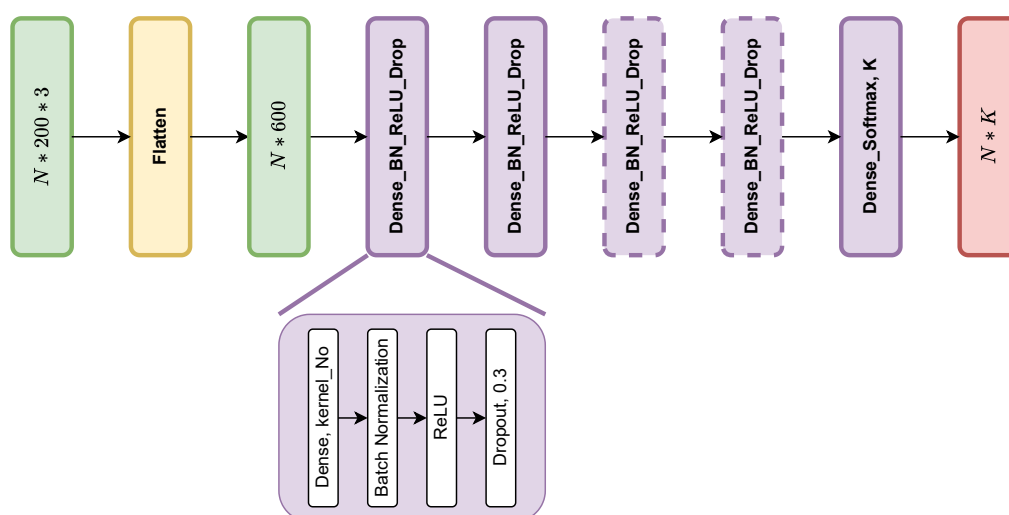


Figure 6.2: The MLP network structure with N samples and K categories. The dotted boxes represent the layers removed for various parameters in evaluation.

three dimensions, shown in green in Figure 6.2, a flatten layer is applied to convert the input to $N \times 600$ before passing it through the dense layers. Then, four dense layers are interconnected with batch normalization layers, ReLU activation layers, and dropout layers. The final dense layer, with a softmax activation function, outputs K people categories ($K = 10$ for the FAUST dataset). The configuration with three hidden dense layers follows a similar structure but omits the two dense layers, as drawn in dotted boxes in Figure 6.2, and has fewer total parameters.

6.2.2.2 CNN + MLP

In classification tasks, CNN networks are typically combined with MLP networks at the end to output the correct number of categories. Following the same strategy in the last section, three configurations that utilize both CNN and MLP are provided. As shown in Figure 6.3, the first configuration includes five convolutional layers with 2274k parameters, each convolutional layer specified with a particular number of kernels and kernel size. These convolutional layers are followed by max pooling layers, which reduce the data scale by combining the outputs of neuron clusters at the current layer into a single neuron in the next layer. A flatten layer is placed after the convolutional layers to stretch the data and reduce the data dimensions. Finally, a partial MLP network is applied after the CNN to perform classification into K categories. The second configuration uses three convolutional layers with 824k parameters, while the third configuration uses only one convolutional layer with 216k parameters. The omitted layers are indicated by dotted boxes in Figure 6.3.

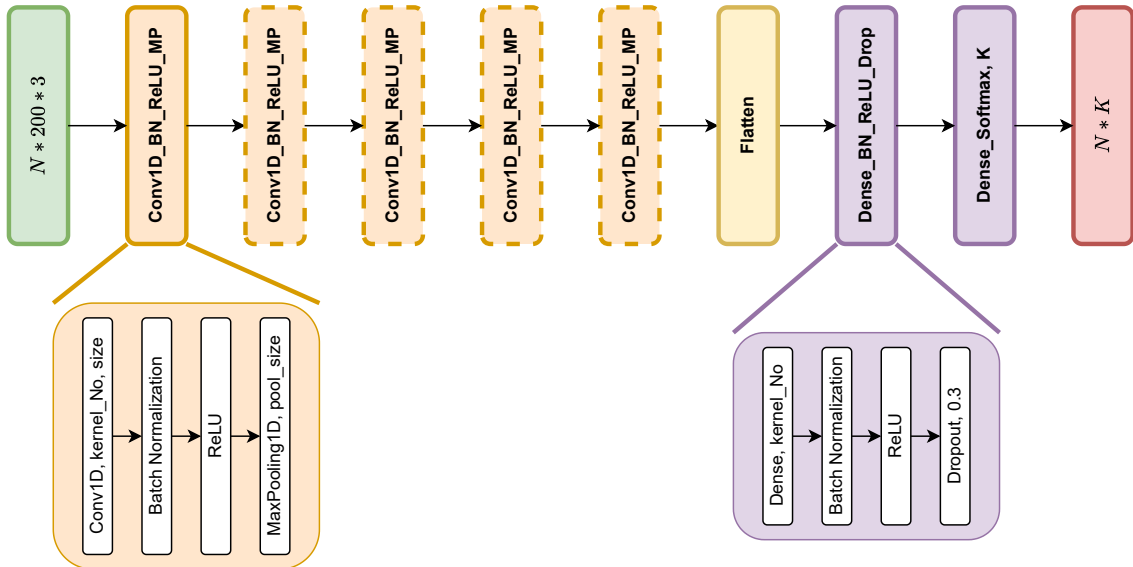


Figure 6.3: The network structure for CNN and MLP with N samples and K categories. The dotted boxes represent the layers removed for various parameters in evaluation.

6.2.3 Results and Comparison

Cross-entropy loss is employed as the loss function to quantify the difference between predicted and true probability distributions in classification tasks. Meanwhile, Adam is utilized as the optimizer, dynamically adjusting learning rates to ensure faster convergence for the networks. Training proceeds for 300 epochs, employing a learning rate of 0.0002 and a batch size of 128. Regarding the MLP network structure, validation accuracy on FAUST (shown in Figure 6.4 (a)) demonstrates that a single MLP network reaches a performance limit of around 40% for both parameter configurations of 1100k (three dense layers) and 3593k (five dense layers). Additionally, (b) depicts the loss for MLP on the validation set, which initially decreases until around 160 epochs before rising in the last 140 epochs. That indicates overfitting tendencies starting from 160 epochs. Overall, the single MLP structure exhibits poor performance in classifying human identification on the FAUST dataset.

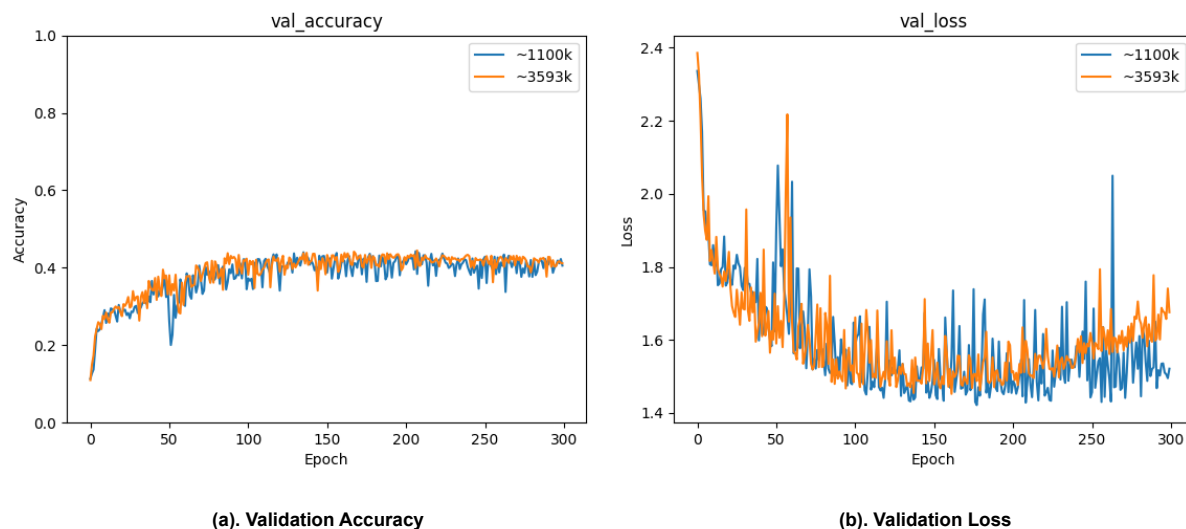


Figure 6.4: The accuracy (a) and loss (b) for a single MLP network on the validation set over epochs.

For the structure of CNN + MLP, there is a significant improvement in performance from 40% to 70% above, compared to the single MLP. As shown in Figure 6.5, on the validation set, networks with parameters of 216k (one convolutional layer) and 824k (three convolutional layers) achieve around 70% accuracy (a) with a loss value approaching 1.4 (b). Moreover, by expanding the network layers to five and increasing the parameter number to around 2274k, the network model can reach 80% accuracy and maintain stability with a lower loss below 1. This model also demonstrates a high accuracy level of 80% for 10 categories on the test set.

In conclusion, the CNN + MLP network structure outperforms the single MLP in the human classification task on the FAUST dataset. The achieved accuracy level of 80% proves the feasibility of applying neural networks to such classification tasks with the input of point clouds. This success

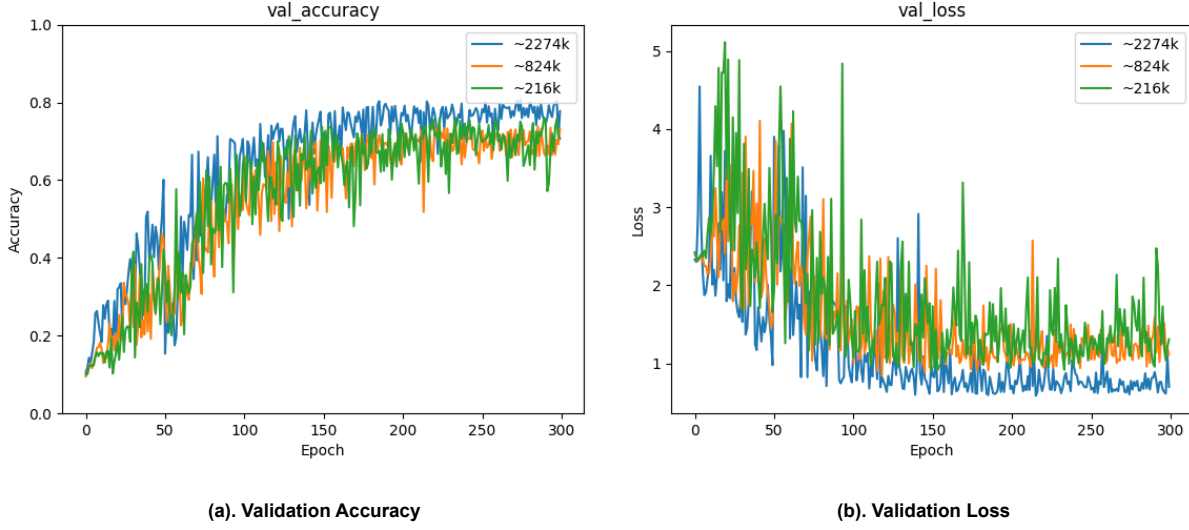


Figure 6.5: The accuracy (a) and loss (b) for CNN + MLP network on the validation set over epochs.

motivates further exploration into designing a network structure that can effectively handle point clouds from mmWave radars for human identification and classification.

6.3 Data Preparation

6.3.1 Data Collection

The data collection process follows the experimental setup described in Section 5.3. Point clouds are gathered from 12 randomly selected individuals, each walking around the radar room for 1-2 minutes per round, with a minimum of 10 rounds, resulting in approximately 20 minutes of data per participant. The subjects, aged 20-30, exhibit a range of heights (160-185 cm) and weights (50-80 kg), representing both male and female participants. The video footage is only utilized to ensure accurate labelling of the data and is not intended for experimental use. This data collection received approval from the Faculty of Engineering Research Ethics Committee at the University of Bristol (ethics approval reference code 17605).

6.3.2 Data Preprocessing

To maintain consistency, all the cloud points are expressed in the coordinate format (x, y, z, v, snr) , where (x, y, z) denotes the point coordinates, and (v, snr) represents velocity and SNR level, respectively. A global coordinate system is established, facilitating the mapping of points collected from different radars through spatial rotation and position shift. Following the concatenation of data points from three radars, the points undergo filtering based on their positions and SNR

levels. Subsequently, the dynamic DBSCAN clustering algorithm from the previous Chapter 5 is applied, as shown in Figure 6.6.

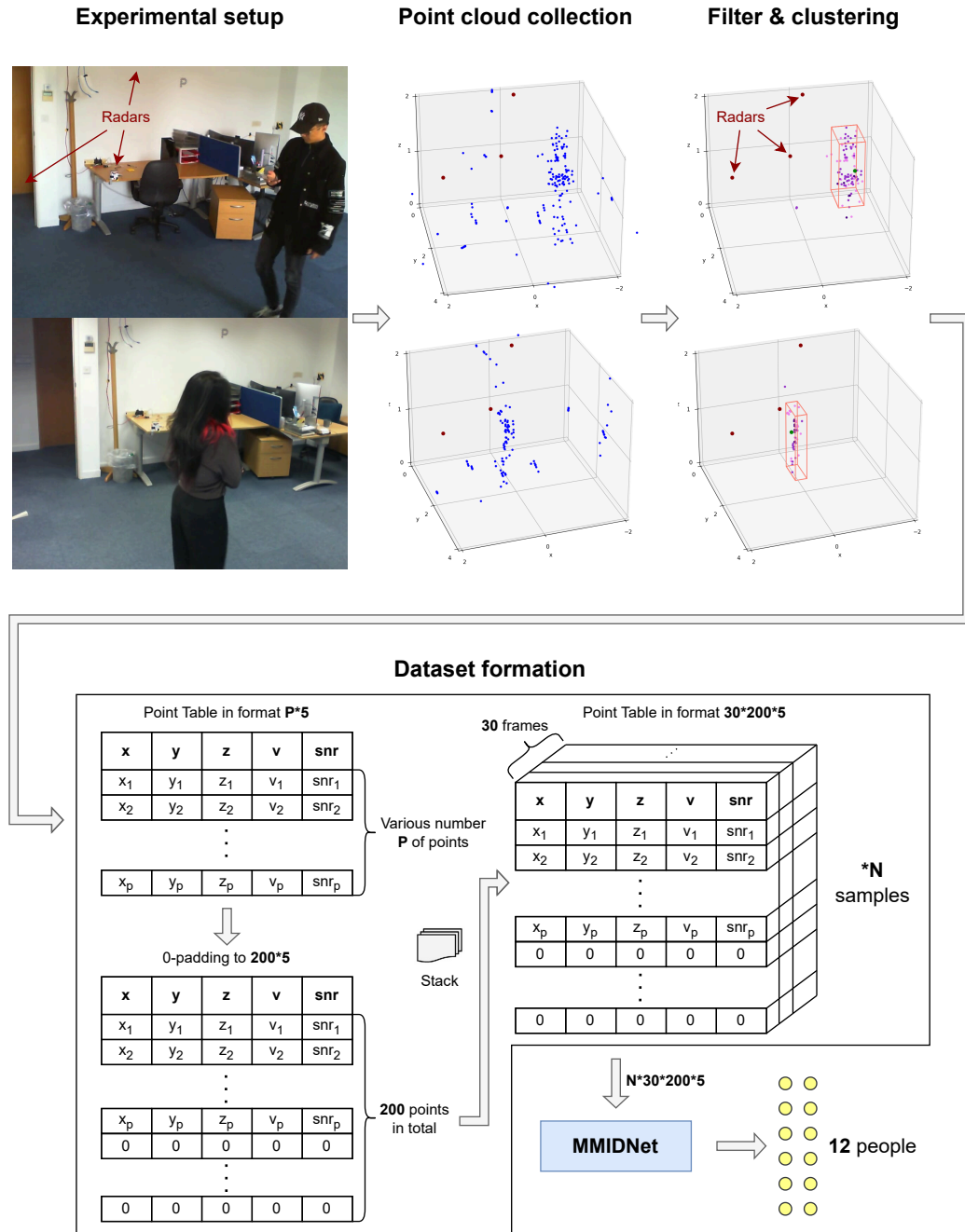


Figure 6.6: The system flowchart encompasses the experimental setup, point cloud collection, data filtering & clustering, and dataset preparation for network training.

In contrast to the approach outlined in [49], the network dataset in this work does not employ voxelization. Voxelization, as described in that work, incurs a high storage cost due to sparse voxelized data, posing challenges for GPU memory during accelerated training. Instead, a more

space-efficient point table representation is utilized. For instance, representing a human body in [49] requires a voxelized data size of $20 \times 20 \times 40$ to capture point cloud positions only for a single frame. In the method used here, the data is represented in a shape of $P \times 5$ (where P is the number of points in a given frame and $P \leq 200$) to represent a body, in terms of coordinates, velocity and SNR levels for the point cloud (depicted in Figure 6.6).

For each frame, the number of points (P) clustered by the DBSCAN algorithm exhibits variability due to the point cloud characteristics, resulting in a shape of $P \times 5$. To enable the network to analyze temporal data, all frames share a consistent shape of 200×5 by using zero-padding. Upon statistical analysis of the data, it is determined that the length of 200 points is optimal. Then, window sliding (length of 30 frames, step of 5 frames, overlap rate of 83%) is used to concatenate 30 neighbouring frames (approximately 1.5 seconds) to form a data sample. Following concatenation, one data sample transforms into the size of $30 \times 200 \times 5$.

The resulting dataset is organized as $N \times T \times P \times C$ as shown in Equation (6.2), where N is the number of samples, T represents the number of frames across time ($T = 30$), P indicates the number of points in each frame ($P = 200$) and C denotes the number of feature channels ($C = 5$). The elements of this tensor can be denoted as $X_{n,t,p,c}$, where n, t, p, c represents corresponding indexes.

$$X \in \mathbb{R}^{N \times T \times P \times C} \quad (6.2)$$

This structured dataset is then utilized to feed the MMIDNet network, which is proficient in classifying the identities of 12 individuals. The data from the 12 individuals used for identity classification is partitioned into 70% for training the network, 10% for validation, and 20% for testing. To enhance speed and reduce GPU memory usage during training, all the data is quantized to float16 data type.

6.4 System Design

6.4.1 Network Structure

MMIDNet is introduced as a neural network structure meticulously tailored and optimized for processing point clouds with five distinct features (x, y, z, v, snr) obtained from mmWave radars. The network comprises five key blocks: the Transformation Block, Residual CNN Block, Global Max Pooling, Bi-LSTM Block, and Dense Block, as illustrated in Figure 6.7.

6.4.1.1 Transformation Block

As mentioned in Section 3.2.4.1, in scenarios where a human body is represented using a 3D point cloud, applying rigid transformations such as translating or rotating on all points together does not lead to information loss or significant alterations, except for changes in the body's relative

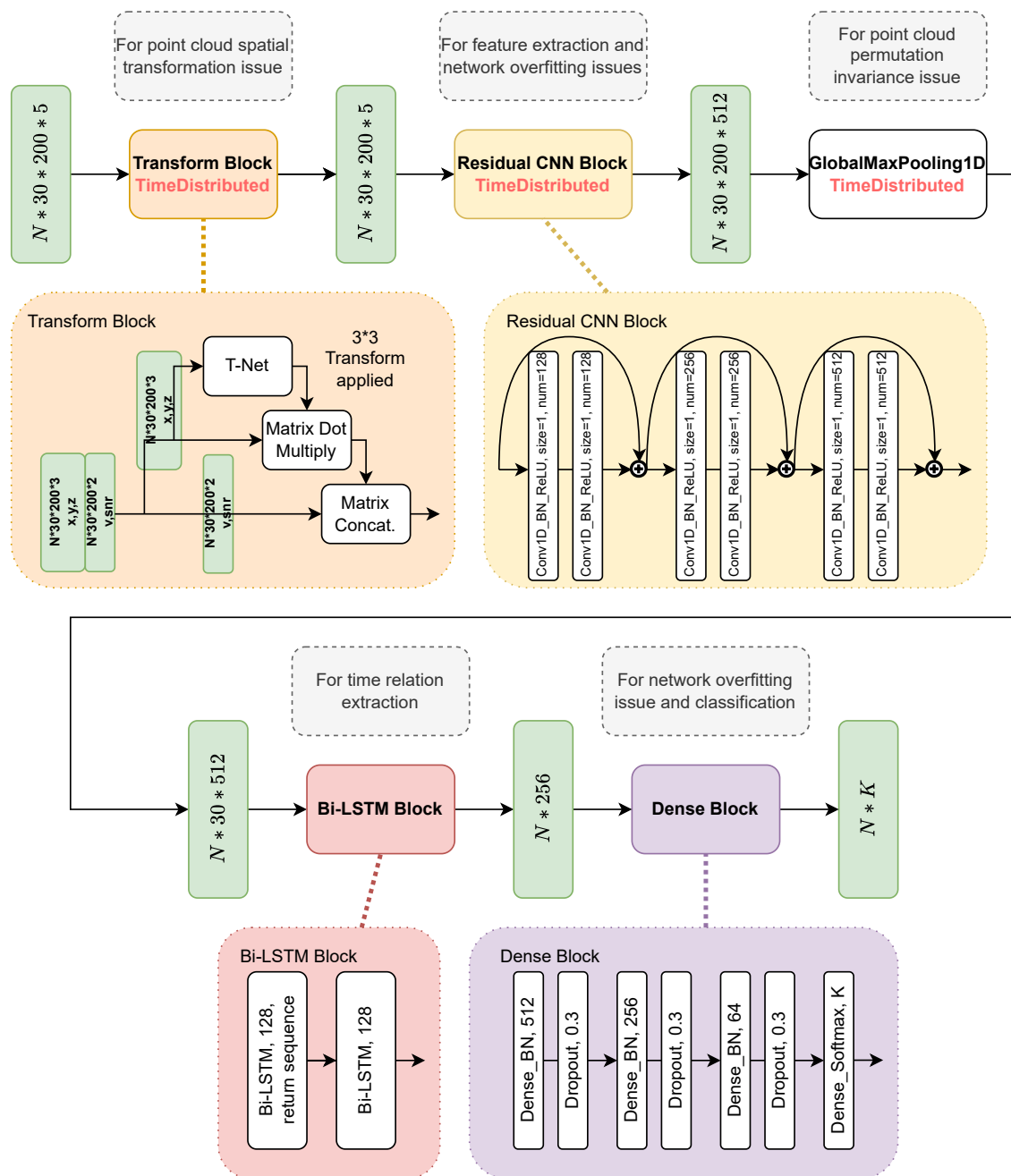


Figure 6.7: The proposed MMIDNet structure for human identify classification. K represents the number of identities ($K = 12$).

position [128]. Consequently, the system should maintain robustness on learned features when processing transformed 3D point cloud inputs.

Inspired by the usage of T-Net in PointNet [128, 202] (mentioned in Section 3.2.4), the data points (X_{input}) are initially separated on feature channels C dimension. The first coordinate segment has three features $C_{\{0,1,2\}}$ (x, y, z) organized as $N \times T \times P \times 3$, while the other two features

$C_{\{3,4\}}(v, snr)$ form a characteristic segment organized as $N \times T \times P \times 2$, as shown in Figure 6.7 and Equation (6.3). After that, the T-Net method capable of predicting an affine transformation matrix with a shape of 3×3 is adopted. This matrix is directly applied to the x , y , and z channels of the points by matrix dot multiplication. Finally, the previously isolated v and snr channels are concatenated back together. This series of actions constitutes the Transform Block.

$$X_{\text{input}} = \begin{bmatrix} X_{:, :, :, 0:3} & \text{(First 3 channels in } C) \\ X_{:, :, :, 3:5} & \text{(Last 2 channels in } C) \end{bmatrix} \quad (6.3)$$

6.4.1.2 Residual CNN Block

Within this block, a residual CNN structure is employed based on the residual structure [161]. Multiple CNN layers are utilized to extract features from the input points. To prevent potential degradation issues caused by stacking too many CNN layers, two CNN layers with identical hyperparameters are grouped into a residual block. The input and output of this group are connected by matrix addition, creating the input for the subsequent residual group, as illustrated in Figure 6.7. This approach helps prevent degradation issues in the network. Furthermore, a batch normalization layer and ReLu activation function following each CNN layer can standardize the input features to accelerate network learning convergence and avoid the overfitting issue [177].

6.4.1.3 Global Max Pooling

As discussed in Section 3.2.4.2, a point cloud is an unordered collection of vectors, which sets it apart from the structured pixel arrays in images or voxel arrays in volumetric grids, as used in [48, 49]. Specifically, as illustrated in Figure 6.8, two identical point cloud sets have the same geometric distribution but can be represented in different orders in table representation. Consequently, when a network processes 3D point clouds, it must preserve invariance to the permutation of the input set's data feeding order. In other words, the system should treat these differently ordered table inputs as equivalent since they represent the same point cloud.

To address this permutation invariance issue, based on [128, 202], a simple symmetric function called global max pooling layer is applied after the CNN in Figure 6.7. This pooling layer takes the maximum value across all dimensions (channels) for each feature, outputting a feature vector that remains invariant to the input vectors' order. This is because the maximum value for each feature is calculated independently of the order in which the points are arranged, ensuring a consistent representation regardless of the point cloud's permutation. The effectiveness of this approach has been demonstrated in point cloud applications [128], showcasing strong performance.

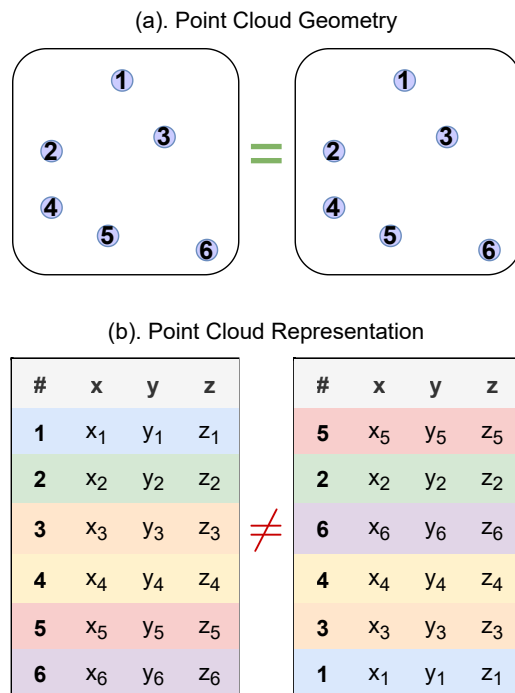


Figure 6.8: The same point distribution in the geometry in (a) can share multiple point cloud representations. Two representations are shown in (b) as an example.

6.4.1.4 Bi-LSTM Block

Classifying individual frames containing only the body-shaped point cloud is less efficient. Instead, employing a data sample that contains multiple adjacent frames in a time sequence offers a more comprehensive representation of the human target, incorporating not only body shape information but movement characteristics (e.g., gait) [49]. To enable the system to extract features from both these aspects, the Bi-LSTM structure which is well-suited for sequential data feature extraction is integrated.

To add the Bi-LSTM Block after the three modules, an additional layer called the TimeDistributed layer, which wraps the previous three modules, is required as shown in Figure 6.7. This TimeDistributed layer allows the layer being wrapped (e.g., the CNN layers) to apply the same operation to each time step of the sequence input and retain the time dimension T . The extracted features are then flattened and provided to the Bi-LSTM model for further processing.

6.4.1.5 Dense Block

Typically positioned at the end of the network structure, dense layers play a role in flattening and reorganizing the previously extracted features. To mitigate overfitting, dropout layers with a dropout rate of 0.3 are interspersed between each dense layer. The final dense layer is followed by a softmax activation function, employed to output the classification categories K (i.e., $K = 12$).

6.4.2 Network Training

The system is built using Python (version 3.8.6) and TensorFlow (version 2.6.5), with key dependencies such as Keras, NumPy, and h5py, on a desktop PC equipped with an Intel(R) Core (TM) i7-10850H CPU @ 2.70GHz and an NVIDIA RTX 2080 GPU with 8GB dedicated memory. CUDA is also required to enable GPU acceleration for the network training process. To access faster training and larger dedicated GPU memory, the network training process is transferred to Google Colab [203], which utilizes an A100 GPU with 40GB of dedicated memory online.

The hyperparameters such as the number of hidden units and kernel size for various network layers (CNN, Bi-LSTM, dense, and dropout) are annotated in Figure 6.7. The loss function is sparse categorical cross-entropy, and the optimizer is Adam. A training batch size of 128 is determined due to limited memory and the network is trained for 200 epochs with a learning rate of 0.0002. For the code implementation, please refer to Appendix A.

6.4.3 Prediction with Window Sliding

The network predictions may exhibit insufficient confidence in assigning the correct label occasionally, resulting in mislabelled segments within the prediction sequence. To further enhance the system accuracy, a window sliding approach is employed on the prediction sequence, opting for a window length of 12 predictions (approximately 3 seconds). This method selects the majority category within the window to reduce the impact of minority wrong predictions, which are highlighted in red in Figure 6.9.

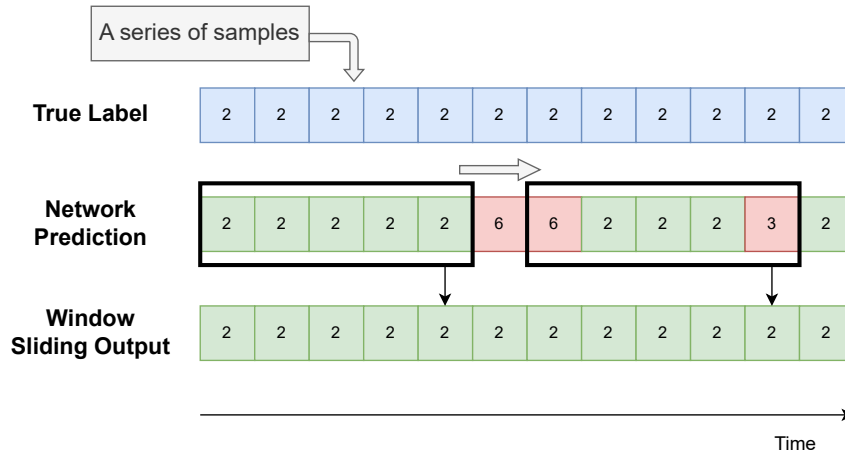


Figure 6.9: The comparison is depicted between the network prediction and the result obtained after applying a window sliding operation, with a window length of 5 predictions as an example.

Given that human identities are consistent when an individual is present in the room, the window sliding approach proves effective. Leveraging the previously developed tracking system in Chapter 5, the network results can be accumulated over time. This system enables the accurate

classification of human identity once a person is detected and tracking is initiated. When the person exits the room, the system deactivates tracking and clears the stacked sample results in the sliding window to prepare for the next tracking instance.

6.5 System Evaluation

Approximately 20 minutes of data is generated for each human subject, resulting in a total of 240 minutes (equivalent to 288,000 frames). The system accuracy in classifying the identities of 12 individuals is evaluated, considering scenarios where only one person is present at any given time.

6.5.1 Identification Performance

By employing the MMIDNet network, the performance of the system on the test set achieves an overall accuracy of 92.4% with the average F1 Score of 92.0%. The accompanying confusion matrix in Figure 6.10 demonstrates a robust performance in distinguishing the identities of the 12 individuals. Most human subjects exhibit high classification accuracy, exceeding 90%.

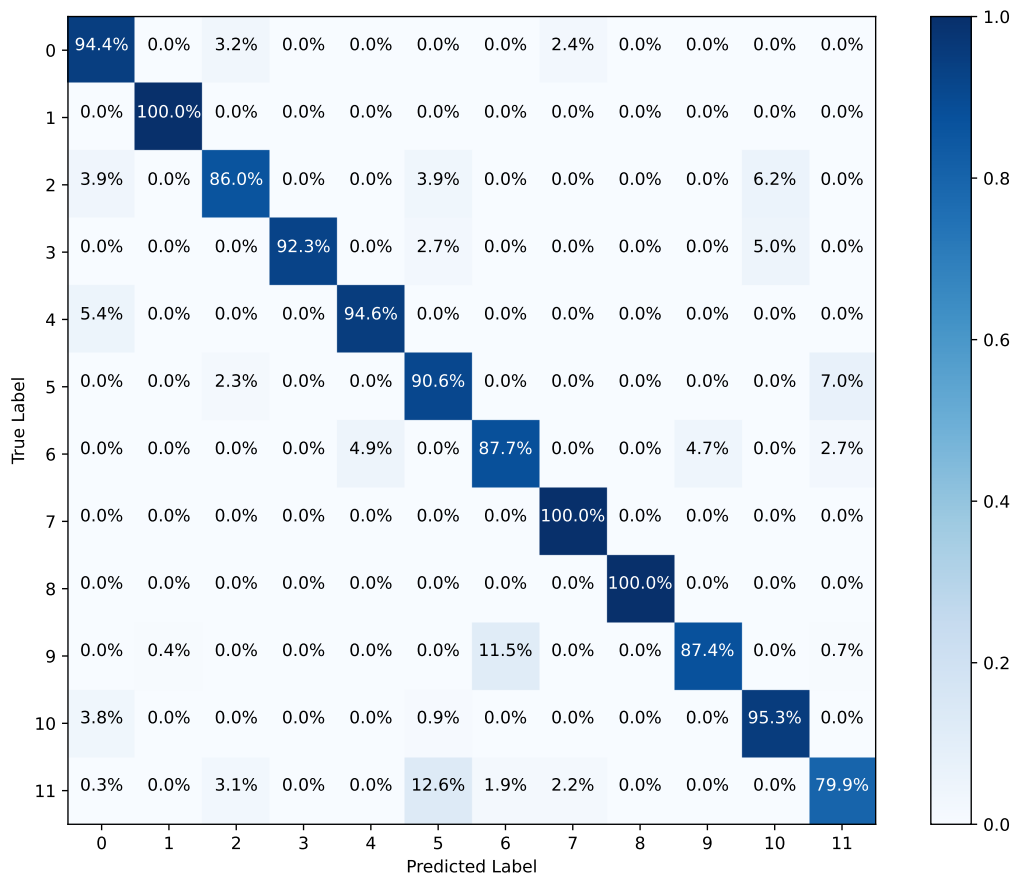


Figure 6.10: The confusion matrix for the classification prediction results of 12 people's identities.

One subject (category label 11) falls below 80% as shown in Figure 6.10. The confusion matrix displays a 12.6% chance of misclassifying label 11 as label 5, suggesting the system struggles to distinguish between these labels due to similar body shapes, similar walking gaits, and insufficient data. On the other hand, label 5 also has a relatively higher chance (7.0%) of being misclassified as label 11. A similar issue occurs between label 6 and label 9, slightly lowering the overall accuracy. To address this and improve accuracy, more data needs to be collected and fed into the network in the future.

Meanwhile, three categories (labels 1, 7, and 8) achieve 100% accuracy in the confusion matrix, indicating the system is highly confident in classifying them. One of these subjects is particularly tall at 185 cm, causing their point cloud to expand more along the z-axis compared to other subjects. Another subject is relatively heavy at 80 kg, resulting in a wider point cloud in both the x and y axes. These distinctive point cloud features make them easier to distinguish.

6.5.2 Evaluation of Network Structure

To investigate the impact of the network structure on human identification performance, various network configurations are evaluated, and the accuracy is measured when specific components are removed. This approach allows for a detailed analysis of the contribution of each component to the overall system performance. Figure 6.11 demonstrates the trajectory of accuracy and loss for the different network configurations on the validation set throughout the training process spanning 200 epochs. Obviously, the basic CNN + Dense structure maintains at 60% accuracy, with its loss rising after 25 epochs, indicating a potential overfitting issue. Based on this, introducing T-Net and Global Max Pooling components adopted from PointNet [128] enhances accuracy to

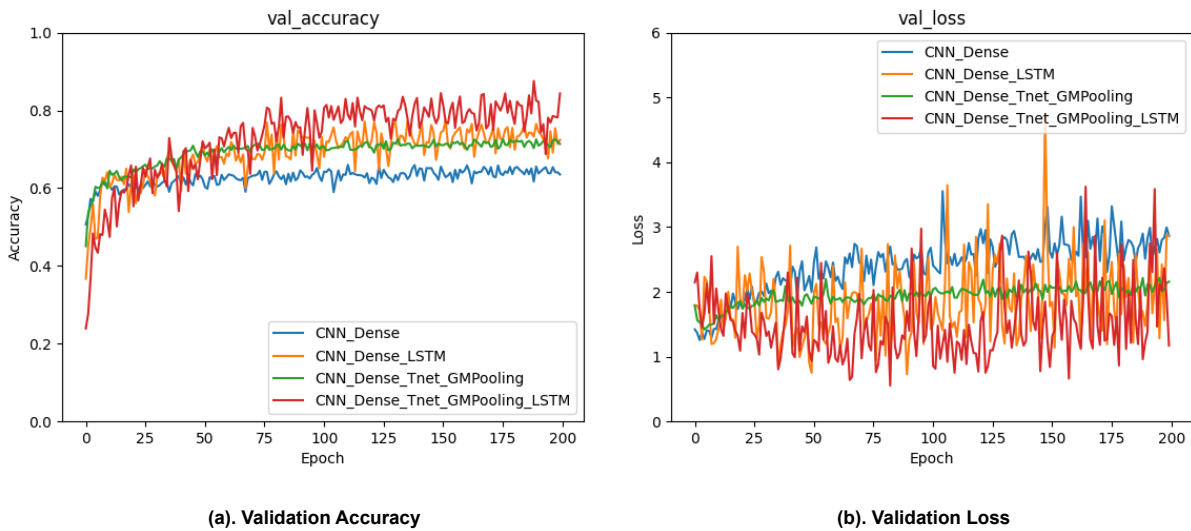


Figure 6.11: The accuracy (a) and loss (b) for different network configurations on the validation set over epochs.

approximately 70%, while applying a Bi-LSTM component yields a comparable outcome with slight variations. The red line in Figure 6.11 represents the combination of all components, resulting in an accuracy of around 80%, peaking at 87% at the epoch of 188.

The weight checkpoints are saved at each epoch during network training. After the training is complete, the epoch checkpoints with the best weights are reloaded for each configuration and used to perform inferences on the test set. Table 6.1 shows the performances of these configurations on the test set. The basic configuration of CNN and Dense layers can only reach an overall accuracy of 61.1%. Incorporating T-Net and Global Max Pooling, enhances accuracy by approximately 10%. The addition of Bi-LSTM further improves accuracy by about 16%. Ultimately, applying the window sliding strategy after the network results in the full system achieving an accuracy of 92.4% for 12 identities.

Table 6.1: Different network structure accuracy comparison on the test set.

Network structure	Accuracy
CNN + Dense	61.1%
CNN + Dense + Tnet + Global Max Pooling	70.5%
CNN + Dense + Bi-LSTM	73.0%
CNN + Dense + Tnet + Global Max Pooling + Bi-LSTM	86.3%
Full System	92.4%

6.5.3 Evaluation of Observation Duration

Common intuition suggests that the longer the sensor monitors, the greater the likelihood of accurately identifying them. To evaluate this, the impact of extended monitoring on system performance is examined in the context of identifying 12 individuals. To be more specific, as outlined in Section 6.4.3, a 3-second observation duration is used to generate predictions, yielding an overall accuracy of 92.4%, as shown in Figure 6.12. When the observation duration is extended to 7 seconds, the system achieves a significantly higher accuracy of over 98% on the test set.

Figure 6.12 illustrates the accuracy of the system across different observation durations, ranging from 3 to 7 seconds. The blue bars represent overall accuracy values, while the black boundaries indicate the range between the highest and lowest accuracy values for different labels. As the observation time increases, there is a consistent rise in system accuracy and a narrowing

in the variability between maximum and minimum accuracy values, demonstrating enhanced robustness and reliability with longer observation periods. Although a longer observation duration slows down the system, this delay only impacts the first prediction when a human subject first appears in the scene. It does not necessitate another full 7-second wait for subsequent predictions.

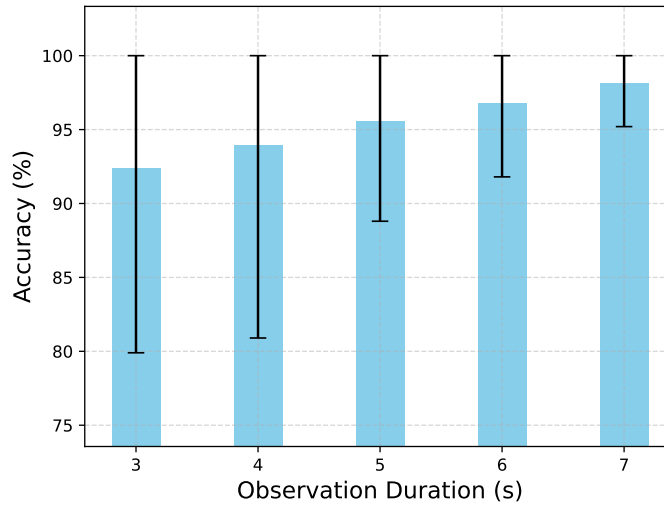


Figure 6.12: The system accuracy across various lengths of observation time, ranging from 3 to 7 seconds.

6.5.4 Evaluation of Number of People

Additionally, an assessment is performed to classify identities within various group sizes, as depicted in Figure 6.13. The system demonstrates strong performance, particularly with smaller groups ranging from 4 to 8 individuals, consistently achieving an accuracy of approximately 97%. The highest overall accuracy surpasses 98% when the group size is 4, with a minimum accuracy of 95%, indicating exceptional reliability in scenarios with fewer occupants.

This contrasts sharply with the results from Zeng et al. [2], who achieved only 80% accuracy when identifying 6 individuals using WiFi CSI. The significant improvement in accuracy using mmWave radar highlights the system’s robustness and precision, especially when faced with smaller groups, where it is able to effectively capture and distinguish the unique characteristics of each individual. This outcome underscores the system’s applicability in smart home and office environments, where the number of individuals present is typically limited. Moreover, even as the group size increases to 12 people, the system’s performance can still remain efficient.

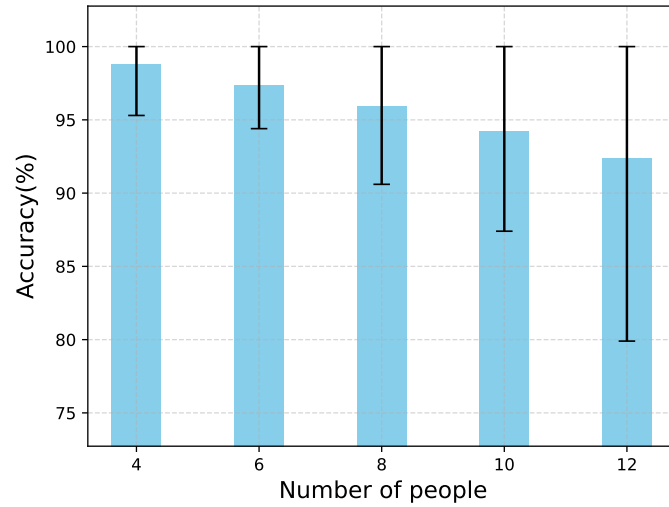


Figure 6.13: The system accuracy across various scenarios involving the classification of different numbers of people, ranging from 4 to 12.

6.5.5 System Comparison

Table 6.2 presents a comparative analysis of human identification systems. Sokolova and Konushin [102] introduced a solution involving cameras and neural networks for human identity classification, achieving outstanding performance exceeding 98% accuracy on the TUM-GAID dataset. Meanwhile, Koide et al. [104] employed cameras with depth information and integrated Bayesian inference methods, achieving a 98.4% accuracy for 6 subjects. However, both approaches in [102] and [104] face severe privacy concerns due to intrusive camera usage.

Table 6.2: Human identification system comparison.

	Camera [102]	RGB-D Camera [104]	Floor sensor [117]	WiFi WiWho [2]	MmWave Radar mID [49]	MmWave Radar MMIDNet (This system)
ID classification	Acc. > 98% on TUM-GAID	87.9% on 6 subjects, 98.4% with Bayesian	Acc. 80% on 15 subjects	92% on 2 subjects, 80% on 6 subjects	Acc. 89% on 12 subjects	Acc. 92.4% on 12 subjects
Multiple people	Not consider	Yes	Not consider	No	Yes	Yes
NN usage	Yes	Yes	No	No	Yes	Yes
Inference speed	-	> 4.2s	22Hz	0.2s	> 2s	3s Acc. 92.4%, 7s Acc. > 98%
Input data size	Large (images)	Moderate (image segments)	Small	Moderate (CSI)	Large (3D voxelized matrix)	Small (Sparse point cloud)
Privacy concerns	Severe	Severe	Negligible	Low	Low	Low
Deployment	Easy	Moderate	Difficult	Easy	Moderate	Easy & extendable

To address these privacy issues, Middleton et al. [117] developed a floor sensor matrix to record human footprints, a feature considered less invasive. Without relying on neural networks,

they achieved 80% accuracy for 15 subjects, operating at 22Hz. However, the deployment of this floor sensor is difficult and troublesome. Another non-intrusive approach is proposed by Zeng et al. [2], who achieved 92% accuracy for 2 subjects and 80% for 6 subjects on human identification by analyzing WiFi CSI. Although deployment is straightforward, it requires human movement between TX and RX which are separate devices. Recently, mmWave radar has garnered attention for non-intrusive human identification by Zhao et al. [49]. They proposed a system called mID that is designed for human tracking and identification. By voxelizing the point cloud provided by mmWave radar and feeding it into the network, mID achieved 89% accuracy for 12 subjects. However, point cloud voxelization leads to a large input dataset which makes it difficult for network training.

In contrast, the MMIDNet proposed in this work is a neural network structure optimized for mmWave radar applications. It utilizes sparse point cloud data, characterized by its small size, as input to expedite the network training process to learn useful information. With a duration of 3 seconds, this system achieves an accuracy of 92.4% for 12 subjects. Extending the observation time to 7 seconds yields a higher accuracy of 98%. Furthermore, the MMIDNet can discern multiple targets within the same scene by implementing clustering algorithms to distinguish between human targets. Integration with the previous work in Chapter 5 allows for extensibility by incorporating additional radars into the system.

6.6 Conclusion

This chapter introduces MMIDNet, a deep neural network structure crafted for mmWave radars in human identification tasks. It begins with a detailed explanation of the data collection process and the formation of datasets for training, validation, and testing. Subsequently, this chapter demonstrates each network component within the MMIDNet architecture and expounds on their respective contributions to the methodology. The proposed method achieves an impressive overall accuracy of 92.4% in classifying 12 human identities. With an extended observation duration of 7 seconds, the accuracy exceeds 98%. Additionally, the chapter explores the effects of different network configurations and their impact on identifying varying numbers of individuals. Furthermore, a comparative analysis with other state-of-the-art approaches is provided in aspects of performance, privacy and deployment. Although additional experiments across various scenarios are necessary for comprehensive human identification practices in the future, this study lays the groundwork for the feasibility of utilizing mmWave radars and deep learning methodologies in smart home and office settings. This work contributes significantly to the preservation of privacy in these environments.

Future work involves expanding the dataset to improve the accuracy and cover more complex situations. Meanwhile, classifying multiple human subjects within the same room at the same time can be achieved by integrating with the previous real-time system in Chapter 5.

CONCLUSIONS AND FUTURE WORK

HAR is a multidisciplinary field intersecting computer science, signal processing, and machine learning, serving as a crucial bridge between human and computer interaction. HAR has evolved beyond simple activity classification to encompass multiple tasks, e.g., human tracking, activity classification, fall detection, and identification. Owing to their benefits of low response latency and strong user privacy protection, edge devices are increasingly being employed in homes and offices for HAR systems, serving as alternatives to PCs and cloud-based computing for processing tasks. This situation creates challenges in achieving both fast runtime and low energy consumption on resource-constrained IoT devices, such as MCUs, many of which are powered by a single battery rather than being connected to a power source. Near-threshold MCUs show great potential for deploying HAR applications in these scenarios, offering significant energy savings while maintaining accuracy and extending battery life compared to standard MCUs. As HAR systems become more widespread and commercially available, privacy concerns associated with vision-based HAR systems have risen, prompting interest in radar-based HAR, which collects only anonymous data. MmWave radars, with their much higher frequency and bandwidth, offer significantly higher resolution for spatial feature capture. Additionally, the ease of deployment due to their small antenna size and significantly lower cost compared to LiDAR make mmWave radars increasingly popular not only in autonomous driving but also in HAR applications.

7.1 Conclusions

This research initially presents a comprehensive background study of HAR using various processors and sensors in Chapter 2. Different near-threshold MCU models from various manufacturers and supplementary frameworks for network acceleration are evaluated. State-of-the-art algo-

rhythmic techniques, including the "big-little" architecture, are discussed to illustrate how similar strategies can enhance MCU performance for HAR tasks. Furthermore, the advantages and drawbacks of various HAR sensors, including wearable and non-wearable sensors, are analyzed. For instance, wearable sensors placed on the human body are inconvenient for elderly individuals, while cameras are affected by lighting conditions and pose privacy concerns. Consequently, mmWave radar is selected as the input sensor for experiments in this thesis due to its suitability for collecting human data. Several HAR applications developed by other researchers are reviewed, focusing on four key tasks such as human tracking, activity classification, fall detection, and human identification. The research compares the contributions in this thesis to existing approaches and highlights how these applications inspired this work. Overall, HAR technology is constantly evolving, bringing new opportunities to application areas such as health monitoring, smart homes, and home security.

Next, the fundamentals of deep learning techniques and mmWave radar using the FMCW method are provided in Chapter 3. The basic principles of deep learning models, including MLP, CNN, LSTM, and PointNet, are introduced along with implementation guidance using the TensorFlow framework for HAR tasks. The working principles of mmWave radar, with a focus on measuring distance, velocity, and angle, are detailed. Various mmWave radar models from TI are compared, leading to the selection of the IWR1843 model for experimentation. Moreover, the configuration of the radar and the retrieval of point cloud data are demonstrated to support HAR applications in real time. This comprehensive introduction to relevant algorithms and hardware technologies provides essential background knowledge for understanding and developing high-performance HAR systems.

Chapter 4 compares the performance and power consumption differences between commercial near-threshold and standard MCUs. At the same operating frequency, the near-threshold MCUs exhibit significantly lower power consumption, approximately 85% less than the standard STM32L4 MCU. With comparable processing speeds, the low-power near-threshold MCUs achieve about 80% to 85% energy savings. Additionally, three adaptive neural network configurations are proposed and examined for deployment on MCUs. The aim is to achieve lower power consumption while maintaining prediction accuracy in classifying six human activities. The experiments on the UCI-HAR dataset demonstrate that the best adaptive network configuration proposed, consisting of one "big" and six "small" networks, reduces power consumption by 80% while maintaining 92.8% accuracy compared to the base method. Combined with the near-threshold MCU, this configuration can extend battery life by 62 times while running HAR tasks, compared to the original non-adaptive method using a standard MCU.

The aforementioned work uses the published standard UCI-HAR dataset. To develop a real-time system, original data collection is necessary, which leads to the choice of mmWave radar as a reliable sensor. Chapter 5 first provides evaluations focused on radar characteristics such as resolution, interference, and coverage range. Then, a real-time system for indoor human position

tracking is established, integrating point clouds from three identical mmWave radars from TI. To enhance the performance, the system features a dynamic DBSCAN clustering algorithm based on the SNR level, a probability matrix for enhanced tracking performance, and a feedback loop for noise suppression. Building on this tracking system, a human fall detection sub-system is developed, capable of sending email alerts upon detecting falls. To maintain real-time operation at 20 FPS while minimizing CPU usage, computationally intensive algorithms like neural networks and Kalman filters are avoided. Instead, the system classifies falls by analyzing point cloud data along the z-axis, distinguishing falls from standing and sitting states. Extensive evaluations are conducted over 300 minutes, including activities such as walking, sitting, and falling in both single-person and multi-person scenarios. The prototype system achieves an F1 score of 98.4% for single-target tracking, and 97.3% and 95.9% for tracking two and three targets, respectively. Meanwhile, the fall detection system demonstrates 99.0% sensitivity, effectively distinguishing falls from other states.

To determine not just human positions and states but also their identities based on the previously built system, the potential of mmWave radar in this context is explored in Chapter 6. Given that point cloud datasets are sparse and contain limited information, applying them to human identification tasks presents challenges. To assess the feasibility of using point clouds for identity classification, MLP and CNN networks are first built and tested on the FAUST point cloud dataset. After the feasibility is verified, an innovative deep learning approach for indoor human identification is developed, using the mmWave radar data. Drawing inspiration from PointNet, the elements of PointNet (i.e., T-Net and global max pooling layer) are integrated with CNNs and Bi-LSTM networks to create a unique neural network architecture called MMIDNet. This network consumes point cloud data directly from mmWave radars. The system achieves a 92.4% accuracy rate in identifying 12 individuals, showcasing a great potential of combining mmWave radar with deep learning for indoor identification in space-secure and smart home applications. Compared to traditional vision-based methods, this approach not only ensures privacy but also delivers high accuracy.

This research explores the advantages and disadvantages of using edge devices and mmWave sensors in multiple HAR tasks. The technical challenges of implementing deep neural networks on resource-constrained edge devices to optimize energy efficiency and inference speed for HAR are discussed. After that, to design real-time HAR systems, the transition from employing a published dataset to the mmWave point cloud dataset is demonstrated. This thesis shows the potential of using mmWave technology as an alternative solution for anonymous data collection when performing HAR applications like human tracking and human fall detection. Human tracking provides essential position information for HAR systems, while fall detection is critical for elderly individuals as falls are typically emergencies. Therefore, the feasibility of a tracking and fall detection system using non-intrusive mmWave radars is examined, leading to the development of such a system. Furthermore, identifying human individuals enhances the ability of HAR systems

to better understand the situation of the scene. Thus, a neural network applying mmWave radar for accurate human identification is designed. The contributions presented in this thesis verify the possibility and feasibility of deploying HAR systems on edge devices for IoT applications and highlight the significant potential of applying mmWave technology to HAR tasks.

7.2 Future Work

This research opens numerous avenues for future work. Firstly, Chapter 4 presents adaptive solutions using the published UCI-HAR dataset. Future work would involve cooperation with IMU sensors placed on human targets to enable real-time activity prediction using edge devices with a battery power supply. Additionally, future work would explore how this approach can be scaled to manage applications with a large number of output categories without incorporating additional network hierarchies, which would significantly increase memory consumption on MCUs. Developing a framework to automatically derive optimal "little" networks from a "big" network based on overall accuracy and energy efficiency could replace manual analysis. Meanwhile, the Mahalanobis distance for the configuration could also be improved by applying separate covariance matrices for each activity subset instead of a single matrix for the entire dataset. This would better detect deviations of a data sample from specific activities, helping to activate the "big" network when necessary. Furthermore, this research can be extended to other application areas, such as human present detection and health monitoring.

As Chapter 5 explores mmWave radar as an alternative sensor for the contactless solution, another research direction could involve applying the developed tracking system in a larger room to verify its robustness. To improve the accuracy of human status detection (i.e., walking, sitting, and fall) in Section 5.4.6, a dynamic threshold can be set in the future based on the measured height of individuals, including children, when they initially enter the room. For the fall detection system, it will be necessary to distinguish between falling and lying down intentionally (sleeping on the ground) to reduce false alarms. Moreover, the system should maintain robustness when pets or animals enter the scene, as their low height could potentially be misclassified as a fallen human. This issue can be addressed by applying filters based on size or movement speed in the future. Other future work could also focus on identifying a broader range of human activities, extending as a module for the system developed using the mmWave technique. To achieve this, incorporating neural networks could help differentiate more activities within complex scenarios. Furthermore, another promising direction for future work involves deploying this tracking and fall alert system using edge MCUs instead of a PC, thereby achieving lower energy consumption during long-term operation. This approach, due to its small size, also facilitates easier integration with other IoT applications in real-world scenarios.

Lastly, future work for the identification system presented in Chapter 6 could expand to more human subjects for system robustness testing. Meanwhile, classifying multiple subjects within

the same room at the same time is also a challenge for future work. This can be achieved by integrating the previous work and employing clustering algorithms to segment human targets separately. The segmented point cloud can then be fed into MMIDNet for identity classification. Further areas for investigation may include intruder detection and real-time processing. Additionally, research could explore integrating human identification functionality with other applications, such as human activity analysis and security monitoring, to create a more intelligent and comprehensive perception system. These research directions can significantly advance the development of HAR applications using edge processors and contactless sensors, contributing to the creation of versatile, low-cost, and non-intrusive HAR systems.



IMPLEMENTATION CODE FOR MMIDNET

This appendix provides the Python code for the MMIDNet, implemented using the Keras/TensorFlow framework. It details the network architecture across five blocks: the Transformation Block, Residual CNN Block, Global Max Pooling, Bi-LSTM Block, and Dense Block. Additionally, the code specifies key hyperparameters, including the learning rate, epochs, loss function, optimizer, and batch size, among others.

```

1  import os
2  import glob
3  import pickle
4
5  from matplotlib import pyplot as plt
6  import tensorflow as tf
7  import numpy as np
8
9  from NN import *
10 from config import *
11 from real_time_tracking.library.utils import *
12
13 # enable GPU for calculation
14 os.environ['CUDA_VISIBLE_DEVICES'] = '0' # 0 for GPU, -1 for CPU
15 print('Num GPUs Available: ', len(tf.config.list_physical_devices('GPU')))
16
17 # allocate GPU memory dynamically
18 gpus = tf.config.list_physical_devices('GPU')
19 if gpus:
20     try:
21         for gpu in gpus:
```

```
22         tf.config.experimental.set_memory_growth(gpu, True)
23     except RuntimeError as e:
24         print(e)
25
26
27     def scheduler(epoch, lr):
28         if epoch < 10:
29             lr = 0.0002
30         elif epoch == 50:
31             lr = 0.0002
32         else:
33             lr = lr * math.exp(-lr * 5)
34         return lr
35
36
37     class TF_callbacks:
38         def __init__(self):
39             self.lr_callback = tf.keras.callbacks. \
40                 LearningRateScheduler(schedule=scheduler, verbose=1)
41             self.cp_callback = tf.keras.callbacks. \
42                 ModelCheckpoint(filepath=os.path.join(CP_DIR, 'ep{epoch:03d}'
43                 _trainacc_{accuracy:.3f}_valloss{val_loss:.2f}_valacc{
44                 val_accuracy:.3f}.h5'),
45                                     verbose=1,
46                                     save_weights_only=True)
47
48
49     class OrthogonalRegularizer(keras.regularizers.Regularizer):
50         """
51         used in tnet block for pointnet structure
52         """
53
54         def __init__(self, num_features, l2reg=0.001):
55             self.num_features = num_features
56             self.l2reg = l2reg
57             self.eye = tf.eye(num_features)
58             pass
59
60         def __call__(self, x):
61             x = tf.reshape(x, (-1, self.num_features, self.num_features))
62             xxt = tf.tensordot(x, x, axes=(2, 2))
63             xxt = tf.reshape(xxt, (-1, self.num_features, self.num_features))
64             return tf.reduce_sum(self.l2reg * tf.square(xxt - self.eye))
```

```

63
64     def get_config(self):
65         return {'num_features': self.num_features,
66                 'l2reg' : self.l2reg}
67
68     @classmethod
69     def from_config(cls, config):
70         return cls(**config)
71
72
73 class CNN(TF_callbacks):
74     """
75     CNN blocks pre-defined
76     """
77
78     @staticmethod
79     def _conv1d_bn_relu(filter_No, kernel_size, layer,
80                         time_related_enable=False):
81         if not time_related_enable:
82             layer = Conv1D(filter_No, kernel_size, strides=1,
83                             padding='same')(layer)
84             layer = BatchNormalization(momentum=0.0)(layer)
85             return Activation("relu")(layer)
86         else:
87             layer = TimeDistributed(Conv1D(filter_No, kernel_size, strides=1,
88                                             padding='same'))(layer)
89             layer = TimeDistributed(BatchNormalization(momentum=0.0))(layer)
90             return TimeDistributed(Activation("relu"))(layer)
91
92     @staticmethod
93     def _dense_bn_relu(filters, layer):
94         layer = Dense(filters)(layer)
95         layer = BatchNormalization(momentum=0.0)(layer)
96         return Activation("relu")(layer)
97
98     def _conv1d_res_block(self, filters, kernel_size, block_input,
99                          first_block=False, time_related_enable=False):
100         if first_block:
101             block_input = self._conv1d_bn_relu(filters, kernel_size,
102                                                block_input, time_related_enable)
103
104         layer = self._conv1d_bn_relu(filters, kernel_size,
105                                     block_input, time_related_enable)

```

```
106         layer = self._conv1d_bn_relu(filters, kernel_size,
107                                     layer, time_related_enable)
108
109     if not time_related_enable:
110         return Add()([layer, block_input])
111     else:
112         return TimeDistributed(Add())([layer, block_input])
113
114     def _tnet_time(self, num_features, block_input, xyz_trans_only=False):
115         if not xyz_trans_only:
116             # Initalize bias as the indentiy matrix
117             bias = keras.initializers.Constant(np.eye(num_features).flatten())
118             reg = OrthogonalRegularizer(num_features)
119
120             layer = self._conv1d_bn_relu(32, 1, block_input)
121             layer = self._conv1d_bn_relu(64, 1, layer)
122             layer = self._conv1d_bn_relu(256, 1, layer)
123             layer = GlobalMaxPooling1D()(layer)
124             layer = self._dense_bn_relu(128, layer)
125             layer = self._dense_bn_relu(64, layer)
126             layer = Dense(num_features * num_features,
127                           kernel_initializer="zeros",
128                           bias_initializer=bias,
129                           activity_regularizer=reg,
130                           )(layer)
131             # calculate and apply affine transformation to input features
132             feat_T = Reshape((num_features, num_features))(layer)
133             return Dot(axes=(2, 1))([block_input, feat_T])
134         else:
135             num_features = 3
136             # split xyz from v and snr
137             layer = Reshape(target_shape=(-1, 5))(block_input)
138             split_layer = Lambda(lambda x: [x[:, :, :3], x[:, :, 3:]])
139             split_output = split_layer(layer)
140             points_xyz = split_output[0]
141             points_vsnr = split_output[1]
142
143             # Initalize bias as the indentiy matrix
144             bias = keras.initializers.Constant(np.eye(num_features).flatten())
145             reg = OrthogonalRegularizer(num_features)
146
147             layer = self._conv1d_bn_relu(32, 1, points_xyz)
148             layer = self._conv1d_bn_relu(64, 1, layer)
```

```

149         layer = self._conv1d_bn_relu(256, 1, layer)
150         layer = GlobalMaxPooling1D()(layer)
151         layer = self._dense_bn_relu(128, layer)
152         layer = self._dense_bn_relu(64, layer)
153         layer = Dense(num_features * num_features,
154                       kernel_initializer="zeros",
155                       bias_initializer=bias,
156                       activity_regularizer=reg,
157                       )(layer)
158         # calculate and apply affine transformation to input features
159         feat_T = Reshape((num_features, num_features))(layer)
160         points_xyz = Dot(axes=(2, 1))([points_xyz, feat_T])
161         layer = Concatenate(axis=-1)([points_xyz, points_vsnr])
162         return Reshape(target_shape=(-1, 200, 5))(layer)
163
164     """
165     CNN main structures defined
166     """
167
168     def CNN1D_Dense_Tnet_GMPooling_LSTM(self, input_shape, output_shape):
169         model_input = Input(shape=input_shape)
170
171         # xyz_trans_only will set num_features=3 and generate a 3*3 trans
172         matrix for xyz dimensions only
173
174         layer = self._tnet_time(3, model_input, xyz_trans_only=True)
175
176         # convolutional blocks
177         layer = self._conv1d_res_block(128, 1, layer, first_block=True,
178                                       time_related_enable=True)
179         layer = self._conv1d_res_block(256, 1, layer, first_block=True,
180                                       time_related_enable=True)
181         layer = self._conv1d_res_block(512, 1, layer, first_block=True,
182                                       time_related_enable=True)
183
184         # symmetric function for point cloud
185         layer = TimeDistributed(GlobalMaxPooling1D())(layer)
186
187         # LSTM
188         layer = Bidirectional(LSTM(128, return_sequences=True))(layer)
189         layer = Bidirectional(LSTM(128))(layer)
190
191         # dense layer
192         layer = self._dense_bn_relu(512, layer)

```

```
191         layer = Dropout(0.3)(layer)
192         layer = self._dense_bn_relu(256, layer)
193         layer = Dropout(0.3)(layer)
194         layer = self._dense_bn_relu(64, layer)
195         layer = Dropout(0.3)(layer)
196         layer = Dense(output_shape, activation='softmax')(layer)
197         return Model(model_input, layer)
198
199
200 if __name__ == '__main__':
201     # load data from stored dataset
202     dataset_file = 'train_val_dataset_12_200_filtered_timelen_20_step5'
203     with open(f'{os.path.join(DATASET_DIR, dataset_file)}', 'rb') as file:
204         data_np, labels_np = pickle.load(file)
205
206     # generate the dataset for train and val
207     train_data_np, train_labels_np, val_data_np, val_labels_np, category_No \
208         = generate_dataset(data_np, labels_np, split_ratio=(0.875, 0.125),
209                           label_categorical=False)
210
211     # clear the last checkpoints
212     folder_clean_recreate(CP_DIR)
213
214     # create the model
215     NN = CNN()
216     model = NN.CNN1D_Dense_Tnet_GMPooling_LSTM(train_data_np.shape[1:],
217                                                category_No)
218
219     # compile the model
220     model.compile(loss=tf.keras.losses.SparseCategoricalCrossentropy(),
221                  optimizer='adam',
222                  metrics=['accuracy'])
223
224     model.summary()
225     history = model.fit(train_data_np, train_labels_np,
226                        validation_data=(val_data_np, val_labels_np),
227                        shuffle=True,
228                        batch_size=128,
229                        callbacks=[NN.lr_callback, NN.cp_callback],
230                        epochs=200,
231                        verbose=1)
232
233     # save model
```

```

234     model.save(os.path.join(MODEL_SAVE_DIR, 'model.h5'))
235
236     # print results
237     print(f"Epoch {np.argmax(history.history['val_accuracy']) + 1} with
           maximum val_accuracy: {history.history['val_accuracy'][np.argmax(
           history.history['val_accuracy'])] * 100:.2f}%")
238
239     # Loads the best weights
240     # avoid to pick the first several checkpoints
241     epoch_remove = int(len(history.epoch) / 4)
242     epoch_val_acc_list = history.history['val_accuracy'][epoch_remove:]
243     h5file = glob.glob(os.path.join(CP_DIR, f'ep{np.argmax(epoch_val_acc_list)
           + 1 + epoch_remove:03d}*.h5'))[0]
244     print(f'Loading weight checkpoint: {os.path.basename(h5file)}')
245     model.load_weights(h5file)
246
247     # save model
248     model.save(os.path.join(MODEL_SAVE_DIR, 'modelbest.h5'))
249
250     # save history
251     with open(os.path.join(MODEL_SAVE_DIR,
252                             'model_filtered_history'), 'wb') as f:
253         pickle.dump(history.history, f)
254
255     # draw the figures
256     plt.figure()
257     plt.plot(history.history['accuracy'], label='accuracy')
258     plt.plot(history.history['val_accuracy'], label='val_accuracy')
259     plt.xlabel('Epoch')
260     plt.ylabel('Accuracy')
261     plt.ylim([0, 1])
262     plt.legend(loc='lower right')
263     plt.figure()
264     plt.plot(history.history['loss'], label='loss')
265     plt.plot(history.history['val_loss'], label='val_loss')
266     plt.xlabel('Epoch')
267     plt.ylabel('Loss')
268     plt.legend(loc='lower right')
269     plt.show()

```

CONFIGURATION CODE FOR MMWAVE RADAR

This appendix contains the configuration file code sent to the mmWave radars to initiate the system [149]. The following code, which should be mainly considered and adjusted, is accompanied by comments. The comments are ignored when transmitting to the radars.

```

1  sensorStop # Stop the radar sensor
2  flushCfg # Flush the config file
3  dfeDataOutputMode 1
4  channelCfg 15 7 0 # Set RX and TX antennas
5      # <rxChannelEn> Rx mask, for 4 antennas, (1111)b = 15
6      # <txChannelEn> Tx mask, for 3 antennas, (0111)b = 7
7      # <cascading>
8  adcCfg 2 1
9  adcbufCfg -1 0 1 1 1
10 profileCfg 0 77 105 7 57.14 0 0 70 1 128 2604 0 0 30 # Set chirps and ADC
11     # <profileId> 0
12     # <startFreq> 77 "Frequency Start" in GHz (float values allowed)
13     # <idleTime> 105 "Idle Time" in  $\mu$ s (float values allowed)
14     # <adcStartTime> 7 "ADC Valid Start Time" in  $\mu$ s (float values allowed)
15     # <rampEndTime> 57.14 "Ramp End Time" in  $\mu$ s (float values allowed)
16     # <txOutPower> 0
17     # <txPhaseShifter> 0
18     # <freqSlopeConst> 70 "Frequency slope" for the chirp in MHz/ $\mu$ s (float
19         values allowed)
20     # <txStartTime> 1 "TX Start Time" in  $\mu$ s (float values allowed)
21     # <numAdcSamples> 128 number of ADC samples collected during "ADC Sampling
22         Time" as shown in the chirp diagram above
23     # <digOutSampleRate> 2604 ADC sampling frequency in ksps. (<numAdcSamples>

```

APPENDIX B. CONFIGURATION CODE FOR MMWAVE RADAR

```
        / <digOutSampleRate> = "ADC Sampling Time")
22      # <hpfCornerFreq1> 0
23      # <hpfCornerFreq2> 0
24      # <rxGain> 30
25      chirpCfg 0 0 0 0 0 0 0 1 # Set chirps
26      chirpCfg 1 1 0 0 0 0 0 4
27      chirpCfg 2 2 0 0 0 0 0 2
28      # chirp start index
29      # chirp end index
30      # profile identifier should match the profileCfg -> <profileId>
31      # start frequency variation in Hz (float values allowed)
32      # frequency slope variation in kHz/ $\mu$ s (float values allowed)
33      # idle time variation in  $\mu$ s (float values allowed)
34      # ADC start time variation in  $\mu$ s (float values allowed)
35      # Tx antenna enable mask
36      frameCfg 0 2 32 0 50 1 0 # Set frames
37      # chirp start index (0-511)
38      # chirp end index (chirp start index-511)
39      # number of loops (1 to 255), chirps
40      # number of frames (valid range is 0 to 65535, 0 means infinite)
41      # frame periodicity in ms (float values allowed)
42      # trigger select
43      # Frame trigger delay in ms (float values allowed)
44      lowPower 0 0
45      guiMonitor -1 1 1 0 0 0 1 # Set GUI
46      cfarCfg -1 0 2 8 4 3 0 15 1 # Set CFAR
47      cfarCfg -1 1 0 4 2 3 1 15 1
48      # <subFrameIdx>
49      # <procDirection> 0-CFAR detection in range direction; 1-CFAR detection in
        Doppler direction
50      # <mode> 0-CFAR_CA (Cell Averaging); 1-CFAR_CAGO (Cell Averaging Greatest
        Of); 2-CFAR_CASO (Cell Averaging Smallest Of)
51      # <noiseWin> noise averaging window length: Length of the one sided noise
        averaged cells in samples
52      # <guardLen> one sided guard length in samples
53      # <divShift> Cumulative noise sum divisor expressed as a shift.
54      # cyclic mode or Wrapped around mode 0-Disabled 1-Enabled
55      # Threshold scale in dB using float representation.
56      # peak grouping 0-Disabled; 1-Enabled
57      multiObjBeamForming -1 1 0.5
58      clutterRemoval -1 0
59      # <subFrameIdx>
60      # <enabled> 1-enable static clutter removal technique
```

```

61  calibDcRangeSig -1 0 -5 8 256
62  extendedMaxVelocity -1 0
63  lvdsStreamCfg -1 0 0 0
64  compRangeBiasAndRxChanPhase 0.0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1
    0
65  measureRangeBiasAndRxChanPhase 0 1.5 0.2
66  CQRxSatMonitor 0 3 5 121 0
67  CQSigImgMonitor 0 63 4
68  analogMonitor 0 0
69  aoaFovCfg -1 -90 90 -90 90 # Set angles
70      # <subFrameIdx>
71      # <minAzimuthDeg>
72      # <maxAzimuthDeg>
73      # <minElevationDeg>
74      # <maxElevationDeg>
75  cfarFovCfg -1 0 0 5
76  cfarFovCfg -1 1 -2 2.00 # Set CFAR
77      # <subFrameIdx>.
78      # <procDirection> 0-point filtering in range direction 1-point filtering
    in Doppler
79      # <min (meters or m/s)>
80      # <max (meters or m/s)>
81  calibData 0 0 0
82  sensorStart # Start the radar sensor

```

BIBLIOGRAPHY

- [1] Gizmochina, “Xiaomi unveils new “people presence sensor” with millimeter-wave, 3 years of battery life.” <https://www.gizmochina.com/2024/05/13/xiaomi-people-presence-sensor-specs-price/>. Accessed August 2024.
- [2] Y. Zeng, P. H. Pathak, and P. Mohapatra, “Wiwho: Wifi-based person identification in smart spaces,” in *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, pp. 1–12, IEEE, 2016.
- [3] W. Wang, A. X. Liu, and M. Shahzad, “Gait recognition using wifi signals,” in *Proceedings of the 2016 ACM International Joint Conference on Pervasive and Ubiquitous Computing*, pp. 363–373, 2016.
- [4] D. Surie, S. Partonia, and H. Lindgren, “Human sensing using computer vision for personalized smart spaces,” in *2013 IEEE 10th international conference on ubiquitous intelligence and computing and 2013 IEEE 10th international conference on autonomic and trusted computing*, pp. 487–494, IEEE, 2013.
- [5] A. Roitberg, N. Somani, A. Perzylo, M. Rickert, and A. Knoll, “Multimodal human activity recognition for industrial manufacturing processes in robotic workcells,” in *Proceedings of the 2015 ACM on International Conference on Multimodal Interaction*, pp. 259–266, 2015.
- [6] A. Roitberg, A. Perzylo, N. Somani, M. Giuliani, M. Rickert, and A. Knoll, “Human activity recognition in the context of industrial human-robot interaction,” in *Signal and Information Processing Association Annual Summit and Conference (APSIPA), 2014 Asia-Pacific*, pp. 1–10, IEEE, 2014.
- [7] S. Park, J. Park, M. A. Al-Masni, M. A. Al-Antari, M. Z. Uddin, and T.-S. Kim, “A depth camera-based human activity recognition via deep learning recurrent neural network for health and social care services,” *Procedia Computer Science*, vol. 100, pp. 78–84, 2016.
- [8] K. De Miguel, A. Brunete, M. Hernando, and E. Gamba, “Home camera-based fall detection system for the elderly,” *Sensors*, vol. 17, no. 12, p. 2864, 2017.

- [9] S.-G. Miaou, P.-H. Sung, and C.-Y. Huang, “A customized human fall detection system using omni-camera images and personal information,” in *1st Transdisciplinary Conference on Distributed Diagnosis and Home Healthcare, 2006. D2H2.*, pp. 39–42, IEEE, 2006.
- [10] Y. Gao, Y. Long, Y. Guan, A. Basu, J. Baggaley, and T. Plötz, “Automated general movement assessment for perinatal stroke screening in infants,” *Smart Assisted Living: Toward An Open Smart-Home Infrastructure*, pp. 167–187, 2020.
- [11] AliExpress, “2024 xiaomi human presence sensor smart human body exists sensor bluetooth 5.0 work with xiaomi bluetooth gateway mi home app.” <https://www.aliexpress.com/item/1005007045530291.html>. Accessed August 2024.
- [12] D. T. Nguyen, W. Li, and P. O. Ogunbona, “Human detection from images and videos: A survey,” *Pattern Recognition*, vol. 51, pp. 148–175, 2016.
- [13] B. Leibe, E. Seemann, and B. Schiele, “Pedestrian detection in crowded scenes,” in *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, vol. 1, pp. 878–885, IEEE, 2005.
- [14] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *2005 IEEE computer society conference on computer vision and pattern recognition (CVPR’05)*, vol. 1, pp. 886–893, Ieee, 2005.
- [15] S. Zhang, C. Bauckhage, and A. B. Cremers, “Informed haar-like features improve pedestrian detection,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 947–954, 2014.
- [16] M. D. Breitenstein, F. Reichlin, B. Leibe, E. Koller-Meier, and L. Van Gool, “Online multi-person tracking-by-detection from a single, uncalibrated camera,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 33, no. 9, pp. 1820–1833, 2010.
- [17] J.-J. Lin, J.-I. Guo, V. M. Shivanna, and S.-Y. Chang, “Deep learning derived object detection and tracking technology based on sensor fusion of millimeter-wave radar/video and its application on embedded systems,” *Sensors*, vol. 23, no. 5, p. 2746, 2023.
- [18] T. Wang, N. Zheng, J. Xin, and Z. Ma, “Integrating millimeter wave radar with a monocular vision sensor for on-road obstacle detection applications,” *Sensors*, vol. 11, no. 9, pp. 8992–9008, 2011.
- [19] H. Cui and N. Dahnoun, “High precision human detection and tracking using millimeter-wave radars,” *IEEE Aerospace and Electronic Systems Magazine*, vol. 36, no. 1, pp. 22–32, 2021.

-
- [20] W. Choi, K. Shahid, and S. Savarese, "What are they doing?: Collective activity classification using spatio-temporal relationship among people," in *2009 IEEE 12th international conference on computer vision workshops, ICCV Workshops*, pp. 1282–1289, IEEE, 2009.
- [21] W. Wang, A. X. Liu, M. Shahzad, K. Ling, and S. Lu, "Device-free human activity recognition using commercial wifi devices," *IEEE Journal on Selected Areas in Communications*, vol. 35, no. 5, pp. 1118–1131, 2017.
- [22] C. Yu, Z. Xu, K. Yan, Y.-R. Chien, S.-H. Fang, and H.-C. Wu, "Noninvasive human activity recognition using millimeter-wave radar," *IEEE Systems Journal*, vol. 16, no. 2, pp. 3036–3047, 2022.
- [23] V. S. Kumar, K. G. Acharya, B. Sandeep, T. Jayavignesh, and A. Chaturvedi, "Wearable sensor-based human fall detection wireless system," in *Wireless Communication Networks and Internet of Things: Select Proceedings of ICNETS2, Volume VI*, pp. 217–234, Springer, 2019.
- [24] R. Cucchiara, A. Prati, and R. Vezzani, "A multi-camera vision system for fall detection and alarm generation," *Expert Systems*, vol. 24, no. 5, pp. 334–345, 2007.
- [25] Y.-H. Hong, S. J. Song, and J. Rho, "Real-time tracking and identification for multi-camera surveillance system," *International Journal of Internet, Broadcasting and Communication*, vol. 10, no. 1, pp. 16–22, 2018.
- [26] T. Xin, B. Guo, Z. Wang, M. Li, Z. Yu, and X. Zhou, "Freesense: Indoor human identification with wi-fi signals," in *2016 IEEE Global Communications Conference (GLOBECOM)*, pp. 1–7, IEEE, 2016.
- [27] Y. Yao, H. Zhang, P. Xia, C. Liu, F. Geng, Z. Bai, L. Du, X. Chen, P. Wang, B. Han, *et al.*, "mmsignature: Semi-supervised human identification system based on millimeter wave radar," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106939, 2023.
- [28] F. Wang, W. Gong, and J. Liu, "On spatial diversity in wifi-based human activity recognition: A deep learning-based approach," *IEEE Internet of Things Journal*, vol. 6, no. 2, pp. 2035–2047, 2018.
- [29] E. Ramanujam, T. Perumal, and S. Padmavathi, "Human activity recognition with smart-phone and wearable sensors using deep learning techniques: A review," *IEEE Sensors Journal*, vol. 21, no. 12, pp. 13029–13040, 2021.
- [30] S. Zhang, Y. Li, S. Zhang, F. Shahabi, S. Xia, Y. Deng, and N. Alshurafa, "Deep learning in human activity recognition with wearable sensors: A review on advances," *Sensors*, vol. 22, no. 4, p. 1476, 2022.

- [31] J. Zhu, A. Pande, P. Mohapatra, and J. J. Han, "Using deep learning for energy expenditure estimation with wearable sensors," in *2015 17th International Conference on E-health Networking, Application & Services (HealthCom)*, pp. 501–506, IEEE, 2015.
- [32] N. Rashid, B. U. Demirel, and M. A. Al Faruque, "Ahar: Adaptive cnn for energy-efficient human activity recognition in low-power edge devices," *IEEE Internet of Things Journal*, vol. 9, no. 15, pp. 13041–13051, 2022.
- [33] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.
- [34] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.
- [35] P.-E. Novac, G. B. Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and deployment of deep neural networks on microcontrollers," *Sensors*, vol. 21, no. 9, p. 2984, 2021.
- [36] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to+ 1 or-1," *arXiv preprint arXiv:1602.02830*, 2016.
- [37] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "Xnor-net: Imagenet classification using binary convolutional neural networks," in *European conference on computer vision*, pp. 525–542, Springer, 2016.
- [38] C.-T. Chu, J.-N. Hwang, H.-I. Pai, and K.-M. Lan, "Tracking human under occlusion based on adaptive multiple kernels with projected gradients," *IEEE Transactions on Multimedia*, vol. 15, no. 7, pp. 1602–1615, 2013.
- [39] B. Benfold and I. Reid, "Stable multi-target tracking in real-time surveillance video," in *CVPR 2011*, pp. 3457–3464, IEEE, 2011.
- [40] R. Hartmann, F. Al Machot, P. Mahr, and C. Bobda, "Camera-based system for tracking and position estimation of humans," in *2010 conference on design and architectures for signal and image processing (DASIP)*, pp. 62–67, IEEE, 2010.
- [41] J. Wu, H. Cui, and N. Dahnoun, "A novel high performance human detection, tracking and alarm system based on millimeter-wave radar," in *2021 10th Mediterranean Conference on Embedded Computing (MECO)*, pp. 1–4, IEEE, 2021.
- [42] CDC, "Older adult falls data." <https://www.cdc.gov/falls/data-research/index.html>.

Accessed May 2024.

- [43] Y. Yao, C. Liu, H. Zhang, B. Yan, P. Jian, P. Wang, L. Du, X. Chen, B. Han, and Z. Fang, "Fall detection system using millimeter-wave radar based on neural network and information fusion," *IEEE Internet of Things Journal*, vol. 9, no. 21, pp. 21038–21050, 2022.
- [44] A. Rezaei, A. Mascheroni, M. C. Stevens, R. Argha, M. Papandrea, A. Puiatti, and N. H. Lovell, "Unobtrusive human fall detection system using mmwave radar and data driven methods," *IEEE Sensors Journal*, vol. 23, no. 7, pp. 7968–7976, 2023.
- [45] Y. Sun, R. Hang, Z. Li, M. Jin, and K. Xu, "Privacy-preserving fall detection with deep learning on mmwave radar signal," in *2019 IEEE Visual Communications and Image Processing (VCIP)*, pp. 1–4, IEEE, 2019.
- [46] ADT, "The 2022 adt burglary report." <https://www.adt.co.uk/blog/the-2022-adt-burglary-report>. Accessed May 2024.
- [47] Z. Meng, S. Fu, J. Yan, H. Liang, A. Zhou, S. Zhu, H. Ma, J. Liu, and N. Yang, "Gait recognition for co-existing multiple people using millimeter wave sensing," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 849–856, 2020.
- [48] P. Zhao, C. X. Lu, J. Wang, C. Chen, W. Wang, N. Trigoni, and A. Markham, "mid: Tracking and identifying people with millimeter wave radar," in *2019 15th International Conference on Distributed Computing in Sensor Systems (DCOSS)*, pp. 33–40, IEEE, 2019.
- [49] P. Zhao, C. X. Lu, J. Wang, C. Chen, W. Wang, N. Trigoni, and A. Markham, "Human tracking and identification through a millimeter wave radar," *Ad Hoc Networks*, vol. 116, p. 102475, 2021.
- [50] Z. Shen, N. Howard, and J. Nunez-Yanez, "Big–little adaptive neural networks on low-power near-subthreshold processors," *Journal of Low Power Electronics and Applications*, vol. 12, no. 2, p. 28, 2022.
- [51] Z. Shen, J. Nunez-Yanez, and N. Dahnoun, "Multiple human tracking and fall detection real-time system using millimeter-wave radar and data fusion," in *2023 12th Mediterranean Conference on Embedded Computing (MECO)*, pp. 1–6, IEEE, 2023.
- [52] Z. Shen, J. Nunez-Yanez, and N. Dahnoun, "Advanced millimeter-wave radar system for real-time multiple-human tracking and fall detection," *Sensors*, vol. 24, no. 11, p. 3660, 2024.

BIBLIOGRAPHY

- [53] Z. Shen, J. Nunez-Yanez, and N. Dahnoun, "Mmidnet: Secure human identification using millimeter-wave radar and deep learning," in *2024 13th Mediterranean Conference on Embedded Computing (MECO)*, pp. 1–7, IEEE, 2024.
- [54] UCI Machine Learning, "Human activity recognition using smartphones data set." <https://archive.ics.uci.edu/ml/datasets/human+activity+recognition+using+smartphones>. Accessed January 2021.
- [55] T. Shany, S. J. Redmond, M. R. Narayanan, and N. H. Lovell, "Sensors-based wearable systems for monitoring of human movement and falls," *IEEE Sensors journal*, vol. 12, no. 3, pp. 658–670, 2011.
- [56] C. Chen, R. Jafari, and N. Kehtarnavaz, "Improving human action recognition using fusion of depth camera and inertial sensors," *IEEE Transactions on Human-Machine Systems*, vol. 45, no. 1, pp. 51–61, 2014.
- [57] H. Cui and N. Dahnoun, "Real-time short-range human posture estimation using mmwave radars and neural networks," *IEEE Sensors Journal*, vol. 22, no. 1, pp. 535–543, 2021.
- [58] G. Li, Z. Zhang, H. Yang, J. Pan, D. Chen, and J. Zhang, "Capturing human pose using mmwave radar," in *2020 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, pp. 1–6, IEEE, 2020.
- [59] Q. Wang, K. Wang, and W. Chen, "Clgnet: A new network for human pose estimation using commodity millimeter wave radar," in *2020 3rd International Conference on Algorithms, Computing and Artificial Intelligence*, pp. 1–5, 2020.
- [60] A. Sengupta, F. Jin, R. Zhang, and S. Cao, "mm-pose: Real-time human skeletal posture estimation using mmwave radars and cnns," *IEEE Sensors Journal*, vol. 20, no. 17, pp. 10032–10044, 2020.
- [61] H. Chaudhary, "Eta compute's ecm3532 board provides ai vision works for months on a single battery." <https://opencloudware.com/eta-computes-ecm3532-board-provides-ai-vision-that-can-work-for-months-on-a-single-battery/>. Accessed June 2021.
- [62] Ambiq Micro, "Apollo3 blue datasheet." https://cdn.sparkfun.com/assets/learn_tutorials/9/0/9/Apollo3_Blue_MCU_Data_Sheet_v0_9_1.pdf. Accessed January 2021.
- [63] F. J. Ordóñez and D. Roggen, "Deep convolutional and lstm recurrent neural networks for multimodal wearable activity recognition," *Sensors*, vol. 16, no. 1, p. 115, 2016.

- [64] A. Toshev and C. Szegedy, “DeepPose: Human pose estimation via deep neural networks,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1653–1660, 2014.
- [65] K. Sun, B. Xiao, D. Liu, and J. Wang, “Deep high-resolution representation learning for human pose estimation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5693–5703, 2019.
- [66] F. Jin, R. Zhang, A. Sengupta, S. Cao, S. Hariri, N. K. Agarwal, and S. K. Agarwal, “Multiple patients behavior detection in real-time using mmwave radar and deep cnns,” in *2019 IEEE Radar Conference (RadarConf)*, pp. 1–6, IEEE, 2019.
- [67] S. Ye, R. Bohush, H. Chen, I. Y. Zakharava, and S. Ablameyko, “Person tracking and reidentification for multicamera indoor video surveillance systems,” *Pattern Recognition and Image Analysis*, vol. 30, pp. 827–837, 2020.
- [68] Y. Cheng and Y. Liu, “Person reidentification based on automotive radar point clouds,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–13, 2021.
- [69] TheLinuxCode, “Powering esp32 projects with batteries: A detailed guide.” <https://thelinuxcode.com/power-esp32-battery/>. Accessed September 2024.
- [70] E. Flamand, D. Rossi, F. Conti, I. Loi, A. Pullini, F. Rotenberg, and L. Benini, “Gap-8: A risc-v soc for ai at the edge of the iot,” in *2018 IEEE 29th International Conference on Application-specific Systems, Architectures and Processors (ASAP)*, pp. 1–4, IEEE, 2018.
- [71] Coral, “Edge tpu.” <https://coral.ai/docs/edgetpu/faq/>. Accessed January 2021.
- [72] Eta Compute, “Eta compute ecm3532 ai sensor product brief.” <https://media.digikey.com/pdf/Data%20Sheets/Eta%20Compute%20PDFs/ECM3532-AI-Vision-Product-Brief-1.0.pdf>. Accessed January 2021.
- [73] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2704–2713, 2018.
- [74] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, “Quantized neural networks: Training neural networks with low precision weights and activations,” *The Journal of Machine Learning Research*, vol. 18, no. 1, pp. 6869–6898, 2017.

- [75] L. Mocerino and A. Calimera, “Coopnet: Cooperative convolutional neural network for low-power mcus,” in *2019 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, pp. 414–417, IEEE, 2019.
- [76] S. Amiri, M. Hosseinabady, S. McIntosh-Smith, and J. Nunez-Yanez, “Multi-precision convolutional neural networks on heterogeneous hardware,” in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 419–424, IEEE, 2018.
- [77] E. Park, D. Kim, S. Kim, Y.-D. Kim, G. Kim, S. Yoon, and S. Yoo, “Big/little deep neural network for ultra low power inference,” in *2015 international conference on hardware/-software codesign and system synthesis (codes+ iss)*, pp. 124–132, IEEE, 2015.
- [78] J. Nunez-Yanez and N. Howard, “Energy-efficient neural networks with near-threshold processors and hardware accelerators,” *Journal of Systems Architecture*, vol. 116, p. 102062, 2021.
- [79] P. Turaga, R. Chellappa, V. S. Subrahmanian, and O. Udrea, “Machine recognition of human activities: A survey,” *IEEE Transactions on Circuits and Systems for Video technology*, vol. 18, no. 11, pp. 1473–1488, 2008.
- [80] L. Lai, N. Suda, and V. Chandra, “Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus,” *arXiv preprint arXiv:1801.06601*, 2018.
- [81] X. Wang, M. Magno, L. Cavigelli, and L. Benini, “Fann-on-mcu: An open-source toolkit for energy-efficient neural network inference at the edge of the internet of things,” *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4403–4417, 2020.
- [82] J. Ma, parai, H. Mabrouk, BaptisteNguyen, idog ceva, J. Xu, and M. T. LÊ, “majianjia/nnom: v0.4.3,” 2021.
- [83] TensorFlow, “Tensorflow lite guide.” <https://www.tensorflow.org/lite/guide>. Accessed March 2021.
- [84] STMicroelectronics, “Artificial intelligence ecosystem for stm32.” https://www.st.com/content/st_com/en/ecosystems/artificial-intelligence-ecosystem-stm32.html. Accessed June 2021.
- [85] Eta Compute, “Tensai@flow.” <https://etacompute.com/tensai-flow/>. Accessed March 2021.
- [86] O. D. Lara and M. A. Labrador, “A survey on human activity recognition using wearable sensors,” *IEEE communications surveys & tutorials*, vol. 15, no. 3, pp. 1192–1209, 2012.

-
- [87] J. Meyer, A. Frank, T. Schlebusch, and E. Kasneci, "U-har: A convolutional approach to human activity recognition combining head and eye movements for context-aware smart glasses," *Proceedings of the ACM on Human-Computer Interaction*, vol. 6, no. ETRA, pp. 1–19, 2022.
- [88] C. Pacchierotti, G. Salvietti, I. Hussain, L. Meli, and D. Prattichizzo, "The hring: A wearable haptic device to avoid occlusions in hand tracking," in *2016 IEEE haptics symposium (HAPTICS)*, pp. 134–139, IEEE, 2016.
- [89] S. Sundaram, P. Kellnhofer, Y. Li, J.-Y. Zhu, A. Torralba, and W. Matusik, "Learning the signatures of the human grasp using a scalable tactile glove," *Nature*, vol. 569, no. 7758, pp. 698–702, 2019.
- [90] S. Zhang, Y. Zhao, D. T. Nguyen, R. Xu, S. Sen, J. Hester, and N. Alshurafa, "Necksense: A multi-sensor necklace for detecting eating activities in free-living conditions," *Proceedings of the ACM on interactive, mobile, wearable and ubiquitous technologies*, vol. 4, no. 2, pp. 1–26, 2020.
- [91] O. Hanton, Z. Shen, M. Fraser, and A. Roudaut, "FabricatINK: Personal fabrication of bespoke displays using electronic ink from upcycled e readers," in *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pp. 1–15, 2022.
- [92] Y. Shi, M. Manco, D. Moyal, G. Huppert, H. Araki, A. Banks, H. Joshi, R. McKenzie, A. Seewald, G. Griffin, *et al.*, "Soft, stretchable, epidermal sensor with integrated electronics and photochemistry for measuring personal uv exposures," *PloS one*, vol. 13, no. 1, p. e0190233, 2018.
- [93] F. Attal, S. Mohammed, M. Dedabrishvili, F. Chamroukhi, L. Oukhellou, and Y. Amirat, "Physical human activity recognition using wearable sensors," *Sensors*, vol. 15, no. 12, pp. 31314–31338, 2015.
- [94] A. Ghandeharioun, S. Fedor, L. Sangermano, D. Ionescu, J. Alpert, C. Dale, D. Sontag, and R. Picard, "Objective assessment of depressive symptoms with machine learning and wearable sensors data," in *2017 seventh international conference on affective computing and intelligent interaction (ACII)*, pp. 325–332, IEEE, 2017.
- [95] S. Gasparrini, E. Cippitelli, E. Gambi, S. Spinsante, J. Wähslén, I. Orhan, and T. Lindh, "Proposal and experimental evaluation of fall detection solution based on wearable and depth data fusion," in *International conference on ICT innovations*, pp. 99–108, Springer, 2015.
- [96] S. Shin and W. Sung, "Dynamic hand gesture recognition for wearable devices with low complexity recurrent neural networks," in *2016 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 2274–2277, IEEE, 2016.

- [97] C. Zhang, Q. Xue, A. Waghmare, R. Meng, S. Jain, Y. Han, X. Li, K. Cunefare, T. Ploetz, T. Starner, *et al.*, “Fingerping: Recognizing fine-grained hand poses using active acoustic on-body sensing,” in *Proceedings of the 2018 CHI conference on human factors in computing systems*, pp. 1–10, 2018.
- [98] Y. Kong and Y. Fu, “Human action recognition and prediction: A survey,” *International Journal of Computer Vision*, vol. 130, no. 5, pp. 1366–1401, 2022.
- [99] S. K. Yadav, K. Tiwari, H. M. Pandey, and S. A. Akbar, “A review of multimodal human activity recognition with special emphasis on classification, applications, challenges and future directions,” *Knowledge-Based Systems*, vol. 223, p. 106970, 2021.
- [100] W. Choi and S. Savarese, “A unified framework for multi-target tracking and collective activity recognition,” in *Computer Vision–ECCV 2012: 12th European Conference on Computer Vision, Florence, Italy, October 7–13, 2012, Proceedings, Part IV 12*, pp. 215–230, Springer, 2012.
- [101] N. T. Cam, N. Van Ninh, and T. H. Trang, “Fall detection system based on pose estimation in videos,” in *International Conference on Intelligent Computing & Optimization*, pp. 162–172, Springer, 2023.
- [102] A. Sokolova and A. Konushin, “Human identification by gait from event-based camera,” in *2019 16th International Conference on Machine Vision Applications (MVA)*, pp. 1–6, IEEE, 2019.
- [103] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [104] K. Koide, E. Menegatti, M. Carraro, M. Munaro, and J. Miura, “People tracking and re-identification by face recognition for rgb-d camera networks,” in *2017 European Conference on Mobile Robots (ECMR)*, pp. 1–7, IEEE, 2017.
- [105] Texas Instruments, “The fundamentals of millimeter wave radar sensors.” <https://www.ti.com/lit/wp/spyy005a/spyy005a.pdf>, 2021.
Accessed May 2022.
- [106] Y. M. Lui, D. Bolme, B. A. Draper, J. R. Beveridge, G. Givens, and P. J. Phillips, “A meta-analysis of face recognition covariates,” in *2009 IEEE 3rd International Conference on Biometrics: Theory, Applications, and Systems*, pp. 1–8, IEEE, 2009.
- [107] B. J. Boom, G. Beumer, L. J. Spreeuwiers, and R. N. Veldhuis, “The effect of image resolution on the performance of a face recognition system,” in *2006 9th international conference on control, automation, robotics and vision*, pp. 1–6, IEEE, 2006.

- [108] Z. Zhuang, D. Landsittel, S. Benson, R. Roberge, and R. Shaffer, "Facial anthropometric differences among gender, ethnicity, and age groups," *Annals of occupational hygiene*, vol. 54, no. 4, pp. 391–402, 2010.
- [109] H. Cui and N. Dahnoun, "Human posture capturing with millimetre wave radars," in *2020 9th Mediterranean Conference on Embedded Computing (MECO)*, pp. 1–4, IEEE, 2020.
- [110] Y. Li, J. Deng, Y. Zhang, J. Ji, H. Li, and Y. Zhang, "Ezfusion: A close look at the integration of lidar, millimeter-wave radar, and camera for accurate 3d object detection and tracking," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11182–11189, 2022.
- [111] V. Venugopal and S. Kannan, "Accelerating real-time lidar data processing using gpus," in *2013 IEEE 56th International Midwest Symposium on Circuits and Systems (MWSCAS)*, pp. 1168–1171, IEEE, 2013.
- [112] L. R. Cenkeramaddi, J. Bhatia, A. Jha, S. K. Vishkarma, and J. Soumya, "A survey on sensors for autonomous systems," in *2020 15th IEEE Conference on Industrial Electronics and Applications (ICIEA)*, pp. 1182–1187, IEEE, 2020.
- [113] K. Qian, S. Zhu, X. Zhang, and L. E. Li, "Robust multimodal vehicle detection in foggy weather using complementary lidar and radar signals," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 444–453, 2021.
- [114] F. Luo, S. Poslad, and E. Bodanese, "Temporal convolutional networks for multiperson activity recognition using a 2-d lidar," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7432–7442, 2020.
- [115] Photonics Spectra, "Integrated photonics looks to advance safety for lidar." https://www.photonics.com/Articles/Integrated_Photonics_Looks_to_Advance_Safety_for/a64791. Accessed September 2024.
- [116] Image Sensors World, "Ieee spectrum on image sensor damage by lasers." <https://image-sensors-world.blogspot.com/2019/01/ieee-spectrum-on-image-sensor-damage-by.html>. Accessed September 2024.
- [117] L. Middleton, A. A. Buss, A. Bazin, and M. S. Nixon, "A floor sensor system for gait recognition," in *Fourth IEEE Workshop on Automatic Identification Advanced Technologies (AutoID'05)*, pp. 171–176, IEEE, 2005.
- [118] J.-S. Fang, Q. Hao, D. J. Brady, B. D. Guenther, and K. Y. Hsu, "Real-time human identification using a pyroelectric infrared detector array and hidden markov models," *Optics express*, vol. 14, no. 15, pp. 6643–6658, 2006.

- [119] J. Roche, V. De-Silva, J. Hook, M. Moencks, and A. Kondo, “A multimodal data processing system for lidar-based human activity recognition,” *IEEE Transactions on Cybernetics*, vol. 52, no. 10, pp. 10027–10040, 2021.
- [120] Q. Yuan and I.-M. Chen, “Localization and velocity tracking of human via 3 imu sensors,” *Sensors and Actuators A: Physical*, vol. 212, pp. 25–33, 2014.
- [121] J. A. Corrales, F. A. Candelas, and F. Torres, “Hybrid tracking of human operators using imu/uwb data fusion by a kalman filter,” in *Proceedings of the 3rd ACM/IEEE international conference on Human robot interaction*, pp. 193–200, 2008.
- [122] R. Tesoriero, J. A. Gallud, M. D. Lozano, and V. M. Penichet, “Tracking autonomous entities using rfid technology,” *IEEE Transactions on Consumer Electronics*, vol. 55, no. 2, pp. 650–655, 2009.
- [123] Q. Ma, X. Li, G. Li, B. Ning, M. Bai, and X. Wang, “Mrliht: Mobile rfid-based localization for indoor human tracking,” *Sensors*, vol. 20, no. 6, p. 1711, 2020.
- [124] X. Li, D. Zhang, Q. Lv, J. Xiong, S. Li, Y. Zhang, and H. Mei, “Indotrack: Device-free indoor human tracking with commodity wi-fi,” *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 1, no. 3, pp. 1–22, 2017.
- [125] K. Qian, C. Wu, Y. Zhang, G. Zhang, Z. Yang, and Y. Liu, “Widar2. 0: Passive human tracking with a single wi-fi link,” in *Proceedings of the 16th annual international conference on mobile systems, applications, and services*, pp. 350–361, 2018.
- [126] Apple, “Apple watch. empowering your patients to live a healthier day.” <https://www.apple.com/healthcare/apple-watch/>. Accessed August 2023.
- [127] B. Zhou, J. Lu, X. Xie, and H. Zhou, “Human identification based on mmwave radar using deep convolutional neural network,” in *2021 3rd International Symposium on Smart and Healthy Cities (ISHC)*, pp. 90–94, IEEE, 2021.
- [128] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, “Pointnet: Deep learning on point sets for 3d classification and segmentation,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 652–660, 2017.
- [129] N. Đăng, “Fmcw vs pulse radar.” <https://www.slideshare.net/slideshow/fmcw-vs-pulse-radar/16168988>. Accessed September 2024.

- [130] EMERSON, “Frequency modulated continuous wave technology.” <https://www.emerson.com/documents/automation/white-paper-frequency-modulated-continuous-wave-technology-rosemount-en-1265738.pdf>. Accessed September 2024.
- [131] Texas Instruments, “Introduction to mmwave sensing: Fmcw radars.” <https://adasauto.blogspot.com/2018/04/introduction-to-mmwave-sensing-fmcw.html>, 2018. Accessed June 2023.
- [132] A. Farina and F. A. Studer, “A review of cfar detection techniques in radar systems,” *Microwave Journal*, vol. 29, p. 115, 1986.
- [133] G. Thiagarajan, S. Hosur, and S. Gurugopinath, “A multi-stage constant false-alarm rate detector for millimeter wave radars,” in *2022 IEEE International Conference on Signal Processing and Communications (SPCOM)*, pp. 1–5, IEEE, 2022.
- [134] Texas Instruments, “xwr1843 evaluation module (xwr1843boost) single-chip mmwave sensing solution.” <https://www.ti.com/lit/ug/spruim4b/spruim4b.pdf>, 2020. Accessed October 2022.
- [135] Texas Instruments, “mmwave demo visualizer.” https://dev.ti.com/gallery/view/mmwave/mmWave_Demo_Visualizer/ver/3.5.0/. Accessed May 2022.
- [136] W. Menzel and A. Moebius, “Antenna concepts for millimeter-wave automotive radar sensors,” *Proceedings of the IEEE*, vol. 100, no. 7, pp. 2372–2379, 2012.
- [137] Karthik Ramasubramanian, “mmwave radar for automotive and industrial applications.” https://www.ti.com/content/dam/videos/external-videos/2/3816841626001/5675916489001.mp4/subassets/Mmwave_webinar_Dec2017.pdf, 2017. Accessed October 2023.
- [138] Texas Instruments, “Awr1443boost: The difference between awr 1443boost and iwr 1443 boost.” <https://e2e.ti.com/support/sensors-group/sensors/f/sensors-forum/861621/awr1443boost-the-difference-between-awr-1443boost-and-iwr-1443-boost>. Accessed July 2022.
- [139] Texas Instruments, “Awr1642: Can’t choose between awr and iwr range.” <https://e2e.ti.com/support/sensors-group/sensors/f/sensors-forum/745969/awr1642-can-t-choose-between-awr-and-iwr-range>. Accessed July 2022.

BIBLIOGRAPHY

- [140] Texas Instruments, "Ccs/iwr6843: Iwr6843 vs awr6843 version question." <https://e2e.ti.com/support/tools/code-composer-studio-group/ccs/f/code-composer-studio-forum/931016/ccs-iwr6843-iwr6843-vs-awr6843-version-question>. Accessed July 2022.
- [141] Texas Instruments, "Iwr1443 single-chip 76- to 81-ghz mmwave sensor." <https://www.ti.com/lit/ds/symlink/iwr1443.pdf>, 2018. Accessed October 2022.
- [142] Texas Instruments, "Iwr1642 single-chip 76- to 81-ghz mmwave sensor." <https://www.ti.com/lit/ds/symlink/iwr1642.pdf>, 2018. Accessed October 2022.
- [143] Texas Instruments, "Iwr1843 single-chip 76- to 81-ghz fmcw mmwave sensor." <https://www.ti.com/lit/ds/symlink/iwr1843.pdf>, 2022. Accessed October 2022.
- [144] Texas Instruments, "Iwr6843, iwr6443 single-chip 60- to 64-ghz mmwave sensor." <https://www.ti.com/lit/ds/symlink/iwr6443.pdf>, 2021. Accessed October 2022.
- [145] Texas Instruments, "Iwr6843, iwr6443 single-chip 60- to 64-ghz mmwave sensor." <https://www.ti.com/lit/ds/symlink/iwr6843.pdf>, 2021. Accessed October 2022.
- [146] Texas Instruments, "Mmwave-sdk." <https://www.ti.com/tool/download/MMWAVE-SDK>. Accessed June 2022.
- [147] Texas Instruments, "mmwave demo visualizer." <https://www.ti.com/lit/ug/swru529c/swru529c.pdf>. Accessed May 2022.
- [148] Texas Instruments, "mmwave radar interface control document." https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/1023/2043.mmWave_2D00_Radar_2D00_Interface_2D00_Control.pdf. Accessed July 2022.
- [149] Texas Instruments, "Mmwave sdk user guide." https://e2e.ti.com/cfs-file/__key/communityserver-discussions-components-files/1023/7801.mmwave_5F00_sdk_5F00_user_5F00_guide.pdf. Accessed June 2022.

-
- [150] Texas Instruments, “Programming chirp parameters in ti radar devices.” <https://www.ti.com/lit/an/swra553a/swra553a.pdf>. Accessed July 2022.
- [151] Texas Instruments, “Mimoradar.” <https://www.ti.com/lit/an/swra554a/swra554a.pdf>. Accessed July 2022.
- [152] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, *et al.*, “Recent advances in convolutional neural networks,” *Pattern recognition*, vol. 77, pp. 354–377, 2018.
- [153] Y. Yu, X. Si, C. Hu, and J. Zhang, “A review of recurrent neural networks: Lstm cells and network architectures,” *Neural computation*, vol. 31, no. 7, pp. 1235–1270, 2019.
- [154] M. W. Gardner and S. Dorling, “Artificial neural networks (the multilayer perceptron)—a review of applications in the atmospheric sciences,” *Atmospheric environment*, vol. 32, no. 14-15, pp. 2627–2636, 1998.
- [155] S. Sharma, S. Sharma, and A. Athaiya, “Activation functions in neural networks,” *Towards Data Sci*, vol. 6, no. 12, pp. 310–316, 2017.
- [156] A. D. Rasamoelina, F. Adjailia, and P. Sinčák, “A review of activation function for artificial neural network,” in *2020 IEEE 18th World Symposium on Applied Machine Intelligence and Informatics (SAMI)*, pp. 281–286, IEEE, 2020.
- [157] G. E. Hinton, S. Osindero, and Y.-W. Teh, “A fast learning algorithm for deep belief nets,” *Neural computation*, vol. 18, no. 7, pp. 1527–1554, 2006.
- [158] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.
- [159] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “Imagenet classification with deep convolutional neural networks,” *Advances in neural information processing systems*, vol. 25, 2012.
- [160] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” *arXiv preprint arXiv:1409.1556*, 2014.
- [161] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.

- [162] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, "Deep networks with stochastic depth," in *Computer Vision—ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pp. 646–661, Springer, 2016.
- [163] K. O'shea and R. Nash, "An introduction to convolutional neural networks," *arXiv preprint arXiv:1511.08458*, 2015.
- [164] S. Albawi, T. A. Mohammed, and S. Al-Zawi, "Understanding of a convolutional neural network," in *2017 international conference on engineering and technology (ICET)*, pp. 1–6, Ieee, 2017.
- [165] H. MrYang, "Convolutional neural networks notes." https://blog.csdn.net/qq_38276972/article/details/114524472, 2021.
Accessed June 2021.
- [166] G. Tolias, R. Sircé, and H. Jégou, "Particular object retrieval with integral max-pooling of cnn activations," *arXiv preprint arXiv:1511.05879*, 2015.
- [167] L. R. Medsker, L. Jain, *et al.*, "Recurrent neural networks," *Design and Applications*, vol. 5, no. 64-67, p. 2, 2001.
- [168] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE transactions on neural networks*, vol. 5, no. 2, pp. 157–166, 1994.
- [169] colah, "Understanding lstm networks." <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>.
Accessed January 2023.
- [170] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [171] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [172] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with lstm," *Neural computation*, vol. 12, no. 10, pp. 2451–2471, 2000.
- [173] A. Graves and J. Schmidhuber, "Framewise phoneme classification with bidirectional lstm and other neural network architectures," *Neural networks*, vol. 18, no. 5-6, pp. 602–610, 2005.
- [174] K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink, and J. Schmidhuber, "Lstm: A search space odyssey," *IEEE transactions on neural networks and learning systems*, vol. 28, no. 10, pp. 2222–2232, 2016.

- [175] M. Jaderberg, K. Simonyan, A. Zisserman, *et al.*, “Spatial transformer networks,” *Advances in neural information processing systems*, vol. 28, 2015.
- [176] P. Cunningham, M. Cord, and S. J. Delany, “Supervised learning,” in *Machine learning techniques for multimedia: case studies on organization and retrieval*, pp. 21–49, Springer, 2008.
- [177] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *International conference on machine learning*, pp. 448–456, pmlr, 2015.
- [178] N. Bjorck, C. P. Gomes, B. Selman, and K. Q. Weinberger, “Understanding batch normalization,” *Advances in neural information processing systems*, vol. 31, 2018.
- [179] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” *arXiv preprint arXiv:1607.06450*, 2016.
- [180] X. Ying, “An overview of overfitting and its solutions,” in *Journal of physics: Conference series*, vol. 1168, p. 022022, IOP Publishing, 2019.
- [181] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *et al.*, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *kdd*, vol. 96, pp. 226–231, 1996.
- [182] ARM, “big.little technology: The future of mobile.” <https://armkeil.blob.core.windows.net/developer/Files/pdf/white-paper/big-little-technology-the-future-of-mobile.pdf>, 2013.
Accessed June 2024.
- [183] STMicroelectronics, “Stm32l4r5xx datasheet.” <https://www.st.com/resource/en/datasheet/stm32l4r5zg.pdf>.
Accessed.
- [184] Ambiq Micro, “Apollo2 mcu datasheet.” <https://ambiq.com/wp-content/uploads/2020/10/Apollo2-MCU-Datasheet.pdf>.
Accessed.
- [185] K.-S. Yeo and K. Roy, *Low voltage, low power VLSI subsystems*. McGraw-Hill, Inc., 2004.
- [186] I. Loshchilov and F. Hutter, “Sgdr: Stochastic gradient descent with warm restarts,” *arXiv preprint arXiv:1608.03983*, 2016.
- [187] R. De Maesschalck, D. Jouan-Rimbaud, and D. L. Massart, “The mahalanobis distance,” *Chemometrics and intelligent laboratory systems*, vol. 50, no. 1, pp. 1–18, 2000.

- [188] G. J. McLachlan, “Mahalanobis distance,” *Resonance*, vol. 4, no. 6, pp. 20–26, 1999.
- [189] TensorFlow, “Converter command line reference.” https://github.com/tensorflow/tensorflow/blob/master/tensorflow/lite/g3doc/r1/convert/cmdline_reference.md.
Accessed January 2021.
- [190] GCC team, “Gcc, the gnu compiler collection.” <https://gcc.gnu.org/>.
Accessed March 2021.
- [191] Arm Developer, “Gnu arm embedded toolchain.” <https://developer.arm.com/tools-and-software/open-source-software/developer-tools/gnu-toolchain/gnu-rm>.
Accessed March 2021.
- [192] J. Pegoraro and M. Rossi, “Real-time people tracking and identification from sparse mm-wave radar point-clouds,” *IEEE Access*, vol. 9, pp. 78504–78520, 2021.
- [193] G. C. Platform, “Oauth api verification faqs.” <https://support.google.com/cloud/answer/13463073>.
Accessed June 2023.
- [194] P. R. Evans, “Rotations and rotation matrices,” *Acta Crystallographica Section D: Biological Crystallography*, vol. 57, no. 10, pp. 1355–1359, 2001.
- [195] C.-K. Shene, “Geometric transformations.” <https://pages.mtu.edu/~shene/COURSES/cs3621/NOTES/geometry/geo-tran.html>.
Accessed May 2023.
- [196] B. Zhuo, “Transformation matrix of points in 3d coordinate system.” <https://zhuanlan.zhihu.com/p/388164543>.
Accessed May 2023.
- [197] S. Jardak, T. Kiuru, M. Metso, P. Pursula, J. Häkli, M. Hirvonen, S. Ahmed, and M.-S. Alouini, “Detection and localization of multiple short range targets using fmcw radar signal,” in *2016 Global Symposium on Millimeter Waves (GSMM) & ESA Workshop on Millimetre-Wave Technology and Applications*, pp. 1–4, IEEE, 2016.
- [198] J. Redmon and A. Farhadi, “Yolov3: An incremental improvement,” *arXiv preprint arXiv:1804.02767*, 2018.
- [199] Ultralytics, “Yolov3, yolov3-ultralytics, and yolov3u.” <https://docs.ultralytics.com/models/yolov3/>.
Accessed October 2023.

- [200] H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid, and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 658–666, 2019.
- [201] F. Bogo, J. Romero, M. Loper, and M. J. Black, “Faust: Dataset and evaluation for 3d mesh registration,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3794–3801, 2014.
- [202] D. Griffiths, “Point cloud classification with pointnet.” <https://keras.io/examples/vision/pointnet/>. Accessed January 2024.
- [203] Google, “Google colaboratory.” <https://colab.google/>. Accessed January 2024.

